

Classical ML and Deep Learning

for the InfoEdge Data Scientist and AI Scientist Selection

Concepts from zero, 675 OA style questions, fully worked solutions

Syllabus covered (exactly as listed in the InfoEdge Pre-Placement Talk 2025-26, Classical ML and Deep Learning block, 66.66% weightage of Rounds 1 and 2): Bagging, Boosting, Gradient Descent, Decision Trees, Linear and Logistic Regression, Random Forests, Gradient Boosting Machines, XGBoost, LightGBM, SVM and the Kernel Trick, Regularisation, Bias-Variance Tradeoff, Metrics (Precision, Recall, F1, TPR and more), Confusion Matrix, Missing Data Imputation, Matrix Factorisations, Density Estimation (KDE, K-Means and more), MAP, MLE, Basic NLP, TF-IDF, Embeddings (Word2Vec, FastText, BERT), Architectures (LSTM, CNN, RNN, Transformers, GANs).

Prepared for Nishant, M.Tech AI, IIT Jodhpur

September 2026

Every page carries a “Stuck on this page?” link in the footer. Every section starts with a companion resource. All links are clickable.

Stuck on this page? See: [StatQuest channel \(short, visual explanations\)](#)

How to use this book

The plan

- Each chapter has two halves: a concept half that assumes nothing, and a question bank of at least 35 questions with full solutions.
- Read the concept half once slowly. Then attempt questions with the solution box covered. Say your answer out loud before checking; InfoEdge Rounds 1 and 2 are pen and paper explanation rounds, so speaking the reasoning is the real skill.
- Questions marked ★ **PYQ (...)** come first in each bank. Treat them as highest priority.
- Question tags: MCQ (one correct option), MSQ (one or more correct), NAT (numerical answer), SUB (short subjective, the way the interview asks it).
- Difficulty tags: E (warm up), M (typical OA level), H (hard OA or interview follow up).
- The “Interview angle” box at the end of each chapter lists the follow ups a senior data scientist asks once your first answer is right.

What we actually know about the InfoEdge ML and DL questions

The following intelligence was collected from public candidate reports. It is the basis for every PYQ mark in this book.

- **OA structure (IIT Roorkee drive):** four sections; Machine Learning had 5 questions on topics like linear regression, SVMs and bias-variance; Deep Learning had 7 questions on RNNs, CNNs, overfitting and word embeddings. Source: [Infoedge test in IITR \(Scribd\)](#).
- **40 MCQ paper (IIT BHU, recalled by an InfoEdge DS hire),** ML and DL items: bag of words; stemming, lemmatization, tokenization; types of ensembles (bagging, boosting, stacking); Xavier and He initialization; ResNets for the vanishing gradient problem; which losses are classification losses (cross entropy, log loss, hinge); deep belief networks and RBMs; is Random Forest bagging or boosting; universal approximation theorem; one vs one and one vs all for multiclass; bias-variance graphs (error vs model complexity, error vs iterations); number of hidden layers in a “deep” network; which is faster, SGD or batch GD; LSTM used in speech recognition; what a false negative means. Source: [Placement guidance repo by an InfoEdge DS hire](#).
- **Interview, same source** (DL was “the most drilled section”): vanishing and exploding gradients and fixes; Xavier vs He; weight decay; dropout; learning rate and how to adjust it; how backprop works; what to do if stuck in a local optimum; CNN filters, stride, padding and the output size formula $\frac{n+2p-f}{s} + 1$; max pooling; is a CNN rotation invariant, translation invariant; which DL model for time series (RNN, GRU, LSTM, BPTT). Classical ML: reducing overfitting in trees (pruning); SVM, hinge loss, kernels, margin, support vectors; heteroscedasticity; multicollinearity and VIF; missing value handling (drop, mean, median, mode, model based imputer); imbalanced data (undersampling, oversampling, SMOTE, class weights); logistic regression, confusion matrix, accuracy, F1, precision, recall in the coding round.
- **Interview (Glassdoor, Info Edge):** Random Forest intuition, bootstrap sampling, random feature selection, how each tree is trained, why RF reduces variance vs a single tree, RF vs Gradient Boosting, feature importance; vanishing gradient and its fix; transformers; outliers; PCA; NLP concepts; and “pseudo coding a decision tree”. Source: [Glassdoor](#), [Info Edge interviews](#).
- **Round format (IIT Kharagpur, 2025):** Round 2 “tested your ability to explain the architecture of ML and DL models” on pen and paper; the candidate advises knowing the maths behind each model and Transformers in depth (CS229, CS224n). Source: [IIT KGP Foresight 2025](#), [InfoEdge](#).

Honesty note on PYQs

InfoEdge does not publish papers and candidates only remember topics, not exact wording. So every PYQ here is a faithful reconstruction of a reported question: the concept and form match the report, the numbers and options are mine. Tags: OA = asked in the online test, INT = asked in an interview. Where no InfoEdge report exists for a chapter, the bank says so and starts with questions modelled on the reported style.

Tactics for a negative marking MCQ/MSQ paper

- ML/DL OA items are mostly one step conceptual checks with traps in the wording (“always”, “only”, “guarantees”). Read every absolute word twice.
- In MSQ, tick an option only if you can justify it in one sentence.
- Numericals are small: Gini, entropy, information gain, precision and recall from a confusion matrix, CNN output shapes and parameter counts, one gradient descent step, TF-IDF of one word, attention scores. Each chapter drills its numerical type.

Quick reference (memorise)

Models

Linear regression: $\hat{y} = w^\top x + b$, OLS $w = (X^\top X)^{-1} X^\top y$. Logistic: $p = \sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma'(z) = \sigma(1 - \sigma)$, log loss $-\log p - (1 - p) \log(1 - p)$, gradient $(p - y)x$.
 Ridge: $+\lambda \|w\|_2^2$, closed form $(X^\top X + \lambda I)^{-1} X^\top y$. Lasso: $+\lambda \|w\|_1$ (sparse). Elastic net: both.
 GD: $w \leftarrow w - \eta \nabla L$. Momentum: $v \leftarrow \beta v + \nabla L$, $w \leftarrow w - \eta v$. Adam: m, v moving averages, bias correction $\hat{m} = m / (1 - \beta_1^t)$.
 Expected error = Bias² + Variance + irreducible noise.

Trees and ensembles

Gini = $1 - \sum p_k^2$ (max 0.5 for 2 classes). Entropy = $-\sum p_k \log_2 p_k$ (max 1 bit for 2 classes). Information gain = parent impurity minus weighted child impurity.
 Bootstrap: $P(\text{a point is left out}) = (1 - 1/n)^n \rightarrow e^{-1} \approx 0.368$, so about 63.2% unique points per bag.
 Variance of the average of B models with variance σ^2 and pairwise correlation ρ : $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
 AdaBoost: $\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$. Gradient boosting: fit tree to negative gradient (residuals for MSE), $F_m = F_{m-1} + v h_m$.
 XGBoost leaf weight $w^* = -\frac{G}{H+\lambda}$, gain = $\frac{1}{2} \left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda} \right] - \gamma$.

SVM and metrics

Hard margin: minimise $\frac{1}{2} \|w\|^2$ s.t. $y_i(w^\top x_i + b) \geq 1$; margin width = $2/\|w\|$. Soft margin: $+C \sum \xi_i$; hinge loss $\max(0, 1 - yf(x))$. RBF kernel $e^{-\gamma \|x-z\|^2}$.
 Precision = $\frac{TP}{TP+FP}$, Recall = TPR = Sensitivity = $\frac{TP}{TP+FN}$, Specificity = $\frac{TN}{TN+FP}$, FPR = $1 - \text{Specificity}$, F1 = $\frac{2PR}{P+R}$, $F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R}$. AUC = P(random positive scored above random negative).

Deep learning

Conv output size: $\left\lfloor \frac{n+2p-f}{s} \right\rfloor + 1$. Conv params: $(f \cdot f \cdot C_{in} + 1) \cdot C_{out}$. Dense params: $(n_{in} + 1)n_{out}$.
 Xavier: $\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$ (tanh, sigmoid). He: $\text{Var}(w) = \frac{2}{n_{in}}$ (ReLU).
 Attention: $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$. LSTM: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $h_t = o_t \odot \tanh(c_t)$.
 TF-IDF: $\text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)}$. Cosine similarity $\frac{a \cdot b}{\|a\| \|b\|}$.

Contents

How to use this book	2
Quick reference	4
1 Linear and Logistic Regression	10
1.1 What “learning” means, in one example	10
1.2 Linear regression: least squares	10
1.3 Assumptions of linear regression (a reported InfoEdge interview topic)	10
1.4 Judging a regression fit	10
1.5 Logistic regression: a linear model for probabilities	11
1.6 More than two classes	11
1.7 Loss functions: which are for classification?	11
1.8 Question bank: Linear and logistic regression	12
2 Gradient Descent and Optimisation	21
2.1 The idea: walk downhill in fog	21
2.2 Choosing the learning rate	21
2.3 Batch, stochastic and mini batch gradient descent	21
2.4 Convexity, local minima and saddle points	22
2.5 Momentum, RMSProp, Adam	22
2.6 Why feature scaling matters for GD	22
2.7 Second order and other ideas (for interview depth)	22
2.8 Question bank: Gradient descent and optimisation	23
3 Bias-Variance Tradeoff and Regularisation	31
3.1 Two ways to be wrong	31
3.2 The two graphs InfoEdge asks about	31
3.3 Modern twist: double descent	31
3.4 Regularisation: penalise complexity	32
3.5 Other regularisers you must be able to name	32
3.6 Cross validation in one paragraph	32
3.7 Question bank: Bias-variance and regularisation	33
4 Decision Trees	41
4.1 A tree is a game of twenty questions	41
4.2 How to choose a question: impurity	41
4.3 The three classic algorithms	41
4.4 Regression trees	41
4.5 Overfitting and pruning (a reported InfoEdge interview question)	42
4.6 Pseudo code (a reported InfoEdge interview task)	42
4.7 Strengths, weaknesses, feature importance	42
4.8 Question bank: Decision trees	43
5 Ensembles, Bagging and Random Forests	51
5.1 Wisdom of crowds	51

5.2	Three families of ensembles (a reported InfoEdge OA question)	51
5.3	Bootstrap sampling	51
5.4	Random Forest = bagged trees + random feature subsets	51
5.5	Why RF reduces variance and overfitting vs one tree (Glassdoor InfoEdge)	52
5.6	Extra Trees, feature importance, and practical notes	52
5.7	Random Forest vs Gradient Boosting (Glassdoor InfoEdge)	52
5.8	Question bank: Ensembles, bagging and random forests	52
6	Boosting: AdaBoost, Gradient Boosting, XGBoost, LightGBM	61
6.1	The core idea: learn from your mistakes, one small step at a time	61
6.2	AdaBoost (adaptive boosting)	61
6.3	Gradient boosting: gradient descent in function space	61
6.4	XGBoost: regularised, second order, engineered for speed	61
6.5	LightGBM and CatBoost: what makes them different	62
6.6	Question bank: Boosting	63
7	Support Vector Machines and the Kernel Trick	71
7.1	Which line is best?	71
7.2	The hard margin optimisation	71
7.3	Soft margin: allowing mistakes, and the C parameter	71
7.4	The dual problem and why it matters	72
7.5	The kernel trick	72
7.6	Practicalities	72
7.7	Question bank: Support vector machines	73
8	Evaluation Metrics and the Confusion Matrix	81
8.1	The confusion matrix	81
8.2	The metrics, each with its question	81
8.3	Threshold free metrics: ROC AUC and PR AUC	81
8.4	Multiclass averaging	82
8.5	Imbalanced data (a reported InfoEdge interview topic)	82
8.6	Regression and ranking metrics (brief)	82
8.7	Question bank: Evaluation metrics	83
9	Maximum Likelihood (MLE) and Maximum a Posteriori (MAP)	91
9.1	Likelihood: flip the question	91
9.2	The recipe (works every time)	91
9.3	Why ML losses are negative log likelihoods	91
9.4	MAP: add a prior belief	91
9.5	Properties of MLE worth quoting	92
9.6	Question bank: MLE and MAP	92
10	Missing Data Imputation (with outliers and imbalance)	100
10.1	Why missing values are a modelling problem, not a cleaning chore	100
10.2	The three missingness mechanisms (Rubin)	100
10.3	Option 1: deletion	100
10.4	Option 2: simple (univariate) imputation	100

10.5	Worked example: what mean imputation does to variance	101
10.6	Option 3: multivariate imputation	101
10.7	Option 4: let the model handle it (XGBoost, LightGBM)	101
10.8	The missing indicator trick	101
10.9	Leakage: fit the imputer on train only	101
10.10	Outliers: the close cousin (reported as asked alongside)	102
10.11	Imbalanced data (reported in the same InfoEdge coding discussion)	102
10.12	Question bank: Missing data, outliers, imbalance	102
11	Matrix Factorisations: SVD, PCA, NMF and Recommenders	112
11.1	What “factorising a matrix” means	112
11.2	SVD from scratch	112
11.3	Low rank approximation (Eckart Young)	112
11.4	PCA is SVD of centred data (reported InfoEdge interview topic)	112
11.5	NMF: non negative parts	113
11.6	Matrix factorisation for recommendations	113
11.7	Question bank: Matrix factorisations	114
12	Density Estimation and Clustering: Histograms, KDE, K-Means, GMM and EM	122
12.1	What is density estimation?	122
12.2	Histograms: the simplest estimator	122
12.3	Kernel Density Estimation (KDE)	122
12.4	K-Means clustering	122
12.5	Gaussian Mixture Models and EM	123
12.6	Other clustering methods worth knowing	123
12.7	Question bank: Density estimation and clustering	124
13	Basic NLP: Preprocessing, Bag of Words and TF-IDF	132
13.1	Why text needs special treatment	132
13.2	Tokenisation (reported InfoEdge OA topic)	132
13.3	Normalisation, stopwords, stemming and lemmatisation (reported OA topic)	132
13.4	Bag of Words (reported OA topic)	132
13.5	TF-IDF: down weight words everyone uses	133
13.6	Classic text classifiers	133
13.7	Other basic NLP vocabulary	133
13.8	Question bank: Basic NLP and TF-IDF	134
14	Embeddings: Word2Vec, GloVe, FastText and BERT	141
14.1	From one hot to dense vectors	141
14.2	Word2Vec (Mikolov et al., 2013)	141
14.3	GloVe	141
14.4	FastText: embeddings from character n-grams	141
14.5	Static vs contextual embeddings	142
14.6	BERT (Devlin et al., 2018)	142
14.7	Sentence and document embeddings	142
14.8	Question bank: Embeddings	143

15 Neural Network Foundations: Backprop, Activations, Initialisation, Regularisation	151
15.1 From a neuron to a network	151
15.2 Universal Approximation Theorem (reported InfoEdge OA topic)	151
15.3 Activation functions	151
15.4 Loss functions (reported OA topic: classification losses)	151
15.5 Backpropagation: the chain rule, organised	152
15.6 Vanishing and exploding gradients (most drilled InfoEdge DL interview topic)	152
15.7 Weight initialisation: Xavier and He (reported OA topic)	152
15.8 Residual connections (ResNet, reported OA topic)	152
15.9 Normalisation layers	152
15.10 Regularisation in deep learning (reported interview topics)	153
15.11 Learning rate and getting unstuck (reported interview topics)	153
15.12 RBMs and Deep Belief Networks (reported OA topic)	153
15.13 Question bank: Neural network foundations	153
16 Convolutional Neural Networks (CNNs)	162
16.1 Why not just use a dense network on images?	162
16.2 The convolution operation, by hand	162
16.3 Output size: the formula every OA asks (reported InfoEdge interview)	162
16.4 Parameters and computation	162
16.5 Pooling (reported InfoEdge interview: max pooling)	163
16.6 Invariance vs equivariance (reported InfoEdge interview: rotation and translation)	163
16.7 Receptive field and hierarchy	163
16.8 Landmark architectures	163
16.9 Transfer learning	163
16.10 Question bank: Convolutional neural networks	164
17 Recurrent Networks: RNN, LSTM, GRU and Sequence Models	172
17.1 Why sequences need memory	172
17.2 The vanilla RNN	172
17.3 Backpropagation Through Time (BPTT) (reported InfoEdge interview topic)	172
17.4 LSTM: a memory with gates	172
17.5 GRU: a lighter LSTM	173
17.6 Variants and training tricks	173
17.7 Where LSTMs shine: speech and time series (reported InfoEdge OA and interview topics)	173
17.8 Why Transformers replaced RNNs	173
17.9 Question bank: RNN, LSTM, GRU	174
18 Transformers and Attention	182
18.1 The idea of attention, in plain words	182
18.2 Scaled dot product attention	182
18.3 Multi head attention	182
18.4 The Transformer block	182
18.5 Positional information	183
18.6 Masks	183
18.7 Three families	183

18.8 Cost and efficiency	183
18.9 Generation and decoding	183
18.10 Transformers beyond text	183
18.11 Question bank: Transformers and attention	184
19 Generative Adversarial Networks (GANs)	191
19.1 The forger and the detective	191
19.2 The minimax objective	191
19.3 Training loop in practice	191
19.4 Why GANs are hard to train	191
19.5 Important variants	191
19.6 Evaluating GANs	192
19.7 GANs vs VAEs vs diffusion models	192
19.8 Question bank: Generative Adversarial Networks	192

1 Linear and Logistic Regression

Companion resource for this section: [StatQuest: Linear Regression, Clearly Explained](#)

1.1 What “learning” means, in one example

- You have data pairs (x_i, y_i) . Example: x = years of experience, y = salary (lakh per year). You want a rule $f(x)$ that predicts y for a new person.
- A **model** is a family of rules with knobs (parameters). Linear regression uses the family $\hat{y} = wx + b$: the knobs are slope w and intercept b .
- A **loss** measures how wrong one prediction is. Training = turning the knobs to make the average loss small.
- With several features $x = (x_1, \dots, x_d)$ we write $\hat{y} = w_1x_1 + \dots + w_dx_d + b = w^\top x + b$. Stack all n rows into a matrix X ($n \times d$, with a column of 1s for b) and predictions become $\hat{y} = Xw$.

1.2 Linear regression: least squares

- Loss: squared error $(y_i - \hat{y}_i)^2$. Average it: $\text{MSE}(w) = \frac{1}{n} \sum_i (y_i - w^\top x_i)^2 = \frac{1}{n} \|y - Xw\|^2$.
- Why square? It is smooth (easy to differentiate), punishes big errors more, and it is exactly what maximum likelihood gives if noise is Gaussian (Chapter on MLE).
- Setting the gradient to zero: $\nabla_w = -\frac{2}{n} X^\top (y - Xw) = 0 \Rightarrow X^\top Xw = X^\top y \Rightarrow w = (X^\top X)^{-1} X^\top y$. These are the **normal equations**.
- One feature shortcut: $w = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = r \frac{s_y}{s_x}$, and $b = \bar{y} - w\bar{x}$. The fitted line always passes through $(\bar{x}, \bar{y})$.
- Tiny worked example: $x = 1, 2, 3$, $y = 2, 4, 5$. $\bar{x} = 2$, $\bar{y} = 11/3$. $\sum (x - \bar{x})(y - \bar{y}) = (-1)(-5/3) + 0 + (1)(4/3) = 3$. $\sum (x - \bar{x})^2 = 2$. So $w = 1.5$, $b = 11/3 - 3 = 2/3$. Line: $\hat{y} = 1.5x + 0.667$.
- Geometric picture: Xw ranges over the column space of X ; least squares picks the point in that space closest to y , i.e. the orthogonal projection. The residual $y - Xw$ is perpendicular to every column of X (so residuals sum to zero when there is an intercept).
- Cost: forming $X^\top X$ is $O(nd^2)$ and inverting is $O(d^3)$. For huge d we use gradient descent instead (Chapter 2).

1.3 Assumptions of linear regression (a reported InfoEdge interview topic)

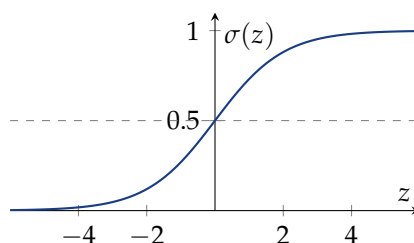
- Linearity**: $E[y | x]$ is linear in the parameters (you may use x^2 , $\log x$ as features; it is still “linear regression”).
- Independence of errors** (no autocorrelation; time series often violates this; Durbin-Watson test).
- Normality of errors** (needed for exact t-tests and confidence intervals, not for the point estimate).
- Equal variance of errors**, called **homoscedasticity**. If the spread of residuals grows with x (salary noise grows with seniority), that is **heteroscedasticity**. Detect: residual vs fitted plot shows a funnel; Breusch-Pagan test. Fix: transform y (log, square root, Box-Cox), weighted least squares, or robust (heteroscedasticity consistent) standard errors. Coefficients stay unbiased; their standard errors become wrong.
- No perfect multicollinearity**: features must not be exact linear combinations of each other, else $X^\top X$ is singular. Near collinearity inflates coefficient variance. Measure with VIF: regress feature j on the other features, get R_j^2 , then $\text{VIF}_j = \frac{1}{1 - R_j^2}$. Rule of thumb: $\text{VIF} > 5$ or > 10 is a problem. Fix: drop or combine features, PCA, or Ridge.
- Mnemonic: LINE plus “no multicollinearity”.

1.4 Judging a regression fit

- $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$ where $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$ and $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$. It is the fraction of variance explained. A model that always predicts $\bar{y}$ scores 0; on test data R^2 can be negative.
- Adding any feature never decreases training R^2 . **Adjusted R^2** = $1 - (1 - R^2) \frac{n-1}{n-d-1}$ penalises useless features.
- MAE (average absolute error) is robust to outliers; RMSE punishes large errors; MAPE is scale free but explodes near $y = 0$.

1.5 Logistic regression: a linear model for probabilities

- Task: predict a class $y \in \{0, 1\}$ (will this Naukri user apply to the job?). A straight line can output -3 or 7 , which are not probabilities.
- Fix: pass the linear score $z = w^\top x + b$ through the **sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$, which squashes any real number into $(0, 1)$. $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$.
- Interpretation: $\log \frac{p}{1-p} = w^\top x + b$. The log odds are linear. Increasing x_j by 1 multiplies the odds by e^{w_j} . If $w_j = 0.7$, odds roughly double.
- Decision boundary: predict 1 when $p \geq 0.5$, i.e. when $w^\top x + b \geq 0$. The boundary is a hyperplane, so logistic regression is a **linear classifier** even though σ is non linear.
- Loss: we cannot use MSE nicely (with sigmoid it becomes non convex). We use **log loss** (binary cross entropy): $L = -[y \log p + (1 - y) \log(1 - p)]$. If $y = 1$ and $p = 0.9$, $L = -\ln 0.9 = 0.105$; if $p = 0.1$, $L = 2.303$. Confident and wrong is punished hard.
- Beautiful gradient: using $\sigma' = \sigma(1 - \sigma)$, $\frac{\partial L}{\partial w} = (p - y)x$. Same shape as linear regression's $(\hat{y} - y)x$.
- No closed form; the loss is convex, so gradient descent (or Newton's method, called IRLS here) finds the global optimum.
- If data are perfectly separable, unregularised weights grow without bound (the loss keeps falling as $\|w\| \rightarrow \infty$). Regularisation fixes this.



1.6 More than two classes

- **Softmax (multinomial) regression**: one score z_k per class, $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$, loss $-\log p_{\text{true}}$ (categorical cross entropy).
- **One vs Rest (OvR / OvA)**: train K binary classifiers, "class k vs everything else"; predict the class with the highest score. K models, each on all data, but each is imbalanced.
- **One vs One (OvO)**: train a classifier for every pair: $\binom{K}{2} = \frac{K(K-1)}{2}$ models, each on only two classes' data; predict by voting. Used by SVC in scikit-learn because SVM training cost grows fast with n , so many small problems are cheaper.

1.7 Loss functions: which are for classification?

- Regression losses: MSE, MAE, Huber (MSE near zero, MAE far away), log-cosh, quantile loss.
- Classification losses: log loss / binary cross entropy, categorical cross entropy, hinge loss $\max(0, 1 - yf(x))$ with $y \in \{-1, +1\}$ (SVM), exponential loss $e^{-yf(x)}$ (AdaBoost), focal loss (imbalanced detection).
- "Cross entropy" and "log loss" are the same thing for classification.

Where this shows up at InfoEdge scale

- Logistic regression is still the default baseline for click and apply prediction because it is fast, calibrated and interpretable (coefficients are log odds ratios).
- Salary estimation is a regression problem with strong heteroscedasticity (noise grows with pay), which is why people model $\log(\text{salary})$.
- In the ML coding round (Round 3) the reported task was exactly: encode, split, fit LogisticRegression, print confusion matrix, precision, recall, F1.

Formula sheet

$w = (X^T X)^{-1} X^T y$; one feature: $w = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sum(x-\bar{x})^2}$, $b = \bar{y} - w\bar{x}$.
 $R^2 = 1 - SS_{res}/SS_{tot}$; adjusted $1 - (1 - R^2)\frac{n-1}{n-d-1}$; VIF = $1/(1 - R_j^2)$.
 $\sigma(z) = 1/(1 + e^{-z})$, $\sigma' = \sigma(1 - \sigma)$; log loss gradient $(p - y)x$; odds multiply by e^{w_j} per unit of x_j .
 OvR: K models; OvO: $K(K - 1)/2$ models.

1.8 Question bank: Linear and logistic regression

Q1.1 MSQ · M

★ PYQ (OA: linear regression) Which statements about ordinary least squares linear regression are true?

- (A) The fitted line always passes through $(\bar{x}, \bar{y})$ when an intercept is included.
- (B) Residuals sum to zero when an intercept is included.
- (C) Normality of errors is required to get unbiased coefficient estimates.
- (D) The OLS solution minimises the sum of absolute residuals.

Solution

- (A) True: $b = \bar{y} - w\bar{x}$ rearranges to $\bar{y} = w\bar{x} + b$.
- (B) True: the normal equation for the intercept column of 1s says $\sum(y_i - \hat{y}_i) = 0$.
- (C) False: unbiasedness needs only linearity and zero mean errors uncorrelated with x . Normality is for exact inference (t-tests, intervals).
- (D) False: that is least absolute deviations (median regression). OLS minimises squared residuals.

Answer: A, B

Q1.2 NAT · E

★ PYQ (OA: linear regression, numerical form) Fit $\hat{y} = wx + b$ by least squares to $(1, 2), (2, 4), (3, 5)$. Find w, b and the prediction at $x = 4$.

Solution

- $\bar{x} = (1 + 2 + 3)/3 = 2$; $\bar{y} = (2 + 4 + 5)/3 = 11/3 \approx 3.667$.
- Deviations: $x - \bar{x} = -1, 0, 1$; $y - \bar{y} = -5/3, 1/3, 4/3$.
- $\sum(x - \bar{x})(y - \bar{y}) = 5/3 + 0 + 4/3 = 3$; $\sum(x - \bar{x})^2 = 1 + 0 + 1 = 2$.
- $w = 3/2 = 1.5$; $b = 11/3 - 1.5 \cdot 2 = 11/3 - 3 = 2/3$.
- $\hat{y}(4) = 6 + 0.667 = 6.667$.

Answer: $w = 1.5$, $b \approx 0.667$, $\hat{y}(4) \approx 6.67$

Q1.3 MSQ · M

★ PYQ (INT: heteroscedasticity and how to fix it) Residuals of a salary model fan out (spread increases) as predicted salary grows. Which are correct?

- (A) This is heteroscedasticity.
- (B) OLS coefficient estimates become biased.
- (C) OLS standard errors (hence p-values) become unreliable.
- (D) Modelling $\log(\text{salary})$ or using weighted least squares are standard fixes.

Solution

- (A) Funnel shaped residuals = non constant error variance = heteroscedasticity.
- (B) False. Coefficients stay unbiased (the zero mean error assumption still holds); they are just no longer the minimum variance estimator.
- (C) True. The usual SE formula assumes constant σ^2 , so t-statistics are off. Robust (HC) standard errors correct this.
- (D) True. A log transform turns multiplicative noise into additive, roughly constant noise. WLS gives noisy points less weight. The interviewer in the reported round wanted a transformation; GLMs were also accepted.

Answer: A, C, D

Q1.4 NAT · M

★ **PYQ (INT: multicollinearity and VIF)** Regressing feature x_3 on all other features gives $R^2 = 0.9$. Compute VIF for x_3 and interpret.

Solution

- $VIF = 1/(1 - R_3^2) = 1/(1 - 0.9) = 1/0.1 = 10$.
- Meaning: the variance of $\hat{w}_3$ is 10 times what it would be if x_3 were uncorrelated with the other features; its standard error is $\sqrt{10} \approx 3.16$ times larger.
- 10 is the classic “serious” threshold. Remedies: drop one of the correlated features, combine them, use PCA, or use Ridge which shrinks the unstable directions.
- Note: multicollinearity hurts interpretation and coefficient stability; it barely hurts prediction accuracy.

Answer: VIF = 10

Q1.5 MSQ · M

★ **PYQ (OA: which are classification losses)** Which of these are losses used for classification?

- (A) Cross entropy / log loss
- (B) Hinge loss
- (C) Huber loss
- (D) Exponential loss

Solution

- (A) Log loss is the negative log likelihood of a Bernoulli / categorical model: the logistic regression and neural network classifier loss.
- (B) Hinge $\max(0, 1 - yf(x))$ is the SVM loss.
- (C) Huber is a robust regression loss: quadratic for small errors, linear for large ones.
- (D) Exponential loss $e^{-yf(x)}$ is what AdaBoost minimises.

Answer: A, B, D

Q1.6 NAT · E

★ **PYQ (OA: one vs one and one vs all)** A problem has 5 classes. How many binary classifiers are trained by (a) One vs All, (b) One vs One?

Solution

- OvA: one per class, class k vs rest: $K = 5$.
- OvO: one per unordered pair: $\binom{5}{2} = \frac{5 \cdot 4}{2} = 10$.
- Trade off: OvO trains more models but each on a smaller data subset (only two classes), which is why it suits kernel SVMs whose training cost grows faster than linearly in n .

Answer: (a) 5, (b) 10

Q1.7 MCQ · M

★ **PYQ (INT: logistic regression in the coding round)** In scikit-learn you fit `LogisticRegression()` with default settings. Which is true?

- (A) It is unregularised by default.
- (B) It applies L2 regularisation by default with $C = 1$, where smaller C means stronger regularisation.
- (C) It applies L1 regularisation by default.
- (D) C is the regularisation strength, larger C means stronger regularisation.

Solution

- scikit-learn's default is `penalty='l2', C=1.0`.
- C is the inverse of regularisation strength ($C = 1/\lambda$). Small C = heavy shrinkage.
- Practical consequence: always scale features before logistic regression, because the penalty treats all coefficients equally.

Answer: B**Q1.8** NAT · E

A logistic regression outputs $z = 1.2$. Compute the predicted probability and the class at threshold 0.5.

Solution

- $e^{-1.2} \approx 0.301$.
- $p = 1/(1 + 0.301) = 1/1.301 \approx 0.769$.
- $0.769 \geq 0.5$, so class 1. Shortcut: $z > 0 \iff p > 0.5$.

Answer: $p \approx 0.769$, class 1**Q1.9** NAT · M

For one example with $y = 1$, compute the log loss when $p = 0.8$ and when $p = 0.2$ (natural log).

Solution

- $y = 1$ keeps only the term $-\log p$.
- $p = 0.8$: $-\ln 0.8 = 0.223$.
- $p = 0.2$: $-\ln 0.2 = 1.609$, more than 7 times larger. Confident wrong answers dominate the loss.

Answer: 0.223 and 1.609**Q1.10** MCQ · M

In logistic regression, coefficient of “has premium account” is $w = 0.693$. What does it mean?

- (A) Probability of applying increases by 0.693.
- (B) Odds of applying double for premium users, other features fixed.
- (C) Probability doubles.
- (D) Log odds halve.

Solution

- Log odds increase by 0.693, so odds multiply by $e^{0.693} = 2$.
- Probability does not double: from $p = 0.5$ (odds 1) to odds 2 gives $p = 2/3$, not 1.
- Coefficients are log odds ratios. This is the standard interview interpretation question.

Answer: B**Q1.11** MSQ · M

Why is MSE not used with a sigmoid output for logistic regression?

- (A) MSE with sigmoid gives a non convex loss in w .
- (B) Gradients vanish when the model is confidently wrong, slowing learning.
- (C) MSE cannot be computed for probabilities.
- (D) Log loss is the maximum likelihood objective for Bernoulli labels.

Solution

- (A) True: composing squared error with sigmoid creates flat regions and local bumps.
- (B) True: MSE gradient contains $\sigma'(z) = p(1 - p)$, near 0 when $p \approx 0$ or 1, even if the answer is wrong. Log loss's gradient $(p - y)x$ cancels that factor.
- (C) False, it can be computed; it is just a poor choice.
- (D) True: $-\log$ likelihood of Bernoulli(p) is exactly log loss.

Answer: A, B, D**Q1.12** NAT · M

Logistic regression, one feature, $w = 0.5$, $b = 0$, learning rate 0.1. One example $x = 2$, $y = 0$. Do one gradient descent step on log loss.

Solution

- $z = 0.5 \cdot 2 + 0 = 1$; $p = \sigma(1) = 0.731$.
- Gradients: $\partial L / \partial w = (p - y)x = 0.731 \cdot 2 = 1.462$; $\partial L / \partial b = p - y = 0.731$.
- Step sizes: $0.1 \times 1.462 = 0.1462$ and $0.1 \times 0.731 = 0.0731$.
- Update: $w = 0.5 - 0.1462 = 0.3538$; $b = 0 - 0.0731 = -0.0731$.
- Sanity check: both moved in the direction that lowers p , which is right because the true label is 0.

Answer: $w = 0.3538$, $b = -0.0731$

Q1.13 MCQ · E

The decision boundary of logistic regression on raw features x_1, x_2 is

- (A) a circle
- (B) a straight line
- (C) a sigmoid curve
- (D) depends on the threshold only through its shape

Solution

- $p \geq t \iff w^\top x + b \geq \log \frac{t}{1-t}$: a linear inequality, so the boundary is a line (hyperplane in higher d).
- Changing the threshold only shifts the line parallel to itself.
- To get curved boundaries you must add features like $x_1^2, x_1 x_2$.

Answer: B

Q1.14 MSQ · M

Data are perfectly linearly separable. Unregularised logistic regression trained by GD:

- (A) converges to a finite optimum quickly
- (B) has weights whose norm keeps growing
- (C) produces predicted probabilities approaching 0 and 1
- (D) can be fixed by adding L2 regularisation

Solution

- Scaling w by a larger factor keeps the same boundary but pushes every p closer to the correct label, lowering log loss further. No finite minimiser exists.
- Hence (B) and (C). L2 penalty makes large $\|w\|$ costly, giving a finite solution (D).

Answer: B, C, D

Q1.15 NAT · M

Compute R^2 for the fit in Q1.2 ($\hat{y} = 1.5x + 2/3$ on $(1, 2), (2, 4), (3, 5)$).

Solution

- Predictions: 2.167, 3.667, 5.167. Residuals: $-0.167, 0.333, -0.167$.
- $SS_{res} = 0.0278 + 0.1111 + 0.0278 = 0.1667$.
- $SS_{tot} = (5/3)^2 + (1/3)^2 + (4/3)^2 = (25 + 1 + 16)/9 = 4.667$.
- $R^2 = 1 - 0.1667/4.667 = 1 - 0.0357 = 0.964$.

Answer: $R^2 \approx 0.964$

Q1.16 MSQ · M

Which statements about R^2 are true?

- (A) Adding a random noise feature never decreases training R^2 .
- (B) Test set R^2 can be negative.
- (C) $R^2 = 0.9$ implies the model is causal.
- (D) For simple linear regression with intercept, $R^2 = r^2$ (squared Pearson correlation).

Solution

- (A) OLS can always set the new coefficient to 0, so the fit can only improve or stay. Hence adjusted R^2 .
- (B) If the model is worse than predicting the training mean on new data, $SS_{res} > SS_{tot}$.
- (C) False: fit quality says nothing about causation.
- (D) True for one predictor with intercept.

Answer: A, B, D

Q1.17 NAT · M

$n = 50$, $d = 4$ features, $R^2 = 0.80$. Compute adjusted R^2 .

Solution

- Adjusted $= 1 - (1 - 0.8) \frac{n-1}{n-d-1} = 1 - 0.2 \cdot \frac{49}{45}$.
- $49/45 = 1.0889$; $0.2 \times 1.0889 = 0.2178$.
- Adjusted $R^2 = 0.782$.

Answer: ≈ 0.782

Q1.18 MCQ · M

Two features are exact copies: $x_2 = x_1$. OLS via normal equations:

- (A) works fine
- (B) fails because $X^T X$ is singular
- (C) gives $w_1 = w_2$ always
- (D) doubles the prediction

Solution

- Columns are linearly dependent, so $\text{rank}(X) < d$ and $X^T X$ is not invertible.
- Infinitely many solutions give the same fit: any $w_1 + w_2 = c$.
- Remedies: drop the duplicate, use the pseudo inverse (minimum norm solution), or Ridge ($X^T X + \lambda I$ is always invertible for $\lambda > 0$).

Answer: B

Q1.19 MSQ · M

You add x^2 and x^3 as features and fit by OLS. Which are true?

- (A) It is still linear regression.
- (B) The model can fit a curve in x .
- (C) The loss is no longer convex in w .
- (D) Features should be scaled because x^3 has a very different range.

Solution

- “Linear” means linear in parameters w . Polynomial features keep the problem a least squares problem, convex in w , so (C) false.
- Scaling matters for GD speed and for regularisation fairness, and it reduces the correlation among x, x^2, x^3 .

Answer: A, B, D

Q1.20 MCQ · E

Which metric is most robust to a few huge outliers in the target?

- (A) MSE
- (B) RMSE
- (C) MAE
- (D) R^2

Solution

- MAE grows linearly with error size; MSE/RMSE grow quadratically, so one outlier can dominate.
- The MAE minimiser is the median; the MSE minimiser is the mean. Medians ignore outliers.

Answer: C**Q1.21** NAT · M

Softmax scores $z = (2, 1, 0)$. Compute class probabilities and cross entropy loss if the true class is the second.

Solution

- $e^2 = 7.389, e^1 = 2.718, e^0 = 1$; sum = 11.107.
- $p = (7.389, 2.718, 1) / 11.107 = (0.665, 0.245, 0.090)$.
- Loss = $-\ln 0.245 = 1.408$.
- Softmax is shift invariant: subtracting max z before exponentiating avoids overflow without changing p .

Answer: $p \approx (0.665, 0.245, 0.090)$, loss ≈ 1.408 **Q1.22** MSQ · H

For OLS, the hat matrix $H = X(X^\top X)^{-1}X^\top$. Which are true?

- (A) $\hat{y} = Hy$.
- (B) $H^2 = H$ (idempotent).
- (C) The residual vector is orthogonal to every column of X .
- (D) The trace of H equals n .

Solution

- (A) By definition of the OLS fit.
- (B) A projection applied twice changes nothing.
- (C) $X^\top(y - Hy) = X^\top y - X^\top y = 0$.
- (D) False: trace equals the number of parameters d (rank of X). Diagonal entries h_{ii} are leverages; high leverage points pull the fit.

Answer: A, B, C**Q1.23** MCQ · M

You regress y on x and get slope w . You then standardise both x and y (z-scores) and refit. The new slope is

- (A) w
- (B) Pearson correlation r
- (C) 1
- (D) w^2

Solution

- Slope = $r s_y / s_x$; after standardising, $s_x = s_y = 1$, so slope = r .

Answer: B**Q1.24** MSQ · M

Logistic regression vs linear regression for a 0/1 target. Which are true?

- (A) Linear regression can predict values outside $[0, 1]$.
- (B) Linear regression's fit is pulled by extreme x values even when their labels are obvious.
- (C) Logistic regression outputs calibrated probabilities if the model is correctly specified.
- (D) Logistic regression cannot use categorical features.

Solution

- (A), (B): the linear probability model is unbounded and a far away, easy point still has large squared error, tilting the line and moving the threshold.
- (C): maximum likelihood for Bernoulli tends to give well calibrated probabilities.
- (D) False: one hot encode categorical.

Answer: A, B, C

Q1.25 NAT · M

Model: $\log \frac{p}{1-p} = -3 + 0.04 \cdot \text{age}$. At what age is $p = 0.5$? What is p at age 100?

Solution

- $p = 0.5 \iff \log \text{odds} = 0 \iff 0.04 \text{age} = 3 \iff \text{age} = 75$.
- Age 100: $\log \text{odds} = -3 + 4 = 1$; $p = \sigma(1) = 0.731$.

Answer: 75 years; $p \approx 0.731$

Q1.26 MCQ · M

The Durbin-Watson statistic checks which assumption?

- (A) Normality of residuals
- (B) Homoscedasticity
- (C) Autocorrelation of residuals
- (D) Multicollinearity

Solution

- $DW \approx 2(1 - \hat{\rho})$ where $\hat{\rho}$ is lag 1 residual autocorrelation. Value near 2 means none, near 0 positive, near 4 negative autocorrelation.
- Homoscedasticity: Breusch-Pagan. Normality: Shapiro-Wilk or Q-Q plot. Multicollinearity: VIF.

Answer: C

Q1.27 MSQ · M

An outlier with very large x (high leverage) and a large residual. Which are true?

- (A) It can strongly change the OLS slope.
- (B) Cook's distance helps detect it.
- (C) MAE-based (least absolute deviation) regression is less affected by it.
- (D) Removing it is always correct.

Solution

- Influence = leverage times residual size; Cook's distance combines both.
- LAD and Huber regression down weight large residuals.
- (D) False: check whether it is a data error or a real rare case. In the reported InfoEdge coding round, the lesson was to look at domain ranges before deleting anything.

Answer: A, B, C

Q1.28 MCQ · E

Which is NOT an assumption of OLS linear regression?

- (A) Errors have constant variance
- (B) Errors are independent
- (C) Features are normally distributed
- (D) No perfect multicollinearity

Solution

- Assumptions concern the *errors*, not the features. Features can have any distribution (even binary).

Answer: C

Q1.29 NAT · H

Solve OLS without intercept for $X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $y = (1, 2, 4)^\top$.

Solution

- $X^T X = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$; $X^T y = (1 + 4, 2 + 4) = (5, 6)$.
- Inverse: determinant = $4 - 1 = 3$; $(X^T X)^{-1} = \frac{1}{3} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$.
- $w = \frac{1}{3}(2 \cdot 5 - 6, -5 + 2 \cdot 6) = \frac{1}{3}(4, 7) = (1.333, 2.333)$.
- Fitted values: $(1.333, 2.333, 3.667)$; residuals $r = y - \hat{y} = (-0.333, -0.333, 0.333)$.
- Check orthogonality: $X^T r = (-0.333 + 0.333, -0.333 + 0.333) = (0, 0)$. Verified.

Answer: $w = (4/3, 7/3)$

Q1.30 MSQ · M

Class imbalance (2% positives) with logistic regression. Sensible actions:

- (A) Use `class_weight='balanced'`.
- (B) Tune the decision threshold on a validation set.
- (C) Report accuracy as the main metric.
- (D) Use PR-AUC or F1 for evaluation.

Solution

- (A) Re-weights the loss so positives count more (same effect as oversampling, no duplicated rows).
- (B) 0.5 is rarely the best threshold for imbalanced data.
- (C) A model predicting all negatives gets 98% accuracy: useless metric here.
- (D) Focus on the rare class. This mirrors the reported InfoEdge interview discussion on SMOTE, undersampling and class weights.

Answer: A, B, D

Q1.31 MCQ · M

Which link function does logistic regression use in the GLM framework?

- (A) Identity
- (B) Log
- (C) Logit
- (D) Probit

Solution

- GLM: $g(E[y]) = w^T x$. Linear regression: identity, Gaussian. Logistic: logit, Bernoulli. Poisson regression: log link for counts (e.g. number of applications per job). Probit uses the normal CDF.

Answer: C

Q1.32 MSQ · H

Newton's method for logistic regression (IRLS). Which are true?

- (A) It uses the Hessian $X^T S X$ with $S = \text{diag}(p_i(1 - p_i))$.
- (B) It typically converges in far fewer iterations than GD.
- (C) Each iteration is cheaper than a GD step for large d .
- (D) It is equivalent to a sequence of weighted least squares problems.

Solution

- Hessian of log loss is $X^T S X$, positive semi definite, confirming convexity.
- Quadratic convergence near the optimum; tens of iterations.
- (C) False: each step solves a $d \times d$ system, $O(d^3)$.
- (D) That is the name: iteratively reweighted least squares.

Answer: A, B, D

Q1.33 SUB · M

Why is logistic regression called "regression" if it classifies?

Solution

- The model's output is a continuous probability; the linear part predicts log odds.
- Historically it is a GLM, the same family as linear regression.

Answer: It regresses the log odds (a continuous quantity) linearly on the features; classification comes only after thresholding.

Q1.34 MCQ · M

Gradient of $\text{MSE } \frac{1}{n} \|y - Xw\|^2$ with respect to w is

- (A) $\frac{2}{n} X^\top (Xw - y)$
- (B) $\frac{2}{n} (Xw - y) X$
- (C) $\frac{2}{n} X (Xw - y)$
- (D) $X^\top Xw$

Solution

- Expand: $\frac{1}{n} (w^\top X^\top Xw - 2y^\top Xw + y^\top y)$. Differentiate: $\frac{1}{n} (2X^\top Xw - 2X^\top y)$.
- Shape check: $X^\top$ is $d \times n$, $(Xw - y)$ is $n \times 1$, product is $d \times 1$ like w .

Answer: A

Q1.35 MSQ · M

Which transformations help linear regression when the true relation is $y = ae^{bx}\epsilon$ (multiplicative noise)?

- (A) Regress $\log y$ on x .
- (B) Regress y on $\log x$.
- (C) Use the model as is; OLS handles it.
- (D) After the log transform, noise becomes additive and roughly homoscedastic.

Solution

- $\log y = \log a + bx + \log \epsilon$: linear in x with additive noise.
- Back transforming predictions needs a small bias correction ($E[e^u] \neq e^{E[u]}$).

Answer: A, D

Q1.36 SUB · H

Interviewer: "Your logistic model has 0.95 train AUC and 0.70 test AUC. Walk me through what you do."

Solution

- First suspect leakage in reverse: is the train set containing future information or duplicate users across split? Split by user or time.
- Overfitting from too many sparse features (one hot IDs): increase regularisation (lower C), try L1 to prune features.
- Check distribution shift between train and test periods.
- Use stratified k fold CV to choose C; look at learning curves (Chapter 3).

Answer: Diagnose overfitting and leakage, then regularise and simplify, validated by cross validation.

Interview angle

- Derive the normal equations; when would you not use them? (large d , singular $X^\top X$.)
- State the assumptions of linear regression and how to test each. What breaks if each fails?
- Why log loss and not MSE for logistic? Derive the gradient $(p - y)x$.
- Interpret a coefficient in logistic regression. What happens with perfectly separable data?
- Heteroscedasticity: detect and fix. Multicollinearity: VIF, and why prediction barely suffers.
- OvR vs OvO vs softmax: when is each preferred?

2 Gradient Descent and Optimisation

Companion resource for this section: [StatQuest: Gradient Descent, Step-by-Step](#)

2.1 The idea: walk downhill in fog

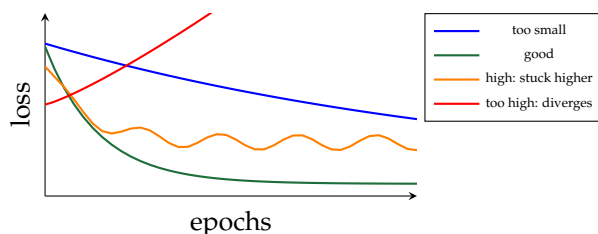
- You stand on a hillside in thick fog and want the valley. You cannot see the whole landscape, but you can feel the slope under your feet. Take a small step in the steepest downhill direction, repeat.
- The landscape is the loss $L(w)$; your position is the parameter vector w ; the slope is the **gradient** $\nabla L(w)$, the vector of partial derivatives. It points uphill (steepest increase), so we step against it:

$$w \leftarrow w - \eta \nabla L(w).$$

- η is the **learning rate** (step size). It is the single most important hyperparameter in deep learning.
- Tiny example: $L(w) = (w - 3)^2$, $\nabla L = 2(w - 3)$. Start $w = 0$, $\eta = 0.1$: gradient -6 , new $w = 0 + 0.6 = 0.6$; then gradient -4.8 , $w = 1.08$; then $1.464 \dots$ approaching 3. Each step shrinks the distance to 3 by the factor $(1 - 2\eta) = 0.8$.

2.2 Choosing the learning rate

- Too small: painfully slow, may stall on plateaus. Too large: overshoots and oscillates or diverges (loss goes to NaN).
- In the example above, the factor is $1 - 2\eta$. If $\eta = 1$ the factor is -1 : you bounce between 0 and 6 forever. If $\eta > 1$, $|1 - 2\eta| > 1$: divergence. General rule for a quadratic with curvature $\lambda_{\max}$ (largest Hessian eigenvalue): stable iff $\eta < 2/\lambda_{\max}$.
- Practical recipe: try a log grid ($10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}$), pick the largest that trains stably; or run a learning rate range test (increase η every batch and look where the loss starts to blow up).
- Schedules** (a reported InfoEdge question: “how to adjust the learning rate”): step decay (divide by 10 at fixed epochs), exponential decay, reduce on plateau (lower when validation loss stops improving), cosine annealing, and **warmup** (start tiny, ramp up over the first steps; standard for Transformers and Adam).



2.3 Batch, stochastic and mini batch gradient descent

- The training loss is an average over n examples: $L = \frac{1}{n} \sum_i \ell_i(w)$.
- Batch GD**: compute the exact gradient using all n examples, then one update. Smooth, but one update costs a full pass over data.
- Stochastic GD (SGD)**: pick one random example, update using $\nabla \ell_i$. That gradient is an *unbiased but noisy* estimate of the true gradient. Each update is n times cheaper, so SGD makes far more progress per second on large data. The noise also helps escape saddle points and sharp minima.
- Mini batch GD**: use B examples (32 to 4096). Averages out some noise (variance falls as $1/B$) and uses vectorised GPU hardware. This is what “SGD” means in deep learning practice.
- Vocabulary: an **iteration** is one update; an **epoch** is one pass over the data. With $n = 10,000$ and $B = 100$, one epoch has 100 iterations.
- Because SGD noise does not vanish at the optimum, plain SGD with a fixed η keeps jittering around the minimum. Decaying η lets it settle.

2.4 Convexity, local minima and saddle points

- A function is **convex** if the line segment between any two points on its graph lies above the graph (a bowl). For convex losses every local minimum is global. Linear regression (MSE), logistic regression (log loss), SVM (hinge) are convex in w .
- Neural network losses are non convex. Surprisingly, in high dimensions most critical points are **saddle points** (downhill in some directions, uphill in others), not bad local minima, and most local minima found in practice have similar loss.
- What if training is stuck (a reported InfoEdge interview question)? Try: restarting from different random initialisations and keeping the best run (what the candidate answered), momentum or Adam, a larger or cyclical learning rate, smaller batch (more noise), better initialisation and normalisation (BatchNorm), and checking for bugs such as dead ReLUs.

2.5 Momentum, RMSProp, Adam

- **Momentum**: keep a velocity $v \leftarrow \beta v + \nabla L$, update $w \leftarrow w - \eta v$ ($\beta \approx 0.9$). Like a ball rolling downhill: it speeds up along consistent directions and damps zig zags across narrow valleys. Effective step in a steady slope is $\eta / (1 - \beta) = 10\eta$.
- **Nesterov momentum**: evaluate the gradient at the look ahead point $w - \eta\beta v$; slightly better correction.
- **AdaGrad**: divide each coordinate's step by the root of the sum of its past squared gradients. Rare features get big steps (good for sparse text). Problem: the sum only grows, so steps shrink to zero.
- **RMSProp**: use an exponential moving average of squared gradients instead: $s \leftarrow \rho s + (1 - \rho)g^2$, $w \leftarrow w - \eta g / (\sqrt{s} + \epsilon)$.
- **Adam** = momentum + RMSProp + bias correction: $m \leftarrow \beta_1 m + (1 - \beta_1)g$, $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$, $\hat{m} = m / (1 - \beta_1^t)$, $\hat{v} = v / (1 - \beta_2^t)$, $w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$. Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$. Bias correction matters because m, v start at 0 and are too small in early steps.
- **AdamW**: applies weight decay directly to the weights instead of adding L2 to the gradient (in Adam those are not equivalent). Standard for Transformers.

2.6 Why feature scaling matters for GD

- If feature 1 is age (20 to 60) and feature 2 is salary (in rupees, 10^5 to 10^7), the loss surface is a very elongated ellipse. The gradient points mostly across the valley, so GD zig zags and needs a tiny η to stay stable along the steep direction.
- Standardising features ($z = (x - \mu) / \sigma$) makes contours closer to circles and GD converges much faster. Tree models do not care about scaling; GD based models (linear, logistic, SVM, neural nets, KNN distances) do.

2.7 Second order and other ideas (for interview depth)

- **Newton's method**: $w \leftarrow w - H^{-1} \nabla L$. Uses curvature, converges in few steps, but H is $d \times d$ (impossible for millions of parameters). L-BFGS approximates it cheaply and is the default solver for scikit-learn's LogisticRegression.
- **Gradient clipping**: cap the gradient norm (e.g. at 1.0) to prevent exploding updates in RNNs and Transformers.
- **Early stopping**: stop when validation loss stops improving; acts as regularisation.

Where this shows up in ML

- Every neural network in the DL chapters is trained by mini batch SGD or Adam; gradients come from backpropagation.
- Gradient boosting (Chapter 6) is literally gradient descent in function space: each new tree is a step along the negative gradient.
- Online learning systems (ranking on job feeds) use SGD style updates as new clicks arrive.

Formula sheet

GD: $w \leftarrow w - \eta \nabla L$. Quadratic stability: $\eta < 2/\lambda_{\max}(H)$. Iterations per epoch = $\lceil n/B \rceil$.
 Momentum: $v \leftarrow \beta v + g$, $w \leftarrow w - \eta v$. RMSProp: $s \leftarrow \rho s + (1 - \rho)g^2$. Adam: $\hat{m} = m/(1 - \beta_1^t)$,
 $\hat{v} = v/(1 - \beta_2^t)$, step $\eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$.
 Mini batch gradient variance $\propto 1/B$. Newton: $w \leftarrow w - H^{-1}g$.

2.8 Question bank: Gradient descent and optimisation

Q2.1 MCQ · E

★ PYQ (OA: which is faster, stochastic or batch gradient descent) On a dataset with 10 million rows, which reaches a good solution faster in wall clock time, typically?

- (A) Batch gradient descent, because its gradient is exact.
- (B) Stochastic (or mini batch) gradient descent, because each update is far cheaper.
- (C) They are always identical.
- (D) Neither converges on large data.

Solution

- One batch GD update needs all 10^7 gradients. In the same time SGD does 10^7 updates (or mini batch GD does $10^7/B$).
- Each SGD step is noisy, but the expected direction is correct, and early in training even a rough direction gives big improvements.
- Batch GD takes fewer *iterations* to converge; SGD takes less *time*. OA wording “faster” usually means per update / wall clock, hence SGD.

Answer: B

Q2.2 MSQ · M

★ PYQ (INT: what is the learning rate and how do you adjust it) Which statements are true?

- (A) A learning rate that is too high can make the loss increase or become NaN.
- (B) Reducing the learning rate when validation loss plateaus is a common schedule.
- (C) Warmup means starting with a large learning rate and decreasing it immediately.
- (D) With Adam, a good default starting learning rate is around 10^{-3} .

Solution

- (A) Overshooting a narrow valley grows the error each step: divergence.
- (B) ReduceLROnPlateau: e.g. halve η after 5 epochs without improvement.
- (C) False: warmup starts small and increases for the first few hundred or thousand steps, then decays. It prevents early instability when Adam's statistics are unreliable.
- (D) True: the widely used default from the Adam paper.

Answer: A, B, D

Q2.3 MSQ · M

★ PYQ (INT: what to do if the network gets stuck at a local optimum) Training loss stops improving at a poor value. Reasonable remedies:

- (A) Restart from several random initialisations and keep the best.
- (B) Use momentum or Adam.
- (C) Set all weights to zero before training.
- (D) Try a learning rate schedule with restarts (cyclical or cosine with restarts).

Solution

- (A) The answer reported by the candidate; different starts land in different basins.
- (B) Momentum carries you across small bumps and through saddles.
- (C) Zero initialisation makes all hidden units identical (symmetry): they receive identical gradients and never differentiate. Harmful.

- (D) Periodically raising η kicks the model out of a basin.

Answer: A, B, D

Q2.4 NAT · E

Minimise $L(w) = (w - 3)^2$ by GD with $w_0 = 0$ and $\eta = 0.1$. Find w_2 .

Solution

- $\nabla L = 2(w - 3)$.
- Step 1: $\nabla L(0) = -6$; $w_1 = 0 - 0.1(-6) = 0.6$.
- Step 2: $\nabla L(0.6) = -4.8$; $w_2 = 0.6 + 0.48 = 1.08$.
- Check with the contraction factor $1 - 2\eta = 0.8$: distance to 3 goes $3 \rightarrow 2.4 \rightarrow 1.92$, so $w_2 = 3 - 1.92 = 1.08$.

Answer: $w_2 = 1.08$

Q2.5 NAT · M

Same $L(w) = (w - 3)^2$. For which η does GD converge from any start?

Solution

- $w_{t+1} - 3 = (1 - 2\eta)(w_t - 3)$.
- Converges iff $|1 - 2\eta| < 1 \iff 0 < \eta < 1$.
- $\eta = 0.5$ jumps to the optimum in one step (factor 0). $\eta = 1$ oscillates forever. This matches the rule $\eta < 2/\lambda_{\max}$ with Hessian $\lambda = 2$.

Answer: $0 < \eta < 1$

Q2.6 NAT · E

A dataset has 50,000 examples, batch size 250, trained for 20 epochs. How many parameter updates?

Solution

- Iterations per epoch = $50000/250 = 200$.
- Total = $200 \times 20 = 4000$ updates.

Answer: 4000

Q2.7 MSQ · M

Which losses are convex in the parameters?

- (A) MSE for linear regression
- (B) Log loss for logistic regression
- (C) Cross entropy for a 2 hidden layer neural network
- (D) Hinge loss for a linear SVM

Solution

- Linear models with convex losses stay convex.
- (C) Hidden layers compose non linearities; permuting hidden units gives many equivalent minima, which is impossible for a strictly convex function with a unique minimum. Non convex.

Answer: A, B, D

Q2.8 MCQ · M

Why does plain SGD with a constant learning rate not settle exactly at the minimum?

- (A) Because the loss is non convex.
- (B) Because the gradient of a single example is non zero at the minimum of the average loss, so updates keep jittering.
- (C) Because of floating point errors.
- (D) Because momentum is missing.

Solution

- At the optimum $\frac{1}{n} \sum \nabla \ell_i = 0$, but individual $\nabla \ell_i \neq 0$. Each SGD step moves by $\eta \nabla \ell_i$, so it wanders in a ball whose size scales with η .
- Remedy: decay η (Robbins-Monro conditions: $\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$), or average iterates.

Answer: B**Q2.9** NAT · M

Momentum with $\beta = 0.9$, $\eta = 0.1$, $v_0 = 0$. The gradient is constantly $g = 1$. Compute v_1, v_2, v_3 and the limiting step size.

Solution

- $v_1 = 0.9 \cdot 0 + 1 = 1$; $v_2 = 0.9 + 1 = 1.9$; $v_3 = 1.71 + 1 = 2.71$.
- Limit: $v^* = 0.9v^* + 1 \Rightarrow v^* = 10$. Step $= \eta v^* = 1.0$, ten times plain GD's 0.1.
- This is why momentum accelerates along consistent slopes.

Answer: $v = 1, 1.9, 2.71$; limit step = 1.0**Q2.10** MSQ · M

Adam's bias correction. Which are true?

- (A) m and v are initialised at zero, so early estimates are biased toward zero.
- (B) $\hat{m}_t = m_t / (1 - \beta_1^t)$.
- (C) Bias correction matters most late in training.
- (D) At $t = 1$, $\hat{m}_1 = g_1$ exactly.

Solution

- At $t = 1$: $m_1 = (1 - \beta_1)g_1$, and $\hat{m}_1 = (1 - \beta_1)g_1 / (1 - \beta_1) = g_1$.
- (C) False: $\beta^t \rightarrow 0$ as t grows, so the correction factor $\rightarrow 1$; it matters early.

Answer: A, B, D**Q2.11** NAT · H

One Adam step: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.01$, $\epsilon \approx 0$, $t = 1$, $g = 4$, $w = 1$. Compute the new w .

Solution

- $m_1 = 0.1 \times 4 = 0.4$; $v_1 = 0.001 \times 16 = 0.016$.
- $\hat{m} = 0.4 / 0.1 = 4$; $\hat{v} = 0.016 / 0.001 = 16$.
- Step $= 0.01 \times 4 / \sqrt{16} = 0.01 \times 1 = 0.01$. $w = 1 - 0.01 = 0.99$.
- Insight: in the first step Adam moves each parameter by about η regardless of gradient scale (the ratio $\hat{m} / \sqrt{\hat{v}} = \text{sign}(g)$). Adam is scale invariant per coordinate.

Answer: $w = 0.99$ **Q2.12** MCQ · M

Why does feature scaling speed up gradient descent for linear regression?

- (A) It makes the loss convex.
- (B) It makes the loss contours rounder, reducing zig zag and allowing a larger stable learning rate.
- (C) It reduces the number of parameters.
- (D) It changes the optimal predictions.

Solution

- The loss was already convex. Scaling improves the condition number $\lambda_{\max} / \lambda_{\min}$ of $X^\top X$; GD's convergence rate depends on it.
- Predictions of the optimal model are unchanged (weights rescale accordingly), except when regularisation is used.

Answer: B

Q2.13 MSQ · M

Saddle points in deep learning. Which are true?

- (A) The gradient is zero at a saddle point.
- (B) In high dimensions, saddle points are far more common than poor local minima.
- (C) Plain GD started exactly on the saddle's stable manifold can get stuck.
- (D) SGD noise helps escape saddles.

Solution

- All are standard results. For a random critical point in d dimensions to be a minimum, all d Hessian eigenvalues must be positive; that is exponentially unlikely, so most critical points are saddles.

Answer: A, B, C, D

Q2.14 MCQ · E

Increasing mini batch size from 32 to 512 (all else equal) typically

- (A) increases gradient noise
- (B) decreases gradient noise and the number of updates per epoch
- (C) increases updates per epoch
- (D) has no effect

Solution

- Variance of the mini batch mean gradient scales as $1/B$: 16 times less noise.
- Updates per epoch drop by 16. Linear scaling rule: when multiplying B by k , multiply η by about k (with warmup).

Answer: B

Q2.15 MSQ · H

Large batch training often generalises slightly worse. Commonly cited reasons:

- (A) Less gradient noise, so it tends to converge to sharper minima.
- (B) Fewer updates per epoch for the same epochs.
- (C) Large batches make the loss non differentiable.
- (D) Learning rate not scaled up accordingly.

Solution

- The sharp vs flat minima argument (Keskar et al.): noise acts as implicit regularisation.
- (C) False.

Answer: A, B, D

Q2.16 MCQ · M

Gradient clipping by norm with threshold 1.0. The raw gradient is (3,4). The clipped gradient is

- (A) (1,1)
- (B) (0.6,0.8)
- (C) (3,4)
- (D) (0.75,1)

Solution

- Norm = $\sqrt{9+16} = 5 > 1$. Rescale: $g \cdot 1/5 = (0.6, 0.8)$. Direction kept, length capped.
- Clipping each coordinate to $[-1, 1]$ would give (1,1) and change direction; that is value clipping, not norm clipping.

Answer: B

Q2.17 MSQ · M

AdaGrad vs RMSProp. Which are true?

- (A) AdaGrad accumulates the full sum of squared gradients.

- (B) AdaGrad's effective learning rate can decay to near zero.
- (C) RMSProp uses an exponential moving average to fix this.
- (D) Both give larger steps to parameters with historically large gradients.

Solution

- (D) False: both divide by the root of accumulated squared gradients, so large historic gradients mean *smaller* steps; rare (sparse) features get relatively larger steps.

Answer: A, B, C**Q2.18** NAT · M

Linear regression, one feature, no bias. Data $(x, y) = (1, 2), (2, 4)$. $w_0 = 0$, $\eta = 0.1$, loss $\text{MSE} = \frac{1}{n} \sum (wx - y)^2$. One batch GD step.

Solution

- Gradient $= \frac{2}{n} \sum x(wx - y)$. At $w = 0$: $\frac{2}{2}[1(0 - 2) + 2(0 - 4)] = -2 - 8 = -10$.
- $w_1 = 0 - 0.1(-10) = 1.0$. (True optimum is $w = 2$.)

Answer: $w_1 = 1.0$ **Q2.19** NAT · M

Same data, SGD using only the example $(2, 4)$, from $w_0 = 0$, $\eta = 0.1$, per example loss $(wx - y)^2$.

Solution

- Gradient $= 2x(wx - y) = 2 \cdot 2(0 - 4) = -16$.
- $w_1 = 0 + 1.6 = 1.6$. A single example's gradient can be much larger than the average: SGD steps are noisier.

Answer: $w_1 = 1.6$ **Q2.20** MCQ · E

One epoch means

- (A) one parameter update
- (B) one pass over the entire training set
- (C) one mini batch
- (D) one validation run

Solution

- Iteration = update; epoch = full pass.

Answer: B**Q2.21** MSQ · M

Early stopping. Which are true?

- (A) It monitors validation loss and stops when it stops improving for some patience.
- (B) It acts like regularisation.
- (C) It requires a validation set separate from test.
- (D) It guarantees the global optimum of the training loss.

Solution

- For linear models with GD, early stopping is approximately equivalent to L2 regularisation (fewer steps = weights stay near their small initial values).
- (D) False: it deliberately stops before the training optimum.

Answer: A, B, C**Q2.22** MCQ · M

Which optimiser decouples weight decay from the gradient based update?

- (A) Adam

- (B) AdamW
- (C) RMSProp
- (D) Nesterov

Solution

- In Adam, adding λw to the gradient gets divided by $\sqrt{\hat{v}}$, so parameters with large gradients get less decay. AdamW does $w \leftarrow w - \eta(\text{Adam step} + \lambda w)$, restoring true weight decay.

Answer: B**Q2.23** NAT · H

$L(w_1, w_2) = w_1^2 + 25w_2^2$. Largest learning rate for GD to converge?

Solution

- Hessian = $\text{diag}(2, 50)$; $\lambda_{\max} = 50$.
- Need $\eta < 2/50 = 0.04$. At $\eta = 0.039$, the w_1 direction contracts by $1 - 2(0.039) = 0.922$ per step: slow. This ill conditioning (ratio 25) is what scaling or Adam fixes.

Answer: $\eta < 0.04$ **Q2.24** MSQ · M

Which are signs your learning rate is too high?

- (A) Loss oscillates wildly or increases
- (B) Loss becomes NaN
- (C) Loss decreases very slowly and smoothly
- (D) Training accuracy jumps around between epochs

Solution

- (C) is the classic sign of a learning rate that is too *low*.

Answer: A, B, D**Q2.25** MCQ · M

Newton's method is rarely used to train large neural networks because

- (A) it does not use derivatives
- (B) computing and inverting the $d \times d$ Hessian is infeasible for millions of parameters
- (C) it always diverges
- (D) it cannot handle convex losses

Solution

- Storage $O(d^2)$, inversion $O(d^3)$. With $d = 10^8$, impossible. Also Newton is attracted to saddle points in non convex problems.

Answer: B**Q2.26** SUB · M

Explain in two lines why momentum helps in a long narrow valley.

Solution

- Across the valley the gradient flips sign each step, so the moving average stays small; along the valley it points the same way, so the average grows up to $1/(1 - \beta)$ times.

Answer: Oscillating components cancel in the velocity while the consistent downhill component accumulates.

Q2.27 MSQ · M

Gradient descent vs closed form normal equations for linear regression:

- (A) Normal equations need no learning rate.
- (B) GD scales better when d is very large.

- (C) Normal equations are $O(d^3)$ for the inversion.
 (D) GD cannot solve linear regression.

Solution

- Rule of thumb: d up to a few thousand, closed form is fine; beyond, or with streaming data, GD.

Answer: A, B, C

Q2.28 NAT · M

Exponential learning rate decay $\eta_t = \eta_0 \cdot 0.9^t$ per epoch, $\eta_0 = 0.1$. Learning rate at epoch 10?

Solution

- $0.9^{10} = 0.3487$; $\eta_{10} = 0.1 \times 0.3487 = 0.0349$.

Answer: ≈ 0.0349

Q2.29 MCQ · M

Cosine annealing from $\eta_{\max} = 0.1$ to 0 over $T = 100$ epochs: $\eta_t = \frac{\eta_{\max}}{2}(1 + \cos \frac{\pi t}{T})$. Value at $t = 50$?

- (A) 0.1
 (B) 0.05
 (C) 0.025
 (D) 0

Solution

- $\cos(\pi/2) = 0$, so $\eta = 0.05(1 + 0) = 0.05$. Halfway in time, halfway in value.

Answer: B

Q2.30 MSQ · H

Which statements about the gradient are true?

- (A) It is perpendicular to the level set (contour) through the point.
 (B) The directional derivative along unit u is $\nabla L \cdot u$.
 (C) $-\nabla L$ is the steepest descent direction in Euclidean norm.
 (D) The gradient always points toward the minimum.

Solution

- (D) False: in an elongated bowl, the gradient points toward the nearest contour, often nearly perpendicular to the direction of the minimum; that is why GD zig zags.

Answer: A, B, C

Q2.31 MCQ · E

Which model does NOT need feature scaling for good training?

- (A) Logistic regression with GD
 (B) SVM with RBF kernel
 (C) Random forest
 (D) K nearest neighbours

Solution

- Tree splits compare a feature with a threshold; any monotone rescaling leaves split order unchanged.

Answer: C

Q2.32 MSQ · M

Stochastic gradient is an unbiased estimator of the full gradient if:

- (A) examples are sampled uniformly at random
 (B) the batch is always the first 100 rows of a sorted file
 (C) the loss is an average of per example losses
 (D) the batch is large

Solution

- $E[\nabla \ell_I] = \frac{1}{n} \sum \nabla \ell_i$ when I is uniform. Sorted data break this: the model sees all class 0 rows, then all class 1 rows, and oscillates. Always shuffle each epoch.
- Batch size affects variance, not bias.

Answer: A, C

Q2.33 SUB · H

Interviewer: “Loss goes to NaN after 3 epochs. Debug it.”

Solution

- Lower the learning rate by 10x; add warmup; add gradient clipping.
- Numerics: $\log(0)$ in a hand written cross entropy (use the fused logits version), division by zero in normalisation, fp16 overflow (use loss scaling or bf16).
- Data: NaN or infinite values in features, unscaled huge features.
- Log gradient norms per layer to see which explodes first.

Answer: Check learning rate, numerics, data and gradients in that order.

Q2.34 MCQ · M

L-BFGS is

- (A) a first order method with momentum
- (B) a quasi Newton method that approximates the inverse Hessian using recent gradient differences
- (C) a learning rate schedule
- (D) an initialisation scheme

Solution

- It stores only the last m (e.g. 10) pairs of parameter and gradient differences: $O(md)$ memory. Great for small to medium, full batch, smooth problems like logistic regression.

Answer: B

Q2.35 MSQ · M

Which are true about Nesterov accelerated gradient?

- (A) It computes the gradient at a look ahead point.
- (B) It can reduce overshooting compared to classical momentum.
- (C) It needs the Hessian.
- (D) In convex smooth optimisation it achieves a faster worst case rate than plain GD.

Solution

- Rate $O(1/t^2)$ vs $O(1/t)$ for GD on smooth convex functions. No Hessian needed.

Answer: A, B, D

Interview angle

- Derive the GD update for logistic regression. Why must we shuffle data every epoch?
- Batch vs SGD vs mini batch: speed, noise, memory, generalisation.
- How would you pick the learning rate for a new model? What is warmup and why do Transformers need it?
- Explain Adam from scratch. Why bias correction? Why AdamW?
- What do you do if training is stuck or the loss becomes NaN?
- Why are saddle points, not local minima, the main obstacle in high dimensions?

3 Bias-Variance Tradeoff and Regularisation

Companion resource for this section: [StatQuest: Machine Learning Fundamentals, Bias and Variance](#)

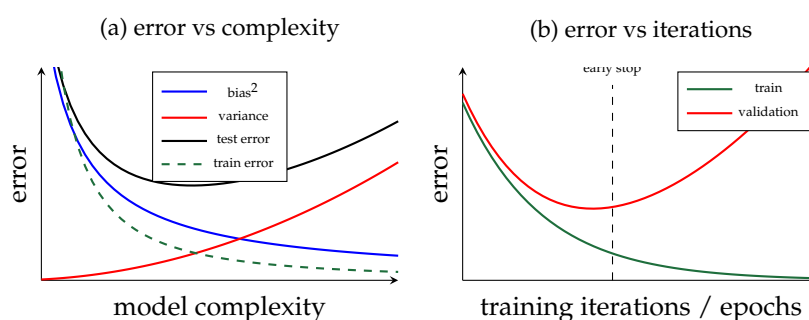
3.1 Two ways to be wrong

- Imagine training the same model type on many different random training sets drawn from the same world, then predicting at one fixed input x .
- Bias**: how far the *average* prediction (over those training sets) is from the truth. A straight line fitted to a curved relationship is wrong on average no matter how much data you give it: high bias, **underfitting**.
- Variance**: how much predictions *jump around* between training sets. A degree 15 polynomial through 20 points changes wildly if you swap a few points: high variance, **overfitting**.
- Dartboard picture: bias = the cluster of darts is off centre; variance = the cluster is spread out.
- For squared error, with $y = f(x) + \epsilon$, $\text{Var}(\epsilon) = \sigma^2$:

$$E[(y - \hat{f}(x))^2] = \underbrace{(E[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{E[(\hat{f}(x) - E\hat{f}(x))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

- The noise term is a floor no model can beat. The first two trade off against each other as model complexity changes.

3.2 The two graphs InfoEdge asks about



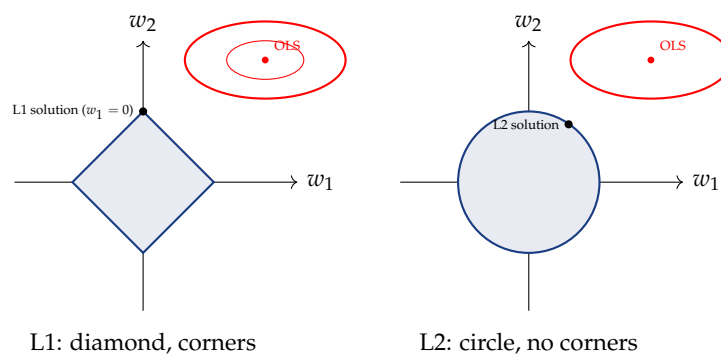
- Graph (a): training error keeps falling with complexity; test error is U shaped. The left side is underfitting (high bias), the right side overfitting (high variance). The sweet spot is the bottom of the U.
- Graph (b): for iterative learners (neural nets, boosting), training error keeps falling with more iterations, while validation error falls then rises. The rise is where the model starts memorising noise; **early stopping** picks the minimum.
- Diagnosis from numbers: train error high and validation error high (close together) = high bias. Train error low, validation error much higher = high variance.
- Fix high bias: more features, bigger or more flexible model, less regularisation, train longer. Fix high variance: more data, regularisation, simpler model, feature selection, bagging, dropout, data augmentation, early stopping.
- Learning curves (error vs training set size): if validation error is still dropping as data grows, more data helps (variance problem). If both curves have flattened at a high level, more data will not help (bias problem).

3.3 Modern twist: double descent

- Very large neural networks can have more parameters than examples and still generalise. Test error can go U shaped, peak near the interpolation threshold (parameters $\approx$ data points), then fall again. This is **double descent**.
- The classical U curve is still the right OA answer; mention double descent only as interview depth.

3.4 Regularisation: penalise complexity

- Idea: minimise $\text{Loss}(w) + \lambda \Omega(w)$. λ controls the trade: $\lambda = 0$ gives the unregularised fit; huge λ forces $w \rightarrow 0$ (a flat, high bias model).
- **L2 / Ridge**: $\Omega = \|w\|_2^2 = \sum w_j^2$. Shrinks all weights smoothly toward 0, rarely exactly 0. Closed form $w = (X^T X + \lambda I)^{-1} X^T y$; the $+\lambda I$ also makes a singular matrix invertible, which is why Ridge tames multicollinearity. In neural nets L2 is called **weight decay**, because the GD update becomes $w \leftarrow (1 - \eta \lambda') w - \eta \nabla L$: weights decay by a constant factor each step.
- **L1 / Lasso**: $\Omega = \|w\|_1 = \sum |w_j|$. Drives many weights *exactly* to 0: built in feature selection. With correlated features it tends to pick one arbitrarily.
- **Elastic net**: $\lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$. Sparse like Lasso but keeps groups of correlated features together like Ridge.
- Why L1 gives zeros (geometry): minimising loss subject to $\|w\|_1 \leq t$ means the loss contours (ellipses) grow until they touch a diamond; diamonds have corners on the axes, and touching at a corner means some $w_j = 0$. The L2 ball is a circle with no corners, so the touch point is generically off axis.
- Why L1 gives zeros (calculus): the L2 penalty's gradient $2\lambda w$ vanishes as $w \rightarrow 0$, so the push weakens; the L1 penalty pushes with constant force $\lambda \text{sign}(w)$ all the way to zero. In one dimension with loss $\frac{1}{2}(w - a)^2$: Ridge gives $w = a / (1 + \lambda)$; Lasso gives soft thresholding $w = \text{sign}(a) \max(|a| - \lambda, 0)$.
- Bayesian view (Chapter on MAP): L2 = Gaussian prior on weights, L1 = Laplace prior.
- Always standardise features before L1/L2, otherwise the penalty unfairly hits features measured in small units. Do not penalise the intercept.



3.5 Other regularisers you must be able to name

- **Dropout** (neural nets): during training, zero each hidden unit with probability p and scale survivors by $1/(1 - p)$ (inverted dropout) so expected activations match test time. It prevents co adaptation and approximates averaging an ensemble of sub networks. At test time dropout is off.
- **Early stopping**; **data augmentation** (flips, crops, noise, back translation); **label smoothing**; **batch normalisation** (a mild side effect); **max norm** constraints.
- Trees: max depth, min samples per leaf, pruning. Boosting: shrinkage (learning rate), subsampling, column sampling, L1/L2 on leaf weights.
- Choosing λ : k fold cross validation over a log grid. Never tune on the test set.

3.6 Cross validation in one paragraph

- k fold: split data into k parts; train on $k - 1$, validate on the held out part; rotate; average the k scores. $k = 5$ or 10 is standard. Stratified k fold keeps class ratios. Time series: use forward chaining (train on past, validate on future), never shuffle. Grouped data (same user in many rows): GroupKFold to avoid leakage.
- Leave one out ($k = n$): nearly unbiased but high variance and expensive.

Where this shows up at InfoEdge

- Sparse one hot features (skills, locations, companies) make linear models prone to variance; L1 or L2 is essential.
- Deep models for resume matching are regularised with dropout, weight decay (AdamW) and early stopping.

- The case study round expects you to diagnose over/underfitting from train and validation numbers and propose fixes.

Formula sheet

$E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. Ridge: $(X^\top X + \lambda I)^{-1} X^\top y$; 1D shrink $a/(1 + \lambda)$. Lasso 1D: $\text{sign}(a) \max(|a| - \lambda, 0)$.

Weight decay: $w \leftarrow (1 - \eta\lambda)w - \eta \nabla L$. Inverted dropout: keep with prob $1 - p$, scale by $1/(1 - p)$.

High bias: train $\approx$ val, both high. High variance: train low, val high.

3.7 Question bank: Bias-variance and regularisation

Q3.1 MCQ · E

★ **PYQ (OA: bias-variance graph, error vs model complexity)** In the plot of error vs model complexity, the test error curve is typically

- (A) monotonically decreasing
- (B) monotonically increasing
- (C) U shaped
- (D) constant

Solution

- At low complexity bias dominates (underfitting); at high complexity variance dominates (overfitting). The sum is U shaped. Training error, in contrast, decreases monotonically.

Answer: C

Q3.2 MSQ · M

★ **PYQ (OA: bias-variance graph, error vs iterations)** For a neural network, as training iterations increase:

- (A) training error generally keeps decreasing.
- (B) validation error first decreases then may increase.
- (C) the point where validation error is minimum is where early stopping should stop.
- (D) validation error always tracks training error exactly.

Solution

- Longer training increases effective model complexity (weights move further from their small initial values), so the right side of the iterations graph behaves like the right side of the complexity graph.
- (D) False: the gap between them is the generalisation gap.

Answer: A, B, C

Q3.3 MSQ · M

★ **PYQ (OA: bias-variance (IIT Roorkee ML section))** A model has training accuracy 99% and validation accuracy 72%. Which are appropriate?

- (A) The model has high variance.
- (B) Add more training data.
- (C) Increase model depth to reduce error.
- (D) Increase regularisation or use dropout / pruning.

Solution

- Big train validation gap = overfitting = high variance.
- (C) makes the model more flexible, which increases variance further.

Answer: A, B, D

Q3.4 MSQ · M

★ **PYQ (INT: weight decay)** Which statements about weight decay are true?

- (A) For plain SGD, weight decay is equivalent to adding an L2 penalty to the loss.
- (B) It multiplies weights by a factor slightly less than 1 each step.
- (C) It encourages exactly zero weights.
- (D) For Adam, weight decay and L2 penalty are not equivalent, which motivated AdamW.

Solution

- Gradient of $\frac{\lambda}{2} \|w\|^2$ is λw ; the update becomes $w(1 - \eta\lambda) - \eta\nabla L$.
- (C) False: that is L1.
- (D) See Chapter 2: Adam rescales the λw term per coordinate.

Answer: A, B, D**Q3.5** MSQ · M

★ **PYQ (INT: how to avoid overfitting in neural networks (OA DL: overfitting))** Which reduce overfitting in a neural network?

- (A) Dropout
- (B) Data augmentation
- (C) Increasing the number of epochs without monitoring validation
- (D) Early stopping

Solution

- Training longer without monitoring usually increases overfitting.
- Also valid: weight decay, smaller network, batch norm, label smoothing, more data.

Answer: A, B, D**Q3.6** MCQ · E

A linear model on a clearly curved dataset gets 60% train and 58% validation accuracy. The problem is

- (A) high variance
- (B) high bias
- (C) data leakage
- (D) too much data

Solution

- Both low and close: underfitting. Add polynomial features, use a non linear model, reduce regularisation.

Answer: B**Q3.7** MSQ · M

Which statements about L1 regularisation are true?

- (A) It can set coefficients exactly to zero.
- (B) It performs implicit feature selection.
- (C) Its penalty is differentiable everywhere.
- (D) Among highly correlated features it tends to pick one and zero the rest.

Solution

- (C) False: $|w|$ has a kink at 0; solvers use subgradients, coordinate descent or proximal steps (soft thresholding).

Answer: A, B, D**Q3.8** NAT · M

One dimensional problem: minimise $\frac{1}{2}(w - 3)^2 + \lambda \cdot \text{penalty}$ with $\lambda = 1$. Find w for (a) Ridge penalty $\frac{1}{2}w^2$, (b) Lasso penalty $|w|$.

Solution

- Ridge: derivative $(w - 3) + w = 0 \Rightarrow w = 3/2 = 1.5$. General: $a/(1 + \lambda)$, a proportional shrink.
- Lasso: for $w > 0$, derivative $(w - 3) + 1 = 0 \Rightarrow w = 2$ (positive, so valid). General: $a - \lambda$ if $a > \lambda$.
- If instead $a = 0.5$ with $\lambda = 1$: Ridge gives 0.25; Lasso gives exactly 0. That is sparsity.

Answer: (a) 1.5, (b) 2

Q3.9 MCQ · M

Geometrically, why does Lasso produce sparse solutions?

- (A) Its constraint region is a circle.
- (B) Its constraint region has corners on the axes where loss contours tend to touch first.
- (C) It uses squared weights.
- (D) It ignores the loss.

Solution

- The L1 ball is a diamond (cross polytope in high d); the first contact with an expanding elliptical loss contour is very often a vertex, where some coordinates are 0.

Answer: B

Q3.10 MSQ · M

Ridge regression. Which are true?

- (A) It always has a unique solution for $\lambda > 0$.
- (B) It shrinks coefficients toward zero.
- (C) It is useful under multicollinearity.
- (D) It increases variance and decreases bias.

Solution

- $X^T X + \lambda I$ has eigenvalues $\geq \lambda > 0$, so invertible.
- (D) Reversed: Ridge increases bias slightly and decreases variance.

Answer: A, B, C

Q3.11 NAT · M

Inverted dropout with $p = 0.2$ (drop probability). A unit's activation is 5 and it is kept. What value is passed forward during training? What is its expected value?

Solution

- Kept units are scaled by $1/(1 - p) = 1/0.8 = 1.25$: $5 \times 1.25 = 6.25$.
- Expected value: $0.8 \times 6.25 + 0.2 \times 0 = 5$. Matches test time (no dropout, no scaling). Linearity of expectation at work.

Answer: 6.25; expected value 5

Q3.12 MCQ · M

At test time, a network trained with inverted dropout should

- (A) drop units with the same probability
- (B) use all units with no scaling
- (C) use all units and multiply by $1 - p$
- (D) randomly choose the dropout rate

Solution

- The scaling was already done during training. (Original, non inverted dropout multiplied weights by $1 - p$ at test time instead.)
- Monte Carlo dropout deliberately keeps dropout on at test time to estimate uncertainty; that is a special technique.

Answer: B

Q3.13 MSQ · H

Which are true about the bias-variance decomposition?

- (A) It holds exactly for squared error loss.
- (B) Irreducible error can be reduced with a better model.
- (C) Bagging mainly reduces variance.
- (D) Boosting mainly reduces bias (early rounds).

Solution

- (B) False by definition: it is the noise variance σ^2 in y given x . Only better features (new information) can reduce it.
- Bagging averages many high variance models; boosting sequentially fits what the current model gets wrong.

Answer: A, C, D

Q3.14 MCQ · E

k nearest neighbours with $k = 1$ compared to $k = 50$ has

- (A) higher bias, lower variance
- (B) lower bias, higher variance
- (C) same bias and variance
- (D) lower bias and lower variance

Solution

- $k = 1$ memorises training points (zero training error) and changes a lot with the data. Large k averages many neighbours: smoother, more biased.

Answer: B

Q3.15 MSQ · M

Learning curves show training error 5% and validation error 25%, and validation error is still decreasing as training set size increases. Best actions:

- (A) Collect more data.
- (B) Use a more complex model.
- (C) Add regularisation.
- (D) Conclude the problem is high bias.

Solution

- Large gap plus a still falling validation curve = variance problem where more data helps.

Answer: A, C

Q3.16 MCQ · M

Elastic net is preferred over Lasso when

- (A) there is only one feature
- (B) features come in correlated groups and you want to keep groups together
- (C) you want no regularisation
- (D) the target is categorical

Solution

- The L2 part gives a grouping effect: correlated features get similar coefficients instead of one being picked at random. It also allows selecting more than n features when $d > n$ (Lasso selects at most n).

Answer: B

Q3.17 MSQ · M

Why must features be standardised before Ridge or Lasso?

- (A) Because the penalty is on coefficient size, which depends on feature units.

- (B) Otherwise a feature in rupees gets a tiny coefficient and is barely penalised, while one in lakhs is heavily penalised.
 (C) Because OLS requires it.
 (D) Because it makes the loss convex.

Solution

- Plain OLS predictions are invariant to scaling; penalised models are not. Both are convex regardless.

Answer: A, B

Q3.18 NAT · M

Ridge closed form with one standardised feature: $\sum x_i^2 = 10$, $\sum x_i y_i = 20$, no intercept. Compute w for $\lambda = 0$ and $\lambda = 10$.

Solution

- $w = \frac{\sum xy}{\sum x^2 + \lambda}$.
- $\lambda = 0$: $20/10 = 2$. $\lambda = 10$: $20/20 = 1$. Halved by the penalty.

Answer: 2 and 1

Q3.19 MCQ · M

From a Bayesian viewpoint, L2 regularisation corresponds to

- (A) a uniform prior
 (B) a Gaussian prior on weights
 (C) a Laplace prior
 (D) no prior

Solution

- MAP: minimise $-\log p(y | w) - \log p(w)$. Gaussian prior gives $\frac{1}{2\tau^2} \|w\|^2$; Laplace prior gives $\frac{1}{b} \|w\|_1$.

Answer: B

Q3.20 MSQ · M

Which are valid ways to regularise a decision tree?

- (A) Limit max depth
 (B) Set a minimum number of samples per leaf
 (C) Cost complexity (post) pruning
 (D) Increase the number of features considered at each split to all features

Solution

- (D) does nothing for a single tree (it already considers all features by default). Pruning is covered in Chapter 4; the reported InfoEdge answer for “reduce overfitting in trees” was pruning.

Answer: A, B, C

Q3.21 MCQ · M

You tune λ by picking the value with the best test set accuracy, then report that accuracy. The problem:

- (A) none
 (B) the reported accuracy is optimistically biased because the test set was used for model selection
 (C) accuracy will be too low
 (D) λ must be tuned on training accuracy

Solution

- Use a validation set or cross validation for tuning, and touch the test set once. Nested CV if data are small.

Answer: B

Q3.22 MSQ · M

Which statements about k fold cross validation are true?

- (A) Every example is used for validation exactly once.
- (B) For time series, shuffled k fold can leak future information.
- (C) Larger k means each model trains on more data.
- (D) Stratification keeps class proportions in each fold.

Solution

- With k folds each model trains on a fraction $(k - 1) / k$ of the data.

Answer: A, B, C, D**Q3.23** MCQ · H

Double descent refers to

- (A) learning rate decaying twice
- (B) test error rising near the interpolation threshold and then falling again as model size grows further
- (C) training error decreasing twice
- (D) two stage fine tuning

Solution

- Observed in neural nets, random features and even linear regression with minimum norm solutions. It does not contradict bias variance; it shows the classical U is not the full story for over parameterised models with implicit regularisation.

Answer: B**Q3.24** MSQ · M

Adding more training data typically

- (A) reduces variance
- (B) reduces bias
- (C) reduces the gap between train and validation error
- (D) never hurts a high variance model

Solution

- Bias is a property of the model family; more data does not change what a straight line can express.

Answer: A, C, D**Q3.25** MCQ · E

Which model is most likely to have HIGH bias?

- (A) Deep unpruned decision tree
- (B) Linear regression on a highly non linear problem
- (C) 1 nearest neighbour
- (D) Large neural network

Solution

- The others are flexible, low bias, high variance models.

Answer: B**Q3.26** NAT · H

At a point x , true $f(x) = 10$. Across many training sets, predictions have mean 12 and variance 3. Noise variance $\sigma^2 = 1$. Expected squared error at x ?

Solution

- Bias² = $(12 - 10)^2 = 4$. Variance = 3. Noise = 1.
- Total = $4 + 3 + 1 = 8$.

Answer: 8**Q3.27** MSQ · M

Data augmentation for an image classifier. Which are appropriate?

- (A) Random horizontal flips for natural photos
- (B) Random crops
- (C) Vertical flips for digit recognition (6 vs 9)
- (D) Colour jitter

Solution

- Augmentations must preserve the label. Flipping a 6 vertically can make a 9. Horizontal flips are fine for cats, not for text.

Answer: A, B, D**Q3.28** MCQ · M

Label smoothing with $\epsilon = 0.1$ and 5 classes turns the one hot target $(0, 1, 0, 0, 0)$ into

- (A) $(0.02, 0.92, 0.02, 0.02, 0.02)$
- (B) $(0.1, 0.9, 0, 0, 0)$
- (C) $(0.025, 0.9, 0.025, 0.025, 0.025)$
- (D) $(0.2, 0.2, 0.2, 0.2, 0.2)$

Solution

- Formula: $y' = (1 - \epsilon)y + \epsilon/K$. Correct class: $0.9 + 0.02 = 0.92$; others: 0.02 . Sum = 1.
- Effect: discourages over confident logits, improves calibration.

Answer: A**Q3.29** MSQ · H

Which statements about regularisation strength λ are true?

- (A) As $\lambda \rightarrow \infty$ in Ridge, all coefficients tend to 0.
- (B) As $\lambda \rightarrow 0$, Ridge approaches OLS.
- (C) The Lasso path: as λ decreases from large values, features enter the model one by one.
- (D) Optimal λ is always 1.

Solution

- The Lasso regularisation path (LARS) is piecewise linear; the first feature to enter is the one most correlated with y .

Answer: A, B, C**Q3.30** MCQ · M

A model's validation loss decreases while validation accuracy also decreases slightly. The most plausible reading is

- (A) a bug always
- (B) the model becomes better calibrated on some examples while a few borderline predictions flip to wrong
- (C) overfitting
- (D) underfitting

Solution

- Loss measures confidence; accuracy measures only the argmax. They can move in different directions for a while. Monitor both; choose the one tied to the business metric.

Answer: B**Q3.31** SUB · M

Explain bias and variance to a non technical product manager in two sentences.

Solution

- Analogy: a stubborn forecaster who always says "sunny" (bias) vs one who changes wildly based on yesterday alone (variance).

Answer: Bias is being consistently wrong because the model is too simple; variance is being inconsistent because the model memorised quirks of the data it saw.

Q3.32 MSQ · M

Which settings increase a gradient boosting model's variance?

- (A) More trees with a high learning rate
- (B) Deeper trees
- (C) Row subsampling of 0.5
- (D) Larger min child weight

Solution

- Subsampling and larger min child weight regularise (reduce variance).

Answer: A, B

Q3.33 MCQ · M

Which is NOT a regularisation technique?

- (A) Dropout
- (B) Early stopping
- (C) Increasing learning rate to 10
- (D) Weight decay

Solution

- A huge learning rate causes divergence, not regularisation (although moderate SGD noise has a mild regularising effect).

Answer: C

Q3.34 SUB · H

Interviewer: "Your validation loss is lower than your training loss. Is something wrong?"

Solution

- Dropout and augmentation are active during training but off during validation, making training loss look worse.
- Training loss is often averaged over the epoch while weights were improving; validation is measured at the end.
- Validation set may be easier or smaller. If the gap is large, check for leakage or duplicates across splits.

Answer: Not necessarily; check regularisation effects, averaging, and leakage.

Q3.35 MSQ · M

Which changes increase a model's bias (and usually lower its variance)?

- (A) Increasing λ in Ridge
- (B) Reducing max depth of a tree
- (C) Increasing k in KNN
- (D) Adding polynomial features of degree 10

Solution

- Each of (A)-(C) restricts or smooths the model. (D) increases flexibility: lower bias, higher variance.

Answer: A, B, C

Interview angle

- Derive the bias variance decomposition for squared loss.
- Draw both graphs (complexity and iterations) and mark under/overfitting and early stopping.
- L1 vs L2: geometry, calculus, Bayesian priors, when to use each, effect on correlated features.
- Why does dropout work? What changes at test time? What is inverted dropout?
- How do you choose λ without leaking the test set? Cross validation for time series?

4 Decision Trees

Companion resource for this section: [StatQuest: Decision and Classification Trees, Clearly Explained](#)

4.1 A tree is a game of twenty questions

- To guess whether a candidate will accept a job offer you might ask: “Is the salary hike above 30%?” If yes, “Is the office in the same city?” Each question splits the data; at the end (a **leaf**) you predict the majority class (classification) or the average value (regression).
- Vocabulary: root node (first question), internal nodes (questions), branches (answers), leaves (predictions), depth (number of questions on the longest path).
- The tree divides feature space into axis aligned rectangles. That is why trees handle non linear patterns and interactions without feature engineering, but draw “staircase” boundaries for diagonal patterns.

4.2 How to choose a question: impurity

- A node is **pure** if all its examples have the same class. We want questions that make children as pure as possible.
- Gini impurity:** $G = 1 - \sum_k p_k^2$. It is the probability that two random draws (with replacement) from the node have different labels. Pure node: 0. 50/50 binary node: $1 - 0.25 - 0.25 = 0.5$ (the binary maximum).
- Entropy:** $H = -\sum_k p_k \log_2 p_k$ bits. Pure: 0. 50/50: 1 bit (binary maximum). With K classes the maximum is $\log_2 K$.
- Information gain (IG)** = impurity(parent) – weighted average impurity(children), weights = fraction of parent examples going to each child.
- Worked example: parent has 10 examples, 5 yes and 5 no, $H = 1$. Split A sends (4 yes, 1 no) left and (1 yes, 4 no) right. Each child: $p = 0.8, 0.2$: $H = -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = 0.8(0.322) + 0.2(2.322) = 0.258 + 0.464 = 0.722$. Weighted: $0.5(0.722) + 0.5(0.722) = 0.722$. $IG = 1 - 0.722 = 0.278$ bits.
- Greedy algorithm: at each node try every feature and every threshold, pick the split with the highest gain, recurse on each child. Stop when a node is pure, too small, or max depth is reached.
- Numeric features: sort the unique values; candidate thresholds are midpoints between consecutive values (only midpoints where the class changes can be optimal).
- Categorical features: CART makes binary splits (a subset vs the rest); ID3 makes one branch per category.

4.3 The three classic algorithms

	ID3	C4.5	CART
Split criterion	Information gain	Gain ratio	Gini (classif.), MSE (regression)
Splits	multiway	multiway	binary only
Numeric features	no	yes (thresholds)	yes
Pruning	no	error based	cost complexity
Regression	no	no	yes

- Information gain is biased toward features with many distinct values (a customer ID column splits every row into its own pure leaf: maximum gain, zero generalisation). **Gain ratio** = IG / split information, where split information = $-\sum_j \frac{n_j}{n} \log_2 \frac{n_j}{n}$ penalises many small branches.
- scikit-learn implements an optimised CART.

4.4 Regression trees

- Leaf prediction = mean of targets in the leaf. Split criterion = reduction in variance (equivalently in sum of squared errors): choose the split minimising $\sum_{\text{left}} (y - \bar{y}_L)^2 + \sum_{\text{right}} (y - \bar{y}_R)^2$.
- Example: targets 1, 2, 10, 11. Splitting between 2 and 10 gives leaves with means 1.5 and 10.5, $SSE = 0.25 + 0.25 + 0.25 + 0.25 = 1$, down from the parent $SSE = 20.25 + 12.25 + 20.25 + 30.25 = 83$ (mean 6).
- A regression tree’s prediction is a step function; it cannot extrapolate beyond the range of training targets.

4.5 Overfitting and pruning (a reported InfoEdge interview question)

- Grown fully, a tree can put every training example in its own leaf: zero training error, high variance.
- **Pre pruning** (early stopping): `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease`, `max_leaf_nodes`. Risk: a weak split now might enable a great split later (XOR pattern), and pre pruning never sees it.
- **Post pruning**: grow the full tree, then remove branches that do not help on validation data. CART's **cost complexity pruning** minimises $R_\alpha(T) = R(T) + \alpha|T|$, where $R(T)$ is training error and $|T|$ the number of leaves. Increasing α gives a nested sequence of smaller trees; pick α by cross validation (scikit-learn: `ccp_alpha`).
- Reduced error pruning: replace a subtree by a leaf if validation accuracy does not drop.

4.6 Pseudo code (a reported InfoEdge interview task)

```
def build(D, depth):
    if pure(D) or depth == MAX or len(D) < MIN:
        return Leaf(majority(D))    # or mean(y) for regression
    best = None
    for f in features:
        for t in midpoints(sorted unique values of f in D):
            L, R = D[f <= t], D[f > t]
            g = imp(D) - (len(L)*imp(L) + len(R)*imp(R)) / len(D)
            if best is None or g > best.g: best = (f, t, g)
    if best.g <= 0: return Leaf(majority(D))
    return Node(best.f, best.t, build(L, depth+1), build(R, depth+1))
```

- Cost: sorting each feature once gives about $O(dn \log n)$ per level, so roughly $O(dn \log n \cdot \text{depth})$ to train. Prediction is $O(\text{depth})$.
- Say this out loud in the interview: “greedy, top down, recursive partitioning; impurity based split; stopping rule; majority or mean at leaves”.

4.7 Strengths, weaknesses, feature importance

- Strengths: interpretable, no scaling needed, handles mixed feature types and interactions, fast prediction, some implementations handle missing values (surrogate splits in CART).
- Weaknesses: high variance (small data changes produce a very different tree), axis aligned staircase boundaries, greedy (not globally optimal; finding the optimal tree is NP hard), biased toward high cardinality features, poor extrapolation.
- **Impurity based feature importance** (MDI): sum over all nodes that split on feature f of (node weight $\times$ impurity decrease), normalised to sum to 1. Fast but biased toward high cardinality and continuous features. **Permutation importance**: shuffle one feature on validation data and measure the score drop; more reliable.
- The cure for variance is averaging many trees: Chapter 5 (bagging, random forests) and Chapter 6 (boosting).

Where this shows up

- Trees are the base learners of Random Forest, XGBoost and LightGBM, which dominate tabular problems like job recommendation features, fraud and churn.
- Interpretable shallow trees are used to explain business rules to stakeholders.

Formula sheet

Gini = $1 - \sum p_k^2$ (binary max 0.5); Entropy = $-\sum p_k \log_2 p_k$ (binary max 1, K class max $\log_2 K$).

IG = $I(\text{parent}) - \sum_j \frac{n_j}{n} I(\text{child}_j)$; Gain ratio = $\text{IG} / \left(-\sum_j \frac{n_j}{n} \log_2 \frac{n_j}{n} \right)$.

Regression split: minimise $\text{SSE}_L + \text{SSE}_R$. Cost complexity: $R(T) + \alpha|T|$.

Handy logs: $\log_2 3 = 1.585$, $\log_2 5 = 2.322$, $\log_2 0.8 = -0.322$, $\log_2 0.75 = -0.415$, $\log_2(1/3) = -1.585$, $\log_2(2/3) = -0.585$.

4.8 Question bank: Decision trees

Q4.1 MSQ · M

★ **PYQ (INT: how to reduce overfitting in trees)** Which techniques reduce overfitting of a single decision tree?

- (A) Cost complexity (post) pruning
- (B) Limiting max depth
- (C) Increasing min samples per leaf
- (D) Growing until every leaf is pure

Solution

- (D) is the recipe for overfitting.
- The reported InfoEdge answer was “pruning”; mention both pre and post pruning and how α is chosen by CV.

Answer: A, B, C

Q4.2 SUB · H

★ **PYQ (INT: pseudo code a decision tree)** Write pseudo code for training a classification tree and state its time complexity.

Solution

- Key points to state: greedy top down recursion; for each feature, for each candidate threshold (mid-points), compute weighted child impurity; pick max gain; stop on purity, depth, min samples or no positive gain; leaf = majority class.
- Follow ups: regression version (mean, SSE), handling categoricals (subset splits), missing values (surrogates or learned default direction like XGBoost).

Answer: See the pseudo code box in 4.6; training is about $O(d n \log n \cdot \text{depth})$, prediction $O(\text{depth})$.

Q4.3 NAT · E

★ **PYQ (OA: entropy formula (applied to a node))** A node has 8 positives and 2 negatives. Compute its entropy and Gini.

Solution

- $p = 0.8, 0.2$. Entropy $= -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = 0.8(0.322) + 0.2(2.322) = 0.258 + 0.464 = 0.722$.
- Gini $= 1 - (0.64 + 0.04) = 0.32$.

Answer: Entropy ≈ 0.722 bits, Gini $= 0.32$

Q4.4 NAT · M

Parent: 6 yes, 6 no. Split: left (5 yes, 1 no), right (1 yes, 5 no). Compute information gain (entropy).

Solution

- Parent entropy: 50/50 gives 1 bit.
- Left: $p = 5/6, 1/6$. $\log_2(5/6) = -0.263$, $\log_2(1/6) = -2.585$. $H = (5/6)(0.263) + (1/6)(2.585) = 0.219 + 0.431 = 0.650$.
- Right is symmetric: 0.650. Weighted: $0.5(0.650) + 0.5(0.650) = 0.650$.
- IG $= 1 - 0.650 = 0.350$.

Answer: ≈ 0.350 bits

Q4.5 NAT · M

Same parent (6 yes, 6 no). Split B: left has 8 examples (6 yes, 2 no), right has 4 (0 yes, 4 no). Compute the Gini gain and compare with a split that has children (4 yes, 2 no) and (2 yes, 4 no).

Solution

- Parent Gini $= 0.5$.
- B left: $p = 0.75, 0.25$; Gini $= 1 - 0.5625 - 0.0625 = 0.375$. Right: pure, 0. Weighted $= (8/12)(0.375) + 0 = 0.25$. Gain $= 0.5 - 0.25 = 0.25$.

- Other: each child $p = 2/3, 1/3$; Gini = $1 - 4/9 - 1/9 = 4/9 = 0.444$. Weighted = 0.444. Gain = 0.056.

Answer: Split B gain = 0.25; other split gain ≈ 0.056 ; choose B

Q4.6 MCQ · E

Maximum entropy of a node with 4 equally likely classes is

- (A) 1 bit
- (B) 2 bits
- (C) 4 bits
- (D) 0.75 bits

Solution

- $\log_2 4 = 2$. (Maximum Gini with 4 classes is $1 - 4(1/16) = 0.75$, the trap option.)

Answer: B

Q4.7 MSQ · M

Which statements about Gini vs entropy are true?

- (A) Both are 0 for a pure node.
- (B) Both are maximised at a uniform class distribution.
- (C) They choose the same split in most practical cases.
- (D) Gini is slower to compute because of logarithms.

Solution

- (D) Reversed: entropy needs logs, Gini only squares, so Gini is slightly faster (scikit-learn default).

Answer: A, B, C

Q4.8 MCQ · M

A “customer_id” column is included as a feature. With information gain, ID3 will likely

- (A) ignore it
- (B) choose it at the root because it perfectly separates training rows
- (C) treat it as noise automatically
- (D) use it only at leaves

Solution

- Every child is a single row, entropy 0, so IG equals parent entropy, the maximum possible. Generalisation is zero. Gain ratio or dropping ID columns fixes this.

Answer: B

Q4.9 NAT · M

A split sends 50% of rows left and 50% right. Compute its split information (the gain ratio denominator). What if it sends rows equally into 8 branches?

Solution

- Two equal branches: $-2(0.5 \log_2 0.5) = 1$. Eight equal: $\log_2 8 = 3$. A many branch split must have 3 times the IG to achieve the same gain ratio.

Answer: 1 bit and 3 bits

Q4.10 NAT · M

Regression targets in a node: 2, 4, 6, 20. Candidate split between 6 and 20. Compute SSE before and after.

Solution

- Parent mean = $32/4 = 8$. Deviations: $-6, -4, -2, 12$; SSE = $36 + 16 + 4 + 144 = 200$.
- Left (2, 4, 6) mean 4: SSE = $4 + 0 + 4 = 8$. Right (20): 0.
- After = 8. Reduction = 192.

Answer: Before 200, after 8

Q4.11 MCQ · M

A regression tree trained on house prices between 20 and 90 lakh is asked to price a mansion. The prediction

- (A) can exceed 90 lakh
- (B) cannot exceed the largest leaf mean, so at most about 90 lakh
- (C) is always the global mean
- (D) is undefined

Solution

- Leaves hold averages of training targets; trees (and forests, boosted trees) cannot extrapolate. Linear models can.

Answer: B

Q4.12 MSQ · M

Which are advantages of decision trees?

- (A) No feature scaling needed
- (B) Captures feature interactions automatically
- (C) Low variance
- (D) Interpretable when shallow

Solution

- Single trees have *high* variance. That is the whole motivation for ensembles.

Answer: A, B, D

Q4.13 MCQ · M

Decision tree decision boundaries in 2D are

- (A) smooth curves
- (B) piecewise constant, axis parallel rectangles
- (C) always a single line
- (D) circles

Solution

- Each split tests one feature against a threshold. A diagonal boundary needs many small steps. Oblique trees (splits on linear combinations) exist but are uncommon.

Answer: B

Q4.14 MSQ · H

The XOR pattern ($y = 1$ iff exactly one of x_1, x_2 is 1) on four points. Which are true?

- (A) Any single split on x_1 or x_2 gives zero information gain.
- (B) A depth 2 tree classifies it perfectly.
- (C) A pre pruning rule “stop if gain ≤ 0 ” stops at the root.
- (D) Logistic regression on x_1, x_2 solves it perfectly.

Solution

- Splitting on x_1 : each child still has one positive and one negative, entropy 1, gain 0.
- Depth 2 separates all four cells.
- This is the classic argument for post pruning over greedy pre pruning.
- (D) False: XOR is not linearly separable (needs the x_1x_2 feature).

Answer: A, B, C

Q4.15 MCQ · E

Which algorithm uses only binary splits and supports regression?

- (A) ID3
- (B) C4.5

- (C) CART
(D) Apriori

Solution

- CART = Classification And Regression Trees.

Answer: C

Q4.16 NAT · M

Cost complexity pruning with $\alpha = 0.02$. Subtree T_1 has training error 0.10 with 5 leaves; pruned T_2 has error 0.14 with 2 leaves. Which has lower R_α ?

Solution

- $R_\alpha(T_1) = 0.10 + 0.02 \cdot 5 = 0.20$. $R_\alpha(T_2) = 0.14 + 0.02 \cdot 2 = 0.18$.
- T_2 wins: the 3 extra leaves buy only 0.04 error, less than their cost 0.06.
- Break even $\alpha = (0.14 - 0.10) / (5 - 2) = 0.0133$.

Answer: T_2 (0.18 vs 0.20)

Q4.17 MSQ · M

Impurity based (MDI) feature importance. Which are true?

- (A) It can be biased toward continuous and high cardinality features.
(B) It is computed from training data.
(C) Permutation importance on validation data is a more reliable alternative.
(D) Importances always imply causation.

Solution

- High cardinality features offer more split points and so more chances to reduce impurity by luck. Also with correlated features, importance gets split between them.

Answer: A, B, C

Q4.18 MCQ · M

Number of candidate thresholds to evaluate for a numeric feature with 100 distinct values in a node:

- (A) 100
(B) 99
(C) 2^{100}
(D) 1

Solution

- Midpoints between consecutive sorted distinct values: $100 - 1 = 99$.
- For a categorical feature with q levels, binary partitions number $2^q - 1$; for binary classification CART sorts levels by positive rate, reducing to $q - 1$ splits.

Answer: B

Q4.19 MSQ · M

Which stopping criteria are common in tree growing?

- (A) Node is pure
(B) Maximum depth reached
(C) Number of samples below a minimum
(D) Validation accuracy of the whole forest

Solution

- (D) is not a per node stopping rule for a single tree.

Answer: A, B, C

Q4.20 NAT · E

A fully grown binary tree has depth 10. What is the maximum number of leaves?

Solution

- Each level doubles: $2^{10} = 1024$. Prediction needs at most 10 comparisons.

Answer: 1024**Q4.21** MCQ · M

Why are decision trees called “greedy”?

- (A) They use all memory.
- (B) They choose the locally best split at each node without looking ahead.
- (C) They always find the globally optimal tree.
- (D) They prefer shallow trees.

Solution

- Finding the optimal tree is NP complete, so greedy search is the practical compromise.

Answer: B**Q4.22** MSQ · M

Handling missing values in trees. Which are valid?

- (A) CART surrogate splits
- (B) Learning a default direction for missing values (XGBoost, LightGBM)
- (C) Imputation before training
- (D) Trees crash with any missing value in every library

Solution

- Surrogate splits use another feature that mimics the primary split. XGBoost learns during training which branch missing values should go.
- scikit-learn trees support NaN natively since version 1.3.

Answer: A, B, C**Q4.23** NAT · H

Parent: 10 examples (7 A, 3 B). Split: left (7 A, 1 B), right (0 A, 2 B). Compute IG with entropy.

Solution

- Parent: $-(0.7 \log_2 0.7 + 0.3 \log_2 0.3) = 0.7(0.515) + 0.3(1.737) = 0.360 + 0.521 = 0.881$.
- Left: $p = 7/8, 1/8$. $\log_2(7/8) = -0.193$, $\log_2(1/8) = -3$. $H = 0.875(0.193) + 0.125(3) = 0.169 + 0.375 = 0.544$.
- Right: pure, 0.
- Weighted: $(8/10)(0.544) = 0.435$. $IG = 0.881 - 0.435 = 0.446$.

Answer: ≈ 0.446 bits**Q4.24** MCQ · M

Which is NOT a hyperparameter of scikit-learn’s DecisionTreeClassifier?

- (A) max_depth
- (B) min_samples_leaf
- (C) ccp_alpha
- (D) learning_rate

Solution

- Learning rate belongs to boosting.

Answer: D**Q4.25** MSQ · M

Decision trees and feature scaling / monotone transforms. Which are true?

- (A) Standardising a feature does not change the learned tree structure.

- (B) Applying log to a positive feature does not change which rows go left and right.
- (C) Trees are sensitive to the units of the target in regression only through the leaf values.
- (D) Trees require one hot encoding in every implementation.

Solution

- Monotone transforms keep the order of values, so the same partitions are achievable (thresholds just move).
- (D) False: LightGBM and CatBoost handle categoricals natively; scikit-learn recently added categorical support in HistGradientBoosting.

Answer: A, B, C**Q4.26** MCQ · H

Two features are almost perfectly correlated. In a single tree:

- (A) both always get equal importance
- (B) the tree may use either one, and importance gets split or concentrated arbitrarily
- (C) the tree fails to train
- (D) it removes one automatically with a warning

Solution

- Tiny data changes can flip which one wins at each split. Do not over interpret importance with correlated features.

Answer: B**Q4.27** MSQ · M

Pre pruning vs post pruning. Which are true?

- (A) Pre pruning is cheaper to compute.
- (B) Post pruning can recover useful splits hidden behind a weak split.
- (C) Post pruning never needs validation data or CV.
- (D) Cost complexity pruning is a post pruning method.

Solution

- (C) False: α is chosen by validation or CV.

Answer: A, B, D**Q4.28** NAT · M

Gini of a node with class counts (2, 3, 5)?

Solution

- $p = 0.2, 0.3, 0.5; \sum p^2 = 0.04 + 0.09 + 0.25 = 0.38; \text{Gini} = 0.62.$

Answer: 0.62**Q4.29** MCQ · E

The leaf prediction of a classification tree is usually

- (A) the median of targets
- (B) the majority class (or class proportions as probabilities)
- (C) a linear model
- (D) random

Solution

- `predict_proba` returns the class fractions in the leaf. These are often poorly calibrated for deep trees (many pure leaves give 0/1).

Answer: B**Q4.30** MSQ · H

Why are single trees unstable (high variance)?

- (A) A small change in data can change the root split.
- (B) Errors at the top propagate to all descendants.
- (C) The hierarchical, discontinuous nature of splits amplifies noise.
- (D) They use gradient descent.

Solution

- A different root reshapes the whole tree. This is why bagging (averaging trees on bootstrap samples) helps so much.

Answer: A, B, C**Q4.31** NAT · M

With 1 million rows and 50 features, roughly how many split evaluations does a pre sorted exact algorithm do at the root?

Solution

- Up to $n - 1 \approx 10^6$ thresholds per feature, times 50 features, so about 5×10^7 . With a histogram of 256 bins (LightGBM), only $256 \times 50 = 12,800$. This is the speed trick of Chapter 6.

Answer: about 5×10^7 **Q4.32** SUB · M

How does a decision tree decide the root node?

Solution

- Mention: weighted child impurity, midpoints for numeric features, tie breaking, and that the choice is greedy.

Answer: It evaluates every feature and threshold and picks the split with the largest impurity decrease (information gain or Gini decrease) on the whole training set.

Q4.33 MCQ · M

Which of these data patterns is hardest for a shallow tree?

- (A) $y = 1$ if $x_1 > 5$
- (B) $y = 1$ if $x_1 > x_2$ (diagonal boundary)
- (C) $y = 1$ if $x_1 > 5$ and $x_2 < 3$
- (D) y depends on a single categorical feature

Solution

- A diagonal needs a staircase of many splits. Adding the engineered feature $x_1 - x_2$ makes it a single split.

Answer: B**Q4.34** MSQ · M

For a regression tree, which split criteria are used in practice?

- (A) Variance reduction / MSE
- (B) MAE (more robust)
- (C) Poisson deviance for counts
- (D) Gini

Solution

- Gini is for classification. scikit-learn supports squared_error, friedman_mse, absolute_error and poisson.

Answer: A, B, C**Q4.35** NAT · M

Entropy of a node with class counts (3, 3, 6)?

Solution

- $p = 0.25, 0.25, 0.5$.
- $H = -[0.25 \log_2 0.25 + 0.25 \log_2 0.25 + 0.5 \log_2 0.5] = 0.25(2) + 0.25(2) + 0.5(1) = 1.5$.

Answer: 1.5 bits

Interview angle

- Pseudo code a tree on paper; give the complexity. How do you handle numeric, categorical and missing features?
- Gini vs entropy vs gain ratio; why information gain favours ID like features.
- Pre vs post pruning, cost complexity pruning and how α is chosen.
- Why are trees high variance, and how do random forests and boosting each fix a different weakness?
- MDI vs permutation importance; what goes wrong with correlated features?

5 Ensembles, Bagging and Random Forests

Companion resource for this section: [StatQuest: Random Forests Part 1, Building, Using and Evaluating](#)

5.1 Wisdom of crowds

- Ask one person to guess the weight of an ox and they may be far off. Average 800 independent guesses and the error shrinks dramatically. Ensembles do this with models.
- Why it works: if B models each have error variance σ^2 and are *independent*, their average has variance σ^2 / B . If they are correlated with pairwise correlation ρ :

$$\text{Var}\left(\frac{1}{B} \sum_b \hat{f}_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

- As $B \rightarrow \infty$ the second term vanishes but $\rho\sigma^2$ remains. So the recipe is: many models (large B), each low bias, and as *decorrelated* as possible (small ρ).
- Majority vote example: 5 independent classifiers, each 70% accurate. The vote is right if at least 3 are right: $P = \binom{5}{3}0.7^30.3^2 + \binom{5}{4}0.7^40.3 + 0.7^5 = 0.3087 + 0.3602 + 0.1681 = 0.837$. Better than any single one, but only because errors are independent.

5.2 Three families of ensembles (a reported InfoEdge OA question)

- **Bagging** (bootstrap aggregating): train the same algorithm on different bootstrap samples *in parallel*, then average (regression) or vote (classification). Mainly reduces **variance**. Examples: bagged trees, Random Forest, Extra Trees.
- **Boosting**: train models *sequentially*, each focusing on the mistakes of the ensemble so far, then combine with weights. Mainly reduces **bias** (and variance to some degree). Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost (Chapter 6).
- **Stacking**: train several different model types (level 0), then train a **meta model** (level 1) on their out of fold predictions to learn how to combine them. Key detail: the meta model must be trained on predictions made for data the base models did not see (k fold), otherwise it learns to trust overfit models.
- **Voting / blending**: simpler cousins of stacking. Hard voting = majority class; soft voting = average predicted probabilities (usually better when probabilities are calibrated). Blending = stacking with a single holdout set instead of k folds.

5.3 Bootstrap sampling

- From n training rows, draw n rows *with replacement*. Some rows appear several times, some not at all.
- Probability a given row is never drawn: $(1 - 1/n)^n \rightarrow e^{-1} \approx 0.368$. So each bootstrap sample contains about 63.2% of the unique rows.
- The left out 36.8% are that tree's **out of bag (OOB)** rows. Predict each training row using only the trees for which it was OOB; the resulting OOB error is a nearly free estimate of test error (similar to cross validation) without a separate validation set.

5.4 Random Forest = bagged trees + random feature subsets

- Plain bagged trees are still highly correlated: if one feature is very strong, every tree uses it at the root, so trees look alike and ρ is large.
- Random Forest's extra trick: at *every split*, consider only a random subset of m features (`max_features`). Defaults: $m = \sqrt{d}$ for classification, $m = d/3$ historically for regression (scikit-learn now uses all features, i.e. 1.0, for regression by default).
- This forces some trees to find alternative good splits, decorrelating them. Each tree gets slightly worse (more bias), but the average gets much better (much lower ρ).
- Trees are usually grown deep and unpruned: low bias, high variance, which is exactly what averaging fixes.
- Algorithm: for $b = 1..B$: draw a bootstrap sample; grow a tree where each split searches only m random features; stop at `min_samples_leaf`. Predict by majority vote or average of probabilities.

- More trees never cause overfitting in the sense of increasing test error; the error just plateaus. Use enough trees for the OOB error to stabilise (hundreds).
- Hyperparameters that matter: `n_estimators`, `max_features` (most important), `max_depth` / `min_samples_leaf`, `bootstrap` on/off, `class_weight`.

5.5 Why RF reduces variance and overfitting vs one tree (Glassdoor InfoEdge)

- A single deep tree has low bias but high variance: it memorises noise specific to its training set.
- Different bootstrap samples contain different noise, so trees make *different* mistakes; averaging cancels mistakes that are not shared. Random feature selection lowers the correlation ρ further.
- Bias of the forest $\approx$ bias of one tree (slightly higher due to feature subsampling); variance drops toward $\rho\sigma^2$.

5.6 Extra Trees, feature importance, and practical notes

- **Extremely Randomised Trees:** also pick the split threshold at random (for each candidate feature) and usually use the full dataset instead of bootstraps. Even lower variance, faster to train, slightly more bias.
- **Feature importance:** MDI (mean impurity decrease across all trees) or permutation importance on OOB or validation data. Same caveats as Chapter 4: MDI favours high cardinality features, correlated features share importance.
- **Proximity:** how often two rows land in the same leaf; used for outlier detection and imputation. Isolation Forest (anomaly detection) uses random trees too: anomalies are isolated in fewer splits.
- **Weaknesses:** large memory and slower prediction than one tree; cannot extrapolate; less interpretable; on many tabular benchmarks well tuned gradient boosting wins.

5.7 Random Forest vs Gradient Boosting (Glassdoor InfoEdge)

	Random Forest (bagging)	Gradient Boosting
Training	parallel, independent trees	sequential, each tree fixes residual errors
Base trees	deep, low bias, high variance	shallow (depth 3 to 8), high bias
Main effect	reduces variance	reduces bias
Overfits with more trees?	no, error plateaus	yes, needs early stopping / learning rate
Tuning effort	low, robust defaults	higher (lr, depth, trees, regularisation)
Noise / outliers	robust	sensitive (keeps chasing hard points)
Typical accuracy	very good	often best on tabular data

Where this shows up

- RF is the go to strong baseline in a 45 minute ML coding round: little tuning, no scaling, OOB score for free.
- Stacking and blending won many Kaggle competitions; in production, a single tuned GBM is usually preferred for latency and maintenance.

Formula sheet

$\text{Var}(\text{avg}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. OOB fraction $(1 - 1/n)^n \rightarrow e^{-1} = 0.368$; unique fraction 0.632.

RF `max_features`: $\sqrt{d}$ (classification). Majority vote accuracy: $\sum_{k>B/2} \binom{B}{k} p^k (1-p)^{B-k}$.

Bagging: parallel, variance down. Boosting: sequential, bias down. Stacking: meta learner on out of fold predictions.

5.8 Question bank: Ensembles, bagging and random forests

Q5.1 MCQ · E

★ **PYQ (OA: is Random Forest bagging or boosting)** Random Forest is an example of

- (A) boosting
- (B) bagging
- (C) stacking
- (D) none of these

Solution

- Bootstrap samples, independent parallel trees, averaging/voting. Plus random feature subsets at each split.

Answer: B**Q5.2** MSQ · M★ **PYQ (OA: types of ensembles, bagging, boosting, stacking)** Which pairings are correct?

- (A) Bagging: models trained in parallel on bootstrap samples.
- (B) Boosting: models trained sequentially, later models focus on earlier errors.
- (C) Stacking: a meta model learns to combine base model predictions.
- (D) Boosting primarily reduces variance while bagging primarily reduces bias.

Solution

- (D) is reversed: bagging mainly reduces variance; boosting mainly reduces bias.

Answer: A, B, C**Q5.3** MSQ · M★ **PYQ (INT: why Random Forests reduce variance compared to one tree)** Which statements explain RF's advantage?

- (A) Averaging many high variance, low bias trees reduces variance.
- (B) Random feature subsets decorrelate the trees.
- (C) Each tree in RF is individually more accurate than a single full tree.
- (D) Bootstrap samples give each tree different noise to fit.

Solution

- (C) False: each RF tree is typically slightly *worse* (it sees a bootstrap sample and restricted features). The forest wins through averaging decorrelated errors.

Answer: A, B, D**Q5.4** MSQ · M★ **PYQ (INT: bootstrap sampling and random feature selection)** In a Random Forest classifier with 64 features (scikit-learn defaults):

- (A) Each tree is trained on a bootstrap sample of the same size as the training set.
- (B) About 8 features are considered at each split.
- (C) Each tree uses the same 8 features throughout.
- (D) Roughly 36.8% of rows are out of bag for each tree.

Solution

- $\sqrt{64} = 8$ features, re sampled at *every split*, not once per tree (C false; that would be the "random subspace" method).

Answer: A, B, D**Q5.5** MSQ · M★ **PYQ (INT: Random Forest vs Gradient Boosting)** Which are true?

- (A) RF trees can be built in parallel; GB trees are built sequentially.
- (B) GB typically uses shallow trees; RF uses deep trees.
- (C) Adding more trees to RF can cause severe overfitting.
- (D) GB usually needs early stopping or a small learning rate to avoid overfitting.

Solution

- (C) False: RF test error plateaus as trees are added. GB can overfit with too many rounds.

Answer: A, B, D

Q5.6 MSQ · M

★ **PYQ (INT: feature importance and how RF makes predictions)** Which are true?

- (A) RF classification prediction is the majority vote (or average of class probabilities) of its trees.
- (B) RF regression prediction is the average of tree outputs.
- (C) MDI importance is always unbiased.
- (D) Permutation importance measures the drop in score when a feature's values are shuffled.

Solution

- scikit-learn averages probabilities (soft voting).
- (C) False: MDI is biased toward high cardinality features and computed on training data.

Answer: A, B, D

Q5.7 NAT · E

With $n = 1000$ training rows, what is the probability that a specific row is out of bag for a given tree?

Solution

- $(1 - 1/1000)^{1000} \approx e^{-1} = 0.3677$.

Answer: ≈ 0.368

Q5.8 NAT · M

Ten models each have prediction variance $\sigma^2 = 4$. Compute the variance of their average if (a) independent, (b) pairwise correlation $\rho = 0.5$.

Solution

- (a) $\sigma^2/B = 4/10 = 0.4$.
- (b) $\rho\sigma^2 + (1 - \rho)\sigma^2/B = 0.5(4) + 0.5(4)/10 = 2 + 0.2 = 2.2$.
- Even infinitely many trees at $\rho = 0.5$ only reach 2. Decorrelation is worth more than more trees.

Answer: (a) 0.4, (b) 2.2

Q5.9 NAT · M

Three independent classifiers each with accuracy 0.8. Accuracy of their majority vote?

Solution

- Vote correct if at least 2 of 3 are correct.
- $P(3) = 0.8^3 = 0.512$. $P(2) = 3 \times 0.8^2 \times 0.2 = 0.384$.
- Total = 0.896.

Answer: 0.896

Q5.10 MCQ · M

Three classifiers each with accuracy 0.4 (worse than chance, binary task) are majority voted, errors independent. Vote accuracy is

- (A) higher than 0.4
- (B) lower than 0.4
- (C) exactly 0.4
- (D) 0.5

Solution

- $0.4^3 + 3(0.4^2)(0.6) = 0.064 + 0.288 = 0.352$. Voting amplifies whichever side of 0.5 the members are on. Ensembles need members better than chance.

Answer: B

Q5.11 MSQ · M

OOB error. Which are true?

- (A) Each training row is predicted only by trees that did not see it.
- (B) It is an approximately unbiased estimate of test error.
- (C) It requires a separate validation set.
- (D) It is available for bagging-based methods.

Solution

- Enable with `oob_score=True` in scikit-learn.

Answer: A, B, D

Q5.12 MCQ · M

In a Random Forest, reducing `max_features` from $\sqrt{d}$ to 1 typically

- (A) increases correlation between trees
- (B) decreases correlation but increases each tree's bias
- (C) has no effect
- (D) turns RF into boosting

Solution

- With one random feature per split, trees are nearly random: very decorrelated but individually weak. The best value is found by tuning.

Answer: B

Q5.13 MSQ · M

Which are true about stacking?

- (A) Base models are usually diverse (e.g. linear, tree ensemble, KNN).
- (B) The meta model should be trained on out of fold predictions.
- (C) Training the meta model on in sample base predictions causes leakage.
- (D) The meta model must be a neural network.

Solution

- A simple linear or logistic regression meta model is common and robust.

Answer: A, B, C

Q5.14 MCQ · E

Soft voting combines classifiers by

- (A) counting predicted labels
- (B) averaging predicted class probabilities
- (C) choosing the most accurate model
- (D) training a meta model

Solution

- Hard voting counts labels. Soft voting can use confidence and usually performs better if models are calibrated.

Answer: B

Q5.15 MSQ · M

Bagging helps most for which base learners?

- (A) Unpruned decision trees
- (B) Linear regression
- (C) High variance neural networks
- (D) k -NN with $k = 50$

Solution

- Bagging stable, low variance learners (linear regression, smooth KNN) gives little benefit: every bootstrap produces nearly the same model.

Answer: A, C

Q5.16 NAT · M

A bootstrap sample of size 5 is drawn from 5 distinct rows. What is the probability that all 5 rows are distinct (sample is a permutation)?

Solution

- Total sequences $5^5 = 3125$; sequences with all distinct = $5! = 120$.
- $120/3125 = 0.0384$. So almost every bootstrap has repeats.

Answer: $5!/5^5 = 0.0384$

Q5.17 NAT · H

Expected number of unique rows in a bootstrap sample of size $n = 100$?

Solution

- Each row appears at least once with probability $1 - (1 - 1/100)^{100} = 1 - 0.366 = 0.634$.
- Linearity of expectation: $100 \times 0.634 = 63.4$.

Answer: ≈ 63.4

Q5.18 MCQ · M

Extra Trees differ from Random Forest mainly by

- (A) using boosting
- (B) choosing split thresholds at random and (by default) not bootstrapping
- (C) using linear models at leaves
- (D) using only one tree

Solution

- More randomness lowers variance and speeds training; bias rises slightly.

Answer: B

Q5.19 MSQ · M

Which are weaknesses of Random Forests?

- (A) Cannot extrapolate beyond training target range
- (B) Large memory footprint and slower prediction than a single tree
- (C) Requires feature scaling
- (D) Less interpretable than a single tree

Solution

- No scaling needed.

Answer: A, B, D

Q5.20 MCQ · M

Your RF has 99% training accuracy and 85% OOB accuracy. Most sensible first step:

- (A) Panic, it overfits.
- (B) Accept that deep trees fit training data almost perfectly; judge by OOB/validation, and tune `min_samples_leaf` or `max_features` if needed.
- (C) Reduce the number of trees to 5.
- (D) Remove bootstrapping.

Solution

- RF training accuracy is nearly always near 100% with deep trees; OOB is the honest number.

Answer: B**Q5.21** MSQ · H

Random Forest on a dataset where only 2 of 500 features are informative. Which are true?

- (A) With $m = \sqrt{500} \approx 22$, many splits will not include an informative feature.
- (B) Performance may suffer compared to a smaller feature set.
- (C) Increasing max_features can help.
- (D) RF is immune to irrelevant features.

Solution

- Chance that a split's 22 candidates include neither informative feature: about $(498/500)^{22} \approx 0.916$. Most splits are on noise. Feature selection or larger m helps.

Answer: A, B, C**Q5.22** MCQ · M

Isolation Forest detects anomalies because anomalies

- (A) have higher Gini
- (B) need fewer random splits to be isolated, giving shorter average path lengths
- (C) always lie at the mean
- (D) are removed by bootstrapping

Solution

- Random axis splits separate rare, far away points quickly. Score is based on average path length relative to $c(n)$.

Answer: B**Q5.23** MSQ · M

Which make a good ensemble?

- (A) Members with accuracy better than chance
- (B) Members whose errors are uncorrelated
- (C) Identical copies of the same model
- (D) Members trained with different algorithms or data views

Solution

- Identical copies give $\rho = 1$: no improvement at all.

Answer: A, B, D**Q5.24** NAT · M

Regression trees predict 12, 15, 9, 14 for a house. RF prediction?

Solution

- Average: $(12 + 15 + 9 + 14)/4 = 50/4 = 12.5$.

Answer: 12.5**Q5.25** NAT · M

Five trees give $P(\text{class 1}) = 0.9, 0.6, 0.4, 0.45, 0.3$. Compare soft voting and hard voting (threshold 0.5).

Solution

- Soft: mean = $(0.9 + 0.6 + 0.4 + 0.45 + 0.3)/5 = 2.65/5 = 0.53$, so class 1.
- Hard: labels 1, 1, 0, 0, 0, majority class 0. The confident tree swings soft voting.

Answer: Soft: 0.53, class 1. Hard: 2 votes vs 3, class 0.

Q5.26 MCQ · E

Which ensemble method's members can be trained fully in parallel?

- (A) AdaBoost
- (B) Gradient Boosting (across trees)
- (C) Random Forest
- (D) Stacking meta model with base models together

Solution

- Boosting is sequential across rounds; XGBoost parallelises *within* a tree (split search), not across trees.

Answer: C

Q5.27 MSQ · H

In the variance formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, which design choices reduce ρ ?

- (A) Bootstrap sampling
- (B) Random feature subsets per split
- (C) Using the full dataset and all features for every tree
- (D) Random split thresholds (Extra Trees)

Solution

- (C) makes all trees identical (for deterministic tree growing): $\rho = 1$.

Answer: A, B, D

Q5.28 MCQ · M

Class imbalance in Random Forest. A reasonable fix is

- (A) `class_weight='balanced_subsample'` or balanced bootstrap sampling
- (B) reducing trees to 1
- (C) removing the minority class
- (D) increasing `max_depth` only

Solution

- Balanced subsample recomputes weights per bootstrap. Balanced Random Forest undersamples the majority in each bootstrap.

Answer: A

Q5.29 MSQ · M

Bagging can be applied to:

- (A) Decision trees
- (B) Neural networks
- (C) Any base learner
- (D) Only linear models

Solution

- scikit-learn's `BaggingClassifier` wraps any estimator. Snapshot ensembles and deep ensembles are the neural network analogues.

Answer: A, B, C

Q5.30 MCQ · M

Pasting differs from bagging in that

- (A) samples are drawn without replacement
- (B) it uses boosting
- (C) it uses all features
- (D) it uses the test set

Solution

- Pasting = random subsets without replacement. Random patches = sample both rows and features.

Answer: A**Q5.31** SUB · M

In one minute, explain Random Forest to an interviewer.

Solution

- Build B trees; each on a bootstrap sample (about 63% unique rows); at each split consider $\sqrt{d}$ random features; grow deep; predict by vote or average; OOB rows give a free validation estimate; importance by MDI or permutation.

Answer: Bagged deep trees with random feature subsets at each split; averaging decorrelated high variance trees reduces variance.

Q5.32 MSQ · M

Which hyperparameters of RandomForestClassifier most directly control individual tree complexity?

- (A) max_depth
- (B) min_samples_leaf
- (C) n_estimators
- (D) max_leaf_nodes

Solution

- n_estimators controls how many trees, not their complexity.

Answer: A, B, D**Q5.33** NAT · H

Five independent classifiers each with accuracy 0.6. Majority vote accuracy?

Solution

- Need at least 3 correct.
- $P(3) = 10(0.6^3)(0.4^2) = 10(0.216)(0.16) = 0.3456$.
- $P(4) = 5(0.6^4)(0.4) = 5(0.1296)(0.4) = 0.2592$.
- $P(5) = 0.6^5 = 0.0778$.
- Total = 0.6826.

Answer: ≈ 0.683 **Q5.34** MCQ · M

Blending differs from stacking in that blending

- (A) uses a single holdout set to create meta features instead of k fold out of fold predictions
- (B) uses boosting
- (C) does not use a meta model
- (D) requires identical base models

Solution

- Simpler and less leakage prone, but the meta model sees less data.

Answer: A**Q5.35** MSQ · H

Why might a Random Forest's probability estimates be poorly calibrated?

- (A) Averaging many trees pushes probabilities away from 0 and 1 (toward the middle).
- (B) Feature subsampling increases variance of probabilities near extremes.
- (C) Probability calibration (Platt or isotonic) can fix it.
- (D) RF probabilities are always perfectly calibrated.

Solution

- To predict exactly 0 or 1, all trees must agree, which randomness prevents; so RF tends to be under confident at the extremes. Use `CalibratedClassifierCV`.

Answer: A, B, C

Interview angle

- Explain bootstrap, OOB, and derive the 63.2% number.
- Derive $\rho\sigma^2 + (1 - \rho)\sigma^2 / B$ and explain what RF does to each term.
- Why do more trees not overfit RF but can overfit GBM?
- Bagging vs boosting vs stacking: when would you choose each for a production system?
- Feature importance pitfalls; how would you explain one prediction (SHAP)?

6 Boosting: AdaBoost, Gradient Boosting, XGBoost, LightGBM

Companion resource for this section: [StatQuest: Gradient Boost Part 1, Regression Main Ideas](#)

6.1 The core idea: learn from your mistakes, one small step at a time

- A **weak learner** is barely better than guessing, like a **decision stump** (a depth 1 tree). Boosting combines many of them *sequentially*: each new learner concentrates on what the current ensemble gets wrong.
- Final model: $F_M(x) = F_0(x) + \sum_{m=1}^M \nu \alpha_m h_m(x)$, a weighted sum of weak learners.
- Analogy: a student takes a mock test, studies only the questions they got wrong, retakes, and repeats.

6.2 AdaBoost (adaptive boosting)

- Labels $y \in \{-1, +1\}$. Start with equal sample weights $w_i = 1/n$.
- Round t : fit a stump h_t minimising *weighted* error $\varepsilon_t = \sum_{i:h_t(x_i) \neq y_i} w_i$.
- Learner weight ("amount of say"): $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$. Error 0.5 gives $\alpha = 0$; smaller error gives larger say; error above 0.5 gives negative say (flip it).
- Reweight: $w_i \leftarrow w_i e^{-\alpha_t y_i h_t(x_i)}$, i.e. multiply by e^{α_t} if misclassified, $e^{-\alpha_t}$ if correct; then normalise to sum 1. After normalisation, misclassified points hold exactly half the total weight.
- Prediction: $\text{sign}(\sum_t \alpha_t h_t(x))$.
- Worked example: 10 points, stump misclassifies 3, $\varepsilon = 0.3$. $\alpha = \frac{1}{2} \ln(0.7/0.3) = \frac{1}{2} \ln 2.333 = \frac{1}{2}(0.847) = 0.424$. Misclassified weights: $0.1 \times e^{0.424} = 0.1 \times 1.528 = 0.1528$; correct: $0.1 \times e^{-0.424} = 0.0655$. Sum = $3(0.1528) + 7(0.0655) = 0.458 + 0.458 = 0.917$. Normalised: wrong ones 0.1667 each (total 0.5), right ones 0.0714 each (total 0.5).
- AdaBoost minimises the **exponential loss** $e^{-yF(x)}$. That loss punishes confident mistakes exponentially, so AdaBoost is sensitive to label noise and outliers.

6.3 Gradient boosting: gradient descent in function space

- Pick any differentiable loss $L(y, F)$. Start with a constant: $F_0 = \arg \min_c \sum L(y_i, c)$ (mean of y for MSE; log odds of the positive rate for log loss).
- Round m : compute **pseudo residuals** $r_i = -\frac{\partial L(y_i, F)}{\partial F} \Big|_{F=F_{m-1}(x_i)}$, the direction each prediction should move to reduce the loss.
- Fit a small regression tree h_m to the r_i . Set each leaf's value to the best constant step for that leaf (line search). Update $F_m = F_{m-1} + \nu h_m$ with **learning rate / shrinkage** ν (0.01 to 0.3).
- For MSE $L = \frac{1}{2}(y - F)^2$, $r_i = y_i - F(x_i)$: literally the residuals. So the next tree predicts "what we are still missing".
- For log loss, $r_i = y_i - p_i$ (label minus current predicted probability).
- Worked example (MSE, $\nu = 0.1$): targets 10, 20, 30. $F_0 = 20$. Residuals $-10, 0, 10$. Suppose the tree fits them exactly. $F_1 = 20 + 0.1(-10, 0, 10) = (19, 20, 21)$. New residuals $(-9, 0, 9)$: each round removes 10% of what remains.
- Why shrinkage: small steps with many trees generalise better (like a small learning rate in GD). Trade off: lower ν needs more trees; tune them together with early stopping on validation loss.
- **Stochastic gradient boosting**: each tree sees a random fraction of rows (subsample 0.5 to 0.8); adds randomness, reduces variance, speeds up.

6.4 XGBoost: regularised, second order, engineered for speed

- Objective at round t : $\sum_i L(y_i, F_{t-1}(x_i) + f_t(x_i)) + \Omega(f_t)$ with $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$ (T = number of leaves, w_j leaf weights). Built in L1 (α) and L2 (λ) regularisation on leaf weights: the "regularised" in regularised boosting.

- Second order Taylor expansion: with $g_i = \partial L / \partial F$ and $h_i = \partial^2 L / \partial F^2$ (gradient and Hessian per example), and $G_j = \sum_{i \in j} g_i$, $H_j = \sum_{i \in j} h_i$ for leaf j :

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma.$$

- For MSE: $g_i = F - y_i$ (negative residual), $h_i = 1$. So $w^* = \frac{\sum r_i}{n_j + \lambda}$: the mean residual shrunk by λ . For log loss: $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$.
- A split is made only if $\text{Gain} > 0$, so γ acts as a minimum gain to split (built in pruning). `min_child_weight` is a minimum H_j in a leaf (for MSE, a minimum number of rows).
- Engineering: weighted quantile sketch for approximate split finding, sparsity aware split finding (**learns a default direction for missing values** at each split), column block structure for parallel split search, cache awareness, out of core computation.
- Other knobs: `max_depth` (default 6), `eta` (learning rate 0.3), `subsample`, `colsample_bytree`, `scale_pos_weight` (imbalance), `early_stopping_rounds`.

6.5 LightGBM and CatBoost: what makes them different

- Histogram based splits:** bin each feature into (default) 255 buckets once; split search scans bins, not sorted values. Much faster and lighter; XGBoost also has `tree_method='hist'`.
- Leaf wise (best first) growth:** LightGBM splits the leaf with the largest gain anywhere in the tree, producing deeper, unbalanced trees with lower loss for the same number of leaves; control with `num_leaves` (default 31) and `max_depth`, or it overfits small data. XGBoost defaults to level wise (depth wise) growth.
- GOSS** (Gradient based One Side Sampling): keep all rows with large gradients (poorly fit), randomly sample rows with small gradients and up weight them. Focuses compute on informative rows.
- EFB** (Exclusive Feature Bundling): bundle sparse features that are rarely non zero at the same time (like one hot columns) into one feature, reducing dimensionality.
- Native categorical handling (optimal split on sorted categories).
- CatBoost:** ordered target statistics for categorical features (avoids target leakage) and ordered boosting; symmetric (oblivious) trees that are fast at inference; strong defaults.

	XGBoost	LightGBM	CatBoost
Tree growth	level wise (default)	leaf wise	symmetric
Split finding	exact / hist	histogram + GOSS + EFB	histogram
Categoricals	encode yourself (recent native support)	native	native, ordered TS
Missing values	learned default direction	learned direction	handled

Where this shows up

- Gradient boosted trees are the workhorse for tabular ranking and prediction features (job apply propensity, recruiter response, fraud scores). LambdaMART, a GBDT with a ranking loss, powers many search rankers.
- The reported InfoEdge coding round answer for missing values included “XGBoost handles/imputes missing values”, which refers to the learned default direction.

Formula sheet

AdaBoost: $\alpha = \frac{1}{2} \ln \frac{1-\epsilon}{\epsilon}$; $w_i \leftarrow w_i e^{-\alpha y_i h(x_i)}$, normalise. Loss e^{-yF} .
 GBM: $r_i = -\partial L / \partial F$; MSE $r = y - F$; log loss $r = y - p$; $F_m = F_{m-1} + \nu h_m$. F_0 : mean or log odds.
 XGBoost: $w^* = -G / (H + \lambda)$; Gain formula above minus γ ; MSE $g = F - y$, $h = 1$; log loss $g = p - y$, $h = p(1 - p)$.
 LightGBM: histogram, leaf wise (`num_leaves`), GOSS, EFB.

6.6 Question bank: Boosting

PYQ status for this chapter

InfoEdge reports mention boosting as an ensemble type in the OA, “Random Forest vs Gradient Boosting” in interviews (Glassdoor), and XGBoost for missing values in the coding round. The first four questions reconstruct those; the rest follow the reported OA style.

Q6.1 MSQ · M

★ **PYQ (OA: types of ensemble, boosting)** Which statements about boosting are true?

- (A) Learners are trained sequentially.
- (B) Each learner focuses on examples the current ensemble handles poorly.
- (C) Boosting typically uses deep, fully grown trees.
- (D) Boosting mainly reduces bias.

Solution

- Boosting uses weak, shallow learners (stumps to depth 8). Deep trees would overfit quickly because each round adds capacity.

Answer: A, B, D

Q6.2 MSQ · M

★ **PYQ (INT: difference between Random Forest and Gradient Boosting)** Which are differences between RF and GBM?

- (A) RF averages independent trees; GBM adds trees that fit residual errors.
- (B) GBM is more sensitive to noisy labels and outliers.
- (C) GBM has a learning rate; RF does not.
- (D) RF requires a differentiable loss.

Solution

- GBM needs a differentiable loss for its gradients; RF uses impurity splits and no global loss.

Answer: A, B, C

Q6.3 MCQ · M

★ **PYQ (INT: XGBoost and missing values (coding round))** How does XGBoost handle a missing feature value during training?

- (A) It drops the row.
- (B) It imputes the mean automatically.
- (C) At each split it learns which child (left or right) missing values should go to, choosing the direction with higher gain.
- (D) It raises an error.

Solution

- Sparsity aware split finding: evaluate gain with missing rows sent left and with them sent right; keep the better default direction.
- Advantage: missingness itself can be informative (e.g. “no salary disclosed”).

Answer: C

Q6.4 MSQ · M

★ **PYQ (INT: why GBM can overfit with more trees while RF does not)** Which are true?

- (A) Each GBM tree reduces training loss further, eventually fitting noise.
- (B) Early stopping on validation loss chooses the number of trees.
- (C) Lowering the learning rate requires more trees for the same training loss.
- (D) RF test error keeps increasing with more trees.

Solution

- RF test error plateaus (D false).

Answer: A, B, C

Q6.5 NAT · M

AdaBoost round has weighted error $\varepsilon = 0.2$. Compute α .

Solution

- $\alpha = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 = \frac{1}{2} (1.386) = 0.693$.

Answer: $\alpha \approx 0.693$

Q6.6 NAT · M

Continue: 5 points with equal weights 0.2, the stump misclassifies exactly one ($\varepsilon = 0.2$, $\alpha = 0.693$). Give normalised new weights.

Solution

- Misclassified: $0.2 \times e^{0.693} = 0.2 \times 2 = 0.4$. Correct: $0.2 \times e^{-0.693} = 0.1$ each, four of them total 0.4.
- Sum = 0.8. Normalised: $0.4/0.8 = 0.5$ and $0.1/0.8 = 0.125$.
- General fact: after reweighting, misclassified points hold half the total weight, so the current stump has weighted error exactly 0.5 on the new weights (it becomes useless, forcing the next stump to differ).

Answer: Misclassified: 0.5; each correct: 0.125

Q6.7 MCQ · E

In AdaBoost, a weak learner with weighted error exactly 0.5 gets

- (A) $\alpha = 1$
- (B) $\alpha = 0$
- (C) $\alpha = \infty$
- (D) $\alpha = -1$

Solution

- $\ln(0.5/0.5) = \ln 1 = 0$. A coin flip gets no say.

Answer: B

Q6.8 MCQ · M

AdaBoost is equivalent to forward stagewise additive modelling with which loss?

- (A) Squared loss
- (B) Hinge loss
- (C) Exponential loss
- (D) Absolute loss

Solution

- Friedman, Hastie, Tibshirani (2000). Its population minimiser is half the log odds.

Answer: C

Q6.9 NAT · E

Gradient boosting with MSE, targets (3, 5, 10). What is F_0 and the first round pseudo residuals?

Solution

- $F_0 = \text{mean} = 18/3 = 6$. Residuals $y - F_0$.

Answer: $F_0 = 6$; residuals $(-3, -1, 4)$

Q6.10 NAT · M

Continue with learning rate 0.1 and a tree that predicts the residuals exactly. Compute F_1 and the new MSE.

Solution

- $F_1 = 6 + 0.1(-3, -1, 4) = (5.7, 5.9, 6.4)$.
- New residuals $(-2.7, -0.9, 3.6)$ (each is 90% of before). $MSE = (7.29 + 0.81 + 12.96)/3 = 21.06/3 = 7.02$.
- Check: initial $MSE = (9 + 1 + 16)/3 = 8.667$, and $8.667 \times 0.9^2 = 7.02$.

Answer: $F_1 = (5.7, 5.9, 6.4)$; $MSE \approx 7.02$

Q6.11 MCQ · M

For binary log loss, the gradient boosting pseudo residual for an example with label 1 and current predicted probability 0.3 is

- (A) 0.3
- (B) 0.7
- (C) -0.7
- (D) $\ln 0.3$

Solution

- $r = y - p = 1 - 0.3 = 0.7$. The next tree pushes this example's log odds up.

Answer: B

Q6.12 MCQ · E

F_0 for gradient boosting with log loss, when 20% of labels are positive, is

- (A) 0.2
- (B) $\ln(0.2/0.8) \approx -1.386$
- (C) 0
- (D) 0.5

Solution

- Boosting for classification works in log odds space; the best constant is the log odds of the base rate.

Answer: B

Q6.13 NAT · M

XGBoost, MSE loss, $\lambda = 1$. A leaf contains 3 examples with residuals $y - F = (2, 3, 4)$. Optimal leaf weight?

Solution

- $g_i = F - y = (-2, -3, -4)$, $G = -9$; $h_i = 1$, $H = 3$.
- $w^* = -G/(H + \lambda) = 9/4 = 2.25$. Without λ it would be the mean residual 3. L2 shrinks it.

Answer: $w^* = 2.25$

Q6.14 NAT · H

XGBoost split, $\lambda = 1$, $\gamma = 0$. Left leaf: $G_L = -6$, $H_L = 2$. Right: $G_R = 4$, $H_R = 3$. Compute gain.

Solution

- $G_L^2/(H_L + \lambda) = 36/3 = 12$.
- $G_R^2/(H_R + \lambda) = 16/4 = 4$.
- Parent: $G = -2$, $H = 5$: $4/6 = 0.667$.
- Gain = $\frac{1}{2}(12 + 4 - 0.667) = \frac{1}{2}(15.333) = 7.667$.

Answer: ≈ 7.67

Q6.15 MCQ · M

In XGBoost, increasing γ (min_split_loss)

- (A) makes trees grow deeper
- (B) makes the algorithm more conservative by requiring larger gain to split
- (C) increases learning rate
- (D) removes regularisation

Solution

- Gain is reduced by γ per split, so weak splits are pruned.

Answer: B**Q6.16** MSQ · M

Which XGBoost parameters help against overfitting?

- (A) Lower eta with more rounds and early stopping
- (B) subsample and colsample_bytree below 1
- (C) Higher lambda and min_child_weight
- (D) Higher max_depth

Solution

- Deeper trees increase capacity (D).

Answer: A, B, C**Q6.17** MSQ · M

Why does XGBoost use second order (Hessian) information?

- (A) It gives a Newton like step for leaf values.
- (B) It lets the same algorithm work for any twice differentiable loss.
- (C) It removes the need for a learning rate.
- (D) It weights examples by curvature in split gain.

Solution

- A learning rate (eta) is still used for shrinkage.

Answer: A, B, D**Q6.18** NAT · MLog loss, one example with $p = 0.8, y = 0$. XGBoost gradient g and Hessian h ?**Solution**

- $g = p - y = 0.8; h = p(1 - p) = 0.8 \times 0.2 = 0.16$.

Answer: $g = 0.8, h = 0.16$ **Q6.19** MCQ · M

LightGBM's leaf wise growth means

- (A) all leaves at the same depth are split together
- (B) the leaf with maximum gain is split next, anywhere in the tree
- (C) trees have exactly one leaf
- (D) it grows trees from leaves to root

Solution

- Lower loss per leaf, but risk of deep, overfit branches on small data; limit num_leaves, max_depth, min_data_in_leaf.

Answer: B**Q6.20** MSQ · M

Which are LightGBM speed techniques?

- (A) Histogram based split finding
- (B) GOSS
- (C) Exclusive Feature Bundling
- (D) Exact greedy search over all sorted values

Solution

- Exact greedy is XGBoost's original, slower method.

Answer: A, B, C

Q6.21 MCQ · H

GOSS keeps the top a fraction of rows by gradient magnitude and samples a b fraction of the rest. Sampled small gradient rows are up weighted by

- (A) 1
- (B) $\frac{1-a}{b}$
- (C) $\frac{a}{b}$
- (D) b

Solution

- The rest contains $(1 - a)$ of the data, represented by b of the data, so each sampled row stands for $(1 - a)/b$ rows. Keeps the gain estimate approximately unbiased.

Answer: B

Q6.22 MCQ · M

CatBoost's ordered target statistics solve which problem?

- (A) Vanishing gradients
- (B) Target leakage when encoding categorical features with the mean target
- (C) Slow histogram building
- (D) Missing values

Solution

- Naive target encoding uses a row's own label to encode its category, leaking the answer. CatBoost encodes each row using only rows before it in a random permutation.

Answer: B

Q6.23 MSQ · M

Learning rate ν in gradient boosting. Which are true?

- (A) Smaller ν usually improves generalisation given enough trees.
- (B) Smaller ν needs more trees.
- (C) $\nu = 1$ is always best.
- (D) ν and number of trees should be tuned together.

Solution

- Classic finding (Friedman): shrinkage below 0.1 improves test error dramatically.

Answer: A, B, D

Q6.24 MCQ · E

Which algorithm is NOT a boosting method?

- (A) AdaBoost
- (B) XGBoost
- (C) Random Forest
- (D) LightGBM

Solution

- Random Forest is bagging with random feature subsets; the other three are boosting.

Answer: C

Q6.25 MSQ · H

Boosting and label noise. Which are true?

- (A) AdaBoost's exponential loss makes it very sensitive to mislabeled points.
- (B) Using Huber or log loss makes gradient boosting more robust than exponential loss.
- (C) Boosting ignores hard examples.
- (D) Early stopping limits how much the model chases noisy points.

Solution

- Boosting focuses *more* on hard examples, including mislabeled ones.

Answer: A, B, D**Q6.26** NAT · M

AdaBoost final classifier: three stumps with $\alpha = (0.4, 0.9, 0.3)$ predict $(+1, -1, +1)$ for a point. Final prediction?

Solution

- Score = $0.4 - 0.9 + 0.3 = -0.2 < 0$, so -1 . The most accurate stump outweighs two weaker ones.

Answer: -1 **Q6.27** MSQ · M

Which are true about `scale_pos_weight` in XGBoost?

- (A) It is used for class imbalance.
- (B) A common value is (number of negatives)/(number of positives).
- (C) It changes gradients of positive examples.
- (D) It affects regression only.

Solution

- It multiplies positive examples' gradient and Hessian, like a class weight.

Answer: A, B, C**Q6.28** MCQ · M

In gradient boosting with MSE, what exactly does each new tree try to predict?

- (A) The original target y
- (B) The residuals $y - F_{m-1}(x)$
- (C) The sign of y
- (D) Random noise

Solution

- Negative gradient of $\frac{1}{2}(y - F)^2$ with respect to F is $y - F$.

Answer: B**Q6.29** MSQ · H

Monotonic constraints in XGBoost / LightGBM. Which are true?

- (A) You can force predictions to be non decreasing in a feature (e.g. experience vs salary).
- (B) They can improve trust and generalisation when domain knowledge is reliable.
- (C) They are enforced by rejecting splits whose leaf values violate the order.
- (D) They make training impossible.

Solution

- Useful for pricing and credit models where regulators expect monotone behaviour.

Answer: A, B, C**Q6.30** MCQ · M

Typical `max_depth` for trees in gradient boosting is

- (A) 1 to 8
- (B) 50 to 100
- (C) unlimited

(D) exactly the number of features

Solution

- XGBoost default 6; depth k captures interactions of up to k features.

Answer: A

Q6.31 MSQ · M

Why is gradient boosting called “gradient” boosting?

- (A) Each tree approximates the negative gradient of the loss with respect to current predictions.
- (B) It trains neural networks.
- (C) It performs gradient descent in the space of functions.
- (D) It uses gradient clipping.

Solution

- Each round is one step of functional gradient descent: $F_m = F_{m-1} - \nu \cdot (\text{tree approximating } \partial L / \partial F)$.

Answer: A, C

Q6.32 NAT · H

XGBoost, MSE, $\lambda = 0$, $\gamma = 0$. Node residuals $(1, 1, -1, -1)$. Split puts the two 1s left, the two -1 s right. Gain?

Solution

- $g = -r$: left $G_L = -2$, $H_L = 2$; right $G_R = 2$, $H_R = 2$; parent $G = 0$, $H = 4$.
- Gain = $\frac{1}{2}(4/2 + 4/2 - 0) = \frac{1}{2}(4) = 2$.
- Check with SSE: parent SSE around mean 0 is 4; children SSE 0 each. Reduction 4, and XGBoost gain for MSE equals half the SSE reduction ($\lambda = 0$): 2. Consistent.

Answer: 2

Q6.33 MCQ · M

Which model would you try first for a 1M row tabular dataset with mixed numeric and categorical features and a strict accuracy target?

- (A) Deep MLP
- (B) Gradient boosted trees (LightGBM / XGBoost / CatBoost)
- (C) KNN
- (D) Naive Bayes

Solution

- GBDTs remain state of the art on most tabular benchmarks; neural nets need more tuning to match them.

Answer: B

Q6.34 MSQ · M

Early stopping in XGBoost requires

- (A) a validation set passed as eval_set
- (B) an evaluation metric
- (C) early_stopping_rounds
- (D) the test set

Solution

- Never early stop on the test set.

Answer: A, B, C

Q6.35 SUB · H

Interviewer: “Explain gradient boosting on a whiteboard in 5 lines.”

Solution

- $F_0 = \bar{y}$. For $m = 1..M$: $r_i = y_i - F_{m-1}(x_i)$; fit tree h_m to r ; $F_m = F_{m-1} + \nu h_m$. Predict $F_M(x)$. For classification, same in log odds space with $r = y - p$.

Answer: Initialise with a constant; repeatedly fit a small tree to the negative gradient (residuals) and add a shrunk copy; stop by validation.

Q6.36 MSQ · M

Which statements about boosting and parallelism are true?

- (A) Trees are built one after another.
- (B) XGBoost parallelises split finding within a tree across features.
- (C) LightGBM can use multiple threads.
- (D) Boosting cannot use GPUs.

Solution

- All three major libraries have GPU implementations.

Answer: A, B, C

Interview angle

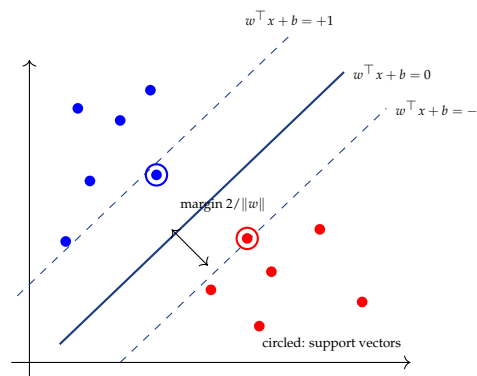
- Walk through AdaBoost weight updates on a toy example. Why does misclassified weight become exactly 0.5?
- Derive the XGBoost leaf weight $-G/(H + \lambda)$ from the second order expansion.
- XGBoost vs LightGBM vs CatBoost: growth strategy, categoricals, speed tricks.
- How do you tune a GBM? (lr and trees with early stopping, then depth / leaves, then sampling, then regularisation.)
- Why do boosted trees beat neural nets on tabular data?

7 Support Vector Machines and the Kernel Trick

Companion resource for this section: [StatQuest: Support Vector Machines Part 1, Main Ideas](#)

7.1 Which line is best?

- Two classes of points that a straight line can separate. Infinitely many lines work. Which one will generalise best? Intuition: the one that stays *as far as possible* from both classes, so small noise in new points does not flip them. That distance is the **margin**.
- The **maximum margin classifier** picks the hyperplane $w^\top x + b = 0$ with the widest empty “street” between the classes.
- Only the points on the edges of the street matter. They are the **support vectors**. Delete any other point and the solution does not change.



7.2 The hard margin optimisation

- Labels $y_i \in \{-1, +1\}$. Scale w, b so that the closest points satisfy $y_i(w^\top x_i + b) = 1$. Then the distance from the hyperplane to those points is $1/\|w\|$ and the street width (margin) is $2/\|w\|$.
- Maximising $2/\|w\|$ is the same as minimising $\frac{1}{2}\|w\|^2$:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 \quad \forall i.$$

- This is a convex quadratic program: a unique global solution.
- Distance of any point to the hyperplane: $\frac{|w^\top x + b|}{\|w\|}$. Example: $w = (3, 4)$, $b = -5$, point $(1, 1)$: $|3 + 4 - 5|/5 = 2/5 = 0.4$.

7.3 Soft margin: allowing mistakes, and the C parameter

- Real data overlap or have outliers; a hard margin may not exist or may be tiny. Introduce slack $\xi_i \geq 0$: $y_i(w^\top x_i + b) \geq 1 - \xi_i$, and pay $C \sum \xi_i$.

$$\min \frac{1}{2}\|w\|^2 + C \sum_i \xi_i.$$

- $\xi_i = 0$: correct and outside the street. $0 < \xi_i \leq 1$: inside the street but correct side. $\xi_i > 1$: misclassified.
- Large C**: mistakes are expensive, so narrow margin, fits training data hard: low bias, high variance. **Small C**: wide margin, tolerates violations: more bias, less variance. $C \rightarrow \infty$ recovers hard margin.
- Equivalent unconstrained form: $\min \frac{\lambda}{2}\|w\|^2 + \frac{1}{n} \sum_i \max(0, 1 - y_i f(x_i))$. The second term is the **hinge loss**. So a linear SVM is “hinge loss + L2 regularisation”, just as logistic regression is “log loss + L2”.
- Hinge loss values: $yf = 2$ gives 0 (confidently right, ignored); $yf = 0.5$ gives 0.5 (right but inside margin); $yf = -1$ gives 2 (wrong). Unlike log loss, hinge is exactly zero beyond the margin, which is why only support vectors matter.

7.4 The dual problem and why it matters

- Using Lagrange multipliers $\alpha_i \geq 0$, the solution has the form $w = \sum_i \alpha_i y_i x_i$, and the dual problem is

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j \quad \text{s.t. } 0 \leq \alpha_i \leq C, \sum_i \alpha_i y_i = 0.$$

- $\alpha_i > 0$ only for support vectors (KKT complementary slackness). Points strictly outside the margin have $\alpha_i = 0$. Points with $\alpha_i = C$ are on the wrong side of their margin.
- Crucial observation: the data appear *only through dot products* $x_i^\top x_j$. Predictions too: $f(x) = \sum_i \alpha_i y_i x_i^\top x + b$.

7.5 The kernel trick

- Some data are not linearly separable in the original space: points inside a circle vs outside. Map $x = (x_1, x_2)$ to $\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$: now the circle $x_1^2 + x_2^2 = r^2$ becomes a plane. Linear separation in the new space = curved boundary in the old one.
- Computing ϕ explicitly can be expensive or impossible (infinite dimensional). But the dual only needs dot products, and for many maps there is a shortcut **kernel** $K(x, z) = \phi(x)^\top \phi(z)$ computable in the original space.
- Check for the map above: $\phi(x)^\top \phi(z) = x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 = (x^\top z)^2$. We never build ϕ ; we just square the dot product.
- Common kernels:
 - Linear: $K = x^\top z$.
 - Polynomial: $K = (\gamma x^\top z + r)^d$.
 - RBF / Gaussian: $K = \exp(-\gamma \|x - z\|^2)$, similarity that decays with distance; corresponds to an infinite dimensional feature space.
 - Sigmoid: $\tanh(\gamma x^\top z + r)$ (not always a valid kernel).
- γ in RBF: large γ means each support vector's influence is very local, giving wiggly boundaries (overfitting); small γ gives smooth, nearly linear boundaries (underfitting). Tune C and γ together on a log grid.
- A valid kernel must correspond to some dot product: its Gram matrix must be symmetric positive semi-definite for any data (Mercer's condition).
- Kernels are not SVM only: kernel ridge regression, kernel PCA, Gaussian processes use the same trick.

7.6 Practicalities

- Scale features:** the margin is a distance, so unscaled features dominate. Standardise first.
- Training cost of kernel SVM is between $O(n^2)$ and $O(n^3)$ and the Gram matrix needs $O(n^2)$ memory, so kernel SVMs struggle beyond about 10^5 rows. Linear SVMs (liblinear, SGD with hinge loss) scale to millions.
- Multiclass: SVMs are binary; scikit-learn's SVC uses one vs one ($K(K-1)/2$ models), LinearSVC uses one vs rest.
- Probabilities: SVM outputs signed distances, not probabilities; Platt scaling (a logistic fit on the scores via internal CV) provides them (`probability=True`, slower).
- SVR (support vector regression): ϵ insensitive loss ignores errors smaller than ϵ ; points outside the ϵ tube become support vectors.
- Imbalance: `class_weight` scales C per class.

Where this shows up

- Linear SVMs on TF-IDF features were the strongest text classifiers before deep learning and remain excellent fast baselines for resume or job category classification.
- The kernel idea (compare examples by similarity instead of explicit features) resurfaces in attention: softmax of dot products is a similarity kernel.

Formula sheet

Margin = $2/\|w\|$; distance of x to plane $|w^\top x + b|/\|w\|$. Primal soft margin: $\frac{1}{2}\|w\|^2 + C \sum \xi_i$.
 Hinge $\max(0, 1 - yf)$. Dual: $w = \sum \alpha_i y_i x_i$, $0 \leq \alpha_i \leq C$, $\sum \alpha_i y_i = 0$.

Kernels: $(x^\top z + r)^d$, $e^{-\gamma\|x-z\|^2}$. Large C or large γ : overfit. Polynomial degree d in p dims: $\binom{p+d}{d}$ features.

7.7 Question bank: Support vector machines

Q7.1 MSQ · M

★ **PYQ (OA: SVMs (IIT Roorkee ML section))** Which statements about SVMs are true?

- (A) The SVM decision boundary depends only on the support vectors.
- (B) Increasing C generally narrows the margin.
- (C) SVMs output calibrated probabilities by default.
- (D) The RBF kernel can model non linear boundaries.

Solution

- (A) Non support vectors have $\alpha_i = 0$ and drop out of $w = \sum \alpha_i y_i x_i$.
- (B) Large C penalises violations heavily, so the optimiser sacrifices margin to classify training points.
- (C) False: raw outputs are distances; probabilities need Platt scaling.

Answer: A, B, D

Q7.2 SUB · M

★ **PYQ (INT: what is SVM; hinge loss, kernels, margin, support vectors)** Explain SVM in the order the reported interview expected.

Solution

- Margin: the street width $2/\|w\|$; maximise it to generalise better.
- Support vectors: points on or inside the margin (or misclassified); only they determine w .
- Soft margin: slack variables, hinge loss $\max(0, 1 - yf)$ plus $\frac{1}{2}\|w\|^2$, C trades margin vs violations.
- Kernels: dual depends only on dot products; replace with $K(x, z) = \phi(x)^\top \phi(z)$ to get non linear boundaries without computing ϕ .

Answer: Maximum margin hyperplane; support vectors define it; soft margin with hinge loss and C ; kernels map to higher dimensions implicitly.

Q7.3 MSQ · M

★ **PYQ (OA: hinge loss is a classification loss)** Hinge loss $\max(0, 1 - yf(x))$ with $y \in \{-1, +1\}$. Which are true?

- (A) It is zero for points correctly classified with $yf(x) \geq 1$.
- (B) It is used by SVMs.
- (C) It is differentiable everywhere.
- (D) It penalises correct predictions inside the margin.

Solution

- Non differentiable at $yf = 1$ (a kink); subgradients are used.
- $0 < yf < 1$: correct side but inside margin, loss $1 - yf > 0$.

Answer: A, B, D

Q7.4 MCQ · M

★ **PYQ (INT: kernels project points to higher dimensions)** The kernel trick lets SVMs

- (A) explicitly compute a high dimensional feature map for every point
- (B) compute dot products in a high dimensional feature space without computing the map
- (C) avoid solving any optimisation problem
- (D) work only on linearly separable data

Solution

- The key sentence to say in the interview: “the dual depends on data only via inner products, and a kernel computes that inner product implicitly”.

Answer: B**Q7.5** NAT · E

A hard margin SVM has $w = (3, 4)$. What is the margin width?

Solution

- $\|w\| = \sqrt{9 + 16} = 5$; margin = $2/5 = 0.4$.

Answer: 0.4**Q7.6** NAT · E

Hyperplane $w = (1, 2)$, $b = -3$. Distance of point $(3, 2)$ to the hyperplane, and predicted class?

Solution

- $w^\top x + b = 3 + 4 - 3 = 4$. $\|w\| = \sqrt{5} = 2.236$.
- Distance = $4/2.236 = 1.789$. Sign positive, so +1.

Answer: ≈ 1.789 , class +1**Q7.7** NAT · M

Compute hinge loss for $f(x) = 0.3$ with $y = +1$; $f(x) = -2$ with $y = +1$; $f(x) = -1.5$ with $y = -1$.

Solution

- $\max(0, 1 - 0.3) = 0.7$ (correct but inside margin).
- $\max(0, 1 + 2) = 3$ (wrong).
- $yf = 1.5 \geq 1$: loss 0.

Answer: 0.7, 3, 0**Q7.8** NAT · M

Compute the polynomial kernel $K(x, z) = (x^\top z + 1)^2$ for $x = (1, 2)$, $z = (3, -1)$.

Solution

- $x^\top z = 3 - 2 = 1$. $K = (1 + 1)^2 = 4$.

Answer: 4**Q7.9** NAT · M

RBF kernel $K = e^{-\gamma \|x - z\|^2}$ with $\gamma = 0.5$, $x = (1, 1)$, $z = (2, 3)$.

Solution

- $\|x - z\|^2 = 1 + 4 = 5$. $K = e^{-2.5} = 0.0821$.
- Same point gives 1; far points give nearly 0.

Answer: $e^{-2.5} \approx 0.082$ **Q7.10** MCQ · M

An RBF SVM has 100% training accuracy and poor test accuracy. Which change most likely helps?

- (A) Increase γ
- (B) Decrease γ and/or decrease C
- (C) Increase C
- (D) Remove feature scaling

Solution

- Both large γ (very local kernels) and large C (penalise every mistake) overfit.

Answer: B**Q7.11** MSQ · M

In the dual solution, which are true?

- (A) $\alpha_i = 0$ for points strictly outside the margin.
- (B) $0 < \alpha_i < C$ for points exactly on the margin.
- (C) $\alpha_i = C$ for points violating the margin.
- (D) α_i can be negative.

Solution

- $\alpha_i \geq 0$ always (Lagrange multipliers of inequality constraints).

Answer: A, B, C**Q7.12** MCQ · E

Why must features be scaled before training an SVM?

- (A) SVMs need Gaussian features
- (B) The margin is a Euclidean distance, so large scale features dominate
- (C) Scaling adds kernels
- (D) It is not needed

Solution

- Same reason as KNN and PCA.

Answer: B**Q7.13** MSQ · M

Linear SVM vs logistic regression. Which are true?

- (A) Both give linear boundaries.
- (B) SVM uses hinge loss; logistic uses log loss.
- (C) Logistic regression gives probabilities naturally.
- (D) In logistic regression only support vectors affect the solution.

Solution

- Log loss is never exactly zero, so every point influences logistic regression (D false).

Answer: A, B, C**Q7.14** NAT · MNumber of features created by an explicit degree 2 polynomial map (with all cross terms, squares, linear terms and a constant) for $p = 3$ input features?**Solution**

- Count monomials of degree ≤ 2 in 3 variables: $\binom{3+2}{2} = 10$ ($1, x_1, x_2, x_3, x_1^2, x_2^2, x_3^2, x_1x_2, x_1x_3, x_2x_3$).
- For $p = 1000$, degree 3: $\binom{1003}{3} \approx 1.7 \times 10^8$ features. The kernel computes the same inner product in $O(p)$.

Answer: 10**Q7.15** MCQ · M

The RBF kernel corresponds to a feature space of dimension

- (A) p
- (B) p^2
- (C) infinite
- (D) 2

Solution

- Taylor expand $e^{2\gamma x^\top z}$: all polynomial degrees appear.

Answer: C**Q7.16** MSQ · M

Which conditions make a function a valid (Mercer) kernel?

- (A) Symmetric: $K(x, z) = K(z, x)$
- (B) Gram matrix positive semi definite for any finite dataset
- (C) Output always in $[0, 1]$
- (D) It equals $\phi(x)^\top \phi(z)$ for some map ϕ

Solution

- The linear kernel can be any real value, so (C) is not required.
- Sums and products of valid kernels, and positive scalar multiples, are valid kernels.

Answer: A, B, D**Q7.17** MCQ · H

In the soft margin SVM, the optimisation problem is

- (A) non convex
- (B) convex quadratic
- (C) linear
- (D) integer

Solution

- Quadratic objective, linear constraints: convex QP with a global optimum. Solved by SMO (sequential minimal optimisation) in libsvm.

Answer: B**Q7.18** MSQ · M

Which SVM implementations/choices scale best to 10 million rows of sparse text?

- (A) LinearSVC (liblinear)
- (B) SGDClassifier with hinge loss
- (C) SVC with RBF kernel
- (D) Kernel SVM with full Gram matrix

Solution

- Gram matrix for 10^7 rows has 10^{14} entries. Impossible. Use linear methods (text is often nearly linearly separable in high dimensions anyway).

Answer: A, B**Q7.19** MCQ · M

Removing a non support vector from the training set and retraining

- (A) changes the boundary a lot
- (B) does not change the boundary
- (C) always increases the margin
- (D) makes the problem infeasible

Solution

- Its $\alpha = 0$; the KKT conditions remain satisfied by the same solution.

Answer: B**Q7.20** NAT · M

One vs one SVC on 10 classes trains how many binary SVMs?

Solution

- $\binom{10}{2} = 45$. Each uses only two classes' data, which keeps kernel training cheap.

Answer: 45**Q7.21** MSQ · M

Support Vector Regression. Which are true?

- (A) Uses an ϵ insensitive loss.
- (B) Errors smaller than ϵ cost nothing.
- (C) Points inside the ϵ tube are support vectors.
- (D) Kernels can be used.

Solution

- Points *outside* (or on the boundary of) the tube are the support vectors.

Answer: A, B, D**Q7.22** MCQ · E $C \rightarrow \infty$ in the soft margin SVM corresponds to

- (A) no regularisation of w
- (B) a hard margin SVM
- (C) a perceptron with zero margin
- (D) logistic regression

Solution

- Any slack becomes infinitely costly, so the constraints must hold exactly (if the data are separable).

Answer: B**Q7.23** NAT · HTwo points: $x_1 = (1, 1)$ with $y = +1$ and $x_2 = (-1, -1)$ with $y = -1$. Find the hard margin SVM w, b and margin.**Solution**

- By symmetry the boundary is the perpendicular bisector, through the origin, normal along $(1, 1)$: $w = a(1, 1), b = 0$.
- Support constraints: $y_1(w^\top x_1) = 2a = 1 \Rightarrow a = 0.5$.
- $\|w\| = 0.5\sqrt{2} = 0.707$; margin = $2/0.707 = 2.83$, which equals the distance between the two points, $\sqrt{4 + 4} = 2.83$. Consistent.

Answer: $w = (0.5, 0.5), b = 0$, margin = $2\sqrt{2} \approx 2.83$ **Q7.24** MSQ · MWhich statements about γ in the RBF kernel are true?

- (A) $\gamma = \frac{1}{2\sigma^2}$ where σ is the kernel width.
- (B) Large γ makes the decision boundary more flexible.
- (C) Small γ makes the model behave closer to a linear model.
- (D) γ has no effect if C is large.

Solution

- γ and C interact but both matter; grid search both on a log scale.

Answer: A, B, C**Q7.25** MCQ · M

Platt scaling for SVM probabilities fits

- (A) a decision tree on the SVM scores
- (B) a sigmoid $1/(1 + e^{Af+B})$ on the SVM scores using held out data

- (C) a softmax over support vectors
 (D) nothing, SVM already outputs probabilities

Solution

- Internal cross validation is used, which is why `probability=True` makes SVC slower.

Answer: B**Q7.26** MSQ · H

KKT conditions for the soft margin SVM include:

- (A) $\alpha_i [y_i f(x_i) - 1 + \xi_i] = 0$
 (B) $(C - \alpha_i) \xi_i = 0$
 (C) $\sum_i \alpha_i y_i = 0$
 (D) $\alpha_i > C$ for outliers

Solution

- (B) means: if $\xi_i > 0$ (violation) then $\alpha_i = C$.
- Box constraint $0 \leq \alpha_i \leq C$, so (D) false.

Answer: A, B, C**Q7.27** MCQ · M

Which dataset is a natural fit for a kernel (RBF) SVM?

- (A) 50 million click logs
 (B) A few thousand samples with complex non linear boundaries and moderate dimensions
 (C) Image classification with ImageNet scale data
 (D) Streaming data

Solution

- Kernel SVMs shine on small to medium data with non linear structure; they do not scale to tens of millions of rows or raw images.

Answer: B**Q7.28** MSQ · M

What happens to an SVM if one class has 99% of samples?

- (A) The boundary may shift toward the minority class.
 (B) Setting `class_weight='balanced'` increases the penalty C for minority errors.
 (C) SVMs are immune to imbalance.
 (D) Evaluating with accuracy may be misleading.

Solution

- The hinge penalty is summed over many majority points, so the optimiser tolerates minority errors; class weights rescale C per class to counter this.

Answer: A, B, D**Q7.29** NAT · M

Primal soft margin objective with $w = (1, 1)$, $C = 2$, slacks $\xi = (0, 0.5, 1.5)$. Compute its value.

Solution

- $\frac{1}{2} \|w\|^2 = \frac{1}{2} (2) = 1$. $C \sum \xi = 2(2) = 4$. Total = 5.

Answer: 5**Q7.30** MCQ · M

A sum of an RBF kernel and a linear kernel, $K = K_{RBF} + K_{lin}$, is

- (A) not a valid kernel
 (B) a valid kernel

- (C) valid only if $\gamma = 1$
 (D) equal to a polynomial kernel

Solution

- Sum of PSD matrices is PSD; it corresponds to concatenating the two feature maps.

Answer: B**Q7.31** MSQ · M

Perceptron vs SVM. Which are true?

- (A) The perceptron finds some separating hyperplane if one exists.
 (B) The SVM finds the maximum margin separating hyperplane.
 (C) The perceptron's solution depends on the order of examples.
 (D) Both always give the same hyperplane.

Solution

- The perceptron stops at the first separating hyperplane it reaches, which depends on example order; the SVM solution is unique.

Answer: A, B, C**Q7.32** MCQ · H

The number of support vectors is often used as a rough indicator of

- (A) training speed only
 (B) model complexity / leave one out error bound
 (C) the kernel type
 (D) number of features

Solution

- Leave one out error $\leq$ (number of support vectors)/ n . Many support vectors suggest a complex boundary or noisy data.

Answer: B**Q7.33** SUB · M

Why is the SVM margin $2/\|w\|$?

Solution

- $|1 - (-1)|/\|w\| = 2/\|w\|$.

Answer: The margin boundaries are $w^\top x + b = \pm 1$; the distance between two parallel planes $w^\top x + b = c_1$ and c_2 is $|c_1 - c_2|/\|w\|$.

Q7.34 MSQ · M

Which models can use the kernel trick?

- (A) SVM
 (B) Kernel ridge regression
 (C) Kernel PCA
 (D) Decision trees

Solution

- Any algorithm expressible purely via inner products. Trees split on single features; no dot products.

Answer: A, B, C**Q7.35** MCQ · M

Kernel SVM prediction time for one test point grows with

- (A) number of support vectors
 (B) number of classes only

- (C) depth of the tree
- (D) nothing

Solution

- $f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$: one kernel evaluation per support vector.

Answer: A

Interview angle

- Derive the margin, write the primal, explain slack and C.
- Why only support vectors matter (KKT). What are α_i values for different point types?
- Derive the dual's dependence on dot products and show $(x^\top z)^2$ equals an explicit map.
- RBF γ and C: effect on bias and variance; how to tune.
- SVM vs logistic regression: losses, probabilities, sparsity, scale. When would you pick each for text?

8 Evaluation Metrics and the Confusion Matrix

Companion resource for this section: [StatQuest: The Confusion Matrix](#)

8.1 The confusion matrix

- For a binary classifier, every prediction lands in one of four cells. Read the name as (was the prediction right?) + (what did we predict?).

		Predicted	
		Positive	Negative
Actual	Positive	TP (true positive)	FN (false negative, Type II)
	Negative	FP (false positive, Type I)	TN (true negative)

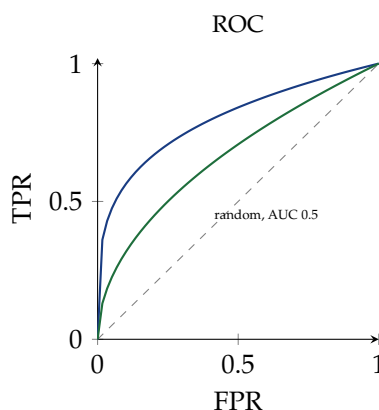
- False negative** (a reported InfoEdge OA question): the model said “negative” but the truth was positive. A fraud transaction passed as legitimate; a cancer missed. **False positive**: the model raised an alarm that was wrong; a genuine email sent to spam.
- Link to hypothesis testing: FP is a Type I error (rejecting a true null “nothing is there”); FN is a Type II error.
- Running example: 100 job applications, 20 genuinely good. The model flags 25; of those, 15 are good. So TP = 15, FP = 10, FN = 5, TN = 70.

8.2 The metrics, each with its question

- Accuracy** = $\frac{TP+TN}{\text{all}}$: how often are we right? Example: $85/100 = 0.85$. Misleading under imbalance: with 1% fraud, “never fraud” scores 99%.
- Precision** = $\frac{TP}{TP+FP}$: when we say positive, how often are we right? Example: $15/25 = 0.60$. Matters when false alarms are costly (spam filter, recommending jobs to a recruiter).
- Recall = Sensitivity = TPR** = $\frac{TP}{TP+FN}$: of the real positives, how many did we catch? Example: $15/20 = 0.75$. Matters when misses are costly (disease screening, fraud).
- Specificity = TNR** = $\frac{TN}{TN+FP}$: of real negatives, how many did we clear? $70/80 = 0.875$. **FPR** = $1 - \text{specificity} = 0.125$.
- F1** = $\frac{2PR}{P+R}$, the harmonic mean: $\frac{2(0.6)(0.75)}{1.35} = 0.667$. Harmonic mean is dragged toward the smaller value, so you cannot game F1 by maximising only one of P or R. Equivalent form: $F1 = \frac{2TP}{2TP+FP+FN} = \frac{30}{45}$.
- $F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R}$: $\beta > 1$ weighs recall more (F_2 for screening), $\beta < 1$ weighs precision.
- Balanced accuracy** = $\frac{TPR+TNR}{2}$; **MCC** (Matthews correlation) = $\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$, a single robust number for imbalanced data, ranging -1 to 1 .
- Precision recall tradeoff: raising the threshold makes the model say “positive” less often: precision usually goes up, recall goes down (recall can only go down or stay).

8.3 Threshold free metrics: ROC AUC and PR AUC

- A probabilistic classifier gives a score; each threshold gives a different confusion matrix. The **ROC curve** plots TPR (y) against FPR (x) as the threshold sweeps from high to low, from $(0,0)$ to $(1,1)$.
- Random guessing gives the diagonal (AUC 0.5). Perfect ranking hugs the top left corner (AUC 1). AUC below 0.5 means the scores are inverted.
- Interpretation: **AUC = probability that a random positive gets a higher score than a random negative**. Example: positives scored 0.9, 0.6, negatives 0.7, 0.2. Pairs: $(0.9 > 0.7)$ yes, $(0.9 > 0.2)$ yes, $(0.6 > 0.7)$ no, $(0.6 > 0.2)$ yes: $\text{AUC} = 3/4$.
- AUC depends only on the *ranking* of scores, not their values: any monotone transform leaves it unchanged, and it says nothing about calibration.
- Under heavy imbalance, ROC AUC can look great because FPR has a huge denominator (many negatives). The **precision recall curve** and its area (**average precision**) focus on the positive class; a random classifier’s PR AUC equals the positive rate, not 0.5.



8.4 Multiclass averaging

- **Macro** average: compute the metric per class and take the plain mean. Every class counts equally, so rare classes matter.
- **Micro** average: pool all TP, FP, FN across classes, then compute. For single label multiclass, micro precision = micro recall = micro F1 = accuracy.
- **Weighted** average: per class metric weighted by class support.

8.5 Imbalanced data (a reported InfoEdge interview topic)

- Choose the right metric first (PR AUC, F1, recall at fixed precision, cost based metric).
- Data level: random undersampling of the majority; random oversampling of the minority; **SMOTE** (create synthetic minority points by interpolating between a minority point and one of its k nearest minority neighbours: $x_{new} = x + u(x_{nm} - x)$, $u \sim U(0,1)$); Borderline SMOTE and ADASYN (generate more near the boundary / hard regions); NearMiss (undersample majority points close to the minority). Apply resampling *only* to the training folds, never before splitting.
- Algorithm level: class weights (the interviewer's suggestion in the report), focal loss, threshold moving, anomaly detection framing.
- After resampling or weighting, predicted probabilities are no longer calibrated to the true base rate; recalibrate if probabilities are used downstream.

8.6 Regression and ranking metrics (brief)

- Regression: MAE, MSE, RMSE, R^2 , MAPE (Chapter 1).
- Ranking (search, recommendations): Precision@k, Recall@k, MAP (mean average precision), MRR (mean reciprocal rank, $\frac{1}{|Q|} \sum 1/\text{rank of first relevant}$), NDCG (discounted gain $\sum \frac{rel_i}{\log_2(i+1)}$, normalised by the ideal ordering).
- Probabilistic quality: log loss, Brier score $\frac{1}{n} \sum (p_i - y_i)^2$, calibration (reliability) curves.

Where this shows up at InfoEdge

- Job recommendation is a ranking problem: offline NDCG, MRR, Recall@k; online click through and apply rates via A/B tests.
- Fraud or fake recruiter detection is extremely imbalanced: PR AUC and recall at a fixed precision.
- The Round 3 coding task ends by printing the confusion matrix, accuracy, precision, recall and F1, then discussing imbalance.

Formula sheet

$$P = \frac{TP}{TP+FP}; R = \text{TPR} = \frac{TP}{TP+FN}; \text{Spec} = \frac{TN}{TN+FP}; \text{FPR} = \frac{FP}{FP+TN}; \text{NPV} = \frac{TN}{TN+FN}.$$

$$F1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}; F\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R}; \text{balanced acc} = \frac{TPR+TNR}{2}.$$

$$\text{AUC} = P(s^+ > s^-); \text{PR AUC baseline} = \text{positive rate. Micro F1} = \text{accuracy (single label multiclass)}.$$

$$\text{MRR} = \text{mean}(1/\text{rank}); \text{DCG} = \sum \text{rel}_i / \log_2(i + 1); \text{Brier} = \frac{1}{n} \sum (p - y)^2.$$

8.7 Question bank: Evaluation metrics

Q8.1 MCQ · E

★ **PYQ (OA: what a false negative means)** In a model that flags fraudulent transactions (positive = fraud), a false negative is

- (A) a legitimate transaction flagged as fraud
- (B) a fraudulent transaction not flagged
- (C) a fraud correctly flagged
- (D) a legitimate transaction correctly passed

Solution

- “False” = the prediction was wrong; “negative” = we predicted negative. So we said “not fraud” when it was fraud. Option A is a false positive.

Answer: B

Q8.2 NAT · E

★ **PYQ (INT/coding round: confusion matrix, accuracy, precision, recall, F1)** TP = 40, FP = 10, FN = 20, TN = 130. Compute accuracy, precision, recall, F1.

Solution

- Total = 200. Accuracy = $(40 + 130) / 200 = 0.85$.
- Precision = $40 / 50 = 0.80$. Recall = $40 / 60 = 0.667$.
- F1 = $2(0.8)(0.667) / (0.8 + 0.667) = 1.0667 / 1.4667 = 0.727$. Check: $2TP / (2TP + FP + FN) = 80 / 110 = 0.727$.

Answer: Acc 0.85, P 0.80, R 0.667, F1 0.727

Q8.3 MSQ · M

★ **PYQ (INT: imbalanced data handling)** A dataset has 1% positives. Which are sound practices?

- (A) Apply SMOTE on the whole dataset before the train/test split.
- (B) Use class weights in the loss.
- (C) Evaluate with PR AUC or F1 rather than accuracy.
- (D) Tune the decision threshold on a validation set.

Solution

- (A) Leakage: synthetic points built from test neighbours leak test information and inflate scores. Resample only inside the training folds (imblearn Pipeline does this).

Answer: B, C, D

Q8.4 MCQ · M

★ **PYQ (OA: what a false negative means, Type I / II link)** A false negative corresponds, in hypothesis testing language, to a

- (A) Type I error
- (B) Type II error
- (C) Type III error
- (D) p-value

Solution

- Null = “no effect / negative”. FN = failing to reject a false null = Type II (β). FP = Type I (α).

Answer: B

Q8.5 NAT · E

From TP = 15, FP = 10, FN = 5, TN = 70, compute specificity and FPR.

Solution

- Specificity = $70 / (70 + 10) = 0.875$; FPR = $10 / 80 = 0.125$.

Answer: 0.875 and 0.125

Q8.6 MCQ · E

For cancer screening, which metric should be prioritised?

- (A) Precision
- (B) Recall (sensitivity)
- (C) Accuracy
- (D) Specificity

Solution

- Missing a cancer (FN) is far worse than a follow up test (FP). Then confirm positives with a high precision second test.

Answer: B

Q8.7 MCQ · E

For a spam filter deciding what to hide from the inbox, which matters most?

- (A) Recall
- (B) Precision
- (C) FNR
- (D) Neither

Solution

- A real email hidden as spam (FP) is costly. High precision: when we say spam, it really is.

Answer: B

Q8.8 NAT · M

A test set has 990 negatives and 10 positives. A model predicts everything as negative. Compute accuracy, recall, precision, balanced accuracy.

Solution

- TP = 0, FN = 10, TN = 990, FP = 0.
- Accuracy = $990 / 1000$. Recall = $0 / 10$. Precision = $0 / 0$, undefined (scikit-learn warns and reports 0).
- Balanced accuracy = $(0 + 1) / 2 = 0.5$: correctly exposes the useless model.

Answer: Acc 0.99, recall 0, precision undefined (treated as 0), balanced acc 0.5

Q8.9 NAT · M

Precision 0.9 and recall 0.1. Compute F1 and compare to the arithmetic mean.

Solution

- $F1 = 2(0.9)(0.1) / (1.0) = 0.18$.
- The harmonic mean punishes imbalance between P and R; the arithmetic mean would hide the terrible recall.

Answer: F1 = 0.18; arithmetic mean 0.5

Q8.10 NAT · M

P = 0.5, R = 0.8. Compute F_2 and $F_{0.5}$.

Solution

- $F_2 = \frac{5PR}{4P+R} = \frac{5(0.4)}{2+0.8} = 2/2.8 = 0.714$.
- $F_{0.5} = \frac{1.25PR}{0.25P+R} = \frac{1.25(0.4)}{0.125+0.8} = 0.5/0.925 = 0.541$.
- F_2 is closer to recall, $F_{0.5}$ closer to precision.

Answer: $F_2 \approx 0.714$, $F_{0.5} \approx 0.541$

Q8.11 NAT · M

Scores: positives (0.8, 0.4), negatives (0.5, 0.3, 0.1). Compute ROC AUC.

Solution

- Pairs = $2 \times 3 = 6$.
- 0.8 beats all three negatives: 3. 0.4 beats 0.3 and 0.1 but not 0.5: 2.
- AUC = $5/6$.

Answer: $5/6 \approx 0.833$

Q8.12 MSQ · M

Which are true about ROC AUC?

- (A) It is invariant to any monotone transformation of scores.
- (B) It measures calibration.
- (C) AUC = 0.5 means random ranking.
- (D) AUC = 0.2 means the scores are useful if inverted.

Solution

- Only ranks matter, so calibration is invisible to AUC. Inverting scores gives $1 - 0.2 = 0.8$.

Answer: A, C, D

Q8.13 MCQ · M

With 0.1% positives, a model has ROC AUC 0.95 but PR AUC 0.08. The most accurate interpretation is

- (A) the model is excellent for the business
- (B) the model ranks positives above negatives often, but at useful recall levels most flagged items are still false positives
- (C) PR AUC is buggy
- (D) ROC AUC is always lower

Solution

- FPR of 1% on a million negatives is 10,000 false alarms, swamping 1,000 positives. PR curves reveal that; ROC hides it. PR AUC baseline here is 0.001, so 0.08 is 80 times random, but precision is still low.

Answer: B

Q8.14 MSQ · M

Raising the classification threshold from 0.5 to 0.8 typically

- (A) decreases recall (or keeps it the same)
- (B) increases precision
- (C) increases the number of predicted positives
- (D) moves the operating point toward the bottom left of the ROC curve

Solution

- Fewer predicted positives: TP and FP both fall. Recall cannot increase. Precision usually increases, though not guaranteed monotone.

Answer: A, B, D

Q8.15 NAT · M

Three class confusion counts (per class TP, FP, FN): A (10, 0, 0), B (5, 5, 5), C (1, 1, 9). Compute macro F1 and micro F1.

Solution

- F1 per class = $2TP / (2TP + FP + FN)$. A: $20/20 = 1$. B: $10/20 = 0.5$. C: $2/12 = 0.167$.
- Macro = $(1 + 0.5 + 0.167) / 3 = 0.556$.
- Micro: pooled TP = 16, FP = 6, FN = 14: $32 / (32 + 20) = 0.615$.
- Macro is lower because the rare, poorly handled class C counts equally.

Answer: Macro ≈ 0.556 , micro ≈ 0.615

Q8.16 MCQ · M

For single label multiclass classification, micro averaged F1 equals

- (A) macro F1
- (B) accuracy
- (C) weighted recall only when balanced
- (D) AUC

Solution

- Each wrong prediction is one FP (for the predicted class) and one FN (for the true class), so pooled FP = pooled FN = errors, and micro P = micro R = accuracy.

Answer: B

Q8.17 NAT · M

Compute MCC for TP = 40, FP = 10, FN = 20, TN = 130.

Solution

- Numerator: $40 \cdot 130 - 10 \cdot 20 = 5200 - 200 = 5000$.
- Denominator: $\sqrt{(50)(60)(140)(150)} = \sqrt{63,000,000} = 7937$.
- MCC = $5000 / 7937 = 0.630$.

Answer: ≈ 0.63

Q8.18 MSQ · M

SMOTE. Which are true?

- (A) It creates synthetic minority samples by interpolating between minority neighbours.
- (B) It duplicates minority samples exactly.
- (C) It can create noisy samples in overlapping regions.
- (D) Borderline SMOTE focuses on minority points near the class boundary.

Solution

- Exact duplication is random oversampling.

Answer: A, C, D

Q8.19 NAT · E

SMOTE: minority point $x = (2, 4)$, chosen neighbour $(4, 8)$, random $u = 0.25$. Synthetic point?

Solution

- $x + u(x_{nn} - x) = (2, 4) + 0.25(2, 4) = (2.5, 5)$.

Answer: (2.5, 5)

Q8.20 MCQ · M

Which metric is most appropriate to compare two job ranking models where only the top 10 results are seen?

- (A) Accuracy
- (B) NDCG@10 or Recall@10
- (C) ROC AUC over all items
- (D) R^2

Solution

- Position matters: NDCG discounts lower ranks by $\log_2(i + 1)$.

Answer: B**Q8.21** NAT · M

Relevances of the top 3 results: (3, 2, 0). Ideal order would be (3, 2, 1) but the item with relevance 1 is at rank 5. Compute DCG@3 and the ideal DCG@3, then NDCG@3.

Solution

- $DCG = 3 / \log_2 2 + 2 / \log_2 3 + 0 / \log_2 4 = 3 + 1.262 + 0 = 4.262$.
- $IDCG = 3 + 1.262 + 1 / \log_2 4 = 4.262 + 0.5 = 4.762$.
- $NDCG = 4.262 / 4.762 = 0.895$.

Answer: DCG $\approx$ 4.262, IDCG 4.762, NDCG $\approx$ 0.895**Q8.22** NAT · E

Three queries; the first relevant result appears at ranks 1, 3 and 2. MRR?

Solution

- $(1 + 1/3 + 1/2) / 3 = 1.833 / 3 = 0.611$.

Answer: $\approx$ 0.611**Q8.23** NAT · M

Brier score for predictions (0.9, 0.2, 0.6) with labels (1, 0, 0).

Solution

- Squared errors: 0.01, 0.04, 0.36; mean = $0.41 / 3 = 0.137$.

Answer: $\approx$ 0.137**Q8.24** MSQ · H

A model's predicted probabilities are all multiplied by 0.5 (e.g. $0.8 \rightarrow 0.4$). Which metrics change?

- (A) ROC AUC
- (B) Log loss
- (C) Brier score
- (D) Accuracy at threshold 0.5

Solution

- Ranking unchanged, so AUC unchanged. Probabilistic losses and threshold dependent metrics change.

Answer: B, C, D**Q8.25** MCQ · M

A reliability (calibration) curve that lies below the diagonal indicates the model is

- (A) under confident
- (B) over confident (predicted probabilities higher than observed frequencies)
- (C) perfectly calibrated
- (D) random

Solution

- When it says 0.8, the event happens less than 80% of the time. Fix with Platt or isotonic calibration.

Answer: B**Q8.26** MSQ · M

Which are true about precision recall curves?

- (A) They do not use true negatives.

- (B) A random classifier's PR curve is a horizontal line at the positive rate.
- (C) They are preferable to ROC for highly imbalanced problems where positives matter.
- (D) Area under PR equals ROC AUC.

Solution

- TN never appears in precision or recall, which is why PR analysis ignores the easy negatives that dominate imbalanced data.

Answer: A, B, C**Q8.27** NAT · M

A classifier has sensitivity 0.9 and specificity 0.95. Prevalence of positives is 2%. Compute precision.

Solution

- Per 1000 people: 20 positive, 980 negative.
- $TP = 0.9(20) = 18$. $FP = 0.05(980) = 49$.
- $Precision = 18 / 67 = 0.269$. Base rates dominate: good sensitivity and specificity, yet most alarms are false (Bayes' theorem in disguise).

Answer: ≈ 0.269 **Q8.28** MCQ · E

NPV (negative predictive value) is

- (A) $TN / (TN + FN)$
- (B) $TN / (TN + FP)$
- (C) $TP / (TP + FN)$
- (D) $FN / (FN + TP)$

Solution

- "When we say negative, how often is it truly negative." (B) is specificity.

Answer: A**Q8.29** MSQ · M

Which statements about accuracy are true?

- (A) It is misleading on imbalanced data.
- (B) It depends on the threshold.
- (C) It equals micro F1 for single label multiclass problems.
- (D) It is always the best metric for business decisions.

Solution

- Accuracy weights every error equally and ignores costs; the business metric decides.

Answer: A, B, C**Q8.30** MCQ · M

A fraud team can manually review 100 cases a day. The best offline metric is

- (A) ROC AUC
- (B) Precision@100 (per day)
- (C) Accuracy
- (D) Macro F1

Solution

- The metric should mirror the decision: only the top 100 scored cases get action, so what matters is how many of those are fraud.

Answer: B**Q8.31** NAT · M

Model A: $P = 0.7$, $R = 0.7$. Model B: $P = 0.9$, $R = 0.5$. Which has higher F1?

Solution

- A: $F1 = 0.7$. B: $2(0.45)/1.4 = 0.643$.

Answer: A (0.70 vs 0.643)

Q8.32 MSQ · H

You oversample positives from 1% to 50% in training. Which are true for the trained classifier's probabilities on real (1%) data?

- (A) They are inflated relative to true probabilities.
- (B) Ranking (AUC) may still be good.
- (C) They can be corrected by adjusting the log odds by $\log \frac{\pi_{true}/(1-\pi_{true})}{\pi_{train}/(1-\pi_{train})}$.
- (D) No correction is ever needed.

Solution

- Prior shift correction: subtract $\log(0.5/0.5) - \log(0.01/0.99) = 4.595$ from the log odds.

Answer: A, B, C

Q8.33 MCQ · M

Cohen's kappa measures

- (A) agreement beyond what would be expected by chance
- (B) ranking quality
- (C) regression error
- (D) calibration

Solution

- $\kappa = (p_o - p_e)/(1 - p_e)$. Common for annotator agreement when building labeled data.

Answer: A

Q8.34 MSQ · M

Which metric pairs are the same quantity?

- (A) Recall and sensitivity
- (B) Recall and TPR
- (C) Specificity and $1 - \text{FPR}$
- (D) Precision and NPV

Solution

- Precision is $TP/(TP + FP)$ while NPV is $TN/(TN + FN)$: different questions.

Answer: A, B, C

Q8.35 SUB · M

Interviewer: "Why is F1 a harmonic mean and not arithmetic?"

Solution

- $F1 = 2TP/(2TP + FP + FN)$: it treats FP and FN symmetrically relative to TP. Arithmetic mean would let $P = 1$, $R = 0.01$ score 0.5.

Answer: Because rates with a common numerator (TP) average correctly with the harmonic mean, and it heavily penalises a low value in either precision or recall.

Interview angle

- Draw the confusion matrix and define every metric from it. Which would you optimise for fraud, for spam, for job recommendations?
- Explain ROC AUC as a probability. Why can it mislead under imbalance?
- Micro vs macro vs weighted averaging; when does each matter?

- How do you handle imbalance end to end, and what happens to calibration?
- Offline metric vs online metric: how would you A/B test a new ranker?

9 Maximum Likelihood (MLE) and Maximum a Posteriori (MAP)

Companion resource for this section: [StatQuest: Maximum Likelihood, Clearly Explained](#)

9.1 Likelihood: flip the question

- Probability: parameters fixed, data random. “If the coin has $p = 0.5$, how likely are 7 heads in 10?”
- **Likelihood**: data fixed (we observed it), parameter varies. “Given 7 heads in 10, how plausible is each p ?”
 $\mathcal{L}(p) = P(\text{data} \mid p) = p^7(1-p)^3$ (times a constant).
- $\mathcal{L}$ is not a probability distribution over p (it need not integrate to 1).
- **MLE**: pick the parameter that makes the observed data most probable: $\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta)$.

9.2 The recipe (works every time)

- Step 1: write the likelihood for i.i.d. data as a product: $\mathcal{L}(\theta) = \prod_i p(x_i \mid \theta)$.
- Step 2: take the log (products become sums, maximiser unchanged because log is increasing, and numerically stable): $\ell(\theta) = \sum_i \log p(x_i \mid \theta)$.
- Step 3: differentiate, set to zero, solve. Check it is a maximum.
- Coin: $\ell(p) = 7 \log p + 3 \log(1-p)$. $\ell' = 7/p - 3/(1-p) = 0 \Rightarrow 7(1-p) = 3p \Rightarrow p = 0.7$. In general $\hat{p} = k/n$.
- Gaussian with known variance: $\ell(\mu) = -\frac{1}{2\sigma^2} \sum (x_i - \mu)^2 + \text{const}$, maximised at $\hat{\mu} = \bar{x}$. Variance: $\hat{\sigma}_{MLE}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ (biased; divides by n not $n-1$).
- Poisson (count data such as applications per hour): $\hat{\lambda} = \bar{x}$. Exponential (waiting times): $\hat{\lambda} = 1/\bar{x}$.

9.3 Why ML losses are negative log likelihoods

- Maximising likelihood = minimising **negative log likelihood (NLL)**.
- Linear regression: assume $y = w^\top x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. $\text{NLL} = \frac{1}{2\sigma^2} \sum (y_i - w^\top x_i)^2 + \text{const}$. So **least squares = MLE under Gaussian noise**.
- Laplace noise gives absolute error (MAE) as NLL.
- Logistic regression: $y \sim \text{Bernoulli}(\sigma(w^\top x))$. $\text{NLL} = \log \text{loss} / \text{binary cross entropy}$. Softmax classifiers: categorical NLL = cross entropy.
- Minimising cross entropy between the empirical distribution and the model is equivalent to minimising the KL divergence $KL(p_{\text{data}} \parallel p_{\theta})$, since the data entropy is a constant.

9.4 MAP: add a prior belief

- Bayes: $p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)} \propto \text{likelihood} \times \text{prior}$.
- **MAP**: $\hat{\theta}_{MAP} = \arg \max_{\theta} [\log p(D \mid \theta) + \log p(\theta)]$. The prior term acts as a regulariser.
- Coin with a Beta(a, b) prior (pseudo counts: $a-1$ imaginary heads, $b-1$ tails): $\hat{p}_{MAP} = \frac{k+a-1}{n+a+b-2}$. With 3 heads in 3 tosses MLE says $p = 1$ (never tails!). With Beta(2,2) MAP gives $(3+1)/(3+2) = 0.8$: sensible shrinkage. Laplace (add one) smoothing in Naive Bayes is exactly this idea.
- Gaussian prior $w \sim \mathcal{N}(0, \tau^2 I)$: $-\log p(w) = \frac{1}{2\tau^2} \|w\|^2$, so **MAP = Ridge** with $\lambda = \sigma^2/\tau^2$. Laplace prior gives **Lasso**.
- As data grow, the likelihood dominates and $\text{MAP} \rightarrow \text{MLE}$. With a uniform (flat) prior, $\text{MAP} = \text{MLE}$.
- **Full Bayesian** inference keeps the whole posterior (not just its peak) and averages predictions over it: $p(y \mid x, D) = \int p(y \mid x, \theta)p(\theta \mid D)d\theta$. MAP is a point estimate; it is not invariant to reparameterisation.
- Conjugate priors keep the posterior in the same family: Beta-Bernoulli, Gamma-Poisson, Normal-Normal, Dirichlet-Multinomial.

9.5 Properties of MLE worth quoting

- Consistent (converges to the true value), asymptotically normal with variance given by the inverse Fisher information, asymptotically efficient.
- Invariance: MLE of $g(\theta)$ is $g(\hat{\theta})$ (MLE of σ is $\sqrt{\hat{\sigma}^2}$).
- Can be biased in small samples ($\hat{\sigma}^2$), can overfit (a word never seen gets probability 0), and can be undefined (separable logistic regression).
- When there are hidden variables (cluster memberships), maximise likelihood with **EM** (Chapter 12).

Where this shows up

- Every classification network is trained by MLE (cross entropy). Weight decay is MAP with a Gaussian prior.
- Smoothing click through rates for new jobs with few impressions (Beta prior) is MAP; it stops a job with 1 click out of 1 view ranking first.

Formula sheet

Bernoulli $\hat{p} = k/n$; Gaussian $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n} \sum (x - \bar{x})^2$; Poisson $\hat{\lambda} = \bar{x}$; Exponential $\hat{\lambda} = 1/\bar{x}$; Uniform(0, θ): $\hat{\theta} = \max x_i$.
 MAP Beta(a, b): $\frac{k+a-1}{n+a+b-2}$; posterior mean $\frac{k+a}{n+a+b}$. Gaussian prior $\Rightarrow$ L2, Laplace $\Rightarrow$ L1.
 Gaussian noise $\Rightarrow$ MSE; Bernoulli $\Rightarrow$ log loss; Laplace noise $\Rightarrow$ MAE.

9.6 Question bank: MLE and MAP

PYQ status for this chapter

No InfoEdge report names an MLE or MAP question explicitly, though MLE and MAP are listed in the syllabus. The reported interview did ask Bayes' theorem and its use in Naive Bayes, and the entropy and cross entropy style OA items; the first questions connect to those. The rest follow the OA style.

Q9.1 MSQ · M

★ **PYQ (OA style: classification losses as likelihoods (cross entropy / log loss))** Which statements are true?

- (A) Minimising log loss in logistic regression is maximum likelihood estimation.
- (B) Minimising MSE in linear regression is MLE under Gaussian noise.
- (C) Cross entropy loss has nothing to do with likelihood.
- (D) Minimising MAE is MLE under Laplace noise.

Solution

- NLL of Bernoulli = log loss; of Gaussian = scaled MSE; of Laplace = scaled MAE.

Answer: A, B, D

Q9.2 NAT · M

★ **PYQ (INT: Bayes' theorem in real life (Naive Bayes smoothing as MAP))** In a spam Naive Bayes model, the word "lottery" appears 0 times in 100 ham emails' word slots, vocabulary size 1000, total ham word count 5000. Using Laplace (add one) smoothing, estimate $P(\text{lottery} \mid \text{ham})$.

Solution

- MLE = $0/5000 = 0$: one unseen word would zero out the entire product of probabilities.
- Add one: $(0 + 1)/(5000 + 1000) = 1/6000 = 0.000167$.
- This is MAP with a symmetric Dirichlet(2) prior (one pseudo count per word).

Answer: $\approx 1.67 \times 10^{-4}$

Q9.3 NAT · E

A coin shows 12 heads in 20 tosses. MLE of p ?

Solution

- $\hat{p} = k/n = 12/20$.

Answer: 0.6

Q9.4 NAT · M

Data: 2, 4, 6, 8 assumed Gaussian. MLE of μ and σ^2 .

Solution

- Mean = $20/4 = 5$. Squared deviations: 9, 1, 1, 9; sum 20.
- MLE divides by n : $20/4 = 5$. (Unbiased sample variance would be $20/3 = 6.67$.)

Answer: $\hat{\mu} = 5, \hat{\sigma}^2 = 5$

Q9.5 NAT · M

Number of job applications per hour observed: 3, 5, 4, 0, 3. Assuming Poisson, MLE of λ ?

Solution

- $\ell(\lambda) = \sum (x_i \log \lambda - \lambda) + \text{const}$; $\ell' = \sum x_i / \lambda - n = 0 \Rightarrow \hat{\lambda} = \bar{x} = 15/5 = 3$.

Answer: 3

Q9.6 NAT · M

Waiting times (minutes) 2, 3, 5 modelled as exponential with rate λ . MLE?

Solution

- $\ell = n \log \lambda - \lambda \sum x_i$; $\ell' = n/\lambda - \sum x = 0 \Rightarrow \hat{\lambda} = n/\sum x = 3/10 = 0.3$.

Answer: 0.3 per minute

Q9.7 NAT · M

3 heads in 3 tosses. Compute MLE, MAP with Beta(2, 2) prior, and posterior mean.

Solution

- MLE = $3/3 = 1$.
- MAP = $(3 + 2 - 1)/(3 + 2 + 2 - 2) = 4/5 = 0.8$.
- Posterior Beta(5, 2) has mean $5/7 = 0.714$.

Answer: MLE 1, MAP 0.8, posterior mean ≈ 0.714

Q9.8 MCQ · M

MAP estimation with a Gaussian prior on weights is equivalent to

- (A) Lasso
- (B) Ridge (L2 regularisation)
- (C) Elastic net
- (D) no regularisation

Solution

- $-\log \mathcal{N}(w; 0, \tau^2) = \frac{\|w\|^2}{2\tau^2} + \text{const}$.

Answer: B

Q9.9 MSQ · M

MLE vs MAP. Which are true?

- (A) With a uniform prior, MAP equals MLE.
- (B) As sample size grows, MAP approaches MLE.
- (C) MAP is a full Bayesian posterior.
- (D) MAP can prevent extreme estimates from small samples.

Solution

- MAP is only the posterior mode, a point estimate.

Answer: A, B, D

Q9.10 MCQ · E

Why do we maximise the log likelihood instead of the likelihood?

- (A) They have different maximisers.
- (B) Log turns products into sums, simplifies derivatives and avoids numerical underflow, without changing the maximiser.
- (C) Log likelihood is always positive.
- (D) It is required for convexity.

Solution

- The product of 10,000 probabilities of about 0.1 is 10^{-10000} , which underflows to 0 in floating point.

Answer: B

Q9.11 MSQ · M

Which are properties of MLE under regularity conditions?

- (A) Consistency
- (B) Asymptotic normality
- (C) Always unbiased for finite n
- (D) Invariance under reparameterisation

Solution

- $\hat{\sigma}_{MLE}^2$ has expectation $\frac{n-1}{n}\sigma^2$: biased for finite n .

Answer: A, B, D

Q9.12 NAT · H

Data from Uniform(0, θ): 1.2, 3.4, 2.7. MLE of θ ?

Solution

- $\mathcal{L}(\theta) = \theta^{-3}$ if $\theta \geq \max x_i$, else 0.
- θ^{-3} decreases in θ , so choose the smallest allowed: $\hat{\theta} = \max = 3.4$. Calculus (setting derivative to 0) fails here; always check the support.
- This MLE is biased low: $E[\hat{\theta}] = \frac{n}{n+1}\theta$.

Answer: 3.4

Q9.13 MCQ · M

Minimising cross entropy between data and model distribution is equivalent to minimising

- (A) $KL(p_{\theta} \| p_{data})$
- (B) $KL(p_{data} \| p_{\theta})$
- (C) the model's entropy
- (D) MSE

Solution

- $H(p_{data}, p_{\theta}) = H(p_{data}) + KL(p_{data} \| p_{\theta})$; the first term does not depend on θ .

Answer: B

Q9.14 NAT · M

Posterior with Gaussian likelihood and Gaussian prior on a mean: prior $\mu \sim \mathcal{N}(0, 1)$, one observation $x = 4$ with noise variance 1. Posterior mean (= MAP)?

Solution

- Precision weighted average: $\frac{(1/1) \cdot 0 + (1/1) \cdot 4}{1+1} = 2$.
- Equal precisions, so the estimate is halfway between prior mean and data.

Answer: 2**Q9.15** MSQ · M

Conjugate prior pairs. Which are correct?

- (A) Beta prior for Bernoulli/Binomial likelihood
- (B) Gamma prior for Poisson rate
- (C) Dirichlet prior for Multinomial
- (D) Gaussian prior for Bernoulli probability

Solution

- A Gaussian on $p \in [0, 1]$ is not conjugate (and puts mass outside $[0, 1]$).

Answer: A, B, C**Q9.16** NAT · M

Gamma($\alpha = 2, \beta = 1$) prior on a Poisson rate; observe counts 3, 5. Posterior mean of λ ? (Posterior is Gamma($\alpha + \sum x, \beta + n$), mean α / β .)

Solution

- Posterior Gamma($2 + 8, 1 + 2$) = Gamma($10, 3$). Mean = $10/3$. MLE would be 4. Shrunk toward prior mean 2.

Answer: $10/3 \approx 3.33$ **Q9.17** MCQ · MThe likelihood function $\mathcal{L}(\theta)$

- (A) always integrates to 1 over θ
- (B) is a function of θ with data fixed, not a probability density over θ
- (C) is the posterior
- (D) is the prior

Solution

- Likelihood $\mathcal{L}(\theta) = p(D | \theta)$: the data are frozen, θ slides.
- Example: 1 head in 1 toss gives $\mathcal{L}(p) = p$; $\int_0^1 p dp = 0.5 \neq 1$. So it is not a density over p .
- Posterior = likelihood $\times$ prior, normalised. Prior is belief before data.

Answer: B**Q9.18** MSQ · H

Logistic regression on perfectly separable data. Which are true?

- (A) The MLE does not exist (weights diverge).
- (B) A Gaussian prior (L2) makes the MAP estimate finite.
- (C) MLE always exists for logistic regression.
- (D) Predicted probabilities approach 0 and 1 under MLE training.

Solution

- If a line separates the classes perfectly, scaling w by 2, 10, 1000 keeps the same boundary but pushes every $\sigma(w^\top x)$ closer to the correct 0 or 1, so the likelihood keeps rising. No finite maximiser exists (A true, C false).
- As $\|w\| \rightarrow \infty$ predictions saturate at 0 and 1 (D true); gradient descent never stops growing w .
- Adding $\frac{\lambda}{2} \|w\|^2$ (Gaussian prior) penalises the growth, so the MAP optimum is finite (B true). This is why sklearn's LogisticRegression uses L2 by default.

Answer: A, B, D

Q9.19 NAT · M

Click through smoothing: a job has 1 click in 2 views. Using a $\text{Beta}(\alpha = 2, \beta = 38)$ prior (prior mean 5%), compute the posterior mean CTR.

Solution

- Posterior $\text{Beta}(2 + 1, 38 + 1) = \text{Beta}(3, 39)$. Mean = $3/42 = 0.0714$.
- Raw CTR 50% would wrongly top the ranking. This empirical Bayes smoothing is standard in recommendation systems.

Answer: $\approx 7.1\%$

Q9.20 MCQ · E

If $\hat{\sigma}^2$ is the MLE of the variance, the MLE of the standard deviation is

- (A) $\hat{\sigma}^2/2$
- (B) $\sqrt{\hat{\sigma}^2}$
- (C) $\hat{\sigma}^2$
- (D) undefined

Solution

- Invariance property of MLE.

Answer: B

Q9.21 MSQ · M

Naive Bayes classifier. Which are true?

- (A) It assumes features are conditionally independent given the class.
- (B) Parameters are usually estimated by MLE (counts) with smoothing.
- (C) It is a discriminative model.
- (D) It works well for text classification despite its assumption.

Solution

- Naive Bayes is generative (models $p(x | y)p(y)$); logistic regression is the discriminative counterpart.

Answer: A, B, D

Q9.22 NAT · H

Linear regression with Gaussian noise $\sigma^2 = 4$ and Gaussian prior $w \sim \mathcal{N}(0, \tau^2 = 2)$. The equivalent Ridge λ (objective $\|y - Xw\|^2 + \lambda\|w\|^2$)?

Solution

- $\text{NLP} = \frac{1}{2\sigma^2}\|y - Xw\|^2 + \frac{1}{2\tau^2}\|w\|^2$. Multiply by $2\sigma^2$: $\|y - Xw\|^2 + \frac{\sigma^2}{\tau^2}\|w\|^2$.
- $\lambda = 4/2 = 2$. Noisier data or tighter prior means stronger regularisation.

Answer: $\lambda = 2$

Q9.23 MCQ · M

EM (expectation maximisation) is used to find MLE when

- (A) the likelihood is convex
- (B) there are latent (hidden) variables or missing data
- (C) data are fully labeled and Gaussian
- (D) you need a MAP estimate only

Solution

- Example: Gaussian mixtures (unknown cluster memberships), HMMs (hidden states). EM monotonically increases the likelihood to a local maximum.

Answer: B

Q9.24 MSQ · M

A model trained by MLE assigns zero probability to an unseen word. Remedies:

- (A) Additive (Laplace) smoothing
- (B) MAP with a Dirichlet prior
- (C) Using subword tokens so words are rarely unseen
- (D) Removing all rare words from test data

Solution

- (D) is cheating on evaluation.

Answer: A, B, C

Q9.25 NAT · M

Log likelihood of data $(1, 0, 1, 1)$ under Bernoulli($p = 0.75$) (natural log).

Solution

- $3 \ln 0.75 + \ln 0.25 = 3(-0.2877) + (-1.3863) = -0.863 - 1.386 = -2.249$.
- This is the MLE, so any other p gives a lower value, e.g. $p = 0.5$: $4 \ln 0.5 = -2.773$.

Answer: ≈ -2.249

Q9.26 MCQ · M

The Fisher information for a Bernoulli(p) observation is $\frac{1}{p(1-p)}$. The approximate standard error of $\hat{p}$ from n tosses is

- (A) $\sqrt{p(1-p)/n}$
- (B) $p(1-p)$
- (C) $1/n$
- (D) $\sqrt{n}$

Solution

- Asymptotic variance $= 1/(nI(p)) = p(1-p)/n$. Same as the binomial proportion formula.

Answer: A

Q9.27 MSQ · M

Generative vs discriminative models. Which are generative?

- (A) Naive Bayes
- (B) Gaussian mixture model / LDA (linear discriminant analysis)
- (C) Logistic regression
- (D) GANs and VAEs

Solution

- Generative models learn $p(x, y)$ or $p(x)$; discriminative learn $p(y | x)$ or a boundary.

Answer: A, B, D

Q9.28 MCQ · H

MAP estimates are not invariant to reparameterisation because

- (A) the likelihood changes
- (B) the prior density transforms with a Jacobian, which can move its mode
- (C) MAP ignores the prior
- (D) they are always equal to MLE

Solution

- Modes of densities move under non linear changes of variable; MLE maximises a function (not a density in θ), so it does not have this issue.

Answer: B

Q9.29 NAT · M

MLE of p for a geometric distribution (number of trials until first success) from observations 2, 4, 3?

Solution

- $P(X = k) = (1 - p)^{k-1}p$. $\ell = n \log p + (\sum k - n) \log(1 - p)$.
- $\ell' = n/p - (\sum k - n)/(1 - p) = 0 \Rightarrow \hat{p} = n/\sum k = 3/9 = 1/3$.

Answer: 1/3

Q9.30 MSQ · M

Which statements connect MLE to deep learning practice?

- (A) Softmax + cross entropy is multinomial MLE.
- (B) Weight decay is a Gaussian prior (MAP).
- (C) Dropout is exactly MLE.
- (D) Label smoothing modifies the target distribution in the cross entropy.

Solution

- Dropout has a Bayesian interpretation (approximate variational inference) but is not plain MLE.

Answer: A, B, D

Q9.31 MCQ · E

A die is rolled 60 times; face 6 appears 15 times. MLE of $P(6)$?

- (A) 1/6
- (B) 0.25
- (C) 0.15
- (D) 0.5

Solution

- 15/60. Categorical MLE = empirical frequencies.

Answer: B

Q9.32 SUB · M

Explain MLE vs MAP vs Bayesian inference to an interviewer in three lines.

Solution

- MLE can overfit on small data; MAP regularises via the prior; full Bayes also quantifies uncertainty but is expensive (MCMC, variational inference).

Answer: MLE maximises $p(D \mid \theta)$; MAP maximises $p(D \mid \theta)p(\theta)$; Bayesian inference keeps the full posterior and averages predictions over it.

Q9.33 NAT · M

Linear regression residuals: 1, -2, 1. Assuming Gaussian noise, the MLE of σ^2 ?

Solution

- $\hat{\sigma}^2 = \frac{1}{n} \sum r_i^2 = (1 + 4 + 1)/3 = 2$.

Answer: 2

Q9.34 MSQ · H

A Laplace prior on weights. Which are true?

- (A) Its negative log is proportional to $\|w\|_1$.
- (B) MAP gives Lasso.
- (C) It has more mass at exactly zero than a Gaussian in the sense of a sharp peak.
- (D) It gives the same estimate as Ridge.

Solution

- Laplace density $p(w) \propto e^{-|w|/b}$, so $-\log p(w) = |w|/b + \text{const}$; summed over weights: $\frac{1}{b} \|w\|_1$ (A).
- Adding it to the Gaussian NLL gives squared error $+\lambda \|w\|_1$: Lasso (B).
- The Laplace density has a sharp cusp at 0, which makes exact zeros optimal for many weights; the Gaussian is flat at 0 (C, in the intended sense).
- Ridge comes from a Gaussian prior and shrinks without zeroing, so D is false.

Answer: A, B, C

Q9.35 NAT · M

Uniform(0, θ) data: 2.1, 4.7, 3.9, 1.2. What is the MLE of θ ?

Solution

- Density of each point is $1/\theta$ if $x_i \leq \theta$, else 0. So $\mathcal{L}(\theta) = \theta^{-4}$ when $\theta \geq \max x_i = 4.7$, and 0 otherwise.
- θ^{-4} decreases as θ grows, so take the smallest allowed value: $\hat{\theta} = 4.7$.
- Note: calculus (setting a derivative to zero) fails here; the maximum is at the boundary. Also the MLE is biased low (the true θ is at least the max).

Answer: 4.7

Interview angle

- Derive MSE from Gaussian noise and log loss from Bernoulli. What loss would Poisson noise give?
- MLE vs MAP vs full Bayes; relate priors to L1/L2.
- Why is $\hat{\theta}_{MLE}^2$ biased? Why does it not matter for large n ?
- Smoothing CTR for new items: which prior, how to set its parameters (empirical Bayes)?
- Where does MLE fail and how does EM help with latent variables?

10 Missing Data Imputation (with outliers and imbalance)

Companion resource for this section: [scikit-learn User Guide: Imputation of missing values](#)

10.1 Why missing values are a modelling problem, not a cleaning chore

- Real tables have holes: a candidate skips “current salary”, a sensor drops a reading, a join finds no match. In pandas they appear as NaN/None; in raw files as blanks, “NA”, “-1”, “999”.
- Most models (Linear/Logistic Regression, SVM, KNN, sklearn neural nets) crash on NaN. Some handle it natively: XGBoost, LightGBM, CatBoost, sklearn’s HistGradientBoosting.
- The *way* you fill the hole changes the data distribution and can bias every downstream number. So first ask: **why** is the value missing?
- First look: `df.isna().sum()` (count per column), `df.isna().mean()` (fraction), and whether missingness correlates with the target: `df.groupby(df['salary'].isna())['placed'].mean()`.

10.2 The three missingness mechanisms (Rubin)

- MCAR** (Missing Completely At Random): the chance of being missing has nothing to do with any value, observed or not. Example: a lab technician randomly spills coffee on 5% of forms. Dropping those rows loses data but does *not* bias estimates.
- MAR** (Missing At Random): missingness depends only on *observed* columns. Example: freshers (experience = 0, observed) tend to skip “current salary”. Within each experience group the missingness is random. Imputation that uses other columns (regression, KNN, MICE) can correct it.
- MNAR** (Missing Not At Random): missingness depends on the missing value itself. Example: people with very high income refuse to report income. No imputation from observed data can fully fix it; you need domain knowledge, a model of the missingness, or a missing indicator.
- Tiny test: salaries [10, 20, 30, 40, NaN] where the NaN is truly 100 (MNAR). Mean imputation fills 25 and the column mean drops from the true 40 to 25. Bias.
- You cannot prove MAR vs MNAR from the data alone (the missing values are, well, missing). You can only check MCAR vs not (compare the observed columns between rows with and without missingness).

10.3 Option 1: deletion

- Listwise deletion** (`df.dropna()`): drop every row with any NaN. Fine if few rows and MCAR. Danger: with 10 columns each 5% missing independently, the fraction of complete rows is $0.95^{10} \approx 0.60$: you lose 40% of data.
- Pairwise deletion**: use all available pairs when computing each correlation. Keeps more data but different statistics use different subsets (correlation matrix may not be positive semidefinite).
- Drop the column**: if a column is, say, more than 60 to 70% missing and not informative, drop it. But first check whether “is missing” itself predicts the target.
- Never drop rows with a missing *feature* from the test set in production: at inference you must still predict. So you need an imputation strategy anyway.

10.4 Option 2: simple (univariate) imputation

- Mean**: numeric, roughly symmetric, no big outliers. Keeps the column mean unchanged; *shrinks variance* (all filled values sit at the centre) and weakens correlations.
- Median**: numeric and skewed or with outliers (salary, income, time on page). Robust.
- Mode** (most frequent): categorical columns (city, degree).
- Constant**: a new category “Unknown” for categoricals, or 0 where 0 has a real meaning (“number of past offers” missing because none).
- Forward fill / backward fill / interpolation**: time series (last known stock price, linear interpolation between sensor readings). Never use backward fill in a forecasting model: it leaks the future.
- Group-wise imputation: fill salary with the median of the same experience band: `df.groupby('exp_band')['salary'].transform('median')`. A cheap way to exploit MAR.

10.5 Worked example: what mean imputation does to variance

- Observed column: 2, 4, 6, NaN, NaN. Observed mean = 4. Observed variance (population) = $((2 - 4)^2 + 0 + (6 - 4)^2)/3 = 8/3 \approx 2.67$.
- After mean imputation: 2, 4, 6, 4, 4. Mean still 4. Variance = $(4 + 0 + 4 + 0 + 0)/5 = 8/5 = 1.6$. The spread fell by 40%.
- Consequence: standard errors look too small, correlations with other columns get diluted, and the model sees a spike at exactly 4.

10.6 Option 3: multivariate imputation

- **KNN imputation**: find the k nearest rows (distance computed on the observed features, so **scale first**), fill with their mean (or weighted mean) of that feature. Uses relationships between columns. Cost: distance computations, $O(n^2)$ naive.
- **Regression imputation**: predict the missing column from the others with a regression. Deterministic regression puts every point exactly on the line (overstates correlation); **stochastic regression imputation** adds random residual noise to restore variance.
- **MICE** (Multiple Imputation by Chained Equations), `sklearn IterativeImputer`: start with mean fill; then loop over columns, regress each incomplete column on all others, re-predict its missing entries; repeat for several rounds until stable.
- **Multiple imputation**: create m different plausible completed datasets (with randomness), fit the model on each, pool results (Rubin's rules). Captures the extra uncertainty caused by missingness. Standard in statistics and clinical trials.
- **MissForest**: iterative imputation with Random Forests; handles non-linearity and mixed types.
- **Matrix factorisation imputation**: treat the table as a low-rank matrix and fill blanks (Chapter 11); the same idea as recommender systems.
- **Deep learning**: autoencoders (denoising) or masked models learn to reconstruct missing entries.

10.7 Option 4: let the model handle it (XGBoost, LightGBM)

- XGBoost's **sparsity aware split finding**: at each split, try sending all missing values left, then all right; keep whichever direction gives higher gain. That learned **default direction** is used at prediction time.
- This means missingness itself can be informative and the model learns it without you writing any imputation code. LightGBM does something similar (`use_missing=true`).
- Classic decision trees (CART) use **surrogate splits**: if the best split feature is missing, use the next most similar split.
- This is exactly the answer InfoEdge candidates reported giving in the coding round: "drop, mean/median/mode, or use a model like XGBoost that handles NaN".

10.8 The missing indicator trick

- Add a binary column `salary_was_missing` before imputing. The model can then learn "missing" as its own signal (very useful for MNAR, e.g. "did not disclose salary" may predict something).
- `sklearn: SimpleImputer(add_indicator=True)`.

10.9 Leakage: fit the imputer on train only

- The mean used to fill must be computed on the **training fold only**, then applied to validation/test. Computing it on the full dataset leaks test information.
- Correct pattern: `Pipeline([('imp', SimpleImputer(strategy='median')), ('scale', StandardScaler()), ('clf', LogisticRegression())])` then `cross_val_score`. The pipeline refits the imputer inside every fold.
- Never use the target to impute features for the test set (you do not have the target there).

10.10 Outliers: the close cousin (reported as asked alongside)

- **Detect:** IQR rule (outside $[Q_1 - 1.5 IQR, Q_3 + 1.5 IQR]$), z score ($|z| > 3$), boxplots, Isolation Forest, Local Outlier Factor, DBSCAN noise points.
- **Decide:** data error (typo “age 250”) → fix or treat as missing; genuine extreme (a CEO’s salary) → keep but make the model robust.
- **Treat:** cap/winsorise at percentiles, log transform skewed columns, use robust models (trees, median, Huber loss, MAE), RobustScaler (uses median and IQR).
- Sensitive to outliers: mean, variance, OLS/MSE, K-Means, PCA, KNN with unscaled distances. Robust: median, trees, Huber, MAE.

10.11 Imbalanced data (reported in the same InfoEdge coding discussion)

- 99% “not fraud”, 1% “fraud”: a model predicting always “not fraud” gets 99% accuracy and zero value.
- **Metrics:** precision, recall, F1, PR AUC (Chapter 8), not accuracy.
- **Resampling:** random oversampling of minority, random undersampling of majority, **NearMiss** (under-sample majority points closest to minority), **SMOTE** (synthesise new minority points on the line between a minority point and one of its k minority neighbours: $x_{new} = x + \lambda(x_{nn} - x)$, $\lambda \sim U(0, 1)$), **ADASYN** (SMOTE that makes more samples where minority points are hard to learn).
- **Cost sensitive:** `class_weight='balanced'` gives each class weight $n / (K \cdot n_k)$; XGBoost `scale_pos_weight` $\approx n_{neg} / n_{pos}$.
- **Threshold tuning:** move the decision threshold away from 0.5 using a validation PR curve.
- Resample **only the training fold**, after the split. Oversampling before splitting puts copies of the same point into train and test: leakage.

Where this shows up

- Naukri profiles: “current CTC”, “notice period”, “skills” are often blank. Missing notice period may itself signal a passive candidate (MNAR): add an indicator.
- Job application prediction: applies are rare events (imbalance). Use PR AUC, class weights and threshold tuning.

Formula sheet

Complete-row fraction with d independent columns each missing with prob q : $(1 - q)^d$.
Mean imputation: mean unchanged; variance scales by (observed count / total count) when using population variance of observed values.
SMOTE: $x_{new} = x + \lambda(x_{nn} - x)$, $\lambda \in [0, 1]$. Balanced class weight: $w_k = n / (Kn_k)$.
IQR fences: $Q_1 - 1.5 IQR$, $Q_3 + 1.5 IQR$. Order: split → fit imputer/scaler/resampler on train → transform test.

10.12 Question bank: Missing data, outliers, imbalance

PYQ status for this chapter

The IIT BHU InfoEdge DS write up reports that the coding round (Logistic Regression on a dataset) involved handling missing values, with drop, mean/median/mode and XGBoost’s native handling discussed, and that imbalanced data techniques (SMOTE, NearMiss, ADASYN, class weights) and outliers were asked. These items come first.

Q10.1 MSQ · E

★ **PYQ (Coding round / INT: ways to handle missing values)** You find 8% NaNs in a numeric column before training Logistic Regression. Which are valid strategies?

- Drop rows with NaN if data are plentiful and missingness looks random.
- Fill with the median of the training data.
- Switch to XGBoost, which can route NaNs natively.
- Leave NaNs in; Logistic Regression ignores them automatically.

Solution

- A: deletion is valid under MCAR with enough data (8% loss is tolerable).
- B: median imputation, fitted on train, is a standard robust fill.
- C: XGBoost learns a default direction for missing values at each split.
- D: sklearn's LogisticRegression raises ValueError: Input contains NaN. False.

Answer: A, B, C**Q10.2** MCQ · E

★ **PYQ (INT: mean vs median imputation)** The column “current salary” is heavily right skewed (a few very high earners). Best simple fill?

- (A) Mean
- (B) Median
- (C) Mode
- (D) Maximum

Solution

- Skew drags the mean toward the tail. Example salaries (LPA): 4, 5, 6, 7, 80: mean = 20.4, median = 6. Filling with 20.4 would make every blank look like a senior hire.
- The median ignores extreme values, so it represents a typical row.

Answer: B**Q10.3** MCQ · E

★ **PYQ (INT: imputing categorical features)** Missing values in the categorical column “city”. Which simple strategy is standard?

- (A) Mean of city codes
- (B) Mode, or a new category “Unknown”
- (C) Median of label-encoded city
- (D) Drop the target column

Solution

- Averaging integer codes (Delhi = 0, Mumbai = 1, Pune = 2) creates meaningless “city 1.3”.
- Most frequent category or a separate “Unknown” level (which also keeps missingness as a signal) are the textbook answers.

Answer: B**Q10.4** MSQ · M

★ **PYQ (INT: how XGBoost handles missing values)** Which statements about XGBoost and NaNs are true?

- (A) At each split it learns a default direction for missing values by trying both sides and picking the one with higher gain.
- (B) It requires mean imputation first.
- (C) At prediction time, a NaN follows the learned default branch.
- (D) It can exploit missingness as informative.

Solution

- XGBoost's sparsity aware algorithm evaluates the gain of sending all missing rows left and all right, and stores the better one (A).
- The stored default branch is used for any NaN at inference (C).
- If “missing” correlates with the target, the learned direction captures that (D).
- Imputation is optional, not required (B false).

Answer: A, C, D**Q10.5** MSQ · M

★ **PYQ (INT: handling imbalanced data)** Fraud is 0.5% of transactions. Which techniques help?

- (A) SMOTE on the training fold

- (B) class_weight = 'balanced'
- (C) Reporting accuracy as the main metric
- (D) Tuning the decision threshold using a precision recall curve

Solution

- SMOTE synthesises minority examples; apply it only on training data after the split (A).
- Class weights make each fraud mistake cost more in the loss (B).
- Accuracy is misleading: always "not fraud" scores 99.5% (C false).
- Lowering the threshold trades precision for recall at an operating point that suits the business (D).

Answer: A, B, D**Q10.6** NAT · E

★ **PYQ (INT: SMOTE mechanics)** SMOTE picks minority point $x = (2, 4)$ and neighbour $x_{nn} = (6, 8)$ with $\lambda = 0.25$. New point?

Solution

- $x_{nn} - x = (4, 4)$.
- $\lambda(x_{nn} - x) = (1, 1)$.
- $x_{new} = (2 + 1, 4 + 1) = (3, 5)$, a point on the segment joining them, one quarter of the way.

Answer: (3, 5)**Q10.7** MSQ · M

★ **PYQ (INT: SMOTE vs NearMiss vs ADASYN)** Which are correct descriptions?

- (A) NearMiss undersamples majority points that are close to minority points.
- (B) ADASYN generates more synthetic samples for minority points that are harder to learn (more majority neighbours).
- (C) SMOTE copies minority rows exactly.
- (D) Random undersampling can throw away useful majority information.

Solution

- NearMiss keeps majority points near the boundary (by distance to minority), discarding the rest (A).
- ADASYN weights each minority point by the fraction of majority points among its neighbours (B).
- SMOTE interpolates new points; exact copying is random oversampling (C false).
- Undersampling discards data (D).

Answer: A, B, D**Q10.8** NAT · M

★ **PYQ (INT: outlier detection)** Data: 3, 5, 7, 8, 9, 11, 13, 40. Using $Q_1 = 6$, $Q_3 = 12$ (median of halves), which values are outliers by the 1.5 IQR rule?

Solution

- $IQR = 12 - 6 = 6$; $1.5 \times 6 = 9$.
- Lower fence = $6 - 9 = -3$; upper fence = $12 + 9 = 21$.
- Only 40 exceeds 21.

Answer: 40 only**Q10.9** MSQ · M

★ **PYQ (INT: dealing with outliers)** Which models or statistics are relatively robust to outliers?

- (A) Median
- (B) Decision trees
- (C) Ordinary least squares
- (D) Huber loss regression

Solution

- Median depends only on the middle order statistic (A).
- Tree splits depend on the order of feature values, so a huge x value just lands in a bucket (B). (A huge y can still affect a regression leaf mean.)
- OLS squares residuals; one big residual dominates (C false).
- Huber is quadratic for small residuals and linear for large, limiting outlier influence (D).

Answer: A, B, D**Q10.10** MCQ · E

A lab technician randomly lost 3% of forms, regardless of content. Mechanism?

- (A) MCAR
- (B) MAR
- (C) MNAR
- (D) Cannot be missing

Solution

- Losing forms at random is independent of every value: Missing Completely At Random.

Answer: A**Q10.11** MCQ · M

In a survey, people with very high incomes tend not to report their income. Mechanism?

- (A) MCAR
- (B) MAR
- (C) MNAR
- (D) None

Solution

- Missingness depends on the value that is missing (high income), so it is Missing Not At Random.
- Mean imputation will underestimate average income.

Answer: C**Q10.12** MCQ · M

Freshers (experience = 0, always recorded) often leave "current salary" blank, but among freshers the blanks are random. Mechanism?

- (A) MCAR
- (B) MAR
- (C) MNAR
- (D) Deterministic

Solution

- Missingness depends on an observed column (experience), not on salary itself: MAR.
- Group-wise or model based imputation using experience fixes the bias.

Answer: B**Q10.13** NAT · E

Column: 2, 4, 6, NaN, NaN. After mean imputation, what is the population variance?

Solution

- Observed mean = $(2 + 4 + 6)/3 = 4$. Filled column: 2, 4, 6, 4, 4.
- Squared deviations: 4, 0, 4, 0, 0; sum 8.
- Variance = $8/5 = 1.6$ (down from $8/3 \approx 2.67$ on the observed values).

Answer: 1.6

Q10.14 MSQ · M

Effects of mean imputation on a numeric feature include

- (A) Column mean unchanged
- (B) Variance reduced
- (C) Correlation with other features weakened
- (D) Bias always removed

Solution

- Filling with the mean keeps the mean (A) but adds points with zero deviation, shrinking variance (B).
- Filled points ignore their relationship with other columns, so covariance is diluted (C).
- Under MAR/MNAR bias remains (D false).

Answer: A, B, C

Q10.15 NAT · M

A dataset has 12 independent columns, each missing with probability 0.05. What fraction of rows are complete (to 2 decimals)?

Solution

- Probability a row is complete = 0.95^{12} .
- $0.95^2 = 0.9025$; $0.95^4 = 0.8145$; $0.95^8 = 0.6634$; $0.95^{12} = 0.6634 \times 0.8145 = 0.5404$.
- So `dropna()` throws away about 46% of rows even though each column is only 5% missing.

Answer: 0.54

Q10.16 MCQ · M

Where must the median used for imputation be computed?

- (A) Full dataset before splitting
- (B) Training set only, then applied to test
- (C) Test set only
- (D) Separately on train and test

Solution

- Fitting on the full data leaks test distribution information (leakage).
- Fitting separately on test makes train and test transformations inconsistent and is impossible for single online predictions.
- Fit on train, transform both. A Pipeline enforces this inside cross validation.

Answer: B

Q10.17 MSQ · M

About KNN imputation, which are true?

- (A) Features should be scaled before computing distances.
- (B) It uses relationships between columns.
- (C) It is cheap for very large datasets.
- (D) The value is filled using neighbours' values of that feature.

Solution

- Without scaling, a salary column in lakhs dominates distance over a 0 to 1 column (A).
- Neighbours are chosen by similarity in other features, so it is multivariate (B).
- Naive KNN needs distances to all rows, costly for large n (C false).
- Fill = mean (or distance weighted mean) of that feature among the k neighbours (D).

Answer: A, B, D

Q10.18 NAT · M

KNN imputation with $k = 3$: the three nearest neighbours of a row have "age" = 24, 27, 30. Imputed age (unweighted)?

Solution

- Mean = $(24 + 27 + 30)/3 = 81/3 = 27$.

Answer: 27**Q10.19** NAT · H

Distance weighted KNN: neighbours have value 10 at distance 1 and value 20 at distance 2. Weights = $1/d$. Imputed value?

Solution

- Weights: $1/1 = 1$ and $1/2 = 0.5$; sum 1.5.
- Weighted mean = $(1 \cdot 10 + 0.5 \cdot 20)/1.5 = 20/1.5 = 13.33$. The closer neighbour counts double.

Answer: $40/3 \approx 13.33$ **Q10.20** MSQ · H

About MICE (IterativeImputer), which are true?

- (A) Each incomplete column is regressed on the others in turn.
- (B) It iterates several rounds until imputations stabilise.
- (C) It starts from an initial simple fill such as the mean.
- (D) It can only handle one missing column.

Solution

- The chained equations cycle through columns, each modelled from all others (A), repeating (B) after an initial fill (C).
- It handles many incomplete columns simultaneously (D false).

Answer: A, B, C**Q10.21** MCQ · H

Why does multiple imputation create several completed datasets rather than one?

- (A) To make training faster
- (B) To reflect uncertainty about the missing values in the final standard errors
- (C) Because one dataset cannot be stored
- (D) To remove outliers

Solution

- A single fill treats guesses as if observed, so confidence intervals come out too narrow.
- Fitting on m plausible versions and pooling (Rubin's rules) adds the between-imputation variance to the uncertainty.

Answer: B**Q10.22** MCQ · M

Deterministic regression imputation puts filled values exactly on the fitted line. The main drawback?

- (A) Underestimates correlation
- (B) Overstates correlation and understates variance
- (C) Always biased mean
- (D) Cannot handle numeric data

Solution

- Filled points have zero residual, so the relationship looks tighter than reality and spread shrinks.
- Fix: stochastic regression imputation (add random residual noise) or multiple imputation.

Answer: B**Q10.23** MSQ · M

Missing indicator columns

- (A) let a model learn that missingness itself is predictive
- (B) are useful under MNAR
- (C) are added with `SimpleImputer(add_indicator=True)`
- (D) remove the need to impute the original column in linear models

Solution

- The binary flag carries the “was missing” signal (A), especially valuable when missingness relates to the value (B). `sklearn` supports it directly (C).
- The original column still contains NaN; linear models still need it filled (D false).

Answer: A, B, C**Q10.24** MCQ · M

For a daily sales time series with a few missing days, a sensible first fill for a forecasting model is

- (A) Backward fill
- (B) Forward fill or linear interpolation using past and neighbouring values
- (C) Global mean over all years
- (D) Random values

Solution

- Interpolation respects local level and trend; forward fill uses only the past.
- Backward fill copies a future value into the past, leaking information in forecasting (A bad). The global mean ignores trend and seasonality.

Answer: B**Q10.25** MSQ · M

Which models in common libraries accept NaN inputs natively?

- (A) XGBoost
- (B) LightGBM
- (C) `sklearn HistGradientBoostingClassifier`
- (D) `sklearn SVC`

Solution

- The three gradient boosting implementations learn missing value routing.
- SVC (kernel distances) cannot compute with NaN (D false).

Answer: A, B, C**Q10.26** MCQ · E

A column is 85% missing and “is missing” shows no relationship with the target. Reasonable action?

- (A) Impute with mean and keep
- (B) Drop the column
- (C) Drop all rows with NaN
- (D) Duplicate the column

Solution

- With 85% missing, any fill is mostly invented, and missingness carries no signal. Dropping the column is sensible.
- Dropping rows would remove 85% of the data.

Answer: B**Q10.27** MSQ · H

Which are examples of leakage in preprocessing?

- (A) Oversampling with SMOTE before the train test split
- (B) Scaling using the mean of the whole dataset
- (C) Imputing a feature using the target column, then applying the same at test time

(D) Fitting the imputer inside each cross validation fold via a Pipeline

Solution

- A: synthetic points built from test rows' neighbours end up in train; near duplicates across the split inflate scores.
- B: test statistics leak into the training transformation.
- C: the target is unavailable at inference, so validation scores are fake.
- D is the correct practice.

Answer: A, B, C

Q10.28 NAT · M

Binary data with 900 negatives and 100 positives. `sklearn class_weight='balanced'` weight for the positive class?

Solution

- $w_k = n / (K n_k) = 1000 / (2 \times 100) = 5$.
- Negative weight = $1000 / (2 \times 900) = 0.556$. Ratio $5 / 0.556 = 9$ = imbalance ratio.

Answer: 5

Q10.29 NAT · E

XGBoost recommended `scale_pos_weight` for 9500 negatives and 500 positives?

Solution

- $n_{neg} / n_{pos} = 9500 / 500 = 19$.

Answer: 19

Q10.30 NAT · M

z score outlier rule with mean 50 and standard deviation 8. Is $x = 76$ an outlier at $|z| > 3$? Give z.

Solution

- $z = (76 - 50) / 8 = 26 / 8 = 3.25 > 3$.
- Caution: mean and SD are themselves inflated by outliers; IQR or MAD based rules are more robust.

Answer: $z = 3.25$, yes

Q10.31 MSQ · M

Which algorithms are sensitive to outliers?

- (A) K-Means
- (B) PCA
- (C) Median
- (D) Random Forest (on features)

Solution

- K-Means centroids are means; one far point drags a centroid (A).
- PCA maximises variance, so a single extreme point can dominate a component (B).
- Median and tree splits on features are robust (C, D false).

Answer: A, B

Q10.32 MCQ · M

Isolation Forest flags anomalies because anomalous points

- (A) need many random splits to isolate
- (B) are isolated in fewer random splits (shorter average path length)
- (C) have the highest density
- (D) are closest to the centroid

Solution

- Random axis splits separate a rare, far point quickly. Shorter average path length across trees means a higher anomaly score.

Answer: B**Q10.33** MCQ · M

Which scaler is most appropriate when features contain outliers?

- (A) StandardScaler
- (B) MinMaxScaler
- (C) RobustScaler
- (D) No scaler

Solution

- RobustScaler centres by the median and scales by IQR, so a single extreme value barely changes the scaling.
- MinMaxScaler is the worst: one outlier squashes all other values near 0.

Answer: C**Q10.34** MSQ · H

A model's validation F1 looks great, but it fails in production on a class that is 1% of traffic. Possible preprocessing causes?

- (A) Resampling before splitting created near duplicates across train and validation.
- (B) Validation set was balanced artificially, unlike production.
- (C) Threshold chosen on a balanced validation set does not suit real prevalence.
- (D) Using stratified splitting.

Solution

- Leakage inflates validation (A). Precision depends on prevalence; a balanced validation set overstates it (B, C).
- Stratification preserves class ratios and is good practice (D false).

Answer: A, B, C**Q10.35** MCQ · M

Missing salary is filled with the median of the candidate's experience band rather than the global median. This exploits

- (A) MCAR
- (B) MAR (missingness and value related to an observed column)
- (C) MNAR
- (D) Nothing

Solution

- Conditioning on an observed column that relates to salary gives better fills and removes bias under MAR.

Answer: B**Q10.36** MSQ · H

Matrix factorisation (low rank completion) for imputation. Which are true?

- (A) It assumes the full matrix is approximately low rank.
- (B) It is the same principle as recommending movies from a sparse ratings matrix.
- (C) It needs no observed entries.
- (D) It fits factors only to observed entries.

Solution

- Low rank means rows are combinations of a few latent patterns (A); ratings completion is the canonical case (B).
- Loss is summed over observed entries only; missing ones are predicted from $u_i^\top v_j$ (D). Without any observed entries nothing can be learned (C false).

Answer: A, B, D

Interview angle

- Walk through how you would handle missing values in a new dataset: inspect, mechanism, drop vs impute, indicator, pipeline without leakage.
- Mean vs median vs mode vs model based: when each? What does mean imputation do to variance and correlations?
- How does XGBoost handle NaN? What about a classic decision tree (surrogate splits)?
- SMOTE: how does it create points? Why only on the training fold? When can SMOTE hurt (noisy minority, high dimensions)?
- Outlier vs anomaly vs data error: how do you decide to drop, cap, or keep?

11 Matrix Factorisations: SVD, PCA, NMF and Recommenders

Companion resource for this section: [StatQuest: Principal Component Analysis \(PCA\), Step by Step](#)

11.1 What “factorising a matrix” means

- Just as $12 = 3 \times 4$ breaks a number into simpler pieces, a matrix factorisation writes A as a product of simpler matrices: $A = BC$ or $A = U\Sigma V^\top$.
- “Simpler” can mean triangular (easy to solve), orthogonal (rotations), diagonal (scalings), low rank (compressed), or non negative (interpretable parts).
- Map of the common ones:
 - LU**: $A = LU$ (lower times upper triangular). Solving linear systems.
 - QR**: $A = QR$ (Q orthonormal columns, R upper triangular). Numerically stable least squares.
 - Cholesky**: $A = LL^\top$ for symmetric positive definite A (covariance matrices, Gaussian sampling, GP regression).
 - Eigendecomposition**: $A = Q\Lambda Q^{-1}$ (square, diagonalisable); symmetric $A = Q\Lambda Q^\top$.
 - SVD**: $A = U\Sigma V^\top$ for any $m \times n$ matrix. The Swiss army knife.
 - NMF**: $A \approx WH$ with $W, H \geq 0$.
 - Recommender MF**: $R \approx PQ^\top$ fitted only on observed ratings.

11.2 SVD from scratch

- Any matrix A ($m \times n$) can be written $A = U\Sigma V^\top$ where U ($m \times m$) and V ($n \times n$) are orthogonal and Σ is diagonal with $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ (singular values).
- Geometric meaning: every linear map is *rotate* ($V^\top$), *stretch along axes* (Σ), *rotate* (U). A circle becomes an ellipse whose semi axes are the σ_i .
- How to compute by hand: eigenvalues of $A^\top A$ are σ_i^2 ; its eigenvectors are the columns of V ; then $u_i = Av_i / \sigma_i$.
- Tiny example: $A = \begin{pmatrix} 3 & 0 \\ 0 & -4 \end{pmatrix}$. $A^\top A = \text{diag}(9, 16)$, so $\sigma = 4, 3$. Singular values are always non negative even though an entry is -4 .
- Outer product form: $A = \sum_i \sigma_i u_i v_i^\top$, a sum of rank one “layers” ordered by importance.
- Facts: rank = number of non zero σ_i ; $\|A\|_2 = \sigma_1$; $\|A\|_F = \sqrt{\sum \sigma_i^2}$; condition number = $\sigma_1 / \sigma_{\min}$; pseudo inverse $A^+ = V\Sigma^+ U^\top$.
- Economy (thin) SVD for tall A ($m > n$): U is $m \times n$, Σ is $n \times n$, V is $n \times n$.

11.3 Low rank approximation (Eckart Young)

- Keep only the top k layers: $A_k = \sum_{i \leq k} \sigma_i u_i v_i^\top$. This is the **best** rank k approximation in both Frobenius and spectral norm.
- Error: $\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots}$, $\|A - A_k\|_2 = \sigma_{k+1}$.
- Storage: an $m \times n$ image needs mn numbers; rank k needs $k(m + n + 1)$. For 1000×1000 and $k = 50$: 100,100 vs 1,000,000, about 10%.
- Uses: compression, denoising (small singular values are often noise), latent semantic analysis, recommenders, low rank adapters (LoRA fine tunes a large model by learning $\Delta W = BA$ of low rank).

11.4 PCA is SVD of centred data (reported InfoEdge interview topic)

- Goal: find new axes (principal components) that capture the most variance, so we can drop the rest.
- Recipe: centre each column (subtract its mean); optionally standardise; compute covariance $C = \frac{1}{n-1} X^\top X$; eigenvectors of C are the PCs, eigenvalues are the variances along them.
- Via SVD: $X = U\Sigma V^\top$; PCs are columns of V ; variance along PC i is $\sigma_i^2 / (n - 1)$; projected data (scores) are $XV = U\Sigma$. SVD is preferred numerically (never forms $X^\top X$).

- **Explained variance ratio:** $\lambda_i / \sum_j \lambda_j$. Pick k to reach, say, 95%, or at the “elbow” of the scree plot.
- Tiny example: eigenvalues 6, 3, 1 $\Rightarrow$ PC1 explains 60%, PC1+PC2 explain 90%.
- Why centre: without it, PC1 just points at the mean. Why scale: a feature in rupees (variance 10^{10}) would swamp a feature in years.
- Limits: linear only; unsupervised (ignores labels, so the highest variance direction may be useless for classification); components are hard to interpret; sensitive to outliers.
- Relatives: **LDA** (supervised: maximises between class / within class variance, at most $K - 1$ components); **t-SNE** and **UMAP** (non linear, preserve local neighbourhoods, for visualisation only, distances between far clusters are not meaningful); **Kernel PCA** (non linear via kernels); **TruncatedSVD** (no centring, works on sparse TF-IDF).

11.5 NMF: non negative parts

- $A \approx WH$ with $A, W, H \geq 0$ (A : documents $\times$ words counts; W : documents $\times$ topics; H : topics $\times$ words).
- Because nothing can be subtracted, the model can only *add* parts: a document is a sum of topics, a face a sum of nose, eyes, mouth. Very interpretable.
- Fitted by multiplicative updates or alternating non negative least squares. Not unique; non convex; components not orthogonal or ordered.
- Use for topic modelling (alongside LDA the topic model), audio source separation, image parts.

11.6 Matrix factorisation for recommendations

- Rating matrix R : users $\times$ items, mostly empty (a user rates 20 of 100,000 jobs). Goal: predict the blanks.
- Model: each user u gets a vector $p_u \in \mathbb{R}^k$, each item i a vector $q_i \in \mathbb{R}^k$. Prediction $\hat{r}_{ui} = p_u^\top q_i$ (plus biases: $\mu + b_u + b_i + p_u^\top q_i$).
- Intuition with $k = 2$: dimensions might be “seniority” and “tech vs non tech”. A senior tech candidate $p = (0.9, 0.8)$ matches a senior tech job $q = (1, 1)$ strongly.
- Loss only on observed entries: $\min \sum_{(u,i) \in \text{obs}} (r_{ui} - p_u^\top q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$. Plain SVD cannot be used because it needs a full matrix; filling blanks with zeros would teach the model that unseen means disliked.
- **SGD**: for each observed rating, $e = r_{ui} - p_u^\top q_i$; $p_u \leftarrow p_u + \eta(e q_i - \lambda p_u)$; $q_i \leftarrow q_i + \eta(e p_u - \lambda q_i)$ (use the old p_u).
- **ALS** (Alternating Least Squares): fix all q ; each p_u is then a ridge regression with closed form $p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u$. Then fix p and solve for q . Each step is embarrassingly parallel (Spark MLlib).
- Parameter count: $k(m + n)$ instead of mn . 10,000 users, 5,000 jobs, $k = 32$: 480,000 parameters vs 50 million cells.
- **Implicit feedback** (clicks, applies, no explicit ratings): treat interactions as preference with confidence weights (weighted ALS of Hu, Koren, Volinsky).
- **Cold start**: new users/items have no vectors. Fix with content features (skills, title embeddings), hybrid models, popularity fallbacks.

Where this shows up

- Naukri/InfoEdge: candidate $\times$ job interaction matrices for “Jobs you may like”; LSA/TruncatedSVD on resume TF-IDF; PCA before clustering candidate profiles.
- Deep learning: LoRA, low rank compression of embedding layers, whitening.

Formula sheet

$A = U \Sigma V^\top$; $\sigma_i = \sqrt{\lambda_i(A^\top A)}$; $\text{rank} = \#\{\sigma_i > 0\}$; $\|A\|_2 = \sigma_1$; $\|A\|_F^2 = \sum \sigma_i^2$.
 Best rank k error: $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$; storage $k(m + n + 1)$.
 PCA: variance of PC $i = \sigma_i^2 / (n - 1)$; $\text{EVR} = \lambda_i / \sum \lambda_j$.
 MF: $\hat{r} = \mu + b_u + b_i + p_u^\top q_i$; SGD $p \leftarrow p + \eta(e q - \lambda p)$; ALS $p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u$.

11.7 Question bank: Matrix factorisations

PYQ status for this chapter

InfoEdge reports mention PCA (Glassdoor DS interviews) and “PCA, t-SNE, LDA” (IIT BHU DS write up). SVD and eigendecomposition also appear in the Maths block of the same syllabus. No report quotes a recommender MF question verbatim; given InfoEdge’s recommendation products (Naukri, 99acres), the rest are framed in OA style.

Q11.1 MSQ · M

★ **PYQ (INT: explain PCA)** Which statements about PCA are true?

- (A) Principal components are eigenvectors of the covariance matrix of centred data.
- (B) PC1 is the direction of maximum variance.
- (C) PCA uses class labels.
- (D) Components are mutually orthogonal.

Solution

- The covariance eigenvectors (equivalently right singular vectors of centred X) are the PCs (A).
- PC1 maximises $\text{Var}(Xw)$ subject to $\|w\| = 1$; the solution is the top eigenvector (B).
- PCA is unsupervised (C false; that is LDA).
- Symmetric matrices have orthogonal eigenvectors (D).

Answer: A, B, D

Q11.2 MSQ · M

★ **PYQ (INT: PCA vs t-SNE vs LDA)** Which are correct?

- (A) LDA is supervised and gives at most $K - 1$ components for K classes.
- (B) t-SNE preserves local neighbourhoods and is mainly for visualisation.
- (C) t-SNE gives a reusable linear projection for new points.
- (D) PCA is linear and unsupervised.

Solution

- LDA maximises between class over within class scatter; the between class scatter has rank at most $K - 1$ (A).
- t-SNE matches neighbour probabilities in low dimensions (B). It learns a free embedding for the given points only, with no transform matrix (C false).
- PCA: linear, label free (D).

Answer: A, B, D

Q11.3 NAT · E

★ **PYQ (INT: explained variance in PCA)** Covariance eigenvalues are 6, 3, 1. What fraction of variance do the first two PCs explain?

Solution

- Total variance = $6 + 3 + 1 = 10$.
- First two: $(6 + 3)/10 = 0.9$, i.e. 90%.

Answer: 0.9

Q11.4 MSQ · M

★ **PYQ (INT: why standardise before PCA)** Why is standardisation often done before PCA?

- (A) Features on large scales would dominate the variance and hence PC1.
- (B) PCA requires features between 0 and 1.
- (C) So that components reflect correlation structure rather than units.
- (D) Standardising removes outliers.

Solution

- Salary in rupees has variance millions of times larger than years of experience, so PC1 would just be “salary” (A).
- After standardising, PCA works on the correlation matrix (C).
- No 0 to 1 requirement (B false); scaling does not remove outliers (D false).

Answer: A, C**Q11.5** MSQ · H★ **PYQ (INT: limitations of PCA)** Which are genuine limitations?

- (A) Captures only linear structure.
 (B) High variance direction may not separate classes.
 (C) Sensitive to outliers.
 (D) Cannot reduce dimensionality.

Solution

- Curved manifolds (swiss roll) are not unrolled (A). Two classes differing along a low variance axis get merged by PC1 (B). Squared deviations let one outlier rotate the axes (C).
- Reducing dimension is its purpose (D false).

Answer: A, B, C**Q11.6** NAT · ESingular values of $A = \begin{pmatrix} 3 & 0 \\ 0 & -4 \end{pmatrix}$, largest first?**Solution**

- $A^T A = \begin{pmatrix} 9 & 0 \\ 0 & 16 \end{pmatrix}$; eigenvalues 16 and 9.
- $\sigma = \sqrt{16}, \sqrt{9} = 4, 3$. Singular values are non negative; the sign goes into U or V .

Answer: 4, 3**Q11.7** NAT · ERank and largest singular value of $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$?**Solution**

- Rows are identical, so rank 1.
- $A^T A = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ with eigenvalues 4 and 0, so $\sigma_1 = 2, \sigma_2 = 0$.
- Check: $A = 2 \cdot uv^T$ with $u = v = (1, 1)/\sqrt{2}$.

Answer: rank 1, $\sigma_1 = 2$ **Q11.8** NAT · M

Singular values 5, 3, 1. Frobenius error of the best rank 1 approximation?

Solution

- Eckart Young: error = $\sqrt{\sigma_2^2 + \sigma_3^2} = \sqrt{9 + 1} = \sqrt{10}$.
- Spectral norm error would be $\sigma_2 = 3$.

Answer: $\sqrt{10} \approx 3.16$ **Q11.9** NAT · M

Same matrix (singular values 5, 3, 1): what fraction of the squared Frobenius norm is retained by rank 2?

Solution

- $\|A\|_F^2 = 25 + 9 + 1 = 35$. Rank 2 keeps $25 + 9 = 34$. Fraction $34/35 = 0.971$.

Answer: $34/35 \approx 0.971$

Q11.10 NAT · M

A 100×200 grayscale image is stored with rank $k = 10$ SVD. How many numbers are stored?

Solution

- k left vectors: 10×100 ; k right vectors: 10×200 ; k singular values: 10.
- Total = $1000 + 2000 + 10 = 3010$ vs 20,000 pixels (about 15%).

Answer: 3010

Q11.11 MSQ · M

About SVD, which are true?

- (A) It exists for every real matrix, square or not.
- (B) The number of non zero singular values equals the rank.
- (C) $\|A\|_2 = \sigma_1$.
- (D) Singular values can be negative.

Solution

- Existence for all matrices is SVD's big advantage over eigendecomposition (A).
- Rank = count of non zero σ (B); spectral norm = largest stretch = σ_1 (C).
- $\sigma_i = \sqrt{\lambda_i(A^T A)} \geq 0$ (D false).

Answer: A, B, C

Q11.12 MCQ · M

For a symmetric positive semidefinite matrix, singular values equal

- (A) the eigenvalues
- (B) the square roots of the eigenvalues
- (C) the diagonal entries
- (D) the absolute row sums

Solution

- For symmetric PSD $A = Q\Lambda Q^T$ with $\lambda \geq 0$, this is already an SVD with $U = V = Q$. So $\sigma_i = \lambda_i$.
- For symmetric but indefinite, $\sigma_i = |\lambda_i|$.

Answer: A

Q11.13 NAT · M

Centred data with $n = 101$ rows has top singular value $\sigma_1 = 20$. Variance along PC1?

Solution

- Variance = $\sigma_1^2 / (n - 1) = 400/100 = 4$.

Answer: 4

Q11.14 MCQ · M

Thin SVD of a 1000×50 full rank matrix has U, Σ, V of sizes

- (A) $1000 \times 1000, 1000 \times 50, 50 \times 50$
- (B) $1000 \times 50, 50 \times 50, 50 \times 50$
- (C) $50 \times 50, 50 \times 50, 1000 \times 50$
- (D) $1000 \times 50, 50 \times 1, 50 \times 1000$

Solution

- Only 50 singular values exist, so only 50 columns of U are needed; Σ is 50×50 ; V is 50×50 .

Answer: B**Q11.15** MSQ · M

Which decompositions fit which job?

- (A) Cholesky for sampling from a multivariate Gaussian with covariance Σ
- (B) QR for stable least squares
- (C) LU for solving $Ax = b$ repeatedly with the same A
- (D) NMF for orthogonal components

Solution

- $x = \mu + Lz$ with $\Sigma = LL^\top$ and $z \sim \mathcal{N}(0, I)$ gives covariance Σ (A).
- QR avoids squaring the condition number that normal equations cause (B).
- Factor once, then two cheap triangular solves per b (C).
- NMF components are non negative and generally not orthogonal (D false).

Answer: A, B, C**Q11.16** NAT · M

Condition number (2 norm) of a matrix with singular values 100, 10, 0.1?

Solution

- $\kappa = \sigma_{\max} / \sigma_{\min} = 100 / 0.1 = 1000$. Large κ means small input errors can be amplified about 1000 times; this is why ridge regularisation (which raises the smallest effective eigenvalues) stabilises regression.

Answer: 1000**Q11.17** MSQ · M

About NMF, which are true?

- (A) All factor entries are non negative.
- (B) It gives parts based, additive representations.
- (C) The factorisation is unique.
- (D) It is used for topic modelling of document term matrices.

Solution

- Non negativity (A) forbids cancellation, so representations are additive parts (B). Topics from word counts (D).
- Scaling W by D and H by D^{-1} gives another solution, and the objective is non convex (C false).

Answer: A, B, D**Q11.18** MCQ · M

Why can't we just run standard SVD on a sparse user item rating matrix?

- (A) SVD is too slow for any matrix
- (B) Standard SVD needs every entry; filling blanks with 0 treats unseen as disliked
- (C) SVD only works for square matrices
- (D) Ratings are integers

Solution

- Most entries are unknown, not zero. Imputing zeros biases every prediction toward zero. Recommender MF fits only observed entries.

Answer: B**Q11.19** NAT · EUser vector $p = (1, 2)$, item vector $q = (3, 0.5)$. Predicted rating $p^\top q$?

Solution

- $1 \times 3 + 2 \times 0.5 = 3 + 1 = 4$.

Answer: 4**Q11.20** NAT · M

With biases: global mean $\mu = 3.5$, user bias $b_u = 0.2$, item bias $b_i = -0.3$, $p_u^\top q_i = 0.4$. Prediction?

Solution

- $3.5 + 0.2 - 0.3 + 0.4 = 3.8$.
- Biases capture “this user rates high” and “this item is generally rated low”; the dot product captures the personal match.

Answer: 3.8**Q11.21** NAT · H

SGD step, no regularisation, $\eta = 0.1$. Rating $r = 4$, $p = (1, 0)$, $q = (2, 1)$. Give updated p and q (use the old p for q 's update).

Solution

- Prediction $= 1 \cdot 2 + 0 \cdot 1 = 2$; error $e = 4 - 2 = 2$.
- $p \leftarrow p + \eta eq = (1, 0) + 0.2(2, 1) = (1.4, 0.2)$.
- $q \leftarrow q + \eta ep_{old} = (2, 1) + 0.2(1, 0) = (2.2, 1)$.
- New prediction $1.4 \cdot 2.2 + 0.2 \cdot 1 = 3.28$, closer to 4.

Answer: $p = (1.4, 0.2)$, $q = (2.2, 1)$ **Q11.22** NAT · M

10,000 users, 5,000 jobs, $k = 32$ latent factors (no biases). Number of parameters?

Solution

- $k(m + n) = 32 \times 15,000 = 480,000$ vs 50,000,000 matrix cells.

Answer: 480,000**Q11.23** NAT · H

ALS with $k = 1$. Item factors of the items a user rated: $q = 1$ and $q = 2$ with ratings 3 and 4. With $\lambda = 1$, the user's factor?

Solution

- Ridge closed form in 1D: $p = \frac{\sum q_i r_i}{\sum q_i^2 + \lambda}$.
- Numerator $= 1 \cdot 3 + 2 \cdot 4 = 11$. Denominator $= 1 + 4 + 1 = 6$. $p = 11/6$.
- Without regularisation it would be $11/5 = 2.2$.

Answer: $11/6 \approx 1.83$ **Q11.24** MSQ · M

Why is ALS popular at scale?

- Each subproblem (fix one side) is convex with a closed form.
- Users can be solved in parallel given item factors.
- It handles implicit feedback with confidence weights naturally.
- It guarantees the global optimum.

Solution

- With Q fixed the loss is quadratic in each p_u (A); users are independent (B); weighted ALS for implicit data is standard (C).
- The joint problem is non convex, so only a local optimum is guaranteed (D false).

Answer: A, B, C

Q11.25 MSQ · M

Handling cold start for a brand new job posting with no interactions:

- (A) Use content features (title, skills embeddings) to build its vector.
- (B) Use popularity or recency based fallback.
- (C) Hybrid model combining collaborative and content signals.
- (D) Do nothing: MF handles it automatically.

Solution

- A new item has no learned q_i because it has no observed ratings. Content, popularity and hybrids fill the gap (A, B, C). D false.

Answer: A, B, C

Q11.26 MCQ · M

Implicit feedback (clicks) differs from explicit ratings mainly because

- (A) there are no negative signals, only interactions with varying confidence
- (B) implicit data are always larger than 5
- (C) implicit data cannot be factorised
- (D) it has no users

Solution

- A non click could mean dislike or never seen. Weighted MF treats all cells as preference 0 or 1 with confidence $1 + \alpha \cdot \text{count}$.

Answer: A

Q11.27 MCQ · M

Latent Semantic Analysis (LSA) is

- (A) K-Means on words
- (B) Truncated SVD of a document term (TF-IDF) matrix
- (C) A neural language model
- (D) NMF with labels

Solution

- Keeping the top k singular directions maps documents and words into a k dimensional “concept” space where synonyms land near each other.

Answer: B

Q11.28 MCQ · M

sklearn TruncatedSVD rather than PCA is recommended for sparse TF-IDF matrices because

- (A) it does not centre the data, so sparsity is preserved
- (B) it is always more accurate
- (C) PCA cannot reduce dimensions
- (D) it uses labels

Solution

- Centring subtracts a non zero mean from every zero, turning a sparse matrix dense (memory blow up). TruncatedSVD skips centring.

Answer: A

Q11.29 MSQ · M

Eigendecomposition vs SVD:

- (A) Eigendecomposition requires a square (diagonalisable) matrix.
- (B) SVD uses two (possibly different) orthogonal bases.
- (C) For non symmetric matrices eigenvectors need not be orthogonal.
- (D) They are always identical.

Solution

- Eigen: square and diagonalisable (A); eigenvectors orthogonal only for normal/symmetric matrices (C).
- SVD: U for the output space, V for the input space (B). They coincide only for symmetric PSD (D false).

Answer: A, B, C

Q11.30 NAT · M

$\|A\|_F$ for a matrix with singular values 3 and 4?

Solution

- $\|A\|_F = \sqrt{9 + 16} = 5$.

Answer: 5

Q11.31 MSQ · H

LoRA fine tuning replaces the update of a $d \times d$ weight ($d = 4096$) by $\Delta W = BA$ with $B: d \times r$, $A: r \times d$, $r = 8$. Which are true?

- (A) Trainable parameters drop from about 16.8M to about 65.5K for this matrix.
- (B) ΔW has rank at most 8.
- (C) The idea relies on the update being approximately low rank.
- (D) Inference must always be slower.

Solution

- Full: $4096^2 = 16,777,216$. LoRA: $2 \times 4096 \times 8 = 65,536$ (A).
- Product of rank r factors has rank at most r (B); low intrinsic rank of updates is the hypothesis (C).
- BA can be merged into W after training, so no extra latency (D false).

Answer: A, B, C

Q11.32 MCQ · H

The Moore Penrose pseudo inverse via SVD is

- (A) $U\Sigma V^\top$
- (B) $V\Sigma^+U^\top$ where Σ^+ inverts non zero singular values
- (C) Σ^{-1}
- (D) $A^\top$

Solution

- Invert each non zero σ_i , leave zeros, transpose shape. $x = A^+b$ gives the minimum norm least squares solution.

Answer: B

Q11.33 MSQ · M

Choosing k for PCA or truncated SVD:

- (A) Keep enough components for a target explained variance (for example 95%).
- (B) Look for the elbow in the scree plot.
- (C) Tune k by downstream validation performance.
- (D) Always use $k = 2$.

Solution

- All three are standard; $k = 2$ is for plotting only (D false).

Answer: A, B, C

Q11.34 MCQ · M

Reconstruction after PCA with k components: $\hat{x} = \bar{x} + V_k V_k^\top (x - \bar{x})$. If k equals the full dimension, the reconstruction error is

- (A) zero

- (B) equal to the variance
- (C) infinite
- (D) equal to σ_1

Solution

- $VV^\top = I$ when all orthonormal components are kept, so $\hat{x} = x$.

Answer: A**Q11.35** MCQ · H

Two features are perfectly correlated ($x_2 = 2x_1$). After PCA on these two (centred), the second eigenvalue is

- (A) 0
- (B) 1
- (C) equal to the first
- (D) negative

Solution

- All points lie on a line, so there is zero variance in the perpendicular direction. The covariance matrix has rank 1.
- This is how PCA removes multicollinearity.

Answer: A**Interview angle**

- Derive PCA as maximising variance with a Lagrange multiplier; show why it leads to eigenvectors of the covariance.
- Relation between SVD of X and eigendecomposition of $X^\top X$.
- How would you build “jobs you may like” with MF? Explicit vs implicit, cold start, biases, evaluation (Recall@K, NDCG).
- SGD vs ALS for MF; where does each shine?
- When would you use NMF over PCA? LSA vs word embeddings?

12 Density Estimation and Clustering: Histograms, KDE, K-Means, GMM and EM

Companion resource for this section: [StatQuest: K-means clustering](#)

12.1 What is density estimation?

- We see samples $x_1, \dots, x_n$ from an unknown distribution and want its density $p(x)$: where data are dense, where they are rare.
- Why care: anomaly detection (low $p(x)$ = suspicious login), generative modelling (sample new data), class conditional densities for Bayes classifiers, understanding shape (bimodal salaries?).
- **Parametric**: assume a family (Gaussian) and fit a few parameters by MLE. Fast, but wrong if the shape is wrong (a two humped distribution fit by one Gaussian).
- **Non parametric**: let the data speak: histograms, KDE, k nearest neighbour density. Flexible, but need more data and suffer badly in high dimensions.
- **Semi parametric / mixtures**: GMM, a sum of a few Gaussians; flexible yet compact.

12.2 Histograms: the simplest estimator

- Cut the line into bins of width h . Density in a bin = $\frac{\text{count in bin}}{n \cdot h}$ (dividing by nh makes total area 1).
- Example: 100 points, 20 fall in $[1, 1.5)$: density = $20 / (100 \times 0.5) = 0.4$.
- Problems: result depends on bin width and bin *origin*; blocky and discontinuous; number of bins explodes in high dimensions (10 bins per axis in 10 dimensions = 10^{10} bins).

12.3 Kernel Density Estimation (KDE)

- Idea: instead of fixed bins, put a small smooth “bump” (kernel) on every data point and add the bumps up.

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- K is a kernel: non negative, integrates to 1, usually symmetric. Gaussian $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$; box (uniform) $K(u) = \frac{1}{2}$ for $|u| \leq 1$; Epanechnikov $K(u) = \frac{3}{4}(1 - u^2)$ for $|u| \leq 1$ (MSE optimal).
- Worked example (box kernel, $h = 1$): data $\{1, 2, 4\}$, estimate at $x = 2.5$. Scaled distances: 1.5, 0.5, 1.5. Only the point 2 is within 1, contributing $K = 0.5$. $\hat{p}(2.5) = \frac{1}{3 \cdot 1} \times 0.5 = 1/6$.
- **Bandwidth h matters far more than the kernel choice.**
 - Small h : spiky, a bump per point, follows noise: low bias, high variance (overfitting).
 - Large h : oversmoothed, merges two modes into one: high bias, low variance.
 - Rules of thumb: Silverman $h = 1.06 \hat{\sigma} n^{-1/5}$ (or with $\min(\hat{\sigma}, IQR/1.34)$); or choose h by cross validated log likelihood.
- Cost: evaluating $\hat{p}$ at one point needs all n points (lazy like KNN); trees or FFT speed it up.
- In d dimensions the error converges like $n^{-4/(4+d)}$: data needed grows exponentially with d (curse of dimensionality). KDE is great in 1 to 3 dimensions.

12.4 K-Means clustering

- Goal: split points into K groups so that points are close to their group centre. Objective (inertia / within cluster sum of squares):

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2.$$

- **Lloyd’s algorithm**: (1) pick K initial centroids; (2) assign each point to its nearest centroid; (3) move each centroid to the mean of its points; repeat (2) and (3) until assignments stop changing.

- Worked 1D example: points $\{1, 2, 6, 7\}$, $K = 2$, start $c_1 = 1, c_2 = 2$.
 - Assign: $1 \rightarrow c_1; 2, 6, 7 \rightarrow c_2$. Update: $c_1 = 1, c_2 = 5$.
 - Assign: $1, 2 \rightarrow c_1$ (2 is at distance 1 from c_1 vs 3 from c_2); $6, 7 \rightarrow c_2$. Update: $c_1 = 1.5, c_2 = 6.5$.
 - Assign again: no change. Converged. Inertia = $0.25 \times 4 = 1$.
- Each step never increases J , so it converges, but only to a **local** minimum. Remedy: several restarts (`n_init`), and **k-means++** initialisation: first centroid random, each next one chosen with probability proportional to $D(x)^2$ (squared distance to nearest chosen centroid), which spreads centroids out.
- Complexity per iteration $O(nKd)$.
- Choosing K : elbow in the inertia vs K plot (inertia always falls as K grows, so never pick K by minimum inertia); **silhouette** $s = \frac{b-a}{\max(a,b)}$ where a = mean distance to own cluster, b = mean distance to the nearest other cluster; range $[-1, 1]$, higher is better; gap statistic; business needs.
- Assumptions/weaknesses: roughly spherical, similar size and density clusters; must scale features; sensitive to outliers (means); cannot find non convex shapes (two moons); Euclidean only. Variants: K-Medoids (robust, any distance), Mini Batch K-Means (big data).

12.5 Gaussian Mixture Models and EM

- Model: $p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$ with mixing weights $\pi_k \geq 0, \sum \pi_k = 1$. A *soft*, probabilistic cousin of K-Means that can model elliptical clusters of different sizes, and gives a real density.
- The hidden variable z (which component made the point) makes direct MLE hard. **EM** alternates:
 - **E step**: responsibilities $\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$ (posterior probability point i belongs to k).
 - **M step**: $N_k = \sum_i \gamma_{ik}; \pi_k = N_k/n; \mu_k = \frac{1}{N_k} \sum_i \gamma_{ik} x_i; \Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top$.
- Worked E step: 1D, $\pi = (0.5, 0.5), \mu = (0, 4), \sigma = 1$, point $x = 1$. Densities $\propto e^{-1/2}$ and $e^{-9/2}$. $\gamma_1 = \frac{1}{1+e^{-4}} = 0.982$.
- EM never decreases the likelihood; it converges to a local optimum. Initialise with K-Means. Choose K with BIC/AIC.
- K-Means is the limit of EM for GMM with equal spherical covariances $\sigma^2 I$ as $\sigma \rightarrow 0$ (responsibilities become hard 0/1).
- Danger: a component collapsing onto one point ($\sigma \rightarrow 0$, likelihood $\rightarrow \infty$); fix with a small covariance floor (`reg_covar`).

12.6 Other clustering methods worth knowing

- **DBSCAN**: density based. Core point: at least `minPts` neighbours within radius ϵ . Clusters = connected core points plus border points; the rest is *noise*. Finds arbitrary shapes, no K needed, flags outliers; struggles with varying densities.
- **Hierarchical (agglomerative)**: start with each point alone, repeatedly merge the closest pair; linkage = single (min, chaining), complete (max), average, Ward (minimise variance increase). Cut the dendrogram at a height to get clusters.
- **Mean shift**: move points uphill on the KDE surface until they reach a mode; each mode is a cluster.

Where this shows up

- Segmenting job seekers by behaviour (K-Means on engagement features), detecting fake recruiter accounts (low density = anomaly), KDE of salary distributions per role for “salary insights”.
- GMMs for speaker verification and background modelling; KDE for density based anomaly scores.

Formula sheet

Histogram: $\text{count}/(nh)$. KDE: $\hat{p}(x) = \frac{1}{nh} \sum K((x - x_i)/h)$; Silverman $h = 1.06\hat{\sigma}n^{-1/5}$.
 K-Means: $J = \sum \|x - \mu_{c(x)}\|^2$; k-means++ $P \propto D^2$; silhouette $(b - a) / \max(a, b)$.
 GMM: $\gamma_{ik} \propto \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$; $\pi_k = N_k/n$; $\mu_k = \sum \gamma_{ik} x_i / N_k$.
 GMM parameter count (full cov, d dims): $K[d + d(d + 1)/2] + (K - 1)$.

12.7 Question bank: Density estimation and clustering

PYQ status for this chapter

No InfoEdge report quotes a KDE or GMM question. Reported adjacent topics are outliers (asked with missing values) and unsupervised dimensionality reduction (PCA, t-SNE). K-Means and KDE are named in the official syllabus, so the questions below follow the OA format (MCQ, MSQ, NAT) with the frequently tested numericals first.

Q12.1 NAT · E

100 points; 20 lie in a histogram bin of width 0.5. Density estimate in that bin?

Solution

- Density = count / ($n \cdot h$) = $20 / (100 \times 0.5) = 20/50 = 0.4$.
- Check units: fraction 0.2 spread over width 0.5 gives height 0.4, so area = 0.2.

Answer: 0.4

Q12.2 NAT · E

Box kernel $K(u) = 1/2$ for $|u| \leq 1$, $h = 1$, data $\{1, 2, 4\}$. $\hat{p}(2.5)$?

Solution

- $(2.5 - 1)/1 = 1.5$ (outside), $(2.5 - 2) = 0.5$ (inside), $(2.5 - 4) = -1.5$ (outside).
- Sum of kernels = 0.5. $\hat{p} = \frac{1}{3 \times 1} \times 0.5 = 1/6$.

Answer: $1/6 \approx 0.167$

Q12.3 NAT · M

Gaussian kernel, $h = 1$, data $\{0, 2\}$. $\hat{p}(1)$? ($\varphi(1) \approx 0.242$)

Solution

- Both points are at scaled distance 1: kernel value $\varphi(1) = \frac{1}{\sqrt{2\pi}} e^{-1/2} = 0.3989 \times 0.6065 = 0.242$.
- $\hat{p}(1) = \frac{1}{2 \times 1} (0.242 + 0.242) = 0.242$.

Answer: ≈ 0.242

Q12.4 NAT · M

Silverman's rule $h = 1.06 \hat{\sigma} n^{-1/5}$ with $\hat{\sigma} = 2$, $n = 32$. h ?

Solution

- $32^{1/5} = 2$, so $n^{-1/5} = 0.5$.
- $h = 1.06 \times 2 \times 0.5 = 1.06$.

Answer: 1.06

Q12.5 MSQ · M

Effect of KDE bandwidth:

- (A) Very small h gives a spiky estimate with high variance.
- (B) Very large h can merge two real modes into one.
- (C) The kernel shape matters more than h .
- (D) h can be chosen by cross validated likelihood.

Solution

- Small h : each point its own spike (overfit) (A). Large h : oversmoothing, bias (B). Likelihood CV on held out points is standard (D).
- In practice Gaussian vs Epanechnikov changes little; h dominates (C false).

Answer: A, B, D

Q12.6 MSQ · M

Which are valid kernels for KDE?

- (A) Gaussian
- (B) Epanechnikov
- (C) Uniform (box)
- (D) $K(u) = u$

Solution

- A kernel must be non negative and integrate to 1. $K(u) = u$ is negative for $u < 0$ and does not integrate to 1 (D false).

Answer: A, B, C

Q12.7 MSQ · M

Histogram vs KDE:

- (A) Histograms depend on the bin origin; KDE does not have that issue.
- (B) KDE gives a smooth estimate with a smooth kernel.
- (C) Both need a smoothing parameter (bin width or bandwidth).
- (D) KDE works well in 100 dimensions with small data.

Solution

- Shifting the bin grid can change a histogram's shape; KDE centres a bump on each point (A). Smooth kernel $\Rightarrow$ smooth estimate (B). Width and bandwidth play the same role (C).
- Curse of dimensionality makes KDE poor in high d (D false).

Answer: A, B, C

Q12.8 MCQ · M

Parametric vs non parametric density estimation: which is non parametric?

- (A) Fitting one Gaussian by MLE
- (B) KDE
- (C) Fitting an exponential
- (D) Poisson MLE

Solution

- KDE's complexity grows with the data (it keeps every point); the others have a fixed number of parameters.

Answer: B

Q12.9 NAT · M

K-Means, 1D points $\{1, 2, 6, 7\}$, $K = 2$, initial centroids 1 and 2. Final inertia after convergence?

Solution

- Iteration 1: assign $1 \rightarrow 1$; $2, 6, 7 \rightarrow 2$. New centroids: 1 and $(2 + 6 + 7)/3 = 5$.
- Iteration 2: 2 is closer to 1 (distance 1) than 5 (distance 3). Clusters $\{1, 2\}$, $\{6, 7\}$. Centroids 1.5, 6.5.
- Iteration 3: unchanged. Inertia $= 0.5^2 \times 4 = 1$.

Answer: 1

Q12.10 NAT · E

Points $(0, 0)$, $(2, 0)$, $(0, 2)$, $(2, 2)$ in one cluster. Its centroid and inertia?

Solution

- Centroid = mean = $(1, 1)$.
- Each point is at squared distance $1 + 1 = 2$; four points give 8.

Answer: Centroid $(1, 1)$, inertia 8

Q12.11 NAT · M

Silhouette of a point with mean intra cluster distance $a = 2$ and nearest other cluster mean distance $b = 5$?

Solution

- $s = (b - a) / \max(a, b) = (5 - 2) / 5 = 0.6$. Positive and fairly large: well clustered.

Answer: 0.6

Q12.12 NAT · M

k-means++: after choosing the first centroid, three remaining points are at distances 1, 2, 3 from it. Probability the next centroid is the farthest point?

Solution

- Probabilities $\propto D^2$: 1, 4, 9, total 14.
- Farthest: 9/14. Far points are favoured, but not forced (so a single outlier is not always chosen).

Answer: $9/14 \approx 0.643$

Q12.13 MSQ · M

Which are true about K-Means?

- (A) Each Lloyd iteration never increases the inertia.
- (B) It always finds the global optimum.
- (C) Results can depend on initialisation.
- (D) Inertia keeps decreasing as K increases.

Solution

- Both the assignment and update steps can only lower (or keep) J (A). Hence convergence, but only to a local optimum (B false), so initialisation matters (C).
- More centroids can always fit better; at $K = n$ inertia is 0 (D). Hence the elbow method rather than minimum inertia.

Answer: A, C, D

Q12.14 MSQ · M

K-Means struggles with

- (A) non convex clusters like two interlocking moons
- (B) clusters of very different sizes and densities
- (C) unscaled features of very different ranges
- (D) well separated spherical blobs

Solution

- Voronoi boundaries are straight lines between centroids, so curved shapes get cut (A). Big clusters get split and small ones absorbed (B). Unscaled features dominate the Euclidean distance (C).
- Spherical blobs are its best case (D false).

Answer: A, B, C

Q12.15 MCQ · E

Time complexity of one K-Means iteration for n points, K clusters, d features?

- (A) $O(n^2d)$
- (B) $O(nKd)$
- (C) $O(K^2)$
- (D) $O(n \log n)$

Solution

- Each of n points computes distances to K centroids in d dimensions.

Answer: B

Q12.16 MCQ · M

Which method picks K by comparing within cluster dispersion to that expected under a reference null distribution?

- (A) Elbow
- (B) Silhouette
- (C) Gap statistic
- (D) BIC for K-Means

Solution

- The gap statistic compares $\log W_K$ with its expectation on uniformly generated reference data; pick the K with the largest gap (with a standard error rule).

Answer: C

Q12.17 NAT · M

GMM, 1D, $\pi = (0.5, 0.5)$, $\mu = (0, 4)$, $\sigma_1 = \sigma_2 = 1$. Responsibility of component 1 for $x = 1$ (3 decimals)?

Solution

- Unnormalised: $0.5 \cdot c e^{-(1-0)^2/2} = 0.5c e^{-0.5}$ and $0.5c e^{-(1-4)^2/2} = 0.5c e^{-4.5}$ (same constant c).
- Ratio: $\gamma_1 = \frac{e^{-0.5}}{e^{-0.5} + e^{-4.5}} = \frac{1}{1 + e^{-4}} = \frac{1}{1.0183} = 0.982$.

Answer: 0.982

Q12.18 NAT · M

Equal variance components with $\pi = (0.8, 0.2)$; a point exactly midway between the two means. γ_1 ?

Solution

- Equal distances and equal variances give equal Gaussian densities, which cancel. $\gamma_1 = 0.8 / (0.8 + 0.2) = 0.8$. The prior weight decides.

Answer: 0.8

Q12.19 NAT · M

M step: points 0, 2, 4 have responsibilities 1, 0.5, 0 for component 1. New μ_1 and π_1 ?

Solution

- $N_1 = 1 + 0.5 + 0 = 1.5$.
- $\mu_1 = (1 \cdot 0 + 0.5 \cdot 2 + 0 \cdot 4) / 1.5 = 1 / 1.5 = 0.667$.
- $\pi_1 = N_1 / n = 1.5 / 3 = 0.5$.

Answer: $\mu_1 = 2/3$, $\pi_1 = 0.5$

Q12.20 NAT · H

Number of free parameters of a GMM with $K = 3$ components, $d = 2$, full covariances?

Solution

- Each component: mean $d = 2$; symmetric covariance $d(d+1)/2 = 3$. So 5 per component, 15 total.
- Mixing weights: $K - 1 = 2$ free (they sum to 1).
- Total = 17. (This count enters BIC.)

Answer: 17

Q12.21 MSQ · M

EM for GMMs:

- (A) Never decreases the data log likelihood.
- (B) Converges to a local maximum, so initialisation matters.
- (C) E step computes responsibilities; M step updates π , μ , Σ .
- (D) Guarantees the global maximum.

Solution

- Monotone ascent (A) to a local optimum (B, D false); step roles as described (C).

Answer: A, B, C

Q12.22 MSQ · H

K-Means vs GMM:

- (A) GMM gives soft assignments; K-Means gives hard assignments.
- (B) GMM can model elliptical clusters with full covariances.
- (C) K-Means is a limiting case of EM for GMMs with shared spherical covariance shrinking to zero.
- (D) K-Means provides a proper probability density.

Solution

- Responsibilities vs argmin (A); Σ_k lets GMM stretch and rotate clusters (B); as $\sigma \rightarrow 0$ responsibilities become 0/1 and the M step becomes the centroid update (C).
- K-Means does not define a normalised density (D false).

Answer: A, B, C

Q12.23 MCQ · H

A GMM component's variance shrinks to zero on a single point during EM. What happens and what is the fix?

- (A) Likelihood goes to zero; increase K
- (B) Likelihood diverges to infinity (degenerate solution); add a small covariance floor
- (C) Nothing
- (D) EM switches to K-Means

Solution

- A Gaussian with $\sigma \rightarrow 0$ centred on a data point has density $\rightarrow \infty$ there, so the likelihood is unbounded. sklearn adds `reg_covar` to the diagonal.

Answer: B

Q12.24 MCQ · M

Which criterion is commonly used to pick the number of GMM components?

- (A) Accuracy
- (B) BIC
- (C) R squared
- (D) Silhouette only

Solution

- $\text{BIC} = -2 \log L + p \log n$ balances fit and parameter count p . Lower is better.

Answer: B

Q12.25 MSQ · M

DBSCAN:

- (A) Does not need the number of clusters.
- (B) Can find arbitrarily shaped clusters.
- (C) Labels low density points as noise.
- (D) Works equally well when clusters have very different densities.

Solution

- Clusters emerge from ϵ and minPts (A), follow dense regions of any shape (B), and non reachable points are noise (C).
- One global ϵ cannot suit both dense and sparse clusters (D false; HDBSCAN addresses this).

Answer: A, B, C

Q12.26 NAT · M

DBSCAN with $\epsilon = 1$ and $\text{minPts} = 3$ (counting the point itself). 1D points: 0, 0.5, 1, 5. How many core points?

Solution

- Point 0: neighbours within 1 are 0, 0.5, 1: 3, core.
- 0.5: 0, 0.5, 1: 3, core. 1: 0, 0.5, 1: 3, core.
- 5: only itself: noise. Core points = 3.

Answer: 3

Q12.27 MCQ · M

Single linkage hierarchical clustering is known for

- (A) compact spherical clusters
- (B) the chaining effect (long, straggly clusters)
- (C) needing K upfront
- (D) being identical to Ward

Solution

- Merging by the closest pair of points lets a chain of intermediate points join distant groups.

Answer: B

Q12.28 MCQ · M

Ward linkage merges the pair of clusters that

- (A) are farthest apart
- (B) causes the smallest increase in total within cluster variance
- (C) have the most points
- (D) have the smallest diameter

Solution

- Ward minimises the increase of the K-Means style objective at each merge, producing compact clusters.

Answer: B

Q12.29 MSQ · M

Using density estimates for anomaly detection:

- (A) Flag points with $\hat{p}(x)$ below a threshold.
- (B) Isolation Forest and LOF are alternatives.
- (C) Density must be estimated on data that are mostly normal.
- (D) High density means anomaly.

Solution

- Low density means rare (A); LOF compares local density to neighbours' (B); a model fit on contaminated data learns the anomalies as normal (C). D reversed.

Answer: A, B, C

Q12.30 MCQ · M

Mean shift clustering finds clusters by

- (A) random splits
- (B) moving points uphill to modes of a KDE
- (C) minimising inertia with fixed K
- (D) spectral decomposition

Solution

- Each point iteratively moves to the weighted mean of points in its window, climbing the density; points reaching the same mode form a cluster. K is implied by bandwidth.

Answer: B

Q12.31 MCQ · M

Why should features be scaled before K-Means?

- (A) K-Means requires integer features
- (B) Euclidean distance is dominated by features with large numeric ranges
- (C) Scaling reduces K
- (D) It makes clusters non convex

Solution

- Salary (lakhs) vs age (tens): unscaled, clusters are essentially salary bands.

Answer: B

Q12.32 MSQ · H

Gaussian Naive Bayes and density estimation:

- (A) It estimates a 1D Gaussian density per feature per class.
- (B) The class posterior uses Bayes' theorem with those densities.
- (C) Replacing Gaussians with 1D KDEs gives a flexible Naive Bayes.
- (D) It models full feature correlations.

Solution

- Per class, per feature Gaussians (A) combine under the independence assumption into $p(x | y)$, then Bayes gives $p(y | x)$ (B). Any 1D density estimator can replace them (C).
- Independence means no correlations (D false).

Answer: A, B, C

Q12.33 MCQ · M

The k nearest neighbour density estimate at x is roughly $k / (n \cdot V)$ where V is

- (A) the volume of the smallest ball around x containing k points
- (B) a fixed bin volume
- (C) the data variance
- (D) always 1

Solution

- Fix k and let the volume adapt: in dense regions the ball is small, so density is large.

Answer: A

Q12.34 MSQ · M

For customer segmentation you run K-Means with $K = 2..10$. Inertia drops sharply until $K = 4$ then slowly; the silhouette peaks at $K = 4$. Which are reasonable?

- (A) Choose $K = 4$.
- (B) Validate segments with business interpretability.
- (C) Choose $K = 10$ because inertia is lowest.
- (D) Check stability across random seeds.

Solution

- Elbow and silhouette agree on 4 (A). Useful segments must make business sense (B), and stable clusters across seeds are more trustworthy (D).
- Lowest inertia always favours the largest K (C false).

Answer: A, B, D

Q12.35 MSQ · M

Salaries for "Data Scientist" look bimodal (a fresher hump and a senior hump). Which estimates can represent this?

- (A) A single Gaussian fitted by MLE
- (B) A 2 component GMM

- (C) KDE with a moderate bandwidth
- (D) A histogram with sensible bins

Solution

- One Gaussian has one peak; it would put its peak in the valley between the humps, where few people actually are (A false).
- Two mixture components capture two humps (B). KDE and histograms are non parametric and follow whatever shape the data have, as long as the smoothing is not so large that the modes merge (C, D).

Answer: B, C, D

Interview angle

- Explain KDE to a non technical stakeholder; what does bandwidth do (bias variance)?
- K-Means algorithm, objective, convergence, k-means++, choosing K , failure cases.
- Derive EM updates for a GMM; relation to K-Means; why use BIC?
- When would you use DBSCAN over K-Means?
- How would you detect fraudulent recruiters with density based methods?

13 Basic NLP: Preprocessing, Bag of Words and TF-IDF

Companion resource for this section: [Jurafsky and Martin, Speech and Language Processing \(free online, Chapters 2, 4, 6\)](#)

13.1 Why text needs special treatment

- Models eat numbers. Text is a variable length sequence of symbols with huge vocabulary, spelling variants (“ML”, “machine learning”), and meaning that depends on order (“not good”).
- Classical NLP pipeline: raw text → clean/normalise → tokenise → (stem/lemmatise, remove stopwords) → vectorise (Bag of Words, TF-IDF) → model (Naive Bayes, Logistic Regression, SVM).
- Modern pipeline: raw text → subword tokeniser → embeddings → Transformer (Chapters 14 and 18). The classical one is still the strong, fast baseline asked in OAs.

13.2 Tokenisation (reported InfoEdge OA topic)

- **Tokenisation** = splitting text into units (tokens). “Don’t apply to 3 jobs!” → [Don’t] [apply] [to] [3] [jobs] [!] (word level) or [Do][n’t]... (Penn Treebank style).
- **Word level**: simple, but huge vocabulary and *out of vocabulary* (OOV) words at test time (“TensorFlowLite”).
- **Character level**: tiny vocabulary, no OOV, but very long sequences and weak semantics per token.
- **Subword** (BPE, WordPiece, Unigram/SentencePiece): frequent words stay whole, rare words split into pieces: “unhappiness” → “un”, “happi”, “ness”. No OOV, manageable vocabulary (30k to 100k). Used by BERT (WordPiece, continuation pieces marked “##”) and GPT (byte level BPE).
- **BPE in one line**: start with characters; repeatedly merge the most frequent adjacent pair into a new symbol; stop at the target vocabulary size.
- **Sentence tokenisation**: splitting into sentences (harder than splitting on “.” because of “Dr.”, “3.5”).

13.3 Normalisation, stopwords, stemming and lemmatisation (reported OA topic)

- **Normalisation**: lowercasing, removing punctuation/HTML, expanding contractions, unicode normalisation, handling numbers (replace with a token).
- **Stopwords**: very frequent function words (the, is, of, a). Removing them shrinks features for BoW models. *Do not* remove them blindly for sentiment (“not” is a stopword in many lists!) or for Transformers.
- **Stemming**: chop suffixes by crude rules to get a *stem*, which need not be a real word. Porter stemmer: “studies” → “studi”, “running” → “run”, “university” and “universe” → “univers” (over stemming). Fast, language specific rules, no dictionary.
- **Lemmatisation**: map a word to its dictionary form (*lemma*) using vocabulary and part of speech: “studies” → “study”, “better” → “good” (adjective), “was” → “be”. Slower, more accurate, output is a real word, needs POS for best results (“meeting” noun vs verb).
- Quick contrast: “caring”: stem “car” (wrong meaning), lemma “care”.
- Both reduce vocabulary so that variants share statistics; both lose some information.

13.4 Bag of Words (reported OA topic)

- Build a vocabulary of all distinct tokens. Represent each document by a vector of counts, one entry per vocabulary word. Word order is thrown away (“a bag”).
- Example: D1 = “python ml python”, D2 = “ml sql”. Vocabulary [ml, python, sql]. D1 = (1, 2, 0), D2 = (1, 0, 1).
- Variants: binary (presence/absence), counts, normalised frequencies.
- **n-grams** restore a little order: bigrams “not good”, “machine learning”. A sentence of T tokens has $T - n + 1$ n -grams. Vocabulary grows fast.
- Vectors are **sparse** and high dimensional (tens of thousands of columns, mostly zeros): stored as sparse matrices (CSR).
- Limits: no word order, no semantics (“car” and “automobile” are unrelated orthogonal axes), huge dimension, frequent words dominate raw counts.

13.5 TF-IDF: down weight words everyone uses

- Intuition: “the” appears in every resume, so it tells us nothing about a resume. “Kubernetes” appears in few, so when it appears it is informative. Weight = how much a word is used *here* $\times$ how rare it is *everywhere*.
- **TF** (term frequency) of term t in document d : raw count $f_{t,d}$, or $f_{t,d}/|d|$, or $1 + \log f_{t,d}$ (sublinear).
- **IDF** (inverse document frequency): $\text{idf}(t) = \log \frac{N}{\text{df}(t)}$, with N = number of documents and df = number of documents containing t . A word in all documents gets $\text{idf} = \log 1 = 0$.
- **TF-IDF** = $\text{tf} \times \text{idf}$.
- scikit-learn default (smooth_idf=True): $\text{idf}(t) = \ln \frac{1 + N}{1 + \text{df}(t)} + 1$ (the +1 inside avoids division by zero as if an extra document contained every word; the +1 outside keeps words in every document from being zeroed). Then each row is L2 normalised.
- Worked example (textbook version, $\log_{10}$): term appears 3 times in a 100 word document; appears in 100 of 10,000 documents. $\text{tf} = 3/100 = 0.03$; $\text{idf} = \log_{10}(10,000/100) = 2$; $\text{tf-idf} = 0.06$.
- Compare documents with **cosine similarity**: $\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$. Length independent, which matters because long documents have large counts.
- Uses: search ranking (BM25 is a tuned TF-IDF), resume to job matching baselines, keyword extraction, features for linear classifiers.

13.6 Classic text classifiers

- **Multinomial Naive Bayes**: $P(c | d) \propto P(c) \prod_{w \in d} P(w | c)$, with Laplace smoothing $P(w | c) = \frac{\text{count}(w, c) + 1}{\text{total words in } c + |V|}$. Fast, strong on short texts (spam).
- **Logistic Regression / Linear SVM** on TF-IDF: the standard strong baseline for intent, sentiment, category classification.
- Work in log space: summing $\log P$ avoids underflow from multiplying many small numbers.

13.7 Other basic NLP vocabulary

- **POS tagging** (noun, verb...), **NER** (named entities: person, organisation, skill, location), **dependency parsing**, **chunking**.
- **n-gram language model**: $P(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$ (bigram MLE); smoothing (Laplace, Kneser Ney) for unseen pairs. **Perplexity** = $P(w_1..w_N)^{-1/N}$: lower is better; a model uniformly guessing among V words has perplexity V .
- **Edit (Levenshtein) distance**: minimum insertions, deletions, substitutions to turn one string into another (spell correction, fuzzy matching of skills “pyhton” $\rightarrow$ “python”).
- **Jaccard similarity**: $|A \cap B| / |A \cup B|$ on token sets.

Where this shows up

- Naukri: skill extraction from resumes (tokenise, normalise “ML”/“machine learning”, NER), TF-IDF/BM25 job search, fuzzy matching of misspelt skills (edit distance), resume classification with TF-IDF + Logistic Regression.
- InfoEdge coding round Part 1 lists “basic text processing” with Pandas: lowercasing, splitting, counting words, `str.contains`.

Formula sheet

BoW: count vector. n-grams in T tokens: $T - n + 1$.
 $\text{idf} = \log(N/\text{df})$; sklearn smooth: $\ln \frac{1+N}{1+\text{df}} + 1$, then L2 row normalisation. $\text{tf-idf} = \text{tf} \times \text{idf}$.
 Cosine = $a \cdot b / (\|a\| \|b\|)$. Laplace NB: $(c + 1) / (N_c + |V|)$. Bigram MLE $c(w_1w_2) / c(w_1)$. Perplexity = $P^{-1/N}$.

13.8 Question bank: Basic NLP and TF-IDF

PYQ status for this chapter

The IIT BHU InfoEdge DS OA (40 MCQs) reportedly included bag of words and questions distinguishing stemming, lemmatisation and tokenisation; Glassdoor InfoEdge interviews mention NLP generally. The coding round syllabus includes basic text processing. These come first.

Q13.1 MCQ · E

★ **PYQ (OA: bag of words)** The Bag of Words representation of a document

- (A) keeps the exact word order
- (B) counts occurrences of vocabulary words, ignoring order
- (C) is a dense 300 dimensional embedding
- (D) stores only the first word

Solution

- BoW maps a document to a vector of term counts (or presence) over the vocabulary; order is discarded, so “dog bites man” = “man bites dog”.

Answer: B

Q13.2 NAT · E

★ **PYQ (OA: bag of words vector)** Vocabulary (in order) [ml, python, sql]. BoW count vector for “python ml python sql python”?

Solution

- ml: 1; python: 3; sql: 1.

Answer: (1, 3, 1)

Q13.3 MCQ · E

★ **PYQ (OA: stemming vs lemmatisation)** Which converts “better” to “good”?

- (A) Stemming
- (B) Lemmatisation
- (C) Tokenisation
- (D) Stopword removal

Solution

- A lemmatiser looks up the dictionary base form (with POS adjective): “better” → “good”. A stemmer only strips suffixes and cannot relate “better” to “good”.

Answer: B

Q13.4 MSQ · M

★ **PYQ (OA: stemming, lemmatisation, tokenisation)** Which statements are true?

- (A) Stemming may produce non words such as “studi”.
- (B) Lemmatisation returns valid dictionary words.
- (C) Tokenisation splits text into units like words or subwords.
- (D) Stemming is generally slower than lemmatisation.

Solution

- Porter maps “studies” → “studi” (A). Lemmas are dictionary entries (B). Tokenisation defines the units (C).
- Stemming is rule based and faster; lemmatisation needs lookups and often POS tagging (D false).

Answer: A, B, C

Q13.5 MCQ · M

★ **PYQ (OA: lemmatisation needs context)** Lemmatising “meeting” correctly depends most on

- (A) its length
- (B) its part of speech in the sentence
- (C) its TF-IDF
- (D) the stopwords list

Solution

- “The meeting was long” (noun, lemma “meeting”) vs “I am meeting her” (verb, lemma “meet”).

Answer: B**Q13.6** MSQ · M★ **PYQ (OA: limitations of bag of words)** Limitations of BoW include

- (A) loss of word order
- (B) no notion of semantic similarity between different words
- (C) very high dimensional sparse vectors
- (D) cannot be fed to Logistic Regression

Solution

- Order lost (A); “car” and “automobile” are orthogonal axes (B); vocabulary sized sparse vectors (C).
- BoW + Logistic Regression is a classic baseline (D false).

Answer: A, B, C**Q13.7** MSQ · M★ **PYQ (Coding round: basic text processing)** In pandas, which help count word frequencies in a text column `df['desc']`?

- (A) `df['desc'].str.lower().str.split().explode().value_counts()`
- (B) `CountVectorizer().fit_transform(df['desc'])`
- (C) `df['desc'].mean()`
- (D) `collections.Counter(" ".join(df['desc']).lower().split())`

Solution

- A: lowercase, split into lists, one row per word, count.
- B: sklearn document term matrix; sum columns for totals.
- D: plain Python counter.
- C: mean of strings is an error.

Answer: A, B, D**Q13.8** NAT · E10 documents; a term appears in 1 of them. idf with $\log_{10}(N/df)$?**Solution**

- $\log_{10}(10/1) = 1$.

Answer: 1**Q13.9** NAT · ESame corpus; a term appears in all 10 documents. idf ($\log_{10}(N/df)$)?**Solution**

- $\log_{10}(10/10) = 0$: the term carries no discriminating information.

Answer: 0**Q13.10** NAT · MA term occurs 3 times in a 100 word document and appears in 100 of 10,000 documents. tf-idf with $tf = f/|d|$ and $\log_{10}$ idf?

Solution

- $tf = 3/100 = 0.03$.
- $idf = \log_{10}(10,000/100) = \log_{10} 100 = 2$.
- $tf-idf = 0.03 \times 2 = 0.06$.

Answer: 0.06**Q13.11** NAT · Mscikit-learn smoothed idf, $N = 3$ documents, $df = 1$. idf (natural log, 3 decimals)?**Solution**

- $\ln \frac{1+3}{1+1} + 1 = \ln 2 + 1 = 0.693 + 1 = 1.693$.

Answer: 1.693**Q13.12** NAT · Mscikit-learn smoothed idf for a term in all $N = 3$ documents?**Solution**

- $\ln \frac{4}{4} + 1 = 0 + 1 = 1$. Unlike the textbook formula, the term is not zeroed out, only given the minimum weight.

Answer: 1**Q13.13** NAT · ECosine similarity between BoW vectors $(1, 1, 0)$ and $(1, 0, 1)$?**Solution**

- Dot product = 1. Norms = $\sqrt{2}$ each. $\cos = 1/2$.

Answer: 0.5**Q13.14** MCQ · M

Why is cosine similarity preferred over Euclidean distance for TF-IDF document vectors?

- (A) It is faster to compute
- (B) It ignores vector length, so a long and a short document on the same topic can still match
- (C) It always returns integers
- (D) Euclidean distance is undefined for sparse vectors

Solution

- A document repeated twice doubles every count; Euclidean distance changes but the angle does not.
- After L2 normalisation, ranking by cosine and by Euclidean distance becomes equivalent.

Answer: B**Q13.15** NAT · E

How many bigrams are in a 7 token sentence? How many trigrams?

Solution

- $T - n + 1$: $7 - 2 + 1 = 6$; $7 - 3 + 1 = 5$.

Answer: 6 bigrams, 5 trigrams**Q13.16** MSQ · M

Using bigrams in addition to unigrams

- (A) captures short phrases like “not good”
- (B) increases the vocabulary size substantially
- (C) fully captures long range word order
- (D) can improve sentiment classification

Solution

- Local order is captured (A, D) at the cost of many more features (B). Long range dependencies are still missed (C false).

Answer: A, B, D

Q13.17 MCQ · M

Removing stopwords is risky for which task?

- (A) Topic classification of news
- (B) Sentiment analysis where “not” matters
- (C) Keyword search indexing
- (D) Document clustering

Solution

- Many stopword lists include “not”, “no”, “nor”. Removing them flips “not good” into “good”.

Answer: B

Q13.18 MSQ · M

Subword tokenisation (BPE, WordPiece):

- (A) Largely eliminates out of vocabulary tokens.
- (B) Splits rare words into frequent pieces.
- (C) Is used by BERT and GPT style models.
- (D) Requires a stemmer first.

Solution

- Any word decomposes into known pieces, down to characters or bytes (A, B). BERT uses WordPiece, GPT byte level BPE (C). No stemming needed (D false).

Answer: A, B, C

Q13.19 NAT · H

BPE on the corpus {“low” ×5, “lower” ×2, “newest” ×6, “widest” ×3} (characters as initial symbols). Frequency of the pair (e, s)?

Solution

- “es” occurs once in “newest” (6 times) and once in “widest” (3 times): $6 + 3 = 9$.
- (s, t) also has 9; (l, o) has $5 + 2 = 7$; (w, e) has $2 + 6 = 8$ (“lower” and “newest”). So “es” (or “st”, tie broken by rule) is merged first.

Answer: 9

Q13.20 MCQ · M

Over stemming means

- (A) different words with different meanings are reduced to the same stem
- (B) a word is not reduced at all
- (C) stopwords are kept
- (D) lemmas are wrong POS

Solution

- “universe” and “university” → “univers”. Under stemming is the opposite: related forms not merged (“data”, “datum”).

Answer: A

Q13.21 NAT · M

Bigram MLE on the corpus “I love NLP . I love ML .”. Compute $P(\text{NLP} \mid \text{love})$.

Solution

- $c(\text{love}) = 2$; $c(\text{love NLP}) = 1$.
- $P = 1/2$.

Answer: 0.5**Q13.22** NAT · M

Laplace smoothed bigram: $c(w_1) = 8$, $c(w_1 w_2) = 0$, vocabulary $|V| = 12$. $P(w_2 | w_1)$?

Solution

- $(0 + 1)/(8 + 12) = 1/20 = 0.05$. Without smoothing it would be 0, making any sentence containing that pair impossible.

Answer: 0.05**Q13.23** NAT · M

A language model assigns probability $1/4$ to every token of a 5 token test sentence. Perplexity?

Solution

- $P = (1/4)^5$. Perplexity $= P^{-1/5} = 4$. Interpretation: as uncertain as choosing among 4 equally likely words.

Answer: 4**Q13.24** NAT · M

Levenshtein distance between “kitten” and “sitting”?

Solution

- $k \rightarrow s$ (substitute), $e \rightarrow i$ (substitute), insert g at the end. Three edits, and no shorter sequence exists.

Answer: 3**Q13.25** NAT · E

Jaccard similarity of skill sets {python, sql, ml} and {python, ml, spark, aws}?

Solution

- Intersection {python, ml}: 2. Union {python, sql, ml, spark, aws}: 5. Jaccard $= 2/5$.

Answer: 0.4**Q13.26** NAT · H

Multinomial Naive Bayes with Laplace smoothing. Class “spam” has 20 total word tokens; the word “offer” appears 4 times in spam; $|V| = 30$. $P(\text{offer} | \text{spam})$?

Solution

- $(4 + 1)/(20 + 30) = 5/50 = 0.1$.

Answer: 0.1**Q13.27** MSQ · M

Why compute Naive Bayes in log space?

- (A) Products of many small probabilities underflow to 0.
- (B) Sums of logs are numerically stable.
- (C) It changes which class wins.
- (D) It turns the model into a linear classifier in log counts.

Solution

- Multiplying 500 probabilities of 0.001 gives 10^{-1500} , below floating point range (A); logs turn it into a sum (B). Since log is monotone the argmax is unchanged (C false). $\log P(c) + \sum_w n_w \log P(w | c)$ is linear in counts (D).

Answer: A, B, D

Q13.28 NAT · M

10,000 resumes, vocabulary 50,000, each resume has about 100 distinct words. Fraction of non zero entries in the document term matrix?

Solution

- Non zeros per row ≈ 100 out of 50,000: $100/50,000 = 0.002$.
- Dense storage would need 5×10^8 numbers; sparse CSR stores about 10^6 values.

Answer: 0.002 (0.2%)

Q13.29 MSQ · M

Which settings of TfidfVectorizer reduce vocabulary noise?

- (A) min_df=5
- (B) max_df=0.9
- (C) stop_words='english'
- (D) ngram_range=(1,5)

Solution

- Drop very rare terms (A), near universal terms (B), and stopwords (C).
- Up to 5 grams massively *grows* the vocabulary (D false).

Answer: A, B, C

Q13.30 MCQ · M

sublinear_tf=True replaces tf by $1 + \log(\text{tf})$. Purpose?

- (A) To give 20 occurrences far less than 20 times the weight of 1 occurrence
- (B) To remove idf
- (C) To make vectors dense
- (D) To add bigrams

Solution

- Repeating a keyword 20 times (keyword stuffing) should not make a resume 20 times more relevant. $1 + \ln 20 \approx 4$.

Answer: A

Q13.31 MSQ · M

Named Entity Recognition on resumes would tag

- (A) "Infosys" as an organisation
- (B) "Bengaluru" as a location
- (C) "the" as a person
- (D) skills like "PyTorch" with a custom SKILL label

Solution

- Standard types ORG and LOC (A, B); custom entity types are trained for domain needs (D). Function words are not entities (C false).

Answer: A, B, D

Q13.32 MCQ · M

In a leak free text classification pipeline, the TF-IDF vocabulary and idf values should be learned from

- (A) train + test
- (B) training data only
- (C) test only
- (D) a random half of test

Solution

- idf depends on document frequencies; including test documents leaks information. Use `Pipeline([('tfidf', TfidfVectorizer()), ('clf', LogisticRegression())])`.

Answer: B**Q13.33** MCQ · M

BM25 improves over plain TF-IDF for search mainly by

- (A) using word embeddings
- (B) saturating term frequency and normalising for document length
- (C) removing idf
- (D) sorting alphabetically

Solution

- BM25 tf term $\frac{f(k_1+1)}{f+k_1(1-b+b|d|/avgdl)}$ saturates as f grows and penalises long documents via b .

Answer: B**Q13.34** MSQ · H

You classify job titles into 500 categories. Which classical baselines are sensible?

- (A) TF-IDF with character n-grams + Linear SVM
- (B) Multinomial Naive Bayes on word counts
- (C) K-Means on raw strings
- (D) Logistic Regression (one vs rest) on TF-IDF

Solution

- Character n-grams handle typos and abbreviations ("Sr. SDE"); linear models scale to many classes (A, D); NB is a fast baseline (B).
- K-Means is unsupervised and needs numeric vectors (C false).

Answer: A, B, D**Q13.35** MCQ · E

Which is a word level normalisation step?

- (A) Lowercasing
- (B) Backpropagation
- (C) Dropout
- (D) Pooling

Solution

- "Python" and "python" become the same token. (Keep case if it matters, e.g. NER.)

Answer: A**Interview angle**

- Walk through preprocessing for resume skill extraction; which steps would you skip for a BERT model and why?
- Stemming vs lemmatisation with examples; when is each preferable?
- Derive TF-IDF, explain idf intuitively, and why L2 normalisation plus cosine.
- BoW limitations and how embeddings fix them.
- How does BPE build its vocabulary? Why do modern models use subwords?

14 Embeddings: Word2Vec, GloVe, FastText and BERT

Companion resource for this section: [Jay Alammar: The Illustrated Word2vec](#)

14.1 From one hot to dense vectors

- One hot: vocabulary of V words, each word is a V dimensional vector with a single 1. “python” = $(0, \dots, 1, \dots, 0)$. Every pair of words is equally far apart: no notion that “python” is closer to “java” than to “banana”.
- **Embedding**: a dense, low dimensional vector (50 to 1024 numbers) per word, learned so that similar words get similar vectors. Similarity is measured with cosine.
- Guiding idea, the **distributional hypothesis**: “you shall know a word by the company it keeps”. Words appearing in similar contexts (“I code in __”: python, java, go) should have similar vectors.
- An embedding layer is just a lookup table $E \in \mathbb{R}^{V \times d}$: row i is word i ’s vector. Multiplying a one hot vector by E selects that row. Parameters = $V \cdot d$ (10,000 words $\times$ 300 dims = 3 million).

14.2 Word2Vec (Mikolov et al., 2013)

- A shallow network with one hidden (projection) layer and no non linearity, trained on a fake task whose by product is the embeddings.
- **Skip-gram**: given the centre word, predict each context word within a window. “the senior data scientist” with window 1: pairs (senior, the), (senior, data).
- **CBOW** (Continuous Bag of Words): average the context vectors and predict the centre word. Faster, smooths over contexts, good for frequent words. Skip-gram works better for rare words and small data.
- Model: $P(o | c) = \frac{\exp(u_o^\top v_c)}{\sum_{w=1}^V \exp(u_w^\top v_c)}$ with two vectors per word (v as centre, u as context). The denominator costs $O(V)$ per pair: too slow.
- **Negative sampling**: turn it into binary classification. For a true pair maximise $\log \sigma(u_o^\top v_c)$; for k random “negative” words maximise $\log \sigma(-u_n^\top v_c)$. Cost $O(k + 1)$ with $k = 5$ to 20. Negatives drawn from the unigram distribution raised to 3/4 (boosts rare words a bit).
- **Hierarchical softmax**: a binary tree over the vocabulary makes the softmax $O(\log V)$.
- **Subsampling frequent words**: discard occurrences of word w with probability $1 - \sqrt{t/f(w)}$ ($t \approx 10^{-5}$), so “the” does not dominate training.
- Famous property: linear analogies. $v_{king} - v_{man} + v_{woman} \approx v_{queen}$. Nearest neighbour search by cosine, excluding the query words.
- Counting training pairs (skip-gram, window m on each side): each word pairs with up to $2m$ neighbours; edge words have fewer.

14.3 GloVe

- Global Vectors: build a word word co occurrence matrix X (how often j appears near i in the whole corpus), then fit $w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$ with a weighting that down weights rare and caps very frequent pairs.
- Count based (global statistics) like LSA, but with Word2Vec quality analogies. Word2Vec is prediction based (local windows).

14.4 FastText: embeddings from character n-grams

- Each word is a bag of character n-grams (typically $n = 3$ to 6) plus the word itself, with boundary symbols. “where”, $n = 3$: <wh, whe, her, ere, re> and <where>.
- Word vector = sum of its n-gram vectors. Trained with skip-gram and negative sampling.
- Wins: **out of vocabulary words** get vectors from their n-grams (“pyspark3” shares n-grams with “pyspark”); robust to typos; good for morphologically rich languages (Hindi, Finnish, Turkish).
- Also a fast linear text classifier (average of word/n-gram embeddings $\rightarrow$ softmax).

14.5 Static vs contextual embeddings

- Word2Vec, GloVe, FastText are **static**: one vector per word type. “bank” in “river bank” and “bank loan” gets the same vector, a blend of meanings.
- ELMo (bi-LSTM) and then **BERT** (Transformer encoder) produce **contextual** embeddings: a different vector for each occurrence, depending on the sentence.

14.6 BERT (Devlin et al., 2018)

- **Bidirectional Encoder Representations from Transformers**: a stack of Transformer encoder layers (Chapter 18) that reads the whole sentence at once, attending left and right.
- **Sizes**: BERT-base 12 layers, hidden 768, 12 heads (64 dims each), about 110M parameters; BERT-large 24 layers, 1024 hidden, 16 heads, about 340M. Max length 512 tokens. WordPiece vocabulary about 30,000.
- **Input** = token embedding + **segment** embedding (sentence A or B) + **position** embedding (learned). Special tokens: [CLS] at the start (its final vector summarises the sequence for classification), [SEP] between/after sentences, [MASK], [PAD].
- **Pre-training task 1, Masked Language Modelling (MLM)**: choose 15% of tokens; of these, 80% replaced by [MASK], 10% by a random token, 10% left unchanged; predict the originals. The 80/10/10 split reduces mismatch with fine tuning (where [MASK] never appears).
- **Pre-training task 2, Next Sentence Prediction (NSP)**: is sentence B the real next sentence (50%) or random (50%)? Later work (RoBERTa) showed NSP is not needed.
- **Fine tuning**: add a small head (for example a linear layer on [CLS]) and train all weights on the task with a small learning rate (2e-5 to 5e-5) for a few epochs. Or use frozen BERT features.
- Why not “bidirectional” language modelling with next word prediction? A word could trivially see itself through the other direction. Masking solves that.
- **Family**: RoBERTa (more data, no NSP, dynamic masking), DistilBERT (40% smaller, about 97% of performance via distillation), ALBERT (parameter sharing, factorised embeddings), domain BERTs (BioBERT, FinBERT).
- **BERT vs GPT**: BERT = encoder, bidirectional, for understanding (classification, NER, retrieval). GPT = decoder, causal (left to right), for generation.

14.7 Sentence and document embeddings

- **Averaging word vectors**: simple strong baseline (loses order; weight by TF-IDF for better results).
- **Sentence-BERT**: fine tune BERT in a siamese (twin) setup so that cosine between pooled outputs reflects semantic similarity. Encode every job description once, store vectors, then search by nearest neighbours (FAISS). This is a **bi-encoder**.
- **Cross-encoder**: feed both texts together into BERT and output a score. More accurate but must run for every pair; use it to re-rank the top results from a bi-encoder.

Where this shows up

- Job to resume semantic matching (“ML engineer” $\approx$ “machine learning developer”), skill normalisation via nearest neighbours in embedding space, FastText for misspelt skills, BERT fine tuned for job title classification, bi-encoder retrieval plus cross-encoder re-ranking for search.

Formula sheet

Embedding params $V \times d$ (Word2Vec keeps two tables: $2Vd$).

Skip-gram negative sampling loss per pair: $-\log \sigma(u_o^\top v_c) - \sum_k \log \sigma(-u_{n_k}^\top v_c)$. Noise $\propto f^{3/4}$. Subsampling discard prob $1 - \sqrt{t/f}$.

FastText: word = sum of char n-gram vectors ($n = 3..6$) + word vector.

BERT-base: 12 layers, 768 hidden, 12 heads, 110M; MLM 15% (80/10/10); input = token + segment + position.

14.8 Question bank: Embeddings

PYQ status for this chapter

The IIT Roorkee InfoEdge OA (Scribd) lists word embeddings among the Deep Learning questions; Glassdoor and the IIT KGP Foresight 2025 report emphasise NLP and Transformers (BERT) in interviews. The syllabus names Word2Vec, FastText and BERT. PYQ tagged items come first.

Q14.1 MSQ · M

★ **PYQ (OA (IITR): word embeddings)** Which statements about word embeddings are true?

- (A) They are dense, low dimensional vectors.
- (B) Words used in similar contexts get similar vectors.
- (C) One hot vectors capture semantic similarity.
- (D) They can be learned as a by product of a prediction task.

Solution

- Dense (A), distributional hypothesis (B), Word2Vec's fake prediction task (D).
- One hot vectors are all orthogonal: cosine 0 between any two different words (C false).

Answer: A, B, D

Q14.2 MCQ · E

★ **PYQ (OA (IITR): word embeddings, CBOW vs skip-gram)** Skip-gram Word2Vec

- (A) predicts the centre word from its context
- (B) predicts context words from the centre word
- (C) counts co occurrences globally
- (D) uses character n-grams

Solution

- CBOW does the opposite (A). GloVe uses global counts (C). FastText uses n-grams (D).

Answer: B

Q14.3 MSQ · M

★ **PYQ (INT: BERT basics)** Which are true about BERT?

- (A) It uses a Transformer encoder and attends in both directions.
- (B) It is pre-trained with masked language modelling.
- (C) It produces the same vector for "bank" in every sentence.
- (D) The [CLS] token's final vector is commonly used for classification.

Solution

- Encoder only, bidirectional (A); MLM (plus NSP originally) (B); [CLS] pooled output feeds a classifier (D).
- BERT embeddings are contextual (C false).

Answer: A, B, D

Q14.4 MSQ · M

★ **PYQ (INT: Word2Vec vs FastText vs BERT)** Match the property to the model. Which pairings are correct?

- (A) FastText: vectors for unseen words from character n-grams
- (B) Word2Vec: contextual vectors
- (C) BERT: contextual vectors from a deep Transformer
- (D) GloVe: global co occurrence statistics

Solution

- Word2Vec is static: one vector per word type (B false). Others correct.

Answer: A, C, D

Q14.5 NAT · E

Vocabulary 10,000, embedding dimension 300. Parameters in one embedding table?

Solution

- $V \times d = 10,000 \times 300 = 3 \times 10^6$. Word2Vec trains two such tables (centre and context), 6 million in total, and usually keeps the centre one.

Answer: 3,000,000

Q14.6 NAT · M

Skip-gram with window 2 (two words each side) on a 5 word sentence. Number of (centre, context) training pairs?

Solution

- Positions 1 to 5 have neighbours within 2: position 1: 2; position 2: 3; position 3: 4; position 4: 3; position 5: 2.
- Total $2 + 3 + 4 + 3 + 2 = 14$.

Answer: 14

Q14.7 NAT · M

Negative sampling distribution $\propto f^{3/4}$. Word A has count 16, word B count 1 (only two words). Probability of sampling B?

Solution

- $16^{3/4} = (2^4)^{3/4} = 2^3 = 8$; $1^{3/4} = 1$.
- $P(B) = 1/(8 + 1) = 1/9$. With raw frequency it would be $1/17 \approx 0.059$: the $3/4$ power boosts rare words.

Answer: $1/9 \approx 0.111$

Q14.8 NAT · M

Subsampling: discard probability $1 - \sqrt{t/f}$ with $t = 10^{-5}$ and word frequency $f = 0.01$. Value (3 decimals)?

Solution

- $t/f = 10^{-3}$; $\sqrt{10^{-3}} = 0.0316$.
- $1 - 0.0316 = 0.968$: about 97% of occurrences of such a common word are skipped.

Answer: 0.968

Q14.9 MCQ · M

Why does full softmax make naive Word2Vec slow?

- (A) Its denominator sums over the entire vocabulary for every training pair
- (B) It has no parameters
- (C) It requires labels
- (D) It is non differentiable

Solution

- $O(V)$ per pair with V in the hundreds of thousands. Negative sampling reduces it to $O(k)$; hierarchical softmax to $O(\log V)$.

Answer: A

Q14.10 NAT · M

With negative sampling $k = 5$, how many dot products (score computations) are needed per positive pair (ignoring the softmax alternative)?

Solution

- One for the true context word and one for each of 5 negatives: $1 + 5 = 6$, vs V for full softmax.

Answer: 6**Q14.11** NAT · M

2D toy vectors: king (3,4), man (2,1), woman (2,3). Compute king – man + woman.

Solution

- $(3 - 2 + 2, 4 - 1 + 3) = (3, 6)$. The nearest vocabulary vector (excluding king, man, woman) by cosine is predicted as the answer (ideally queen).

Answer: (3,6)**Q14.12** NAT · E

Cosine similarity of embeddings (1,2,2) and (2,1,2)?

Solution

- Dot = $2 + 2 + 4 = 8$. Norms = 3 and 3. Cosine = $8/9$.

Answer: $8/9 \approx 0.889$ **Q14.13** MSQ · M

CBOW vs skip-gram:

- (A) CBOW is faster to train.
- (B) Skip-gram tends to represent rare words better.
- (C) CBOW averages context vectors.
- (D) Skip-gram cannot use negative sampling.

Solution

- CBOW makes one prediction per window (A), averaging contexts (C); skip-gram makes one per context word, giving rare words more updates (B).
- Negative sampling was introduced with skip-gram (D false).

Answer: A, B, C**Q14.14** NAT · M

FastText with $n = 3$ and boundary symbols $\langle, \rangle$: how many character trigrams for “where” (not counting the whole word token)?

Solution

- Padded string $\langle \text{where} \rangle$ has 7 characters, giving $7 - 3 + 1 = 5$ trigrams: $\langle \text{wh, whe, her, ere, re} \rangle$.
- Plus the special sequence $\langle \text{where} \rangle$ itself.

Answer: 5**Q14.15** MSQ · M

Advantages of FastText over Word2Vec:

- (A) Handles out of vocabulary words
- (B) Robust to misspellings
- (C) Helps morphologically rich languages
- (D) Produces contextual embeddings

Solution

- Subword n-grams give vectors for unseen or misspelt words and share information across word forms (A, B, C). Still static (D false).

Answer: A, B, C

Q14.16 MCQ · M

Which problem do static embeddings fail on?

- (A) “bank” meaning river side vs financial institution
- (B) Spelling of “bank”
- (C) Counting words
- (D) Tokenising “bank”

Solution

- A single vector per type cannot represent polysemy; contextual models produce different vectors per occurrence.

Answer: A

Q14.17 MCQ · M

GloVe fits word vectors to

- (A) the log of global co occurrence counts
- (B) next sentence labels
- (C) character n-grams
- (D) masked tokens

Solution

- $w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$, weighted least squares.

Answer: A

Q14.18 NAT · M

BERT MLM on a 512 token sequence. About how many tokens are selected for prediction, and about how many of those become [MASK]?

Solution

- $15\% \times 512 = 76.8 \approx 77$.
- 80% of those: $0.8 \times 76.8 = 61.4 \approx 61$ become [MASK]; about 8 random tokens; about 8 unchanged.

Answer: about 77 selected, about 61 masked

Q14.19 MCQ · H

Why does BERT not replace all selected tokens with [MASK]?

- (A) To save memory
- (B) Because [MASK] never appears at fine tuning time; mixing in random and unchanged tokens reduces this pre-train / fine-tune mismatch and forces representations for every token
- (C) Because the vocabulary lacks [MASK]
- (D) To speed up tokenisation

Solution

- If only [MASK] positions were predicted, the model could ignore real tokens' representations. Random and kept tokens mean any position might need correcting.

Answer: B

Q14.20 NAT · M

BERT-base: hidden 768, 12 heads. Dimension per head?

Solution

- $768/12 = 64$.

Answer: 64

Q14.21 NAT · M

BERT-base token embedding table with vocabulary 30,522 and hidden 768. Parameters (millions, 1 decimal)?

Solution

- $30,522 \times 768 = 23,440,896 \approx 23.4\text{M}$, about a fifth of BERT-base's 110M.

Answer: 23.4**Q14.22** MSQ · M

BERT input representation is the sum of

- (A) token embeddings
- (B) segment (sentence A/B) embeddings
- (C) position embeddings
- (D) TF-IDF weights

Solution

- The three learned embeddings are summed elementwise (each 768 dims for base). TF-IDF is not used (D false).

Answer: A, B, C**Q14.23** MCQ · M

RoBERTa's key changes to BERT include

- (A) adding NSP
- (B) removing NSP, dynamic masking, more data and longer training
- (C) switching to a decoder
- (D) using character level tokens

Solution

- Ablations showed NSP did not help; masks are resampled every epoch; far more data and compute.

Answer: B**Q14.24** MSQ · M

DistilBERT:

- (A) is trained by knowledge distillation from BERT
- (B) has about 40% fewer parameters
- (C) keeps about 97% of BERT's GLUE performance
- (D) is a decoder model

Solution

- Student mimics teacher soft labels; 6 layers instead of 12; faster inference (A, B, C). Still an encoder (D false).

Answer: A, B, C**Q14.25** MCQ · M

BERT vs GPT:

- (A) BERT is a causal decoder; GPT is a bidirectional encoder
- (B) BERT is a bidirectional encoder for understanding; GPT is a causal decoder for generation
- (C) Both are identical
- (D) GPT uses MLM

Solution

- GPT is trained with next token prediction and a causal mask; BERT with MLM and full attention.

Answer: B**Q14.26** MSQ · H

Semantic search over 5 million job postings:

- (A) Encode postings once with a bi-encoder (Sentence-BERT) and index them with approximate nearest neighbours.

- (B) Re-rank the top 100 with a cross-encoder.
- (C) Run a cross-encoder on all 5 million pairs per query.
- (D) Normalise vectors so inner product equals cosine.

Solution

- Bi-encoders allow precomputation (A); cross-encoders are accurate but costly, so only for re-ranking (B, C false). Unit vectors make dot product = cosine, which ANN libraries search fast (D).

Answer: A, B, D

Q14.27 MCQ · M

A simple sentence embedding from Word2Vec vectors is

- (A) their (optionally TF-IDF weighted) average
- (B) their product
- (C) the first word's vector
- (D) a random vector

Solution

- Averaging is a strong baseline; weighting by TF-IDF or SIF down weights frequent words.

Answer: A

Q14.28 MSQ · M

Fine tuning BERT for resume classification:

- (A) Add a classification head on the [CLS] output.
- (B) Use a small learning rate like 2e-5.
- (C) Train for a few epochs.
- (D) Retrain the WordPiece tokenizer from scratch every time.

Solution

- Standard recipe (A, B, C). Keep the pre-trained tokenizer; changing it would break the learned embeddings (D false).

Answer: A, B, C

Q14.29 MCQ · M

Maximum input length of original BERT is

- (A) 128
- (B) 256
- (C) 512 tokens
- (D) Unlimited

Solution

- Learned position embeddings exist for 512 positions. Longer documents need truncation, sliding windows or long context models.

Answer: C

Q14.30 MSQ · M

Word embedding biases:

- (A) Embeddings can encode societal stereotypes present in training text.
- (B) Such bias can affect downstream hiring systems.
- (C) Bias can be measured with association tests and mitigated.
- (D) Embeddings are always neutral.

Solution

- Classic finding: "man:programmer :: woman:homemaker". For a jobs platform this is a real fairness risk (A, B, C). D false.

Answer: A, B, C

Q14.31 MCQ · M

Typical Word2Vec embedding dimensions are

- (A) 2 to 5
- (B) 100 to 300
- (C) 10,000
- (D) equal to vocabulary size

Solution

- Enough to capture many semantic directions while staying compact; the original Google News vectors are 300 dimensional.

Answer: B

Q14.32 MCQ · H

In Word2Vec, the hidden layer has

- (A) a ReLU activation
- (B) no non linearity; it is a linear projection (lookup)
- (C) a sigmoid
- (D) a softmax

Solution

- The “hidden layer” is just the embedding row selected by the one hot input (or the mean for CBOW). The only non linearity is at the output (softmax or sigmoid for negative sampling).

Answer: B

Q14.33 MSQ · M

Intrinsic vs extrinsic evaluation of embeddings:

- (A) Word analogy and word similarity benchmarks are intrinsic.
- (B) Downstream task accuracy (for example NER F1) is extrinsic.
- (C) Good intrinsic scores always mean best downstream results.
- (D) Extrinsic evaluation is ultimately what matters for a product.

Solution

- Intrinsic scores do not always correlate with task performance (C false).

Answer: A, B, D

Q14.34 NAT · H

Skip-gram negative sampling loss for one positive pair with score $u_o^\top v_c = 0$ and one negative with score $u_n^\top v_c = 0$ (natural log, 3 decimals).

Solution

- $\sigma(0) = 0.5$. Loss = $-\log \sigma(0) - \log \sigma(-0) = -\ln 0.5 - \ln 0.5 = 2 \times 0.693 = 1.386$.
- Training raises the positive score and lowers the negative score to reduce it.

Answer: 1.386

Q14.35 MCQ · M

ELMo produced contextual embeddings using

- (A) a Transformer encoder
- (B) bidirectional LSTMs
- (C) character CNN only
- (D) K-Means

Solution

- Forward and backward LSTM language models, whose layer outputs are combined per task. BERT replaced LSTMs with Transformers.

Answer: B

Interview angle

- Explain Word2Vec end to end: training task, architecture, negative sampling, what the embeddings are.
- Why can FastText handle OOV words? When does that matter in resumes?
- Explain BERT pre-training (MLM 80/10/10, NSP) and fine tuning; BERT vs GPT.
- Design semantic job search: bi-encoder vs cross-encoder, ANN index, evaluation.
- How would you detect and reduce gender bias in embeddings used for job recommendations?

15 Neural Network Foundations: Backprop, Activations, Initialisation, Regularisation

Companion resource for this section: [3Blue1Brown: But what is a neural network? \(Deep Learning, Chapter 1\)](#)

15.1 From a neuron to a network

- A **neuron** computes a weighted sum plus bias and passes it through an activation: $a = g(w^\top x + b)$. With g = sigmoid this is exactly logistic regression.
- Example: $x = (1, 2)$, $w = (0.5, -1)$, $b = 1$: $z = 0.5 - 2 + 1 = -0.5$. ReLU gives 0; sigmoid gives $\sigma(-0.5) = 1/(1 + e^{0.5}) = 0.378$.
- **Perceptron** (Rosenblatt): step activation, update $w \leftarrow w + \eta(y - \hat{y})x$. Can only draw a straight line, so it cannot solve XOR.
- **Multi layer perceptron (MLP)**: layers of neurons: input $\rightarrow$ hidden layers $\rightarrow$ output. $h^{(1)} = g(W^{(1)}x + b^{(1)})$, $h^{(2)} = g(W^{(2)}h^{(1)} + b^{(2)})$, ... One hidden layer solves XOR by first making two lines and then combining them.
- **Why non linear activations**: stacking linear layers gives $W_2 W_1 x = W'x$, still linear. Without non linearity depth adds nothing.
- **Parameters of a dense layer** with n_{in} inputs and n_{out} outputs: $n_{in}n_{out} + n_{out}$. MLP 784-128-10: $784 \cdot 128 + 128 + 128 \cdot 10 + 10 = 101,770$.
- **Deep network**: more than one hidden layer (depth lets the network build features hierarchically: edges $\rightarrow$ shapes $\rightarrow$ objects).

15.2 Universal Approximation Theorem (reported InfoEdge OA topic)

- Statement: a feedforward network with a **single hidden layer**, a suitable non polynomial activation (sigmoid, ReLU) and **enough hidden units** can approximate any continuous function on a closed bounded region to any desired accuracy.
- What it does *not* say: how many units are needed (could be astronomically many), that gradient descent will find those weights, or that the network will generalise from finite data.
- Why go deep then? Deep networks can represent many functions with exponentially fewer units than shallow ones, and hierarchical features generalise better in practice.

15.3 Activation functions

- **Sigmoid** $\sigma(z) = 1/(1 + e^{-z})$, range $(0, 1)$, derivative $\sigma(1 - \sigma) \leq 0.25$. Saturates for large $|z|$ (gradient ≈ 0), outputs not zero centred. Use: output layer for binary probabilities, gates in LSTMs.
- **Tanh**: range $(-1, 1)$, zero centred, derivative $1 - \tanh^2 \leq 1$; still saturates.
- **ReLU** $\max(0, z)$: derivative 1 for $z > 0$, 0 otherwise. No saturation for positives, cheap, sparse activations. Problem: **dying ReLU** (a neuron whose input is always negative gets zero gradient forever, often after a huge learning rate step).
- **Leaky ReLU** $\max(\alpha z, z)$ with $\alpha = 0.01$; **ELU**; **GELU** $z\Phi(z)$ (used in BERT/GPT); **Swish/SiLU** $z\sigma(z)$.
- **Softmax** for K classes: $p_k = e^{z_k} / \sum_j e^{z_j}$. Example $z = (1, 2, 3)$: $e^z = (2.718, 7.389, 20.086)$, sum 30.19, $p = (0.090, 0.245, 0.665)$. Subtract $\max z$ before exponentiating for numerical stability.

15.4 Loss functions (reported OA topic: classification losses)

- Regression: MSE (Gaussian noise), MAE (robust), Huber (both).
- Binary classification: **binary cross entropy / log loss** $-[y \log p + (1 - y) \log(1 - p)]$ with sigmoid output. Its gradient w.r.t. the logit is simply $p - y$.
- Multi class: **categorical cross entropy** $-\log p_{\text{true}}$ with softmax; gradient w.r.t. logits $p - y_{\text{onehot}}$.
- **Hinge loss** $\max(0, 1 - y s)$ with $y \in \{-1, +1\}$ (SVM). Zero once the margin exceeds 1.
- Others: focal loss (imbalance), KL divergence (distillation), contrastive/triplet (embeddings).

15.5 Backpropagation: the chain rule, organised

- Training = gradient descent on the loss. We need $\partial L / \partial w$ for millions of weights. Backprop computes all of them in one backward sweep, reusing intermediate results, at roughly the cost of two forward passes.
- Forward pass: compute and store every z and a . Backward pass: start from $\partial L / \partial \hat{y}$ and multiply local derivatives layer by layer back to the input.
- Tiny example: $\hat{y} = wx$, $L = (\hat{y} - t)^2$, $w = 2$, $x = 3$, $t = 4$. Forward: $\hat{y} = 6$, $L = 4$. Backward: $\partial L / \partial \hat{y} = 2(6 - 4) = 4$; $\partial \hat{y} / \partial w = x = 3$; so $\partial L / \partial w = 12$. With $\eta = 0.01$: $w \leftarrow 2 - 0.12 = 1.88$.
- Layer form: $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot g'(z^{(l)})$ and $\partial L / \partial W^{(l)} = \delta^{(l)} a^{(l-1)\top}$.
- Frameworks (PyTorch autograd) build the computation graph and apply this automatically.

15.6 Vanishing and exploding gradients (most drilled InfoEdge DL interview topic)

- The gradient reaching layer 1 is a **product** of many per layer factors (weights times activation derivatives). Product of many numbers below 1 $\rightarrow$ vanishes; above 1 $\rightarrow$ explodes.
- Sigmoid: each factor at most 0.25. Ten sigmoid layers: $0.25^{10} \approx 9.5 \times 10^{-7}$. Early layers barely learn.
- Exploding: factor 1.5 over 20 layers (or 20 RNN time steps): $1.5^{20} \approx 3325$. Loss becomes NaN.
- **Symptoms**: vanishing: early layer weights hardly change, loss plateaus. Exploding: loss spikes or NaN, huge weight updates.
- **Fixes**: ReLU family activations; proper initialisation (Xavier/He); **Batch Normalisation**; **residual connections** (ResNet); gradient clipping (exploding); LSTM/GRU gates for sequences; smaller learning rate; layer normalisation in Transformers.

15.7 Weight initialisation: Xavier and He (reported OA topic)

- All zeros: every neuron in a layer gets identical gradients forever (**symmetry** never broken). So initialise randomly.
- Too large $\rightarrow$ activations explode/saturate; too small $\rightarrow$ signals shrink to zero layer after layer. Goal: keep the variance of activations (and gradients) roughly constant across layers.
- **Xavier/Glorot** (for tanh/sigmoid): $\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$; uniform version $U\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right]$.
- **He/Kaiming** (for ReLU): $\text{Var}(w) = \frac{2}{n_{in}}$. The factor 2 compensates for ReLU zeroing half of the inputs.
- Example: $n_{in} = n_{out} = 256$. Xavier std = $\sqrt{2/512} = 1/16 = 0.0625$. He std = $\sqrt{2/256} = 0.0884$.

15.8 Residual connections (ResNet, reported OA topic)

- A block outputs $y = F(x) + x$: the layers learn only the *residual* F on top of an identity shortcut.
- Gradient: $\partial y / \partial x = \partial F / \partial x + I$. The identity term lets gradient flow straight back through many blocks, so 50, 101, 152 layer networks train. Also, if extra layers are unnecessary, they can learn $F \approx 0$ easily.
- Solved the **degradation problem**: before ResNet, a 56 layer plain network had higher *training* error than a 20 layer one (not overfitting, an optimisation failure).

15.9 Normalisation layers

- **Batch Normalisation**: for each feature, over the mini batch: $\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then $y = \gamma \hat{x} + \beta$ (learnable scale and shift). At inference use running averages of μ, σ^2 . Allows higher learning rates, less sensitivity to initialisation, mild regularisation. Needs reasonable batch sizes.
- Example batch for one feature (1, 2, 3): mean 2, variance 2/3, $\hat{x} = (-1.225, 0, 1.225)$; with $\gamma = 2$, $\beta = 1$: $(-1.449, 1, 3.449)$.
- **Layer Normalisation**: normalise across features of a single example; independent of batch size; standard in Transformers and RNNs.

15.10 Regularisation in deep learning (reported interview topics)

- **Weight decay:** add $\frac{\lambda}{2} \|w\|^2$; update $w \leftarrow w - \eta(\nabla L + \lambda w)$, i.e. shrink weights by factor $(1 - \eta\lambda)$ every step. (AdamW decouples decay from the adaptive step.)
- **Dropout:** during training, zero each unit with probability p (e.g. 0.5 in dense layers, 0.1 in Transformers). Prevents co-adaptation; like training an ensemble of thinned networks. **Inverted dropout** scales kept units by $1/(1 - p)$ during training so nothing changes at test time, when dropout is off (`model.eval()`).
- **Early stopping:** stop when validation loss stops improving (patience), restore best weights.
- **Data augmentation:** flips, crops, noise, mixup; the strongest regulariser for vision.
- Others: label smoothing, batch norm's noise, smaller networks, gradient noise.

15.11 Learning rate and getting unstuck (reported interview topics)

- Too high: loss oscillates or diverges. Too low: painfully slow, may stall. Find it with a learning rate range test; use schedules (step decay, cosine, warmup then decay), and `ReduceLROnPlateau`.
- Local minima: in high dimensions most critical points are **saddle points**, and most local minima are nearly as good as the global one. Practical remedies when stuck: momentum/Adam, learning rate warm restarts, different random initialisation ("rerun with another seed"), batch size changes, better normalisation.
- Gradient clipping: if $\|g\| > c$, set $g \leftarrow c \cdot g / \|g\|$.

15.12 RBMs and Deep Belief Networks (reported OA topic)

- **Restricted Boltzmann Machine:** a two layer, undirected, energy based generative model: visible units v and hidden units h , connected only *between* layers (no visible-visible or hidden-hidden edges, which is the "restriction"). Energy $E(v, h) = -a^\top v - b^\top h - v^\top W h$; $p(v, h) \propto e^{-E}$. Trained with **contrastive divergence** (CD-k, Gibbs sampling).
- **Deep Belief Network:** a stack of RBMs trained **greedily layer by layer** (unsupervised pre-training), then fine tuned with backprop. Historically (Hinton, 2006) this was what made deep networks trainable before ReLU, good initialisation and BatchNorm.
- RBMs were also used in collaborative filtering (Netflix Prize).

Where this shows up

- Every model at InfoEdge built in PyTorch/TensorFlow relies on these defaults: He init with ReLU, AdamW with weight decay, dropout, LayerNorm, residuals, cosine schedule with warmup, gradient clipping for sequence models.

Formula sheet

Dense params $n_{in}n_{out} + n_{out}$. Sigmoid' ≤ 0.25 ; tanh' ≤ 1 .
 BCE gradient w.r.t. logit $= p - y$; softmax+CE $= p - y$. Hinge $\max(0, 1 - ys)$.
 Xavier Var $= 2/(n_{in} + n_{out})$, uniform limit $\sqrt{6/(n_{in} + n_{out})}$; He Var $= 2/n_{in}$.
 Inverted dropout scale $1/(1 - p)$. BN: $\gamma(x - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta$. Weight decay factor $(1 - \eta\lambda)$. Clip: $g \cdot c / \|g\|$.

15.13 Question bank: Neural network foundations

PYQ status for this chapter

This chapter carries the most reported InfoEdge DL items. OA (IIT BHU, 40 MCQs): Xavier and He initialisation, ResNets for vanishing gradients, classification losses (cross entropy, log loss, hinge), DBN and RBM, universal approximation theorem, number of hidden layers in a deep network. OA (IITR): overfitting. Interviews (IIT BHU, Glassdoor): vanishing/exploding gradients, weight decay, dropout, learning rate and adjusting it, backpropagation intuition, what to do when stuck in a local optimum. They appear first.

Q15.1 MCQ · E

★ **PYQ (OA: universal approximation theorem)** The universal approximation theorem states that a feedforward network

- (A) with one hidden layer and enough units can approximate any continuous function on a compact domain
- (B) always generalises perfectly
- (C) needs at least 10 layers to approximate any function
- (D) trained with SGD always finds the global optimum

Solution

- It is an existence statement about representation. It says nothing about the number of units required, trainability by SGD, or generalisation (B, D false). A single hidden layer suffices in principle (C false).

Answer: A

Q15.2 MCQ · E

★ **PYQ (OA: number of hidden layers in a deep network)** A neural network is typically called “deep” when it has

- (A) no hidden layer
- (B) exactly one hidden layer
- (C) more than one hidden layer
- (D) only an output layer

Solution

- Zero hidden layers is a linear/logistic model; one hidden layer is “shallow”. Two or more hidden layers is the conventional meaning of deep.

Answer: C

Q15.3 MSQ · M

★ **PYQ (OA: Xavier and He initialisation)** Which statements are correct?

- (A) Xavier initialisation uses variance $2/(n_{in} + n_{out})$.
- (B) He initialisation uses variance $2/n_{in}$ and suits ReLU.
- (C) Initialising all weights to zero is fine because biases break symmetry.
- (D) Good initialisation keeps activation variance roughly constant across layers.

Solution

- Formulas (A, B); the aim (D).
- With zero weights, all hidden units in a layer compute the same function and receive the same gradient; identical biases do not break that symmetry (C false).

Answer: A, B, D

Q15.4 NAT · M

★ **PYQ (OA: He initialisation numeric)** A ReLU layer has $n_{in} = 512$ inputs. Standard deviation for He normal initialisation?

Solution

- $\text{Var} = 2/512 = 1/256$; $\text{std} = 1/16 = 0.0625$.

Answer: 0.0625

Q15.5 NAT · M

★ **PYQ (OA: Xavier uniform numeric)** Xavier uniform limit for $n_{in} = 100$, $n_{out} = 50$ (3 decimals)?

Solution

- Limit $= \sqrt{6/(100 + 50)} = \sqrt{0.04} = 0.2$. Weights drawn from $U[-0.2, 0.2]$.
- Check: variance of $U[-a, a]$ is $a^2/3 = 0.04/3 = 2/150$, matching $2/(n_{in} + n_{out})$.

Answer: 0.2

Q15.6 MCQ · M

★ **PYQ (OA: ResNets and vanishing gradients)** Residual connections mainly help because

- (A) they reduce the number of parameters
- (B) the identity shortcut gives gradients a direct path, easing the vanishing gradient/degradation problem
- (C) they remove the need for activations
- (D) they make the network shallower

Solution

- $y = F(x) + x$ gives $\partial y / \partial x = I + \partial F / \partial x$; the gradient cannot vanish through the identity term, so very deep networks train.

Answer: B

Q15.7 MSQ · M

★ **PYQ (OA: loss functions for classification)** Which are classification losses?

- (A) Cross entropy
- (B) Log loss (binary cross entropy)
- (C) Hinge loss
- (D) Mean absolute error

Solution

- Cross entropy and log loss (same thing in the binary case) for probabilistic classifiers; hinge for max margin (SVM).
- MAE is a regression loss (D false).

Answer: A, B, C

Q15.8 NAT · E

★ **PYQ (OA: hinge loss value)** Hinge loss for true label $y = +1$ and score $s = 0.3$?

Solution

- $\max(0, 1 - 1 \times 0.3) = 0.7$. Correct side but inside the margin, so still penalised. With $s = 1.5$ the loss would be 0.

Answer: 0.7

Q15.9 MSQ · M

★ **PYQ (OA: DBN and RBM)** Which are true?

- (A) An RBM has connections only between the visible and hidden layers.
- (B) RBMs are trained with contrastive divergence.
- (C) A DBN is built by stacking RBMs and training them greedily layer by layer.
- (D) An RBM is a supervised discriminative classifier trained only by backprop.

Solution

- The “restricted” bipartite structure (A) makes Gibbs sampling easy for CD (B). DBN = stacked RBMs with greedy pre-training, then optional fine tuning (C).
- RBMs are unsupervised generative energy based models (D false).

Answer: A, B, C

Q15.10 MSQ · M

★ **PYQ (INT: vanishing and exploding gradients)** Which help against vanishing gradients?

- (A) ReLU instead of sigmoid in hidden layers
- (B) Residual connections
- (C) Batch Normalisation
- (D) Stacking more sigmoid layers

Solution

- ReLU derivative is 1 on the active side (A); shortcuts (B); BN keeps pre-activations out of saturated regions (C).
- More sigmoid layers multiply more factors ≤ 0.25 (D worsens it).

Answer: A, B, C

Q15.11 NAT · M

★ **PYQ (INT: why sigmoid causes vanishing gradients)** Upper bound on the gradient factor contributed by 8 stacked sigmoid activations (ignoring weights)? Give as a power of 4 and decimal.

Solution

- Max of $\sigma'(z) = \sigma(1 - \sigma)$ is at $z = 0$: $0.5 \times 0.5 = 0.25$.
- $0.25^8 = 4^{-8} = 1/65,536 \approx 1.53 \times 10^{-5}$.

Answer: $4^{-8} \approx 1.5 \times 10^{-5}$

Q15.12 MSQ · M

★ **PYQ (INT: fixing exploding gradients)** Loss suddenly becomes NaN while training an RNN. Reasonable fixes:

- (A) Gradient clipping
- (B) Lower learning rate
- (C) Use LSTM/GRU cells
- (D) Increase the learning rate tenfold

Solution

- Clipping caps update size (A); smaller steps (B); gated cells stabilise gradient flow (C). D makes explosion worse.

Answer: A, B, C

Q15.13 NAT · M

★ **PYQ (INT: gradient clipping by norm)** Gradient $g = (3, 4)$, clip norm $c = 1$. Clipped gradient?

Solution

- $\|g\| = 5 > 1$, so $g \leftarrow g \cdot 1/5 = (0.6, 0.8)$. Direction kept, length capped at 1.

Answer: (0.6, 0.8)

Q15.14 MSQ · M

★ **PYQ (INT: dropout)** Which are true about dropout?

- (A) Units are randomly zeroed during training only.
- (B) Inverted dropout scales kept activations by $1/(1 - p)$ during training.
- (C) It reduces co-adaptation of neurons and acts like an ensemble.
- (D) Dropout should stay on at test time for normal predictions.

Solution

- Train only (A), scaling keeps expectations equal (B), ensemble interpretation (C).
- At test time dropout is off (`model.eval()`); keeping it on (MC dropout) is only for uncertainty estimation (D false).

Answer: A, B, C

Q15.15 NAT · E

★ **PYQ (INT: inverted dropout scale)** Dropout probability $p = 0.2$. Scale factor applied to kept units during training?

Solution

- $1/(1 - 0.2) = 1/0.8 = 1.25$. On average 80% of units survive; scaling by 1.25 keeps the expected activation unchanged.

Answer: 1.25**Q15.16** NAT · M

★ **PYQ (INT: weight decay)** Weight $w = 1$, learning rate $\eta = 0.1$, weight decay $\lambda = 0.01$, loss gradient 0 at this step. New w ?

Solution

- $w \leftarrow w - \eta\lambda w = 1 - 0.1 \times 0.01 \times 1 = 0.999$. Each step multiplies weights by $(1 - \eta\lambda)$, hence “decay”.

Answer: 0.999**Q15.17** MSQ · M

★ **PYQ (INT: learning rate and adjusting it)** Training loss oscillates wildly and does not decrease. Which are sensible?

- (A) Reduce the learning rate
- (B) Use a schedule with warmup and decay
- (C) Use ReduceLROnPlateau
- (D) Remove all normalisation

Solution

- Oscillation is the classic sign of too large a step (A); schedules adjust over time (B, C). D false.

Answer: A, B, C**Q15.18** MSQ · H

★ **PYQ (INT: stuck in a local optimum)** Your network’s loss plateaus early at a poor value. Which actions are reasonable?

- (A) Restart with a different random initialisation (seed).
- (B) Use momentum or Adam.
- (C) Try a learning rate warm restart or cyclical schedule.
- (D) Conclude that the global optimum has been reached.

Solution

- A new seed explores a different basin (the reported InfoEdge interview answer). Momentum carries through flat regions and saddles (B); restarts kick the optimiser out (C).
- Plateaus in deep nets are usually saddles or poor conditioning, not proof of global optimality (D false).

Answer: A, B, C**Q15.19** MSQ · M

★ **PYQ (OA (IITR): overfitting in neural networks)** Training accuracy 99%, validation 70%. Which help?

- (A) Dropout and weight decay
- (B) Data augmentation
- (C) Early stopping
- (D) Training longer with a bigger network

Solution

- Classic high variance gap: regularise (A), add effective data (B), stop at the best validation point (C). D increases variance.

Answer: A, B, C**Q15.20** MCQ · M

★ **PYQ (INT: backpropagation intuition)** Backpropagation is best described as

- (A) a new optimisation algorithm replacing gradient descent
- (B) an efficient application of the chain rule that computes all weight gradients in one backward pass
- (C) random search over weights
- (D) a method to choose the architecture

Solution

- Backprop computes gradients; an optimiser (SGD, Adam) uses them to update weights.

Answer: B**Q15.21** NAT · ENeuron with $x = (1, 2)$, $w = (0.5, -1)$, $b = 1$. Output with ReLU? With sigmoid (3 decimals)?**Solution**

- $z = 0.5 \cdot 1 + (-1) \cdot 2 + 1 = -0.5$.
- ReLU: $\max(0, -0.5) = 0$. Sigmoid: $1/(1 + e^{0.5}) = 1/(1 + 1.6487) = 0.378$.

Answer: ReLU 0; sigmoid 0.378**Q15.22** NAT · E

Number of parameters in an MLP 784-128-10 (dense, with biases)?

Solution

- Layer 1: $784 \times 128 + 128 = 100,352 + 128 = 100,480$.
- Layer 2: $128 \times 10 + 10 = 1,290$.
- Total = 101,770.

Answer: 101,770**Q15.23** NAT · M

Softmax of logits (1, 2, 3): probability of the third class (3 decimals)?

Solution

- $e^1 = 2.718$, $e^2 = 7.389$, $e^3 = 20.086$; sum = 30.193.
- $20.086/30.193 = 0.665$. (Others 0.090 and 0.245.)

Answer: 0.665**Q15.24** NAT · M

Cross entropy loss if the true class is the third one in the previous question (natural log, 3 decimals)?

Solution

- $-\ln 0.665 = 0.408$.

Answer: 0.408**Q15.25** NAT · MSigmoid output $p = 0.8$, true label $y = 1$, binary cross entropy. Gradient of the loss with respect to the logit z ?**Solution**

- For sigmoid + BCE, $\partial L / \partial z = p - y = 0.8 - 1 = -0.2$. Negative, so gradient descent increases z , pushing p toward 1.

Answer: -0.2**Q15.26** NAT · M $\hat{y} = wx$, $L = (\hat{y} - t)^2$, $w = 2$, $x = 3$, $t = 4$, $\eta = 0.01$. Updated w ?**Solution**

- $\hat{y} = 6$; $\partial L / \partial \hat{y} = 2(6 - 4) = 4$; $\partial \hat{y} / \partial w = 3$; gradient 12.
- $w = 2 - 0.01 \times 12 = 1.88$.

Answer: 1.88

Q15.27 MCQ · M

Why can't a network without non linear activations solve XOR?

- (A) It has too few parameters
- (B) A composition of linear maps is linear, and XOR is not linearly separable
- (C) XOR needs softmax
- (D) It lacks biases

Solution

- $W_2(W_1x + b_1) + b_2 = W'x + b'$. Any linear boundary misclassifies at least one XOR point.

Answer: B

Q15.28 MSQ · M

About ReLU:

- (A) Its gradient is 1 for positive inputs.
- (B) Neurons can "die" if their inputs stay negative.
- (C) It is zero centred.
- (D) Leaky ReLU keeps a small gradient for negative inputs.

Solution

- Outputs are ≥ 0 , so not zero centred (C false).

Answer: A, B, D

Q15.29 NAT · E

Leaky ReLU with $\alpha = 0.01$ at input -2 ?

Solution

- $\max(\alpha z, z) = \max(-0.02, -2) = -0.02$.

Answer: -0.02

Q15.30 NAT · M

Batch Normalisation of one feature over the batch $(1, 2, 3)$ with $\epsilon = 0$, $\gamma = 2$, $\beta = 1$. Output for the value 3 (3 decimals)?

Solution

- Mean = 2; variance = $((1 - 2)^2 + 0 + (3 - 2)^2) / 3 = 2/3$; std = 0.8165.
- $\hat{x} = (3 - 2) / 0.8165 = 1.2247$.
- $y = 2 \times 1.2247 + 1 = 3.449$.

Answer: 3.449

Q15.31 MSQ · M

Batch Norm vs Layer Norm:

- (A) BatchNorm normalises each feature across the batch.
- (B) LayerNorm normalises across features within one example.
- (C) LayerNorm is preferred in Transformers and with small or variable batches.
- (D) BatchNorm uses the same statistics at training and inference by recomputing on each test batch.

Solution

- At inference BatchNorm uses running averages collected during training, not test batch statistics (D false).

Answer: A, B, C

Q15.32 NAT · M

Trainable parameters of a BatchNorm layer on 64 features (exclude running statistics)?

Solution

- γ and β per feature: $2 \times 64 = 128$. (Running mean and variance are another 128 buffers, not trained.)

Answer: 128

Q15.33 NAT · M

An RNN multiplies the gradient by a factor 1.5 at each of 20 time steps. Approximate total factor?

Solution

- $1.5^{10} = 57.67$; $1.5^{20} = 57.67^2 \approx 3325$. Exploding gradient; clip it.

Answer: ≈ 3325

Q15.34 MCQ · M

Why do we not initialise all weights to the same non zero constant?

- (A) It makes training too fast
- (B) Neurons in a layer stay identical (symmetry), so they learn the same feature
- (C) It is illegal in PyTorch
- (D) Biases would explode

Solution

- Same weights $\Rightarrow$ same outputs $\Rightarrow$ same gradients $\Rightarrow$ same updates. Random init breaks symmetry.

Answer: B

Q15.35 MCQ · M

Early stopping monitors

- (A) training loss only
- (B) validation loss/metric, stopping when it stops improving for some patience
- (C) the learning rate
- (D) the number of parameters

Solution

- Training loss keeps decreasing while validation loss rises when overfitting begins; stop at the best validation epoch and restore those weights.

Answer: B

Q15.36 MSQ · H

The degradation problem that ResNet solved refers to

- (A) deeper plain networks having higher training error than shallower ones
- (B) a failure of optimisation, not overfitting
- (C) test error only, with training error lower
- (D) identity mappings being hard for stacks of non linear layers to learn

Solution

- He et al. observed a 56 layer plain net with *higher training* error than a 20 layer one (A), so it is an optimisation issue (B, C false). Learning $F(x) = 0$ is easy, learning identity through non linear layers is not (D).

Answer: A, B, D

Q15.37 MSQ · M

Adam vs SGD with momentum (quick recap from Chapter 2):

- (A) Adam adapts per parameter step sizes using squared gradient averages.
- (B) AdamW applies weight decay decoupled from the adaptive update.

- (C) SGD with momentum sometimes generalises better on vision tasks.
 (D) Adam needs no learning rate.

Solution

- Adam has a learning rate (default 1e-3) (D false).

Answer: A, B, C

Q15.38 MCQ · M

In a multi class classifier, which output activation and loss pair is standard?

- (A) Sigmoid + MSE
 (B) Softmax + categorical cross entropy
 (C) ReLU + hinge
 (D) Tanh + MAE

Solution

- Softmax gives a distribution over classes; cross entropy is its negative log likelihood with a clean gradient $p - y$. For multi label (several labels can be true) use independent sigmoids + BCE.

Answer: B

Q15.39 NAT · H

A 3 layer chain $h_1 = w_1x$, $h_2 = w_2h_1$, $\hat{y} = w_3h_2$ with $x = 1$, $w_1 = w_2 = w_3 = 0.5$, loss $L = \hat{y}$ (just the output). Compute $\partial L / \partial w_1$.

Solution

- $\hat{y} = w_3w_2w_1x$. $\partial \hat{y} / \partial w_1 = w_3w_2x = 0.5 \times 0.5 \times 1 = 0.25$.
- Note how the gradient for the first layer is a product of later weights: with many layers of 0.5 it shrinks geometrically (vanishing).

Answer: 0.25

Q15.40 MSQ · M

Label smoothing with $\epsilon = 0.1$ over 10 classes:

- (A) The true class target becomes 0.91.
 (B) Every wrong class target becomes 0.01.
 (C) It discourages over confident logits.
 (D) It increases the number of classes.

Solution

- Target = $(1 - \epsilon)y + \epsilon/K$: true $0.9 + 0.01 = 0.91$, others 0.01 (A, B). Prevents pushing logits to infinity (C). D false.

Answer: A, B, C

Interview angle

- Explain backprop on a 2 layer network on paper; where do the stored activations get used?
- Vanishing vs exploding gradients: causes, symptoms, five fixes. Why does ResNet help mathematically?
- Derive why He init has the factor 2. What happens with zero init?
- Dropout at train vs test time; inverted dropout. BatchNorm at train vs test time.
- Your loss is stuck: walk through your debugging checklist (learning rate, init, normalisation, data, overfit one batch first).

16 Convolutional Neural Networks (CNNs)

Companion resource for this section: [StatQuest: Neural Networks Part 8, Image Classification with Convolutional Neural Networks](#)

16.1 Why not just use a dense network on images?

- A small 224×224 RGB image has $224 \cdot 224 \cdot 3 = 150,528$ inputs. A dense layer to only 1000 units needs 150 million weights. Too many parameters, too much overfitting.
- Dense layers also ignore that nearby pixels are related, and that a cat in the top left is still a cat in the bottom right.
- CNNs build in two assumptions (inductive biases):
 - **Local connectivity:** each output looks only at a small neighbourhood (for example 3×3).
 - **Weight sharing:** the same small filter slides over the whole image, so a “vertical edge detector” learned once works everywhere.
- Result: far fewer parameters and **translation equivariance** (shift the input, and the feature map shifts the same way).

16.2 The convolution operation, by hand

- A **filter** (kernel) is a small grid of weights. Place it on a patch, multiply elementwise, sum, add bias: one output number. Slide and repeat to get a **feature map**.
- Example: patch $\begin{pmatrix} 3 & 1 & 0 \\ 4 & 2 & 1 \\ 5 & 1 & 0 \end{pmatrix}$, vertical edge kernel $\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$. Output = $(3 + 4 + 5) - (0 + 1 + 0) = 12 - 1 = 11$. A large value: bright on the left, dark on the right, i.e. a vertical edge.
- (Deep learning “convolution” is technically cross correlation: the kernel is not flipped. Since kernels are learned, it does not matter.)
- Input with C_{in} channels: each filter has shape $f \times f \times C_{in}$ and produces *one* output channel. Using K filters gives K output channels.

16.3 Output size: the formula every OA asks (reported InfoEdge interview)

- Input size n , filter f , padding p , stride s :

$$\text{out} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1.$$

- **Padding:** add p rings of zeros around the input. “Valid” = no padding (shrinks). “Same” = $p = (f - 1)/2$ for stride 1 (keeps size). Padding also lets border pixels influence as many outputs as central ones.
- **Stride:** step size of the slide. Stride 2 roughly halves each spatial dimension.
- Examples: $32, f = 5, p = 0, s = 1$: 28. $28, f = 3, p = 1, s = 1$: 28 (same). AlexNet: $227, f = 11, s = 4, p = 0$: $(227 - 11)/4 + 1 = 55$. $7, f = 3, s = 2$: 3.
- Output volume: $\text{out}_h \times \text{out}_w \times K$ (number of filters).

16.4 Parameters and computation

- Conv layer parameters: $(f \cdot f \cdot C_{in} + 1) \cdot K$ (the $+1$ is one bias per filter). Independent of image size!
- Example: 3×3 filters on RGB, 64 filters: $(27 + 1) \cdot 64 = 1,792$.
- Multiplications: $\text{out}_h \cdot \text{out}_w \cdot K \cdot f^2 C_{in}$.
- 1×1 **convolution:** mixes channels at each pixel; cheap way to reduce channels ($256 \rightarrow 64$ costs $256 \cdot 64 + 64 = 16,448$ parameters). Used in Inception and ResNet bottlenecks.
- **Depthwise separable** (MobileNet): a $f \times f$ filter per input channel (depthwise), then a 1×1 to mix (pointwise). Cost $f^2 C_{in} + C_{in} C_{out}$ instead of $f^2 C_{in} C_{out}$.
- **Dilated** convolution: gaps of $d - 1$ between kernel taps; effective size $f + (f - 1)(d - 1)$. Larger receptive field without more parameters.

16.5 Pooling (reported InfoEdge interview: max pooling)

- Downsample each channel independently. **Max pooling** keeps the largest value in each window; **average pooling** the mean. Typical: 2×2 window, stride 2, halving height and width. No learnable parameters.
- Example, 2×2 max pool stride 2 on $\begin{pmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 9 & 1 \\ 3 & 4 & 0 & 8 \end{pmatrix}$ gives $\begin{pmatrix} 6 & 4 \\ 7 & 9 \end{pmatrix}$; average pool gives $\begin{pmatrix} 3.75 & 2.25 \\ 4 & 4.5 \end{pmatrix}$.
- Why: reduces computation and memory, grows the receptive field, and gives **small translation invariance** (a feature moved by one pixel within the window gives the same max).
- Backprop through max pool: gradient goes only to the position that was the max.
- Global average pooling**: average each channel over all positions ($7 \times 7 \times 512 \rightarrow 512$). Replaces big dense layers, fewer parameters, works with any input size.

16.6 Invariance vs equivariance (reported InfoEdge interview: rotation and translation)

- Equivariance**: transform the input, and the output transforms the same way. Convolution is translation *equivariant*.
- Invariance**: output unchanged. Pooling plus global pooling plus the classifier give (approximate) translation *invariance* of the final prediction.
- CNNs are not rotation invariant** (or scale invariant) by design: a filter for vertical edges does not fire on horizontal ones. Remedies: data augmentation (random rotations, flips, scaling), rotation equivariant architectures (group CNNs), spatial transformer networks, or test time augmentation.
- Strided convolution and pooling can even break exact shift equivariance (aliasing); anti aliased CNNs blur before downsampling.

16.7 Receptive field and hierarchy

- Receptive field: the input region affecting one output unit. Two stacked 3×3 convs (stride 1) see 5×5 ; three see 7×7 .
- VGG's insight: two 3×3 layers have the receptive field of one 5×5 with fewer parameters ($2 \cdot 9C^2 = 18C^2$ vs $25C^2$) and an extra non linearity.
- Hierarchy: early layers learn edges and colours; middle layers textures and parts; deep layers object level concepts.

16.8 Landmark architectures

- LeNet-5** (1998): digits; conv, pool, dense.
- AlexNet** (2012): ReLU, dropout, GPUs, data augmentation; won ImageNet by a large margin.
- VGG** (2014): only 3×3 convs, deep and uniform; about 138M parameters (mostly in the dense layers).
- Inception/GoogLeNet** (2014): parallel 1×1 , 3×3 , 5×5 branches with 1×1 bottlenecks.
- ResNet** (2015): residual connections, 50 to 152 layers, BatchNorm, global average pooling.
- MobileNet, EfficientNet**: depthwise separable convs, compound scaling of depth, width and resolution.
- Vision Transformers (ViT) now rival CNNs with enough data; CNN inductive biases help on small data.

16.9 Transfer learning

- Start from ImageNet pre-trained weights. Small dataset similar to ImageNet: freeze the backbone, train a new head. More data or different domain: fine tune later layers (or everything) with a small learning rate.
- Normalise inputs with the same statistics as pre-training.

Where this shows up

- Property photos on 99acres (room type classification, duplicate/fake listing detection), document images (resume parsing, OCR), profile photo moderation, 1D CNNs over text or time series.

Formula sheet

Output $\lfloor (n + 2p - f)/s \rfloor + 1$; same padding $p = (f - 1)/2$ (stride 1).
 Conv params $(f^2 C_{in} + 1)K$; mults $\text{out}_h \text{out}_w K f^2 C_{in}$. Pooling: 0 params.
 Depthwise separable $f^2 C_{in} + C_{in} C_{out}$. Dilated effective size $f + (f - 1)(d - 1)$.
 Receptive field of L stacked 3×3 (stride 1): $2L + 1$. Transposed conv output $(n - 1)s - 2p + f$.

16.10 Question bank: Convolutional neural networks

PYQ status for this chapter

Reported InfoEdge items: in interviews (IIT BHU write up) CNN filters, stride and padding with the formula $(n + 2p - f)/s + 1$, max pooling, and whether CNNs are rotation and translation invariant; the IITR OA lists CNNs among the DL questions. These come first.

Q16.1 NAT · E

★ PYQ (INT: output size formula) Input 32×32 , filter 5×5 , padding 0, stride 1. Output spatial size?

Solution

- $(32 + 0 - 5)/1 + 1 = 28$. Output 28×28 .

Answer: 28

Q16.2 NAT · M

★ PYQ (INT: output size with stride) AlexNet's first layer: input 227×227 , filter 11×11 , stride 4, padding 0. Output size?

Solution

- $(227 - 11)/4 + 1 = 216/4 + 1 = 54 + 1 = 55$.

Answer: 55

Q16.3 NAT · E

★ PYQ (INT: same padding) Input 28×28 , filter 3×3 , stride 1. What padding keeps the output 28×28 ?

Solution

- Need $28 + 2p - 3 + 1 = 28 \Rightarrow 2p = 2 \Rightarrow p = 1$. In general $p = (f - 1)/2$.

Answer: 1

Q16.4 NAT · M

★ PYQ (INT: stride 2 output) Input 7×7 , filter 3×3 , stride 2, padding 0. Output size?

Solution

- $(7 - 3)/2 + 1 = 2 + 1 = 3$.

Answer: 3

Q16.5 NAT · M

★ PYQ (INT: floor in the output formula) Input 8×8 , filter 3×3 , stride 2, padding 0. Output size?

Solution

- $(8 - 3)/2 + 1 = 2.5 + 1$; take the floor of 2.5: $2 + 1 = 3$. The last column/row that does not fit a full window is dropped.

Answer: 3

Q16.6 NAT · E

★ **PYQ (INT: max pooling)** 2×2 max pooling, stride 2 on $\begin{pmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 9 & 1 \\ 3 & 4 & 0 & 8 \end{pmatrix}$. Sum of the output entries?

Solution

- Windows: top left $\{1, 3, 5, 6\} \rightarrow 6$; top right $\{2, 4, 1, 2\} \rightarrow 4$; bottom left $\{7, 2, 3, 4\} \rightarrow 7$; bottom right $\{9, 1, 0, 8\} \rightarrow 9$.
- Output $\begin{pmatrix} 6 & 4 \\ 7 & 9 \end{pmatrix}$, sum = 26.

Answer: 26**Q16.7** NAT · E

★ **PYQ (INT: average vs max pooling)** Same matrix, 2×2 average pooling stride 2. Bottom right output?

Solution

- $(9 + 1 + 0 + 8) / 4 = 18 / 4 = 4.5$.

Answer: 4.5**Q16.8** MSQ · M

★ **PYQ (INT: purpose of max pooling)** Which are true about max pooling?

- (A) It has no learnable parameters.
- (B) It reduces spatial size and computation downstream.
- (C) It provides some invariance to small translations.
- (D) It is applied across channels, mixing them.

Solution

- Fixed operation (A); halving each side cuts later compute by 4 (B); moving a feature within the window keeps the same max (C).
- Pooling acts per channel independently (D false; that is 1×1 conv).

Answer: A, B, C**Q16.9** MSQ · M

★ **PYQ (INT: are CNNs rotation and translation invariant?)** Which statements are correct?

- (A) Convolution is translation equivariant.
- (B) Pooling and global pooling add approximate translation invariance.
- (C) Standard CNNs are rotation invariant by design.
- (D) Rotation robustness is usually obtained by data augmentation.

Solution

- Shifting the input shifts the feature map (A). Pooling and global aggregation discard exact position (B).
- A vertical edge filter does not respond to a rotated edge; nothing in the architecture enforces rotation invariance (C false). Random rotations during training teach it (D).

Answer: A, B, D**Q16.10** MCQ · M

★ **PYQ (INT: filters in a CNN)** A conv layer with 64 filters of size 3×3 on a $32 \times 32 \times 16$ input (same padding, stride 1) produces an output of shape

- (A) $32 \times 32 \times 16$
- (B) $32 \times 32 \times 64$
- (C) $30 \times 30 \times 64$
- (D) $32 \times 32 \times 1024$

Solution

- Same padding keeps 32×32 ; each filter (shape $3 \times 3 \times 16$) gives one channel; 64 filters $\rightarrow$ 64 channels.

Answer: B**Q16.11** NAT · M

★ **PYQ (INT: parameters in a conv layer)** Parameters of a conv layer with 64 filters of size 3×3 on an RGB input (with biases)?

Solution

- Each filter: $3 \cdot 3 \cdot 3 = 27$ weights + 1 bias = 28.
- $28 \times 64 = 1,792$.

Answer: 1,792**Q16.12** NAT · M

Conv layer: input channels 32, 64 filters of size 5×5 , with biases. Parameters?

Solution

- $(5 \cdot 5 \cdot 32 + 1) \cdot 64 = (800 + 1) \cdot 64 = 51,264$.

Answer: 51,264**Q16.13** NAT · E

Compute the output of the kernel $\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$ on the patch $\begin{pmatrix} 3 & 1 & 0 \\ 4 & 2 & 1 \\ 5 & 1 & 0 \end{pmatrix}$ (no bias).

Solution

- Left column contributes $3 + 4 + 5 = 12$; middle column times 0; right column contributes $-(0 + 1 + 0) = -1$.
- Total 11: strong vertical edge response.

Answer: 11**Q16.14** NAT · E

A 1×1 convolution reducing 256 channels to 64 (with biases). Parameters?

Solution

- $(1 \cdot 1 \cdot 256 + 1) \cdot 64 = 257 \cdot 64 = 16,448$.

Answer: 16,448**Q16.15** MCQ · M

The main purpose of 1×1 convolutions in Inception and ResNet bottlenecks is

- (A) spatial downsampling
- (B) cheaply changing (usually reducing) the number of channels and mixing them
- (C) edge detection
- (D) adding padding

Solution

- They act like a dense layer applied at every pixel across channels, cutting cost before expensive 3×3 or 5×5 convs.

Answer: B**Q16.16** NAT · M

Receptive field of three stacked 3×3 conv layers (stride 1, no pooling)?

Solution

- Each layer adds $f - 1 = 2$: $1 + 2 + 2 + 2 = 7$.

Answer: 7×7

Q16.17 NAT · H

Parameters (weights only, C input and output channels) of two stacked 3×3 convs vs one 5×5 conv, for $C = 64$?

Solution

- Two 3×3 : $2 \cdot 9 \cdot C^2 = 18 \cdot 4096 = 73,728$.
- One 5×5 : $25 \cdot C^2 = 102,400$.
- Same receptive field (5×5), 28% fewer parameters, and an extra ReLU. This is VGG's rationale.

Answer: 73,728 vs 102,400

Q16.18 NAT · H

Depthwise separable conv, 3×3 , 64 input channels, 128 output channels (weights only). Parameters, and those of a standard 3×3 conv?

Solution

- Depthwise: $9 \times 64 = 576$. Pointwise: $64 \times 128 = 8,192$. Total 8,768.
- Standard: $9 \times 64 \times 128 = 73,728$. About 8.4 times fewer.

Answer: 8,768 vs 73,728

Q16.19 NAT · M

Effective kernel size of a 3×3 conv with dilation 2?

Solution

- $f + (f - 1)(d - 1) = 3 + 2 \cdot 1 = 5$. Same 9 weights, wider view.

Answer: 5

Q16.20 NAT · M

A 28×28 feature map goes through 2×2 max pool (stride 2), then a 3×3 conv (valid, stride 1), then another 2×2 max pool. Final spatial size?

Solution

- Pool: $28 \rightarrow 14$. Conv: $14 - 3 + 1 = 12$. Pool: $12 \rightarrow 6$.

Answer: 6

Q16.21 NAT · H

Multiplications for a conv layer producing $28 \times 28 \times 64$ from a 3 channel input with 3×3 filters?

Solution

- Per output value: $3 \cdot 3 \cdot 3 = 27$ multiplications.
- Output values: $28 \cdot 28 \cdot 64 = 50,176$.
- Total $50,176 \times 27 = 1,354,752$.

Answer: 1,354,752

Q16.22 NAT · M

A dense layer from a flattened $32 \times 32 \times 3$ image to 100 units has how many weights (excluding biases)? Compare to a 3×3 , 100 filter conv.

Solution

- Dense: $3072 \times 100 = 307,200$.
- Conv: $3 \cdot 3 \cdot 3 \times 100 = 2,700$. Weight sharing makes the difference.

Answer: 307,200 vs 2,700

Q16.23 MSQ · M

Advantages of CNNs over dense networks for images:

- (A) Weight sharing reduces parameters.
- (B) Local connectivity exploits spatial structure.
- (C) Translation equivariance.
- (D) They need no training data.

Solution

- The three inductive biases (A, B, C). D false.

Answer: A, B, C

Q16.24 MCQ · M

Global average pooling on a $7 \times 7 \times 512$ tensor produces

- (A) 7×7
- (B) 512 values
- (C) 25,088 values
- (D) 1 value

Solution

- One average per channel. Flattening would give $7 \cdot 7 \cdot 512 = 25,088$ inputs to a dense layer; GAP removes that huge parameter block.

Answer: B

Q16.25 MCQ · M

During backprop through a max pooling layer, the gradient

- (A) is split equally across the window
- (B) flows only to the input that was the maximum
- (C) is zero everywhere
- (D) is multiplied by the window size

Solution

- Max is locally a selection; its derivative is 1 for the argmax and 0 elsewhere. (Average pooling splits it equally.)

Answer: B

Q16.26 NAT · M

Transposed convolution (upsampling): input 4×4 , kernel 4, stride 2, padding 1. Output size?

Solution

- $(n - 1)s - 2p + f = 3 \cdot 2 - 2 + 4 = 8$. Common in U-Net decoders and GAN generators to double resolution.

Answer: 8

Q16.27 MSQ · M

Which innovations are associated with AlexNet?

- (A) ReLU activations
- (B) Dropout in the dense layers
- (C) GPU training
- (D) Residual connections

Solution

- Residual connections came with ResNet in 2015 (D false).

Answer: A, B, C

Q16.28 MCQ · M

VGG-16 has about 138M parameters. Most of them are in

- (A) the first conv layer
- (B) the fully connected layers
- (C) the pooling layers
- (D) the biases

Solution

- The first dense layer alone maps $7 \cdot 7 \cdot 512 = 25,088$ inputs to 4096 units: about 103M weights.

Answer: B

Q16.29 MSQ · M

What typically happens as we go deeper in a CNN?

- (A) Spatial size decreases.
- (B) Number of channels increases.
- (C) Features become more abstract (parts, objects).
- (D) Receptive field shrinks.

Solution

- Receptive fields grow with depth (D false).

Answer: A, B, C

Q16.30 MSQ · M

Transfer learning with 2,000 labelled property photos:

- (A) Start from ImageNet pre-trained weights.
- (B) Freeze early layers and train a new classifier head first.
- (C) Use a small learning rate when unfreezing more layers.
- (D) Train a ResNet-152 from scratch.

Solution

- 2,000 images are far too few to train 60M parameters from scratch without severe overfitting (D false).

Answer: A, B, C

Q16.31 MSQ · M

Data augmentation for image classification:

- (A) Random crops and flips
- (B) Colour jitter
- (C) Random rotations to gain rotation robustness
- (D) Flipping digits "6" and "9" vertically in digit recognition

Solution

- Augmentations must preserve the label; flipping a 6 vertically can create a 9 (D is harmful).

Answer: A, B, C

Q16.32 MCQ · H

A strided convolution with stride 2 is best described as

- (A) a learnable alternative to pooling for downsampling
- (B) a way to increase resolution
- (C) a normalisation layer

(D) identical to dilation

Solution

- Many modern nets (ResNet downsampling blocks, all convolutional nets) downsample with stride 2 convs instead of pooling.

Answer: A

Q16.33 MSQ · M

1D CNNs are useful for

- (A) text classification over word embeddings
- (B) time series and sensor signals
- (C) audio
- (D) only images

Solution

- Filters slide along one axis to detect local patterns (n-gram like features in text, motifs in signals).

Answer: A, B, C

Q16.34 MCQ · M

Batch Normalisation in a CNN normalises

- (A) each channel over batch and spatial positions
- (B) each pixel independently
- (C) the whole tensor with one mean
- (D) only the output layer

Solution

- One γ , β and one mean/variance per channel, computed over $N \times H \times W$ values, so the conv's weight sharing is respected.

Answer: A

Q16.35 NAT · M

Output size for input 64×64 , filter 5×5 , stride 3, padding 2?

Solution

- $(64 + 4 - 5)/3 + 1 = 63/3 + 1 = 21 + 1 = 22$.

Answer: 22

Q16.36 MCQ · H

Vision Transformers vs CNNs, which is true?

- (A) ViTs have weaker built in locality bias, so they usually need more data or strong augmentation
- (B) CNNs cannot be pre-trained
- (C) ViTs use no attention
- (D) CNNs always beat ViTs

Solution

- ViT treats patches as tokens with global attention from layer one; CNNs' locality and weight sharing help with small data. With large scale pre-training ViTs match or exceed CNNs.

Answer: A

Interview angle

- Derive $(n + 2p - f)/s + 1$ on paper; compute shapes and parameter counts through a small CNN end to end.
- Why weight sharing? Why do two 3×3 convs beat one 5×5 ?
- Max vs average pooling; how does gradient flow through max pooling?

- Are CNNs translation invariant, equivariant, rotation invariant? How do you get rotation robustness?
- Walk through ResNet, 1×1 convs, depthwise separable convs, and transfer learning strategy for a small dataset.

17 Recurrent Networks: RNN, LSTM, GRU and Sequence Models

Companion resource for this section: [Christopher Olah: Understanding LSTM Networks](#)

17.1 Why sequences need memory

- Text, speech, clickstreams, stock prices and sensor logs are **ordered** and of **variable length**. Meaning depends on what came before: “not” changes “good”; yesterday’s price informs today’s.
- A dense network needs a fixed size input and has no notion of order. We need a model that reads one step at a time and carries a summary (memory) forward.

17.2 The vanilla RNN

- At each time step t : $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$, output $y_t = W_{hy}h_t + b_y$. The hidden state h_t is the memory.
- The **same weights** are used at every step (weight sharing over time), so the model handles any length.
- Unrolling: draw the network once per time step; it becomes a deep feedforward network whose depth equals the sequence length.
- Worked scalar example: $w_{hh} = 0.5$, $w_{xh} = 1$, $b = 0$, $h_0 = 0$, inputs $x_1 = 1$, $x_2 = -1$.
 - $h_1 = \tanh(0.5 \cdot 0 + 1) = \tanh 1 = 0.762$.
 - $h_2 = \tanh(0.5 \cdot 0.762 - 1) = \tanh(-0.619) = -0.551$.
- Parameters (input d , hidden h , excluding output layer): $hd + h^2 + h$. With $d = 10$, $h = 20$: $200 + 400 + 20 = 620$.
- Shapes of problems: **many to one** (sentiment of a review), **one to many** (image captioning), **many to many aligned** (POS tagging, frame labelling in speech), **many to many unaligned / seq2seq** (translation).

17.3 Backpropagation Through Time (BPTT) (reported InfoEdge interview topic)

- Unroll over T steps, compute the loss (summed over steps), and apply ordinary backprop on the unrolled graph. Gradients for the shared weights are **summed** over all time steps.
- Gradient from step t back to step k contains the product $\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod W_{hh}^\top \text{diag}(\tanh'(\cdot))$.
- If the factors are below 1: **vanishing** (for example $0.5^{30} \approx 9 \times 10^{-10}$), so the RNN cannot learn long range dependencies (“The candidate who applied last year from Pune ... was selected”). Above 1: **exploding**.
- Truncated BPTT**: backpropagate only through the last k steps (for example 50 to 200) to limit memory and computation, while still carrying the hidden state forward.
- Fixes: gradient clipping (exploding), gated cells LSTM/GRU (vanishing), good initialisation (orthogonal W_{hh}), attention and Transformers.

17.4 LSTM: a memory with gates

- Adds a **cell state** c_t , a conveyor belt that information can travel along with only small, controlled edits. Three **gates** (sigmoids between 0 and 1) decide what to erase, write and reveal. With $z = [h_{t-1}, x_t]$:
 - Forget gate $f_t = \sigma(W_f z + b_f)$: how much old memory to keep.
 - Input gate $i_t = \sigma(W_i z + b_i)$ and candidate $\tilde{c}_t = \tanh(W_c z + b_c)$: what new information to write.
 - Cell update: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (additive).
 - Output gate $o_t = \sigma(W_o z + b_o)$; hidden state $h_t = o_t \odot \tanh(c_t)$.
- Worked example: $c_{t-1} = 2$, $f = 0.5$, $i = 0.8$, $\tilde{c} = 0.5$, $o = 0.5$. $c_t = 0.5 \cdot 2 + 0.8 \cdot 0.5 = 1.4$; $h_t = 0.5 \tanh(1.4) = 0.5 \times 0.885 = 0.443$.
- Why it fixes vanishing gradients: $\partial c_t / \partial c_{t-1} = f_t$ (plus smaller terms). If the network learns $f \approx 1$, the gradient passes almost unchanged over many steps: $0.9^{10} = 0.35$ vs a vanilla factor like $0.5^{10} = 0.001$. Initialising the forget bias to 1 helps early training.
- Parameters: four gate blocks, each $h(d + h) + h$: total $4[h(d + h) + h]$. With $d = 10$, $h = 20$: $4 \times 620 = 2,480$.

17.5 GRU: a lighter LSTM

- Two gates, no separate cell state. Update gate z_t (how much to replace the state), reset gate r_t (how much past to use when proposing new content):

$$\tilde{h}_t = \tanh(Wx_t + U(r_t \odot h_{t-1})), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

- About 3/4 of an LSTM's parameters ($3[h(d + h) + h]$), faster, often similar accuracy. LSTM can be better on long, complex dependencies; try both.

17.6 Variants and training tricks

- Bidirectional RNN**: one pass left to right, one right to left; concatenate states (size $2h$). Great when the whole sequence is available (NER, classification, offline speech); not for real time forecasting or generation.
- Stacked** (deep) RNNs: the hidden sequence of layer 1 is the input of layer 2.
- Padding and masking**: batch sequences of different lengths by padding, and mask pads so they do not affect loss or state (PyTorch `pack_padded_sequence`).
- Teacher forcing**: in seq2seq training feed the true previous token to the decoder rather than its own prediction; fast, but causes exposure bias at inference.
- Seq2seq with encoder decoder**: the encoder compresses the input into a vector; the fixed size bottleneck hurts long inputs, which motivated **attention** (Bahdanau 2014), and later Transformers (Chapter 18).

17.7 Where LSTMs shine: speech and time series (reported InfoEdge OA and interview topics)

- Speech recognition**: audio becomes a sequence of spectrogram frames; (bidirectional) LSTMs map frames to characters/phonemes, trained with **CTC loss** (Connectionist Temporal Classification), which handles the unknown alignment between many frames and few characters. LSTMs powered Google voice search (2015) and many ASR systems before Transformers (Whisper, Conformer).
- Other classic LSTM uses: handwriting recognition, language modelling, machine translation (pre 2017), music generation.
- Time series with deep learning**: build supervised samples with a sliding window (last L values $\rightarrow$ next value). With N points, lookback L and one step ahead: $N - L$ samples. Scale using training data only; split by time (never shuffle across the split); walk forward validation.
- Options: RNN, LSTM, GRU, 1D CNN / TCN (dilated causal convs), Transformers (Informer, TFT). Strong baselines to beat first: naive last value, seasonal naive, ARIMA/ETS, gradient boosting with lag features.

17.8 Why Transformers replaced RNNs

- RNNs are sequential: step t waits for step $t - 1$, so training cannot be parallelised over time. Transformers process all positions at once.
- Path length between distant tokens: $O(T)$ in RNNs vs $O(1)$ in self attention, which makes long range dependencies easier.
- RNNs still have advantages: $O(1)$ memory per step at inference, natural streaming; modern state space models (Mamba) revive this idea.

Where this shows up

- Session based job recommendation (sequence of viewed jobs $\rightarrow$ next job), forecasting application volumes, typing autocomplete for job search, voice search.

Formula sheet

RNN: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$; params $hd + h^2 + h$.
 LSTM: $c_t = f \odot c_{t-1} + i \odot \tilde{c}$; $h_t = o \odot \tanh c_t$; params $4[h(d + h) + h]$.
 GRU: $h_t = (1 - z)h_{t-1} + z\tilde{h}$; params $3[h(d + h) + h]$. BiRNN state $2h$.
 Sliding window samples $N - L$ (one step ahead). BPTT gradient $\propto \prod \partial h_j / \partial h_{j-1}$.

17.9 Question bank: RNN, LSTM, GRU

PYQ status for this chapter

Reported InfoEdge items: OA (IIT BHU) "LSTM is used in which of these" (speech recognition); OA (IITR) RNN questions; interviews (IIT BHU) deep learning for time series with RNN, GRU, LSTM and Backpropagation Through Time, plus vanishing and exploding gradients. These come first.

Q17.1 MCQ · E

★ **PYQ (OA: where LSTMs are used)** LSTMs are best known for which application among these?

- (A) Speech recognition
- (B) K-Means clustering
- (C) Image thresholding
- (D) Sorting numbers

Solution

- Speech is a long sequence of audio frames with long range context; (bi)LSTMs with CTC loss were the standard ASR model for years. The others are not sequence modelling tasks.

Answer: A

Q17.2 MSQ · M

★ **PYQ (OA: LSTM applications)** Which are typical LSTM applications?

- (A) Speech recognition
- (B) Machine translation (pre Transformer)
- (C) Handwriting recognition
- (D) Principal component analysis

Solution

- All three are sequence to sequence or sequence labelling problems. PCA is linear algebra, not a sequence model (D false).

Answer: A, B, C

Q17.3 MCQ · M

★ **PYQ (INT: Backpropagation Through Time)** Backpropagation Through Time is

- (A) a separate optimiser for RNNs
- (B) ordinary backprop on the RNN unrolled over time, with gradients of shared weights summed across steps
- (C) a way to avoid computing gradients
- (D) only used for CNNs

Solution

- Unroll, backprop, sum the per step contributions for W_{xh} , W_{hh} , b (they are the same parameters at every step).

Answer: B

Q17.4 MSQ · M

★ **PYQ (INT: vanishing gradients in RNNs)** Why do vanilla RNNs struggle with long range dependencies?

- (A) Gradients are products of many Jacobians across time and can shrink exponentially.
- (B) tanh derivatives are at most 1.
- (C) They have too few time steps.
- (D) They cannot share weights.

Solution

- Repeated multiplication by $W_{hh}^T \text{diag}(\tanh')$ with norm below 1 decays exponentially with distance (A);

$\tanh' \leq 1$ (and near 0 when saturated) contributes (B).

- C and D false: they handle any length and do share weights.

Answer: A, B

Q17.5 MSQ · M

★ **PYQ (INT: DL for time series (RNN, GRU, LSTM))** Forecasting daily job applications with an LSTM. Which practices are correct?

- (A) Split train/validation by time, not randomly.
- (B) Fit the scaler on training data only.
- (C) Create samples with a sliding window of past values.
- (D) Shuffle and split randomly to maximise data use.

Solution

- Random splitting lets the model peek at the future (leakage), inflating validation scores (D false).

Answer: A, B, C

Q17.6 NAT · E

★ **PYQ (INT: sliding window samples for time series)** 1,000 daily values, lookback 30, predict the next single day. Number of training samples?

Solution

- First sample uses days 1 to 30 to predict day 31; last uses days 970 to 999 to predict day 1000. Count = $1000 - 30 = 970$.

Answer: 970

Q17.7 MSQ · M

★ **PYQ (INT: GRU vs LSTM)** Which are true?

- (A) GRU has two gates (update, reset); LSTM has three (forget, input, output).
- (B) GRU has no separate cell state.
- (C) GRU typically has about 25% fewer parameters than an LSTM of the same size.
- (D) GRU cannot be bidirectional.

Solution

- 3[·] vs 4[·] blocks, so GRU has 3/4 of the parameters (C). Any RNN cell can be made bidirectional (D false).

Answer: A, B, C

Q17.8 MCQ · M

★ **PYQ (INT: how the LSTM avoids vanishing gradients)** The main reason LSTMs mitigate vanishing gradients is

- (A) they use ReLU
- (B) the additive cell state update, whose gradient path is scaled by the forget gate and can stay close to 1
- (C) they have fewer parameters
- (D) they use dropout

Solution

- $c_t = f_t c_{t-1} + i_t \tilde{c}_t$, so $\partial c_t / \partial c_{t-1} \approx f_t$; with $f \approx 1$ information and gradient flow across many steps (constant error carousel).

Answer: B

Q17.9 MSQ · M

★ **PYQ (OA (IITR): RNN basics)** About vanilla RNNs, which are true?

- (A) The same weights are applied at every time step.

- (B) They can process sequences of variable length.
- (C) The hidden state summarises the past.
- (D) Training is easily parallelised across time steps.

Solution

- Each step depends on the previous hidden state, so steps are computed sequentially (D false).

Answer: A, B, C**Q17.10** NAT · M

Scalar RNN $h_t = \tanh(0.5h_{t-1} + x_t)$, $h_0 = 0$, $x_1 = 1$, $x_2 = -1$. Compute h_2 (3 decimals).

Solution

- $h_1 = \tanh(1) = 0.762$.
- $h_2 = \tanh(0.381 - 1) = \tanh(-0.619) = -0.551$.

Answer: -0.551**Q17.11** NAT · E

Vanilla RNN with input size 10, hidden size 20 (with one bias vector, no output layer). Parameters?

Solution

- W_{xh} : $20 \times 10 = 200$; W_{hh} : $20 \times 20 = 400$; b : 20. Total 620.

Answer: 620**Q17.12** NAT · M

LSTM with input 10, hidden 20 (one bias per gate). Parameters?

Solution

- Each gate: $20 \times (10 + 20) + 20 = 620$. Four gate blocks (f, i, o, candidate): $4 \times 620 = 2,480$.
- (PyTorch keeps two bias vectors per gate, giving 2,560.)

Answer: 2,480**Q17.13** NAT · M

Keras LSTM(128) on inputs with 64 features. Parameters?

Solution

- $4 \times [128 \times (64 + 128) + 128] = 4 \times [24,576 + 128] = 4 \times 24,704 = 98,816$.

Answer: 98,816**Q17.14** NAT · M

GRU with input 10, hidden 20 (single bias per gate). Parameters?

Solution

- $3 \times 620 = 1,860$.

Answer: 1,860**Q17.15** NAT · M

LSTM step: $c_{t-1} = 2$, $f = 0.5$, $i = 0.8$, $\tilde{c} = 0.5$, $o = 0.5$. Compute c_t and h_t (3 decimals).

Solution

- $c_t = 0.5 \cdot 2 + 0.8 \cdot 0.5 = 1 + 0.4 = 1.4$.
- $\tanh(1.4) = 0.885$; $h_t = 0.5 \times 0.885 = 0.443$.

Answer: $c_t = 1.4$, $h_t = 0.443$

Q17.16 NAT · M

Gradient factor through the cell state across 10 steps with forget gate constant 0.9, compared with a vanilla factor 0.5 per step (3 decimals each)?

Solution

- $0.9^{10} = 0.349$. $0.5^{10} = 1/1024 \approx 0.001$. Over 10 steps the LSTM retains about 350 times more gradient.

Answer: 0.349 vs 0.001

Q17.17 NAT · M

Vanilla RNN gradient factor 0.5 per step. After how many steps does it first fall below 10^{-3} ?

Solution

- $0.5^9 = 1/512 \approx 0.00195$; $0.5^{10} = 1/1024 \approx 0.00098 < 10^{-3}$.

Answer: 10

Q17.18 MSQ · M

LSTM gates:

- (A) The forget gate decides how much of the previous cell state to keep.
- (B) The input gate controls how much new candidate content is written.
- (C) The output gate controls how much of the cell is exposed as h_t .
- (D) Gates use ReLU so they can exceed 1.

Solution

- Gates use sigmoid to lie in $(0, 1)$, acting as soft switches (D false).

Answer: A, B, C

Q17.19 MCQ · M

Why is the forget gate bias often initialised to 1?

- (A) To forget everything at the start
- (B) So the gate starts near open (keep memory), helping gradients flow early in training
- (C) To speed up inference
- (D) It is required for GRUs

Solution

- $\sigma(1) \approx 0.73$ vs $\sigma(0) = 0.5$; starting biased toward remembering eases learning long dependencies.

Answer: B

Q17.20 MCQ · M

A bidirectional LSTM with hidden size 128 per direction outputs vectors of size

- (A) 64
- (B) 128
- (C) 256
- (D) 512

Solution

- Forward and backward states are concatenated: $2 \times 128 = 256$.

Answer: C

Q17.21 MCQ · M

Bidirectional RNNs are unsuitable for

- (A) named entity recognition on complete sentences
- (B) real time next step forecasting
- (C) sentence classification
- (D) offline transcription

Solution

- The backward direction needs future inputs, which are unavailable when predicting the next step in real time.

Answer: B**Q17.22** MCQ · MTruncated BPTT with window k means

- (A) the sequence is cut and the hidden state reset every k steps
- (B) gradients are propagated back at most k steps while the hidden state may still be carried forward
- (C) only k parameters are trained
- (D) the learning rate is divided by k

Solution

- It bounds compute and memory; dependencies longer than k can still influence the forward state but receive no direct gradient.

Answer: B**Q17.23** MSQ · M

Match problem shape to example:

- (A) Many to one: sentiment of a review
- (B) One to many: image captioning
- (C) Many to many (aligned): POS tagging
- (D) One to one: machine translation

Solution

- Translation is many to many with unequal lengths (seq2seq), not one to one (D false).

Answer: A, B, C**Q17.24** MCQ · M

Teacher forcing in seq2seq training means

- (A) the teacher model generates labels
- (B) feeding the ground truth previous token to the decoder instead of its own prediction
- (C) freezing the encoder
- (D) using a larger learning rate

Solution

- Speeds up and stabilises training; the mismatch at inference (where it must use its own outputs) is called exposure bias; scheduled sampling eases it.

Answer: B**Q17.25** MCQ · M

The fixed length context vector in a basic encoder decoder RNN causes

- (A) better long sentence translation
- (B) an information bottleneck for long inputs, which attention solves
- (C) no problems
- (D) faster training

Solution

- Squeezing a 60 word sentence into one vector loses detail. Attention lets the decoder look at all encoder states at every step.

Answer: B**Q17.26** MCQ · H

CTC loss in speech recognition solves

- (A) class imbalance
- (B) unknown alignment between many input frames and a shorter output label sequence
- (C) overfitting
- (D) exploding gradients

Solution

- CTC sums over all alignments (with a blank symbol and merging repeats) that collapse to the target transcript, so no frame level labels are needed.

Answer: B**Q17.27** MSQ · M

Handling variable length sequences in a batch:

- (A) Pad to a common length.
- (B) Mask padded positions in the loss.
- (C) Use packed sequences so the RNN skips pads.
- (D) Truncate every sequence to length 1.

Solution

- D destroys the information (false).

Answer: A, B, C**Q17.28** MSQ · H

Advantages of Transformers over RNNs:

- (A) Parallel computation across positions during training
- (B) Constant path length between any two positions
- (C) Lower memory for very long sequences at every layer
- (D) Easier modelling of long range dependencies

Solution

- Self attention is $O(T^2)$ in memory, so for very long sequences it uses *more* memory than an RNN's $O(1)$ state per step (C false).

Answer: A, B, D**Q17.29** MCQ · M

Why is tanh (rather than ReLU) the default activation in vanilla RNN and LSTM hidden updates?

- (A) It is faster
- (B) Its bounded, zero centred output keeps the recurrent state from blowing up over many steps
- (C) ReLU is not differentiable
- (D) Tradition only; no reason

Solution

- Repeatedly applying an unbounded ReLU with W_{hh} of norm above 1 can explode. (IRNN with ReLU and identity init works but needs care.)

Answer: B**Q17.30** MSQ · M

Strong baselines to beat before trusting an LSTM forecaster:

- (A) Naive (last value)
- (B) Seasonal naive (same day last week)
- (C) ARIMA / exponential smoothing
- (D) Gradient boosting with lag features

Solution

- Deep models often lose to these on small, univariate business series. All four are standard.

Answer: A, B, C, D

Q17.31 MCQ · M

In PyTorch `nn.LSTM(input_size=64, hidden_size=128, batch_first=True)`, an input of shape $(32, 50, 64)$ gives output of shape

- (A) $(32, 50, 128)$
- (B) $(32, 128)$
- (C) $(50, 32, 64)$
- (D) $(32, 50, 512)$

Solution

- Batch 32, 50 time steps, hidden 128 per step. The final states h_n, c_n have shape $(1, 32, 128)$.

Answer: A

Q17.32 MSQ · M

Gradient clipping in RNN training:

- (A) Rescales the gradient when its norm exceeds a threshold.
- (B) Addresses exploding gradients.
- (C) Addresses vanishing gradients.
- (D) Is a one line call in PyTorch (`clip_grad_norm_`).

Solution

- Clipping cannot enlarge a vanishing gradient (C false).

Answer: A, B, D

Q17.33 MCQ · M

A stacked 2 layer LSTM means

- (A) two LSTMs run on different inputs
- (B) the output sequence of layer 1 is the input sequence of layer 2
- (C) the hidden size is doubled
- (D) the sequence is processed twice in reverse

Solution

- Depth over layers gives more abstract temporal features; dropout between layers is common.

Answer: B

Q17.34 MSQ · H

For session based “next job to view” prediction, which models are reasonable?

- (A) GRU4Rec style RNN over the session’s job ID embeddings
- (B) Transformer (SASRec) over the session
- (C) Plain K-Means
- (D) Markov chain / item to item co occurrence baseline

Solution

- GRU4Rec and SASRec are standard; co occurrence and Markov models are strong baselines. K-Means does not predict the next item (C false).

Answer: A, B, D

Q17.35 MCQ · H

In a GRU, if the update gate $z_t = 0$ then

- (A) $h_t = \tilde{h}_t$

- (B) $h_t = h_{t-1}$ (the state is copied unchanged)
 (C) $h_t = 0$
 (D) the reset gate is ignored for all future steps

Solution

- $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t = h_{t-1}$. This copy path is what lets GRUs carry information and gradients across long gaps. (Some papers swap the roles of z and $1 - z$; the idea is the same.)

Answer: B**Q17.36** NAT · H

Seq2seq: an input sentence has 20 tokens and the output 15. With Bahdanau attention, how many attention weights are computed in total during decoding?

Solution

- Each of 15 decoder steps computes one weight per encoder position: $15 \times 20 = 300$.

Answer: 300**Interview angle**

- Draw an unrolled RNN, write BPTT, and show where vanishing/exploding comes from.
- Draw the LSTM cell, explain each gate, and why the cell state helps gradients. LSTM vs GRU.
- How would you forecast weekly applications with deep learning? Windowing, scaling, validation, baselines.
- Why is LSTM used in speech recognition? What is CTC?
- Why did Transformers replace RNNs, and where are RNNs still useful?

18 Transformers and Attention

Companion resource for this section: [Jay Alammar: The Illustrated Transformer](#)

18.1 The idea of attention, in plain words

- When you read “The recruiter called the candidate because she liked her resume”, you resolve “she” by *looking back* at the relevant words. Attention lets every token look at every other token and pull in the information it needs.
- Library analogy: each token issues a **query** (what am I looking for?). Every token also has a **key** (what do I contain?) and a **value** (the content I hand over). Compare the query to all keys, turn the match scores into weights, and take the weighted average of the values.
- Queries, keys and values are learned linear projections of the token vectors: $Q = XW_Q$, $K = XW_K$, $V = XW_V$.

18.2 Scaled dot product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- Step 1: scores $QK^\top$ ($n \times n$ for n tokens): dot product = similarity.
- Step 2: divide by $\sqrt{d_k}$.
- Step 3: softmax each row, so each token’s weights are positive and sum to 1.
- Step 4: multiply by V : each output row is a weighted mix of value vectors.
- Worked example** ($d_k = 2$): query $q = (1, 0)$; keys $k_1 = (1, 0)$, $k_2 = (0, 1)$; values $v_1 = (10, 0)$, $v_2 = (0, 10)$.
 - Scores: $q \cdot k_1 = 1$, $q \cdot k_2 = 0$. Scaled: $1/\sqrt{2} = 0.707$ and 0.
 - Softmax: $e^{0.707} = 2.028$, $e^0 = 1$; weights $2.028/3.028 = 0.670$ and 0.330 .
 - Output = $0.670(10, 0) + 0.330(0, 10) = (6.70, 3.30)$.
- Why divide by $\sqrt{d_k}$:** if the components of q and k are independent with mean 0 and variance 1, then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k . For $d_k = 64$ scores have standard deviation 8; softmax of such large numbers is nearly one hot, its gradients are nearly zero (saturation), and training stalls. Dividing by $\sqrt{d_k}$ brings the variance back to 1.

18.3 Multi head attention

- Run h attention operations in parallel, each with its own $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$ projecting to $d_k = d_{\text{model}}/h$ dimensions. Concatenate the h outputs and project with W_O .
- Why: different heads can specialise (one tracks syntax such as subject verb, another coreference, another nearby tokens). Same total cost as one big head.
- Parameters (with biases): $4(d_{\text{model}}^2 + d_{\text{model}})$. For $d = 512$: $4(262,144 + 512) = 1,050,624$.
- Base Transformer: $d_{\text{model}} = 512$, $h = 8$, $d_k = 64$, $d_{ff} = 2048$, 6 encoder + 6 decoder layers.

18.4 The Transformer block

- Encoder layer: multi head self attention $\rightarrow$ add residual $\rightarrow$ LayerNorm $\rightarrow$ position wise feed forward network (FFN: $\max(0, xW_1 + b_1)W_2 + b_2$, expand to $d_{ff} = 4d$ then back) $\rightarrow$ add residual $\rightarrow$ LayerNorm.
- FFN parameters: $2d d_{ff} + d_{ff} + d$. For $d = 512$, $d_{ff} = 2048$: 2,099,712.
- Rule of thumb: one layer has about $12d^2$ parameters ($4d^2$ attention + $8d^2$ FFN). BERT-base: $12 \times 768^2 \times 12 \approx 85\text{M}$ plus about 24M embeddings $\approx 110\text{M}$.
- Pre-LN** (LayerNorm before each sub layer, used in GPT-2 and later) trains more stably than the original Post-LN, often without warmup tricks.
- Decoder layer: **masked** self attention (each position sees only earlier positions), then **cross attention** (queries from the decoder, keys and values from the encoder output), then FFN.

18.5 Positional information

- Attention is permutation invariant: shuffle the tokens and each output is just shuffled. Order must be injected.
- **Sinusoidal** encoding (original paper): $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$, added to the embeddings. Each dimension is a wave of a different wavelength; relative offsets become linear functions; no parameters; may extrapolate.
- **Learned** absolute position embeddings (BERT, GPT-2): a table of size $\text{max_len} \times d$.
- **Relative / rotary (RoPE)** (LLaMA and most modern LLMs): rotate q and k by position dependent angles so their dot product depends on the relative distance. ALiBi adds a distance based bias.

18.6 Masks

- **Padding mask**: ignore [PAD] tokens by setting their scores to $-\infty$ before softmax (weight becomes 0).
- **Causal (look ahead) mask**: upper triangle set to $-\infty$ so token t attends only to positions $\leq t$. For n tokens, allowed entries $= n(n+1)/2$.

18.7 Three families

- **Encoder only** (BERT, RoBERTa): bidirectional; classification, NER, embeddings, retrieval.
- **Decoder only** (GPT, LLaMA): causal; generation; trained on next token prediction.
- **Encoder decoder** (original Transformer, T5, BART): translation, summarisation.

18.8 Cost and efficiency

- Self attention costs $O(n^2d)$ time and $O(n^2)$ memory per head for the score matrix. $n = 512$: 262,144 scores per head per layer; doubling length quadruples it.
- Remedies: FlashAttention (exact, IO aware, avoids materialising the matrix), sparse/local attention (Longformer), linear attention approximations, sliding windows.
- **KV cache** at generation time: store past keys and values so each new token computes attention in $O(n)$ instead of recomputing everything.
- Compared with RNNs: all positions processed in parallel during training; path length between any two tokens is 1.

18.9 Generation and decoding

- Greedy (pick the argmax), beam search (keep top B partial sequences), sampling with temperature T (divide logits by T : $T < 1$ sharper, $T > 1$ flatter), top- k , top- p (nucleus).
- Example: logits $(2, 0)$. $T = 1$: $p = 1/(1 + e^{-2}) = 0.881$. $T = 2$: logits $(1, 0)$, $p = 0.731$.

18.10 Transformers beyond text

- **ViT**: split an image into 16×16 patches, flatten and linearly embed each as a token, add a [CLS] token and position embeddings. 224×224 gives $14 \times 14 = 196$ patches $+1 = 197$ tokens.
- Speech (Whisper, Conformer), tabular (FT-Transformer), recommendation (SASRec, BERT4Rec), multimodal (CLIP).

Where this shows up

- Fine tuned BERT style encoders for resume parsing and job classification; bi-encoders for semantic job search; LLMs (decoder only) for job description generation, candidate summaries and chat assistants; SASRec style sequential recommendation.

Formula sheet

$\text{softmax}(QK^\top / \sqrt{d_k})V$; $\text{Var}(q \cdot k) = d_k$ for unit variance components.
 $d_k = d_{\text{model}}/h$. MHA params $4(d^2 + d)$; FFN $2dd_{ff} + d_{ff} + d$; layer $\approx 12d^2$.
 Attention memory $O(n^2)$ per head; causal allowed entries $n(n+1)/2$.
 Sinusoidal PE: $\sin(\text{pos}/10000^{2i/d})$, $\cos(\cdot)$. ViT tokens $(H/P)(W/P) + 1$. Temperature: $\text{softmax}(z/T)$.

18.11 Question bank: Transformers and attention

PYQ status for this chapter

Transformers were reported as an InfoEdge interview topic on Glassdoor, and the IIT KGP Foresight 2025 write up says Round 2 required explaining architectures on pen and paper with the maths, specifically knowing Transformers in depth. These interview style items come first, followed by OA style numericals.

Q18.1 MSQ · M

★ PYQ (INT: explain the Transformer) Which statements about the original Transformer are true?

- (A) It uses no recurrence or convolution; it relies on attention.
- (B) Each layer has self attention followed by a position wise feed forward network, each with a residual connection and LayerNorm.
- (C) Positional encodings are needed because attention is order agnostic.
- (D) It processes tokens strictly one at a time during training.

Solution

- “Attention is All You Need” (A); block structure (B); permutation invariance of attention requires position information (C).
- Training processes all positions in parallel (with a causal mask in the decoder) (D false).

Answer: A, B, C

Q18.2 MCQ · M

★ PYQ (INT: why scale by $\sqrt{d_k}$) The $\sqrt{d_k}$ scaling in attention is used because

- (A) it makes softmax faster
- (B) dot products grow in variance with d_k , pushing softmax into saturated regions with tiny gradients
- (C) it normalises the values V
- (D) it is needed for positional encodings

Solution

- With unit variance components, $\text{Var}(q \cdot k) = d_k$. Dividing by $\sqrt{d_k}$ restores unit variance so the softmax stays in a region with useful gradients.

Answer: B

Q18.3 NAT · M

★ PYQ (INT (pen and paper): compute attention) $q = (1, 0)$, $k_1 = (1, 0)$, $k_2 = (0, 1)$, $v_1 = (10, 0)$, $v_2 = (0, 10)$, $d_k = 2$. First component of the attention output (2 decimals)?

Solution

- Scaled scores: $1/\sqrt{2} = 0.707$, 0.
- Softmax: $e^{0.707} = 2.028$; weights $2.028/3.028 = 0.670$, 0.330.
- Output = $(6.70, 3.30)$; first component 6.70.

Answer: 6.70

Q18.4 NAT · M

★ PYQ (INT: effect of scaling) Same scores as above but without the $\sqrt{d_k}$ scaling. Weight on k_1 (3 decimals)?

Solution

- Softmax of $(1, 0)$: $e/(e+1) = 2.718/3.718 = 0.731$. Without scaling the distribution is sharper; with $d_k = 64$ the effect would be far stronger.

Answer: 0.731**Q18.5** NAT · M

★ **PYQ (INT: variance of dot products)** $q, k \in \mathbb{R}^{64}$ with independent components, mean 0, variance 1. Standard deviation of $q \cdot k$?

Solution

- Each product $q_i k_i$ has variance $1 \cdot 1 = 1$; 64 independent terms: variance 64; std = 8. After dividing by $\sqrt{64} = 8$, std is 1.

Answer: 8**Q18.6** MSQ · M

★ **PYQ (INT: multi head attention)** Which are true?

- (A) Each head uses its own learned projections for Q, K, V.
- (B) With $d_{model} = 512$ and 8 heads, each head has dimension 64.
- (C) Head outputs are concatenated and linearly projected.
- (D) More heads always multiply total compute by the number of heads.

Solution

- Because each head's dimension is d/h , total cost stays about the same as one full width head (D false).

Answer: A, B, C**Q18.7** MSQ · M

★ **PYQ (INT: encoder vs decoder)** Which are correct?

- (A) The decoder's self attention is masked so a position cannot see future tokens.
- (B) Cross attention takes queries from the decoder and keys/values from the encoder.
- (C) BERT is encoder only; GPT is decoder only.
- (D) The encoder uses a causal mask.

Solution

- The encoder attends bidirectionally (D false).

Answer: A, B, C**Q18.8** MSQ · M

★ **PYQ (INT: positional encodings)** Which statements about positional information are true?

- (A) Sinusoidal encodings use sines and cosines of different frequencies.
- (B) BERT uses learned absolute position embeddings.
- (C) Rotary embeddings make attention scores depend on relative position.
- (D) Without any positional signal a Transformer encoder is permutation equivariant.

Solution

- All four are correct. (D: permuting the input tokens simply permutes the outputs, so "dog bites man" and "man bites dog" would look the same as bags.)

Answer: A, B, C, D**Q18.9** MSQ · H

★ **PYQ (INT: why Transformers beat RNNs)** Which are advantages of self attention over recurrence?

- (A) Parallel training across positions
- (B) Constant maximum path length between tokens
- (C) Linear memory in sequence length
- (D) Better modelling of long range dependencies in practice

Solution

- Attention scores are $O(n^2)$ in memory, not linear (C false).

Answer: A, B, D

Q18.10 NAT · E

$d_{model} = 768$, 12 heads. Per head dimension?

Solution

- $768/12 = 64$.

Answer: 64

Q18.11 NAT · M

Parameters of one multi head attention layer with $d_{model} = 512$ (Q, K, V, O projections with biases)?

Solution

- Each projection: $512 \times 512 + 512 = 262,656$.
- Four of them: $4 \times 262,656 = 1,050,624$. (Number of heads does not change this.)

Answer: 1,050,624

Q18.12 NAT · M

FFN parameters for $d = 512$, $d_{ff} = 2048$ (with biases)?

Solution

- W_1 : $512 \times 2048 = 1,048,576$, b_1 : 2048. W_2 : $2048 \times 512 = 1,048,576$, b_2 : 512.
- Total $2,097,152 + 2,560 = 2,099,712$.

Answer: 2,099,712

Q18.13 NAT · M

Using the $12d^2$ rule, parameters of 12 layers with $d = 768$ (millions, 1 decimal)?

Solution

- $12 \times 768^2 = 12 \times 589,824 = 7,077,888$ per layer.
- $\times 12 = 84,934,656 \approx 84.9$ M. Add about 24M embeddings: BERT-base's 110M.

Answer: 84.9

Q18.14 NAT · M

Sequence length 512, 12 heads. Attention scores computed per layer?

Solution

- Per head $512^2 = 262,144$. Times 12 = 3,145,728.

Answer: 3,145,728

Q18.15 NAT · E

By what factor does attention score memory grow when sequence length goes from 1,024 to 4,096?

Solution

- Quadratic: $(4096/1024)^2 = 4^2 = 16$.

Answer: 16

Q18.16 NAT · E

Causal mask on 4 tokens: how many (query, key) pairs are allowed?

Solution

- Token 1 sees 1, token 2 sees 2, token 3 sees 3, token 4 sees 4: $1 + 2 + 3 + 4 = 10 = 4 \cdot 5/2$.

Answer: 10**Q18.17** MCQ · M

How is a causal mask applied?

- (A) Multiply attention weights by 0 after softmax and do not renormalise
- (B) Add $-\infty$ to scores of future positions before softmax
- (C) Remove future tokens from the vocabulary
- (D) Set future values to 1

Solution

- $e^{-\infty} = 0$, so those weights vanish and the remaining weights still sum to 1.

Answer: B**Q18.18** NAT · MSinusoidal positional encoding: value of dimension $2i = 0$ at position 1 (3 decimals)?**Solution**

- $\sin(1/10000^0) = \sin(1) = 0.841$. At position 0 every sine dimension is 0 and every cosine dimension is 1.

Answer: 0.841**Q18.19** MSQ · M

Residual connections and LayerNorm in Transformers:

- (A) Residuals help gradients flow through deep stacks.
- (B) LayerNorm normalises across the features of each token.
- (C) Pre-LN places LayerNorm before each sub layer and usually trains more stably.
- (D) LayerNorm depends on the batch size.

Solution

- LayerNorm works per token, independent of batch (D false). That is why it replaced BatchNorm here.

Answer: A, B, C**Q18.20** MCQ · M

Each row of the attention weight matrix (after softmax)

- (A) sums to 1
- (B) sums to n
- (C) is one hot
- (D) contains negative values

Solution

- Softmax gives a probability distribution over keys for each query.

Answer: A**Q18.21** MSQ · M

The position wise FFN in a Transformer:

- (A) is applied independently to every token with shared weights
- (B) typically expands to $4d$ then projects back
- (C) mixes information across tokens
- (D) holds about two thirds of the layer's parameters

Solution

- Token mixing happens only in attention (C false). FFN $\approx 8d^2$ of $12d^2$ (D).

Answer: A, B, D

Q18.22 MCQ · M

The KV cache speeds up autoregressive generation by

- (A) caching the output tokens only
- (B) storing keys and values of previous tokens so each new step computes attention only for the new query
- (C) shrinking the vocabulary
- (D) removing the FFN

Solution

- Without it every step recomputes K and V for the whole prefix. The cost is memory proportional to $\text{layers} \times \text{length} \times d$.

Answer: B

Q18.23 NAT · M

Temperature sampling over two tokens with logits (2, 0). Probability of the first token at $T = 2$ (3 decimals)?

Solution

- Divide logits by 2: (1, 0). Probability = $e/(e+1) = 0.731$. At $T = 1$ it would be 0.881; higher temperature flattens.

Answer: 0.731

Q18.24 MSQ · M

Decoding strategies:

- (A) Greedy decoding picks the most likely token each step.
- (B) Beam search keeps several partial hypotheses.
- (C) Top- p sampling samples from the smallest set of tokens whose cumulative probability exceeds p .
- (D) Temperature above 1 makes outputs more deterministic.

Solution

- $T > 1$ flattens the distribution, making outputs more random (D false).

Answer: A, B, C

Q18.25 NAT · E

ViT with 224×224 images and 16×16 patches, plus a [CLS] token. Sequence length?

Solution

- $(224/16)^2 = 14^2 = 196$ patches, plus 1 = 197.

Answer: 197

Q18.26 MCQ · M

Which model family fits “generate a job description from a few bullet points” best?

- (A) Encoder only (BERT)
- (B) Decoder only or encoder decoder (GPT, T5)
- (C) K-Means
- (D) CNN

Solution

- Text generation needs an autoregressive decoder.

Answer: B

Q18.27 MCQ · M

Time complexity of self attention for sequence length n and model width d is

- (A) $O(nd)$
- (B) $O(n^2d)$
- (C) $O(nd^2)$ only
- (D) $O(\log n)$

Solution

- QK^T is $n \times d$ times $d \times n$: n^2d . (Projections add $O(nd^2)$.)

Answer: B

Q18.28 MSQ · H

FlashAttention:

- (A) computes exact attention (not an approximation)
- (B) avoids storing the full $n \times n$ matrix in GPU high bandwidth memory by tiling
- (C) changes model outputs significantly
- (D) speeds up training and reduces memory

Solution

- Same result up to floating point (C false); IO aware tiling with online softmax (B).

Answer: A, B, D

Q18.29 MCQ · M

In cross attention of an encoder decoder model, the keys and values come from

- (A) the decoder's previous layer
- (B) the encoder's output
- (C) the vocabulary
- (D) positional encodings only

Solution

- Queries from the decoder ask "which source tokens matter for the token I am generating now?"

Answer: B

Q18.30 MCQ · M

Decoder only LLMs are trained with

- (A) masked language modelling
- (B) next token prediction (causal language modelling)
- (C) next sentence prediction only
- (D) contrastive clustering

Solution

- Maximise $\sum_t \log p(x_t | x_{<t})$: cross entropy on every position in parallel thanks to the causal mask.

Answer: B

Q18.31 MSQ · H

Learning rate warmup in Transformer training:

- (A) Starts with a small learning rate and increases it over the first steps.
- (B) Helps stability, especially with Post-LN and Adam's early noisy estimates.
- (C) The original paper used warmup then inverse square root decay.
- (D) Is never needed.

Solution

- D false; warmup is still standard.

Answer: A, B, C

Q18.32 MSQ · M

Padding masks:

- (A) prevent attention to [PAD] tokens
- (B) are implemented by adding $-\infty$ to their scores
- (C) are needed when batching sequences of different lengths
- (D) change the model's parameters

Solution

- Masks are inputs, not parameters (D false).

Answer: A, B, C

Q18.33 MCQ · H

Self attention vs a CNN on the same sequence: the self attention layer has

- (A) a fixed local receptive field
- (B) a global receptive field in one layer, with input dependent weights
- (C) no learnable parameters
- (D) no need for position information

Solution

- Attention weights are computed from the content (dynamic), unlike fixed convolution kernels, and every token can see every other token immediately.

Answer: B

Q18.34 NAT · H

Attention with two identical keys and $q \cdot k = 3$ for both, values $v_1 = 4$ and $v_2 = 8$ (scalars). Output?

Solution

- Equal scores give equal weights 0.5 regardless of scaling.
- Output = $0.5 \cdot 4 + 0.5 \cdot 8 = 6$.

Answer: 6

Q18.35 MSQ · M

LoRA and parameter efficient fine tuning of Transformers:

- (A) Freeze the base weights and learn low rank updates for some matrices (often W_Q, W_V).
- (B) Train a tiny fraction of the parameters.
- (C) Adapters can be merged into weights for inference.
- (D) Always require retraining the tokenizer.

Solution

- The tokenizer is unchanged (D false). See Chapter 11 for the parameter count.

Answer: A, B, C

Interview angle

- Derive scaled dot product attention and the variance argument for $\sqrt{d_k}$; compute a small example by hand.
- Draw a full encoder and decoder block; where do masks, residuals, LayerNorm, cross attention go?
- Positional encodings: sinusoidal vs learned vs RoPE.
- Count parameters of BERT-base roughly; why is attention $O(n^2)$ and what helps (FlashAttention, KV cache, sparse attention)?
- BERT vs GPT vs T5: which for classification, search, generation at Naukri?

19 Generative Adversarial Networks (GANs)

Companion resource for this section: [Google Machine Learning: Generative Adversarial Networks course](#)

19.1 The forger and the detective

- A **generator** G is a forger: it takes random noise z (for example 100 numbers from $\mathcal{N}(0, I)$) and produces a fake sample $G(z)$, such as an image.
- A **discriminator** D is a detective: it looks at a sample and outputs $D(x) \in (0, 1)$, the probability that x is real.
- They train against each other. D gets better at spotting fakes; G gets better at fooling D . If all goes well, G 's samples become indistinguishable from real data and D is reduced to guessing 0.5.
- Discriminative vs generative: a classifier learns $p(y | x)$; a generative model learns to produce samples from $p(x)$. GANs do this *implicitly*: no explicit density, just a sampler.

19.2 The minimax objective

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

- D maximises: push $D(x) \rightarrow 1$ on real, $D(G(z)) \rightarrow 0$ on fake. This is just binary cross entropy with labels real = 1, fake = 0.
- G minimises: push $D(G(z)) \rightarrow 1$.
- **Optimal discriminator** for a fixed G : $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$. Example: $p_{data}(x) = 0.6$, $p_g(x) = 0.2$ gives $D^* = 0.75$.
- Plugging D^* in: $V = -\log 4 + 2 JS(p_{data} \parallel p_g)$. Minimising over G minimises the **Jensen Shannon divergence**; the global optimum is $p_g = p_{data}$, where $D^* = 0.5$ everywhere and $V = -\log 4 \approx -1.386$.

19.3 Training loop in practice

- Alternate: (1) sample real batch and fake batch; update D with BCE (real label 1, fake label 0). (2) sample noise; update G to make D output "real" on its fakes, keeping D fixed.
- **Non saturating generator loss**: instead of minimising $\log(1 - D(G(z)))$, maximise $\log D(G(z))$ (minimise $-\log D(G(z))$). Early in training D easily rejects fakes ($D(G(z)) \approx 0$) and $\log(1 - D)$ is flat there, giving G almost no gradient; $-\log D$ is steep there.
- Example: $D(G(z)) = 0.1$: saturating loss $\log(0.9) = -0.105$ (tiny slope); non saturating $-\log 0.1 = 2.303$ with a strong gradient.

19.4 Why GANs are hard to train

- **Mode collapse**: G finds a few outputs that fool D and produces only those (every generated face looks alike). Fixes: minibatch discrimination, unrolled GANs, WGAN, diversity terms.
- **Non convergence / oscillation**: a two player game, not a single loss minimisation; losses may oscillate without improving.
- **Vanishing gradients** when D becomes too good (and supports of real and fake distributions barely overlap, where JS divergence is constant at $\log 2$).
- Loss values are poor indicators of sample quality; inspect samples and use metrics.
- Stabilising tricks: DCGAN guidelines (strided convs instead of pooling, BatchNorm, ReLU in G with tanh output, LeakyReLU in D), one sided label smoothing (real label 0.9), spectral normalisation, two time scale learning rates (TTUR), Adam with $\beta_1 = 0.5$.

19.5 Important variants

- **DCGAN**: convolutional GAN; G upsamples with transposed convolutions ($4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$).

- **Conditional GAN (cGAN)**: feed a label y to both G and D to control what is generated (“generate a 7”).
- **WGAN**: replaces JS with the **Wasserstein (Earth Mover’s) distance**; D becomes a *critic* outputting an unbounded score; requires the critic to be 1-Lipschitz (weight clipping in WGAN, gradient penalty in WGAN-GP). Gives meaningful gradients even with non overlapping supports and a loss that tracks quality; typically 5 critic steps per generator step.
- **pix2pix**: paired image to image translation (sketch $\rightarrow$ photo) with conditional GAN + L1 loss.
- **CycleGAN**: unpaired translation (horse $\leftrightarrow$ zebra) with **cycle consistency**: $F(G(x)) \approx x$.
- **StyleGAN**: mapping network and style injection at each resolution; photorealistic faces.

19.6 Evaluating GANs

- **Inception Score (IS)**: high when each sample is confidently classified (quality) and classes are diverse. Higher is better. Ignores real data statistics.
- **Fréchet Inception Distance (FID)**: fit Gaussians to Inception features of real and generated images and compute the Fréchet distance between them. **Lower is better**; sensitive to both quality and diversity. Standard metric.
- Precision/recall for generative models separate fidelity from coverage.

19.7 GANs vs VAEs vs diffusion models

- **VAE**: encoder decoder trained with a likelihood bound (ELBO = reconstruction + KL to prior). Stable, has a latent space and approximate likelihood; samples often blurry.
- **GAN**: sharp samples, fast single pass sampling; unstable training, mode collapse, no likelihood.
- **Diffusion models**: learn to reverse a gradual noising process; very high quality and diversity, stable training; slower sampling (many denoising steps, reduced by DDIM and distillation). Dominant for images today (Stable Diffusion).

Where this shows up

- Synthetic tabular data (CTGAN) for privacy or augmentation of rare classes, image enhancement and super resolution for property photos, and GAN detection in deepfake and fake profile photo screening.

Formula sheet

$V(D, G) = \mathbb{E} \log D(x) + \mathbb{E} \log(1 - D(G(z)))$. $D^* = p_{data} / (p_{data} + p_g)$.
 At optimum $p_g = p_{data}$, $D^* = 1/2$, $V = -\log 4$; $V = -\log 4 + 2JS$. $0 \leq JS \leq \log 2$.
 D loss (BCE) = $-\log D(x) - \log(1 - D(G(z)))$; non saturating G loss = $-\log D(G(z))$.
 WGAN: critic, Wasserstein distance, Lipschitz via clipping or gradient penalty. FID lower is better; IS higher is better.

19.8 Question bank: Generative Adversarial Networks

PYQ status for this chapter

GANs appear in the official InfoEdge syllabus (architectures list) but no report quotes a specific GAN question; the IIT BHU OA did include generative energy models (RBM/DBN), covered in Chapter 15. The IIT KGP report stresses explaining architectures with their maths, so the questions below emphasise the objective, the optimal discriminator and training failure modes in OA format.

Q19.1 MSQ · E

Which statements describe a GAN?

- (A) A generator maps random noise to samples.
- (B) A discriminator classifies samples as real or fake.
- (C) The two networks are trained adversarially.
- (D) The generator is trained with real labels for each image class.

Solution

- The vanilla GAN is unsupervised: the generator never sees class labels (conditional GANs add them) (ID false).

Answer: A, B, C

Q19.2 MCQ · E

In the GAN minimax game the discriminator

- (A) minimises $V(D, G)$
- (B) maximises $V(D, G)$
- (C) does not have a loss
- (D) generates images

Solution

- D maximises $\mathbb{E} \log D(x) + \mathbb{E} \log(1 - D(G(z)))$, i.e. minimises binary cross entropy. G minimises V .

Answer: B

Q19.3 NAT · M

For fixed G , at a point x with $p_{data}(x) = 0.6$ and $p_g(x) = 0.2$, the optimal discriminator output is?

Solution

- $D^* = 0.6 / (0.6 + 0.2) = 0.75$.

Answer: 0.75

Q19.4 NAT · E

At the global optimum of a GAN ($p_g = p_{data}$), what is $D^*(x)$?

Solution

- $D^* = p / (p + p) = 1/2$: the discriminator can only guess.

Answer: 0.5

Q19.5 NAT · M

Value of $V(D^*, G^*)$ at the global optimum (natural log, 3 decimals)?

Solution

- $V = \log 0.5 + \log(1 - 0.5) = 2 \log 0.5 = -\log 4 = -1.386$.

Answer: -1.386

Q19.6 MCQ · M

With the optimal discriminator, minimising the GAN objective over G is equivalent to minimising

- (A) the KL divergence $KL(p_g \| p_{data})$ only
- (B) the Jensen Shannon divergence between p_{data} and p_g
- (C) the mean squared error
- (D) the Wasserstein distance

Solution

- $V(D^*, G) = -\log 4 + 2 JS(p_{data} \| p_g)$. WGAN uses Wasserstein instead.

Answer: B

Q19.7 NAT · M

Discriminator BCE loss for one real sample with $D(x) = 0.9$ and one fake with $D(G(z)) = 0.2$ (natural log, 3 decimals).

Solution

- Real term: $-\ln 0.9 = 0.105$. Fake term: $-\ln(1 - 0.2) = -\ln 0.8 = 0.223$.
- Sum = 0.329 (0.328 with more rounding).

Answer: 0.329**Q19.8** NAT · M

Non saturating generator loss $-\log D(G(z))$ when $D(G(z)) = 0.1$ (3 decimals)?

Solution

- $-\ln 0.1 = 2.303$. Large loss with a steep gradient: exactly when G most needs a learning signal.

Answer: 2.303**Q19.9** MCQ · M

Why do practitioners train G to maximise $\log D(G(z))$ instead of minimising $\log(1 - D(G(z)))$?

- (A) It is cheaper to compute
- (B) Early on $D(G(z)) \approx 0$ where $\log(1 - D)$ is flat, so G gets vanishing gradients; $-\log D$ gives strong gradients there
- (C) It changes the optimum
- (D) It removes the need for D

Solution

- Both have the same fixed point, but very different gradients at the start of training.

Answer: B**Q19.10** MSQ · M

Mode collapse:

- (A) The generator produces limited variety, ignoring many modes of the data.
- (B) It can occur even when individual samples look realistic.
- (C) WGAN and minibatch discrimination can mitigate it.
- (D) It means the discriminator outputs NaN.

Solution

- D false; mode collapse is about lack of diversity.

Answer: A, B, C**Q19.11** MSQ · M

Why is GAN training unstable?

- (A) It is a two player minimax game, not simple minimisation.
- (B) A too strong discriminator can give the generator vanishing gradients.
- (C) Loss curves do not reliably reflect sample quality.
- (D) GANs have no parameters.

Solution

- D false.

Answer: A, B, C**Q19.12** MSQ · M

DCGAN architectural guidelines include

- (A) strided (transposed) convolutions instead of pooling
- (B) BatchNorm in both networks
- (C) tanh at the generator output and LeakyReLU in the discriminator
- (D) large fully connected hidden layers everywhere

Solution

- DCGAN removed fully connected hidden layers (D false).

Answer: A, B, C

Q19.13 NAT · M

A DCGAN generator starts from a 4×4 map and each transposed conv (kernel 4, stride 2, padding 1) doubles size. Layers needed to reach 64×64 ?

Solution

- $(n - 1) \cdot 2 - 2 + 4 = 2n$: $4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$, four layers.

Answer: 4

Q19.14 MCQ · M

A conditional GAN

- (A) conditions both G and D on extra information such as a class label
- (B) uses no noise
- (C) has no discriminator
- (D) is a VAE

Solution

- $G(z, y)$ and $D(x, y)$: you can ask for a specific digit or category.

Answer: A

Q19.15 MSQ · H

About WGAN:

- (A) It uses the Wasserstein (Earth Mover's) distance.
- (B) The critic outputs an unbounded real score rather than a probability.
- (C) The critic must be (approximately) 1-Lipschitz, enforced by weight clipping or a gradient penalty.
- (D) It uses sigmoid cross entropy for the critic.

Solution

- No sigmoid or log in the critic loss: critic maximises $\mathbb{E}[C(x)] - \mathbb{E}[C(G(z))]$ (D false).

Answer: A, B, C

Q19.16 MCQ · H

Why does WGAN give useful gradients when real and fake distributions do not overlap?

- (A) JS divergence is constant ($\log 2$) there, but Wasserstein distance still decreases smoothly as distributions move closer
- (B) Wasserstein distance is always zero
- (C) It uses larger learning rates
- (D) It removes the generator

Solution

- Two point masses at distance θ : JS is $\log 2$ for any $\theta \neq 0$ (flat), Wasserstein equals $|\theta|$ (informative slope).

Answer: A

Q19.17 NAT · M

Range of the Jensen Shannon divergence (natural log): give the maximum value (3 decimals).

Solution

- $0 \leq JS \leq \ln 2 = 0.693$; the maximum occurs for disjoint supports.

Answer: 0.693

Q19.18 MCQ · M

In WGAN training the critic is typically updated

- (A) once every 5 generator steps
- (B) about 5 times per generator step
- (C) never
- (D) only at the end

Solution

- A well trained critic gives a good estimate of the distance, so it gets more updates ($n_{critic} = 5$).

Answer: B

Q19.19 MCQ · M

CycleGAN enables unpaired image translation mainly through

- (A) paired L1 loss
- (B) cycle consistency: translating to the other domain and back should reconstruct the input
- (C) class labels
- (D) K-Means

Solution

- $F(G(x)) \approx x$ and $G(F(y)) \approx y$ constrain the mappings without paired examples.

Answer: B

Q19.20 MCQ · M

pix2pix differs from CycleGAN because it

- (A) needs paired training images
- (B) uses no discriminator
- (C) is unsupervised
- (D) uses RNNs

Solution

- Paired (input, target) images plus conditional GAN + L1 loss.

Answer: A

Q19.21 MSQ · M

FID (Fréchet Inception Distance):

- (A) Lower is better.
- (B) Compares feature statistics of real and generated images.
- (C) Penalises both poor quality and poor diversity.
- (D) Needs human raters.

Solution

- Fully automatic with Inception features (D false).

Answer: A, B, C

Q19.22 MCQ · M

The Inception Score is high when

- (A) samples are blurry
- (B) each sample is confidently classified and samples cover many classes
- (C) all samples are identical
- (D) images are grayscale

Solution

- $IS = \exp(\mathbb{E}_x KL(p(y | x) || p(y)))$: sharp conditionals, broad marginal.

Answer: B

Q19.23 MSQ · M

One sided label smoothing in GANs:

- (A) Replace real labels 1 with about 0.9 for the discriminator.
- (B) Keep fake labels at 0.
- (C) Prevents the discriminator from becoming over confident.
- (D) Also smooths the generator's labels to 0.1.

Solution

- Smoothing only the real side avoids rewarding fakes (hence "one sided"); D false.

Answer: A, B, C

Q19.24 MSQ · M

GAN vs VAE:

- (A) GANs typically produce sharper images.
- (B) VAEs optimise a likelihood lower bound (ELBO).
- (C) VAEs have an encoder giving a latent representation of inputs.
- (D) GANs give an explicit likelihood for any image.

Solution

- GANs are implicit models: no tractable likelihood (D false).

Answer: A, B, C

Q19.25 MSQ · M

Diffusion models vs GANs:

- (A) Diffusion training is typically more stable.
- (B) Diffusion sampling traditionally needs many denoising steps, so it is slower.
- (C) Diffusion models suffer less from mode collapse.
- (D) Diffusion models use a discriminator.

Solution

- They learn to denoise with a simple regression loss; no adversary (D false).

Answer: A, B, C

Q19.26 MCQ · M

During the generator's update step

- (A) the discriminator's weights are also updated to help G
- (B) the discriminator is held fixed; gradients flow through it into G
- (C) real images are fed to G
- (D) the noise is not used

Solution

- D acts as a differentiable critic; only G 's optimiser steps.

Answer: B

Q19.27 MCQ · M

Generator output layer typically uses tanh because

- (A) images are scaled to $[-1, 1]$
- (B) it is faster
- (C) it prevents mode collapse
- (D) it gives probabilities

Solution

- Inputs to D (real images) are normalised to $[-1, 1]$, so fakes must be in the same range.

Answer: A

Q19.28 MSQ · H

Spectral normalisation in the discriminator:

- (A) divides each weight matrix by its largest singular value
- (B) constrains the Lipschitz constant of D
- (C) stabilises GAN training
- (D) removes the need for any discriminator

Solution

- Links to Chapter 11: $\|W\|_2 = \sigma_1$, estimated cheaply by power iteration. D false.

Answer: A, B, C

Q19.29 MCQ · M

If the discriminator's accuracy quickly reaches 100% and stays there, the most likely issue is

- (A) the generator receives vanishing gradients and stops improving
- (B) the GAN has converged perfectly
- (C) mode collapse is impossible
- (D) learning rates are too small for D

Solution

- Rebalance: fewer D steps, lower D learning rate, label smoothing, noise on inputs, non saturating loss or WGAN.

Answer: A

Q19.30 MSQ · M

Common latent space facts:

- (A) Latent vectors are usually sampled from $\mathcal{N}(0, I)$ or uniform.
- (B) Interpolating between two latent codes often gives smooth transitions between images.
- (C) Vector arithmetic in latent space can change attributes (for example adding glasses).
- (D) The latent dimension must equal the image size.

Solution

- Typical latent size is 100 to 512, much smaller than the image (D false).

Answer: A, B, C

Q19.31 MSQ · M

Useful applications of GANs:

- (A) Image super resolution (SRGAN)
- (B) Synthetic tabular data for rare classes (CTGAN)
- (C) Image to image translation
- (D) Computing exact data likelihoods

Solution

- GANs have no tractable likelihood (D false).

Answer: A, B, C

Q19.32 MCQ · M

Adam settings commonly recommended for DCGAN training are

- (A) $\beta_1 = 0.9$, learning rate 0.1
- (B) $\beta_1 = 0.5$, learning rate 0.0002
- (C) no momentum, learning rate 1
- (D) SGD only

Solution

- Lower momentum reduced oscillation in the adversarial game (DCGAN paper).

Answer: B**Q19.33** MSQ · H

Deepfake and fake profile photo detection:

- (A) GAN generated faces can leave frequency domain artefacts that detectors exploit.
- (B) A discriminator trained with one generator may not generalise to images from other generators.
- (C) Detection is a binary classification task that CNNs or ViTs can learn.
- (D) Detectors never need retraining.

Solution

- New generators change artefacts, so detectors must be updated (D false).

Answer: A, B, C**Q19.34** MCQ · E

The “adversarial” in GAN refers to

- (A) adversarial attacks on classifiers
- (B) the competition between generator and discriminator
- (C) a regulariser
- (D) the dataset

Solution

- The two networks compete in a zero sum game: G tries to fool D , D tries not to be fooled.
- Adversarial attacks (tiny perturbations that fool classifiers) are a separate topic with a similar name, a common OA distractor.

Answer: B**Q19.35** MCQ · M

A GAN’s generator and discriminator losses both look flat, yet samples keep improving visually. The best conclusion is

- (A) training has failed
- (B) GAN losses are not reliable progress measures; track FID and inspect samples
- (C) increase the learning rate tenfold
- (D) remove the discriminator

Solution

- In an adversarial game both players improve together, so each loss can stay roughly constant while quality rises. Use FID on a held out set and periodic sample grids.

Answer: B**Interview angle**

- Write the GAN objective and derive the optimal discriminator; show the JS divergence connection.
- Why the non saturating loss? What is mode collapse and how do you fight it?
- WGAN: what changes and why does it help?
- GAN vs VAE vs diffusion: trade offs; which would you pick for synthetic tabular data or image generation today?
- How do you evaluate a generative model (FID, IS, precision/recall)?